Reversing en Windows

Tomo II — Ramas Especializadas: Kernel, Drivers e Internals

Profundización avanzada en el sistema operativo Windows a nivel Ring 0 y subsistemas de plataforma: arquitectura del kernel NT, drivers WDM/KMDF, IOCTLs, depuración de kernel con WinDbg (KDNET), minifiltros de sistema de archivos, mitigaciones HVCI/VBS, Windows Filtering Platform (WFP), kCFG a nivel ISA y CET/IBT, Fuzzing de drivers en kernel (WTF, kAFL/Nyx), objetos del ejecutivo, procesos/hilos, Registro, servicios, Task Scheduler, COM/DCOM, RPC, WMI, ALPC, excepciones x64 (.pdata/UNWIND_INFO), ETW/ETWti, virtualización con Hyper-V/VMBus, mitigación Win32k System Call Lockdown y cadena de arranque segura UEFI/bootkits.

Contenido y Balance Editorial de este Volumen: Comprende la **Etapa 6 completa (Lecciones 6.1 a 6.10)**, la **Rama 2D completa (Lecciones 2D.1 a 2D.17)** y el **Anexo A (Fe de Erratas Técnica y Trazabilidad con Tomo I)**, en su versión íntegra, exhaustiva y sin condensación de formato.

Requisitos previos: Este tomo asume el dominio del tronco principal (Etapas 0 a 4 del **Tomo I**). Para las sesiones prácticas de laboratorio en Ring 0 se recomienda tener abierta la **Separata de Escritorio** (Hojas 1 a 6).

> **Seguridad de Laboratorio Obligatoria:** Toda práctica con controladores de kernel (`.sys`), depuración con `bcdedit /debug on` , Driver Verifier, Special Pool o interceptores de red debe realizarse exclusivamente dentro de una **máquina virtual aislada y descartable**, nunca en la estación de trabajo física.

Contenido — Tomo II

Etapa 6 · Kernel y drivers en Windows (Lecciones 6.1 a 6.10) pp. 4–5 (2 págs.)

6.1 Arquitectura del kernel y modelo de E/S **pp. 6–11 (6 págs.)**

6.2 KMDF: driver propio con IOCTL inocuo **pp. 12–21 (10 págs.)**

6.3 Debugging de kernel y bugchecks **pp. 22–26 (5 págs.)**

6.4 Revisión defensiva de un driver **pp. 27–37 (11 págs.)**

6.5 Minifilters y callbacks **pp. 38–45 (8 págs.)**

6.6 Mitigaciones de Windows para drivers **pp. 46–50 (5 págs.)**

6.7 Seguridad profunda en kernel: VBS/HVCI, Callbacks y BYOVD **pp. 51–62 (12 págs.)**

6.8 Windows Filtering Platform (WFP): capas ALE y callouts **pp. 63–69 (7 págs.)**

6.9 kCFG a nivel ISA, Bitmap y contraste con CET/IBT **pp. 70–76 (7 págs.)**

6.10 Fuzzing de drivers en modo kernel (IOCTL, WTF, kAFL) **pp. 77–86 (10 págs.)**

Rama 2D · Windows Internals y Virtualización (Lecciones 2D.1 a 2D.17) pp. 87–88 (2 págs.)

2D.1 Objetos del ejecutivo y tokens de seguridad **pp. 89–93 (5 págs.)**

2D.2 Memoria virtual y el loader de Windows **pp. 94–98 (5 págs.)**

2D.3 Arquitectura del Registro de Windows **pp. 99–102 (4 págs.)**

2D.4 Servicios de Windows (SCM) **pp. 103–106 (4 págs.)**

2D.5 Programador de tareas (Task Scheduler) **pp. 107–109 (3 págs.)**

2D.6 Component Object Model (COM): Interfaces **pp. 110–119 (10 págs.)**

2D.7 Activación y servidores COM **pp. 120–125 (6 págs.)**

2D.8 DCOM y RPC sobre ALPC/TCP **pp. 126–134 (9 págs.)**

2D.9 Windows Management Instrumentation (WMI) **pp. 135–144 (10 págs.)**

2D.10 Comunicación entre procesos (IPC) y puertos ALPC **pp. 145–153 (9 págs.)**

2D.11 Manejo de excepciones x64 (.pdata, SEH, VEH) **pp. 154–161 (8 págs.)**

2D.12 Event Tracing for Windows (ETW) y ETWti **pp. 162–168 (7 págs.)**

2D.13 Laboratorio integrador: análisis guiado **pp. 169–172 (4 págs.)**

2D.14 Subsistema gráfico Win32k y Shadow SSDT **pp. 173–176 (4 págs.)**

2D.15 Cadena de arranque UEFI, winload y bootkits **pp. 177–181 (5 págs.)**

2D.16 Hyper-V, particiones, VMBus y virtualización **pp. 182–187 (6 págs.)**

2D.17 Arquitectura profunda de VBS, Secure Kernel y HVCI **pp. 188–193 (6 págs.)**

Anexos Editoriales

Anexo A · Fe de Erratas Técnica y Matriz de Trazabilidad con Tomo I **pp. 194–198 (5 págs.)**

Etapa 6 · Rama optativa · Kernel y drivers

> **Objetivo medible**
>
> Comprender, construir y revisar un driver KMDF didáctico con una interfaz inocua, depurarlo solo en VM descartable y explicar mitigaciones actuales.

Prerrequisitos: Etapa 0 (laboratorio aislado), C, assembly, memoria y sincronización, WinDbg y 2D.1 (objetos/tokens), 2D.2 (memoria/loader) y 2D.11 (excepciones/debugging). Etapa 2C es recomendada, no requisito; no hace falta completar toda 2D. **Duración estimada:** 45–65 horas como estimación inicial; puede requerir más tiempo si C, WinDbg o WDK son nuevos.. Es una estimación de esta guía, derivada de la suma de las lecciones; no es una duración rígida de laboratorio.

> **Rama optativa; fuera de la aprobación obligatoria**
>
> La Etapa 6 profundiza en kernel/drivers y no cambia los requisitos de las Etapas 0–4. Sus prácticas de carga requieren una VM descartable; nunca modifiques las protecciones del equipo principal.

Lecciones

#	Lección	Qué aprendés
6.1	Arquitectura del kernel y modelo de E/S	user/kernel mode, IRQL, pool, IRP, I/O Manager, sincronización y bugchecks.
6.2	KMDF: driver propio con IOCTL inocuo	KMDF/UMDF y una respuesta fija con permisos restrictivos; sin acceso arbitrario a memoria.
6.3	Debugging de kernel y bugchecks	WinDbg en target VM, stacks/dumps y Driver Verifier con rollback.
6.4	Revisión defensiva de un driver	device interfaces, IOCTLs, SDDL y validación de entradas.
6.5	Minifilters y callbacks	Pila de filesystem, callbacks pre/post y límites de observación.
6.6	Mitigaciones de Windows para drivers	firma, Secure Boot, HVCI/VBS, WDAC y vulnerable driver blocklist.
6.7	Seguridad profunda en kernel: VBS/HVCI, Callbacks y BYOVD	Secure Kernel (VTL 1), telemetría defensiva (ETWti, ObRegisterCallbacks, InstrumentationCallback), PiDDBCacheTable y doble lectura en MDLs.
6.8	Windows Filtering Platform (WFP): capas ALE y callouts	BFE, fwpkclnt.sys, filtrado e inspección de paquetes en Ring 0 y clasificación con classifyFn.
6.9	kCFG a nivel ISA, Bitmap y contraste con CET/IBT	guard_dispatch_icall, bitmap CFG, tablas GuardCFFunctionTable, diagnóstico bugcheck 0x139 y endbr64.
6.10	Fuzzing de drivers en modo kernel: IOCTL, WTF y kAFL/Nyx	Superficie IOCTL, Special Pool en PTEs, fuzzing por snapshots con WTF y cobertura Intel PT con kAFL.

Recursos y lecturas de referencia

- 2D · Windows internals (solo lecciones requeridas para esta rama)
- Extensión Hyper-V (optativa, no requisito)

Evaluación de salida

- Explicar la ruta de una solicitud y los límites de IRQL, memoria y sincronización.
- Revisar un IOCTL propio de salida fija, con acceso limitado y validación completa; si no se puede cargar en VM conforme a la política vigente, entregar revisión estática.
- Leer un stack/dump de driver propio o muestra didáctica, indicando límites de símbolos y causalidad.
- Verificar mitigaciones en solo lectura y explicar qué protegen, sin desactivarlas.
- Demostrar rollback/limpieza de la VM; Driver Verifier no es requisito de aprobación.

> **Entregable de la etapa**
>
> Un archivo en tu bitácora con: lecciones completadas, ejercicios resueltos, dudas abiertas y evidencia (capturas, valores, hashes). Sin evidencia no se considera aprobada.

Errores frecuentes

- Probar un driver en el equipo principal o desactivar protecciones para forzar su carga.
- Crear IOCTLs de lectura/escritura arbitraria o asumir que una ACL reemplaza la validación.
- Interpretar el módulo del bugcheck como causa raíz sin revisar stack y contexto.

Qué no entra en esta etapa

Los temas que no tienen material revisado o que exceden el marco se listan en Kernel y otros territorios. No los uses como atajo: cada rama tiene prerrequisitos.

Arquitectura del kernel y modelo de E/S

Rama optativa de alto impacto

Todo el bloque requiere VM descartable con snapshot. Un error en kernel puede colgar o detener el invitado; nunca pruebes un driver en el host ni en un equipo de uso diario.

Propósito

Construir el mapa mental de la ruta de una solicitud desde user mode al driver y reconocer qué estado/memoria es válido según IRQL. Al terminar deberías poder mirar una rutina de un driver y decir **a qué IRQL** corre, **en el contexto de qué hilo**, **qué memoria** puede tocar y **qué** podría hacer que el sistema se detenga.

Prerrequisitos

- *Obligatorios para Etapa 6:* C básico, assembly, memoria/stack, sincronización elemental, WinDbg y 2D.1 objetos/tokens, 2D.2 memoria/loader, 2D.11 excepciones/debugging.
- *Recomendados:* 2C WinDbg/heap y fundamentos de kernel; no exige COM, Scheduler ni toda 2D.

Puente Pedagógico: De Modo Usuario a Modo Kernel (Transición Etapa 5 $\to$ Etapa 6)

Al culminar la Etapa 5, el analista domina la disección de malware, el análisis estático y dinámico de ejecutables y la auditoría de vulnerabilidades en **espacio de usuario (Ring 3)**. Al ingresar a la Etapa 6 y a la rama de controladores de dispositivo (**Modo Kernel / Ring 0**), las reglas del juego cambian drásticamente. Esta transición exige reconectar los modelos mentales aprendidos previamente:

Concepto en Modo Usuario (Etapa 5)	Equivalente en Modo Kernel (Etapa 6)	Impacto Arquitectural y Diferencia Crítica
Fallo de Acceso / Crash (`0xC0000005`)	**Bugcheck / BSOD (`0x50`, `0xD1`, `0x3B`)**	En user-mode, un crash termina únicamente el proceso afectado. En kernel-mode, cualquier desreferencia nula o corrupción de memoria detiene por completo el sistema operativo para evitar la corrupción de datos en disco.
Vulnerabilidad en Parser de Archivos (CWE-190/787)	**Fallo de Validación en Handler de IOCTLs**	Un desbordamiento o cálculo de tamaño incorrecto en un buffer de `DeviceIoControl` permite a un usuario sin privilegios escribir en la memoria del kernel o ejecutar código arbitrario en Ring 0.
Depuración Local con x64dbg	**Depuración Remota con WinDbg y KDNET**	No se puede pausar un driver desde la misma máquina porque detener el kernel congela la pantalla, el teclado y el scheduler. Es obligatorio depurar con dos máquinas enlazadas por red (VM objetivo + Host depurador).
Asignación de Memoria (`malloc` / `VirtualAlloc`)	**Asignación en Pools (`ExAllocatePool2` Paged/NonPaged)**	En kernel no existe un heap gestionado automáticamente con protección de usuario: el desarrollador debe elegir explícitamente entre memoria paginable (*PagedPool*) o memoria física no paginable (*NonPagedPoolNx*), condicionada por el **IRQL**.
Mitigaciones de Proceso (DEP, ASLR, CFG)	**Mitigaciones de Kernel (SMEP, SMAP, kCFG, HVCI/VBS)**	El procesador y el hipervisor impiden activamente que el kernel ejecute código en páginas de usuario (SMEP) o lea memoria de usuario sin intermediación (SMAP).

Mapa del kernel

Las aplicaciones corren en **user mode** con privilegios limitados; el kernel y drivers operan en **kernel mode**, donde un fallo puede afectar al sistema entero. El **I/O Manager** traduce solicitudes en IRP y las dirige por una pila de drivers. El **IRQL** limita qué rutinas/operaciones son válidas en cada contexto; a IRQL elevado no se puede asumir que una operación bloqueante o memoria paginable sea segura.

El **pool** es memoria del kernel con políticas de paginación/NX. La sincronización protege estado compartido entre CPU, callbacks e interrupciones. Un **bugcheck** detiene Windows al detectar una condición grave; el dump y stack ayudan a atribuirla, aunque el driver señalado puede no ser el origen.

IRQL: qué se puede hacer en cada nivel

El IRQL al que corre una rutina determina qué funciones de soporte puede llamar. Los niveles que más vas a ver, de menor a mayor:

IRQL	Qué rutinas típicas corren ahí	Qué se puede y qué no
`PASSIVE_LEVEL`	`DriverEntry`, `AddDevice`, `Unload`, la mayoría de las rutinas de despacho, hilos del driver, callbacks de *worker threads*	Hay contexto de hilo; se puede esperar y el código/datos pueden ser paginables
`APC_LEVEL`	Algunas rutinas de despacho	Igual que `PASSIVE_LEVEL`, salvo que no se reciben APC; el código puede ser paginable
`DISPATCH_LEVEL`	`StartIo`, DPC (`DpcForIsr`, `CustomDpc`), timers, rutinas de completion de drivers de nivel superior, `Cancel` con el spin lock tomado	**No** se puede esperar un objeto con intervalo no nulo; **no** se puede tocar memoria paginable; los spin locks suben a este nivel
DIRQL	Rutina de interrupción (ISR), `SynchCritSection`	Lo mínimo indispensable; se difiere el resto a un DPC

Dos reglas documentadas que explican muchos bugchecks: una rutina por encima de `APC_LEVEL` no puede asignar ni acceder a pool paginado (un fallo de página ahí es fatal), y esperar con intervalo no nulo a `DISPATCH_LEVEL` o más también es fatal. Cuanto más alto el IRQL, más corta debe ser la rutina.

Contexto de hilo

La mayoría de las rutinas de un driver corren en un **contexto de hilo arbitrario**: el del hilo que estaba ejecutándose cuando se las llamó, que puede no tener nada que ver con la aplicación que pidió la operación. Solo los drivers de más alto nivel (por ejemplo, la parte de un sistema de archivos que atiende la llamada) corren en el contexto del hilo que hizo la solicitud. Consecuencias:

- Una dirección de **user mode** solo tiene sentido en el contexto del proceso que la pasó; por eso los métodos de buffer de 6.2 y el patrón de `METHOD_NEITHER` de 6.4.
- El estado de cada dispositivo se guarda en la **extensión del dispositivo** (en KMDF, en el contexto del objeto), no en variables del hilo.

Objetos de dispositivo, pilas y stack locations

Cada dispositivo se representa con una **pila de objetos de dispositivo**: el **PDO** que crea el driver de bus, el **FDO** del driver de función y, opcionalmente, **filtros** por encima o por debajo. Un IRP entra por arriba y baja por la pila; cada driver tiene en el IRP su propia **I/O stack location** con los parámetros que le corresponden (código de función, longitudes, código de IOCTL). Al completarse, el IRP sube y los drivers que registraron rutinas de *completion* las ejecutan, normalmente a IRQL elevado. El resultado viaja en `IoStatus.Status` y la cantidad de bytes en `IoStatus.Information`.

En KMDF todo esto existe igual, pero el framework lo encapsula en objetos request, colas y callbacks (6.2).

Pool y NX

El pool **no paginado** se puede usar a cualquier IRQL, pero es un recurso escaso; el **paginado** solo por debajo de `DISPATCH_LEVEL`. Desde Windows 8, la documentación pide asignar la memoria no paginada desde el pool **no ejecutable** (`NonPagedPoolNx`) y usar el pool ejecutable solo cuando haga falta ejecutar código ahí. En un análisis, una asignación de pool ejecutable en un driver moderno es algo a explicar; con integridad de memoria (HVCI, ver 6.6) las asignaciones de kernel están más restringidas.

Ejemplo trabajado

(Diagrama conceptual.)

```
app: DeviceIoControl
  → I/O Manager crea IRP
  → pila: filtro → function driver → bus
  → completion/status vuelve al cliente
```

En KMDF, el framework recibe/organiza IRP y llama a callbacks del driver. En WDM, el código trabaja más directamente con dispatch routines e IRPs. No deduzcas que una app se comunica con hardware solo por llamar `DeviceIoControl`: puede ser un dispositivo software.

Lectura de un bugcheck **ilustrativo** con los conceptos de esta lección (no es un dump real):

```
Bug check 0xD1 DRIVER_IRQL_NOT_LESS_OR_EQUAL
  Arg1: dirección referenciada
  Arg2: 2              ; IRQL en ese momento = DISPATCH_LEVEL
  Arg3: 0              ; operación de lectura
  Arg4: dirección de la instrucción que leyó
```

Interpretación prudente: a `DISPATCH_LEVEL` se leyó una dirección paginable o inválida. Microsoft lista como causas típicas código o datos paginables usados a ese IRQL (por ejemplo, una función marcada paginable que corre con un spin lock tomado), un puntero inválido o liberado, o una llamada a un driver que ya se descargó. El IRQL no suele ser el problema en sí: lo es la memoria accedida. La 6.3 muestra cómo seguir cada parámetro.

Práctica de lectura

Usá un diagrama/documentación de un driver propio o una muestra WDK ya compilada. Identificá user/kernel mode, entrada IRP, callback, cola, IRQL declarado y finalización. No cargues ni modifiques drivers en esta lección.

1. Para cada callback de la muestra, anotá a qué IRQL puede llamarse según la documentación.
2. Marcá cada asignación de memoria como paginada, no paginada o no paginada NX, y verificá que las que se usan a `DISPATCH_LEVEL` no sean paginadas.
3. Señalá dónde el driver depende del contexto del hilo llamador, si lo hace.

Entregable

Diagrama de la ruta de una solicitud (con pila de dispositivos y stack locations), tabla callback–IRQL–memoria usada, glosario IRP/IRQL/pool/DPC/bugcheck y dos condiciones que podrían invalidar una rutina en IRQL elevado.

Repaso

▼ **¿Qué transporta un IRP?**

> Una solicitud de E/S y su estado/resultado a través de la pila de drivers.

▼ **¿Por qué importa IRQL?**

> Restringe qué operaciones y memoria son válidas en el contexto actual.

▼ **¿Qué no puede hacer una rutina a `DISPATCH_LEVEL`?**

> Esperar con intervalo no nulo ni acceder a memoria paginable; un fallo de página ahí es fatal.

▼ **¿Qué significa que un driver corre en contexto de hilo arbitrario?**

> Que el hilo actual puede no ser el que pidió la operación; las direcciones de user mode del solicitante no son válidas en general.

▼ **¿Qué es una I/O stack location?**

> La parte del IRP con los parámetros para un driver concreto de la pila.

▼ **¿Por qué se prefiere `NonPagedPoolNx`?**

> Porque es pool no paginado no ejecutable; el ejecutable solo se usa si hace falta ejecutar código en esa memoria.

▼ **¿Un bugcheck identifica siempre al culpable?**

> No necesariamente; muestra dónde se detectó el problema. El análisis debe reconstruir el estado y las llamadas previas.

Puerta optativa

Podés explicar la ruta user mode → I/O Manager → pila de dispositivos → driver, decir a qué IRQL corre un callback y qué memoria puede usar, y leer los parámetros de un 0xD1 sin atribuirle una causa única.

Lo que queda fuera

El scheduler, APC en detalle, la arquitectura de la memoria virtual del kernel, PnP y energía (ver el recuadro de 6.4), DMA e interrupciones de hardware no se desarrollan.

Fuentes

- Microsoft Learn: Overview of the Windows I/O Model, IRQL, POOL_TYPE (pool paginado, no paginado y NX), Synchronization.
- Microsoft Learn, contexto y pilas: Driver Thread Context, Introduction to Device Objects, Device nodes and device stacks, I/O Stack Locations, Introduction to DPC Objects, Introduction to Spin Locks, No-Execute Nonpaged Pool, Bug Check 0xD1.
- **Fundamentos previos:** stack/memoria 1.2; fundamentos user-mode en 2D.1.

Etiquetas

Núcleo conceptual: conceptos de memoria y procesos. **Conocimiento externo:** arquitectura del kernel/driver, IRQL, contexto de hilo, pilas de dispositivos y pool. **Ejemplo ilustrativo:** diagrama IRP y lectura de 0xD1. **Laboratorio verificado:** no; lectura conceptual.

KMDF: driver propio con IOCTL inocuo

> **Solo en VM de pruebas**
>
> La carga de drivers altera el kernel. Necesitás VM descartable, snapshot conocido, consola de recuperación y un paquete propio. No cambies Secure Boot, firmas ni protecciones del equipo principal. Si la VM no puede probar un driver firmado según la documentación vigente del WDK, no cargues uno sin firma: completá el análisis estático.

> **Propósito**
>
> Entender cómo WDF traduce solicitudes a callbacks y practicar un canal mínimo con límites estrictos: un IOCTL consulta una estructura fija, sin lectura/escritura arbitraria de memoria. Al terminar deberías poder seguir una solicitud desde la aplicación hasta el driver, decir **en qué puntos** se controla el acceso y **dónde** se puede observar cada paso.

Prerrequisitos

- *Obligatorios:* 6.1 arquitectura, C/estructuras, IRP/IRQL y 2D.1 ACL.
- *Recomendados:* 2D.2 memoria/loader; 6.3 debugging de kernel; documentación de instalación WDK para el build exacto de la VM.

WDF y la interfaz

WDF ofrece dos marcos: **KMDF** ejecuta el driver en kernel; **UMDF** hospeda drivers compatibles en user mode, con menor impacto potencial ante fallos. El marco encapsula parte del modelo WDM, pero no elimina responsabilidades de seguridad.

El laboratorio se limita a un dispositivo software KMDF. Una única solicitud `IOCTL_BLOCLAB_QUERY_VERSION` acepta cero bytes y devuelve una estructura fija `{Size, Version, State}`. El callback verifica código, longitudes y tamaño del buffer; cualquier código desconocido se rechaza. Elegí `METHOD_BUFFERED` para este intercambio pequeño y aplica SDDL restrictiva (por ejemplo, acceso solo a Administrators/SYSTEM en la VM). No incluyas IOCTL de lectura/escritura de direcciones, mapas físicos, procesos o memoria arbitraria.

Recorrido de una solicitud IOCTL

Seguí el camino completo; cada paso tiene su propio control y su propio punto de observación.

1. **Apertura del dispositivo.** La aplicación abre el dispositivo con `CreateFile`, normalmente a través de una **interfaz de dispositivo** (o, en drivers antiguos, de un nombre o enlace simbólico). El I/O manager compara el token del llamador con el **descriptor de seguridad del dispositivo** y entrega un handle con los derechos concedidos. WDF siempre marca sus objetos de dispositivo con `FILE_DEVICE_SECURE_OPEN`, para que ese descriptor también se aplique a cualquier nombre "dentro" del namespace del dispositivo.
2. **Envío del código.** La aplicación llama a `DeviceIoControl(handle, código, entrada, salida…)`. El I/O manager mira el campo **Access** del código: si exige `FILE_READ_DATA` o `FILE_WRITE_DATA` y el handle no tiene ese derecho, **la solicitud no llega al driver**. Con `FILE_ANY_ACCESS`, basta con tener un handle.

3. **Construcción del IRP.** El I/O manager crea un IRP `IRP_MJ_DEVICE_CONTROL`. Con `METHOD_BUFFERED`, reserva **un único buffer de sistema** del tamaño del mayor entre entrada y salida, copia ahí la entrada y lo expone en `Irp->AssociatedIrp.SystemBuffer`. El driver lee la entrada **antes** de escribir la salida en el mismo buffer.
4. **Del IRP al objeto request.** Como el driver es de WDF, el framework atiende el despacho, crea un **objeto request** (`WDFREQUEST`) que representa el IRP y lo coloca en una **cola** (`WDFQUEUE`). Según cómo se configuró la cola (secuencial, paralela o manual), el framework entrega la solicitud.
5. **Callback del driver.** El framework llama a `EvtIoDeviceControl(Queue, Request, OutputBufferLength, InputBufferLength, IoControlCode)`. El driver ya recibe las longitudes y el código: ahí valida.
6. **Acceso al buffer.** `WdfRequestRetrieveOutputBuffer(Request, MinimumRequiredSize, &buffer, &length)` devuelve el buffer solo si mide al menos `MinimumRequiredSize`; si no, devuelve `STATUS_BUFFER_TOO_SMALL`. Pasar `sizeof(BLOCLAB_INFO)` como mínimo hace que el propio framework rechace buffers cortos.
7. **Finalización.** El driver completa con `WdfRequestCompleteWithInformation(Request, status, bytes)`. El I/O manager copia `bytes` del buffer de sistema al buffer de salida del usuario y `DeviceIoControl` devuelve, con `lpBytesReturned` igual a esos bytes. Después de completar, el driver ya no puede tocar el buffer.

```
App (user)        CreateFile ──► [ACL del dispositivo] ──► handle (derechos)
                  DeviceIoControl(code) ──► [bits Access del código vs. handle]
I/O manager       IRP_MJ_DEVICE_CONTROL + SystemBuffer (METHOD_BUFFERED)
WDF               WDFREQUEST ──► WDFQUEUE ──► EvtIoDeviceControl
Driver            valida código/longitudes ──► RetrieveOutputBuffer(min) ──► llena todo
                  ──► CompleteWithInformation(status, bytes)
I/O manager       copia bytes al buffer de la app ──► DeviceIoControl retorna
```

Qué recibe el driver según el método del IOCTL

Los dos bits **Method** del código deciden cómo describe el I/O manager los dos buffers de `DeviceIoControl` (el primero, “de entrada”, y el segundo, “de salida” en los nombres de la API). Desde el lado del driver:

Método	Primer buffer (entrada)	Segundo buffer (salida)	Qué hizo el I/O manager antes	Cómo lo obtiene un driver KMDF
`METHOD_BUFFERED`	`Irp->AssociatedIrp.SystemBuffer`	El **mismo** `SystemBuffer`	Reservó un buffer de sistema del tamaño mayor y copió la entrada; al completar copia `Information` bytes a la app.	`WdfRequestRetrieveInputBuffer` y `WdfRequestRetrieveOutputBuffer` (mismo buffer de sistema).
`METHOD_IN_DIRECT` / `METHOD_OUT_DIRECT`	`SystemBuffer` (copia, `InputBufferLength`)	Un **MDL** en `Irp->MdlAddress` que describe las páginas del buffer de la app (`OutputBufferLength`)	Verificó el acceso al segundo buffer y bloqueó sus páginas. `IN_DIRECT` asegura acceso de lectura (datos hacia el driver); `OUT_DIRECT`, de escritura (datos hacia la app).	Primer buffer como en buffered; el segundo, con `WdfRequestRetrieveOutputBuffer` (dirección de sistema) o `WdfRequestRetrieveOutputWdmMdl` (el MDL).
`METHOD_NEITHER`	`Parameters.DeviceIoControl.Type3InputBuffer`: dirección **de usuario**	`Irp->UserBuffer`: dirección **de usuario**	**Nada**: no copia, no valida, no bloquea.	Solo en `EvtIoInCallerContext` (contexto del hilo llamador): `WdfRequestRetrieveUnsafeUserInputBuffer` / `…OutputBuffer` y luego `WdfRequestProbeAndLockUserBufferForRead` / `…ForWrite`, antes de devolver la solicitud a una cola.

Tres consecuencias para leer un driver:

- Con `METHOD_BUFFERED` el driver no toca memoria de usuario… **salvo** que la estructura de entrada contenga punteros. Un puntero embebido es una dirección de usuario sin validar, con los mismos riesgos que `METHOD_NEITHER`.
- Con direct I/O las páginas del segundo buffer están bloqueadas y accesibles, pero siguen perteneciendo al proceso: conviene tratar su contenido como dato no confiable que puede cambiar mientras el driver lo lee (conocimiento externo; la documentación citada describe el bloqueo, no esta garantía).
- Con `METHOD_NEITHER`, el driver asume todo el trabajo que en los otros métodos hace el I/O manager, y solo un driver del nivel más alto, en el contexto del hilo que llamó, puede hacerlo. Por eso en la 6.4 se trata como **patrón de revisión de riesgo**.

El laboratorio de esta lección usa `METHOD_BUFFERED` y no tiene punteros embebidos a propósito.

Controles de acceso: cuatro capas que no se reemplazan

Capa	Qué controla	Cómo se fija	Límite
Exposición del dispositivo	Qué caminos existen para abrirlo.	Preferir **interfaces de dispositivo**; nombrar el objeto o crear enlaces simbólicos solo si hace falta.	Cada nombre adicional es otro camino de apertura.
Descriptor de seguridad del dispositivo	**Quién** puede abrir un handle y con qué derechos.	`WdfDeviceInitAssignSDDLString` en el driver o seguridad declarada en el INF. Si el driver nombra el dispositivo sin SDDL propia, WDF aplica `SDDL_DEVOBJ_SYS_ALL_ADM_ALL`.	No valida el contenido de las solicitudes.
Bits **Access** del código IOCTL	**Qué derechos** del handle exige cada operación.	En `CTL_CODE`: `FILE_ANY_ACCESS`, `FILE_READ_DATA`, `FILE_WRITE_DATA`. Microsoft advierte que `FILE_ANY_ACCESS` solo se use si se está seguro de que no abre un camino de abuso. `IoValidateDeviceIoControlAccess` permite controles más estrictos.	Solo mira derechos del handle, no los datos.
Validación en el callback	**Qué datos** acepta el driver.	Lista cerrada de códigos, longitudes exactas, tamaño mínimo al pedir buffers, inicialización completa de la salida.	Es la última línea: si falla, las capas anteriores no compensan.

Ejemplo trabajado

(Contrato ilustrativo, no salida de compilación ni de una VM.)

```
IOCTL: BLOCLAB_QUERY_VERSION
  = CTL_CODE(0x8000, 0x800, METHOD_BUFFERED, FILE_READ_DATA)
  = (0x8000 << 16) | (1 << 14) | (0x800 << 2) | 0
  = 0x80006000
Input bytes: 0
Output bytes: sizeof(BLOCLAB_INFO) = 12   ; tres ULONG
Respuesta fija: { Size=12, Version=1, State=READY }
Otros códigos / buffer corto → error y sin escritura fuera del buffer
```

Lectura del código: `DeviceType` `0x8000` (rango de fabricante, con el bit *Common* encendido), función `0x800` (rango de fabricante, bit *Custom*), `METHOD_BUFFERED` y `FILE_READ_DATA`: un handle abierto sin derecho de lectura **no** llega al driver con esta solicitud. El mismo desglose sirve en reversing para leer constantes que aparecen en el `switch` de un driver ajeno.

Tabla de casos que debería producir el diseño:

Caso	Dónde se decide	Resultado esperado
Handle sin `FILE_READ_DATA`	I/O manager (bits Access)	La solicitud no llega a `EvtIoDeviceControl`; error de acceso.
Código desconocido	Callback (`default:` del `switch`)	`STATUS_INVALID_DEVICE_REQUEST`, 0 bytes.
Salida menor que 12 bytes	`WdfRequestRetrieveOutputBuffer` con mínimo 12	`STATUS_BUFFER_TOO_SMALL`, 0 bytes, sin escritura.
Entrada distinta de 0 bytes	Callback (comparación de `InputBufferLength`)	`STATUS_INVALID_PARAMETER` (decisión de diseño del laboratorio).
Caso válido	Callback	`STATUS_SUCCESS`, 12 bytes, estructura completa.

La verificación del tamaño debe ocurrir antes de obtener o copiar buffers. La respuesta se inicializa por completo para no filtrar pool; no contiene punteros ni direcciones del kernel.

Puntos de observación defensiva

Sirven tanto para tu driver como para revisar uno ajeno en la 6.4. Separá lo que se puede ver **sin cargar nada** de lo que requiere la VM.

Estático (sin cargar el driver):

- **INF y paquete:** clase de dispositivo, seguridad declarada, servicio que crea.
- **Cadenas SDDL** en el binario o en el INF, y llamadas a `WdfDeviceInitAssignSDDLString`, `WdfDeviceInitAssignName`, `WdfDeviceCreateSymbolicLink` o `WdfDeviceCreateDeviceInterface`.
- **Configuración de colas:** dónde se asigna `EvtIoDeviceControl` y con qué modo de despacho.
- **Códigos IOCTL:** constantes del `switch`, desglosadas en `DeviceType`, función, método y acceso. Prestá atención a `METHOD_NEITHER` y a `FILE_ANY_ACCESS`.
- **Validaciones:** si cada rama compara longitudes antes de pedir buffers y si usa un `MinimumRequiredSize` razonable.

Dinámico (solo en la VM de pruebas, con el debugger de kernel de la 6.3):

- Extensiones de WDF (`wdfkd.dll`): `!wdfldr` para ver la versión del framework cargada y `!wdflogdump` para leer el **In-flight Recorder** (IFR), un registro circular que WDF mantiene por driver con el progreso de IRPs y requests.
- Breakpoint en tu `EvtIoDeviceControl` para comparar longitudes y código recibidos con lo que envió la app.
- Driver Verifier y WDF Verifier sobre **tu** driver, en la VM, para detectar usos incorrectos de buffers o de IRQL.
- Desde user mode: el `GetLastError` de `DeviceIoControl` y `lpBytesReturned` para cada caso de la tabla.

Plantilla 6: esqueleto mínimo de `BlocLab.sys` para análisis estático

La lección describe el *contrato* de `BlocLab.sys`, pero el código de referencia vive acá, para que puedas **compilarlo y abrirlo en IDA/Ghidra** (revisión estática de 6.4) o cargarlo en una VM descartable. Es el KMDF mínimo: `DriverEntry`, `EvtDeviceAdd` y `EvtIoDeviceControl`.

```
// bloclab_kmdf_min.c — KMDF mínimo. El SDDL restringe la apertura a SYSTEM + Administradores.
#include <ntddk.h>
#include <wdf.h>
#include <wdmsec.h>

#define IOCTL_BLOCLAB_QUERY_VERSION \
    CTL_CODE(0x8000, 0x800, METHOD_BUFFERED, FILE_READ_DATA)   // 0x80006000

typedef struct _BLOCLAB_INFO { ULONG Size; ULONG Version; ULONG State; } BLOCLAB_INFO,
*PBLOCLAB_INFO;

EVT_WDF_DRIVER_DEVICE_ADD          BlocLabEvtDeviceAdd;
EVT_WDF_IO_QUEUE_IO_DEVICE_CONTROL BlocLabEvtIoDeviceControl;

NTSTATUS DriverEntry(PDRIVER_OBJECT DriverObject, PUNICODE_STRING RegistryPath) {
    WDF_DRIVER_CONFIG config;
    WDF_DRIVER_CONFIG_INIT(&config, BlocLabEvtDeviceAdd);      // +0x08 en x64 =
EvtDriverDeviceAdd
    return WdfDriverCreate(DriverObject, RegistryPath, WDF_NO_OBJECT_ATTRIBUTES, &config,
WDF_NO_HANDLE);
}

NTSTATUS BlocLabEvtDeviceAdd(WDFDRIVER Driver, PWDFDEVICE_INIT DeviceInit) {
    UNREFERENCED_PARAMETER(Driver);
    DECLARE_CONST_UNICODE_STRING(ntName,  L"\\Device\\BlocLab");
    DECLARE_CONST_UNICODE_STRING(symLink, L"\\DosDevices\\BlocLab");
    NTSTATUS st = WdfDeviceInitAssignName(DeviceInit, &ntName);
    if (!NT_SUCCESS(st)) return st;
    st = WdfDeviceInitAssignSDDLString(DeviceInit, &SDDL_DEVOBJ_SYS_ALL_ADM_ALL);  // solo SYSTEM
+ Admin
    if (!NT_SUCCESS(st)) return st;
    WDFDEVICE device;
    st = WdfDeviceCreate(&DeviceInit, WDF_NO_OBJECT_ATTRIBUTES, &device);
    if (!NT_SUCCESS(st)) return st;
    st = WdfDeviceCreateSymbolicLink(device, &symLink);
    if (!NT_SUCCESS(st)) return st;
    WDF_IO_QUEUE_CONFIG q;
    WDF_IO_QUEUE_CONFIG_INIT_DEFAULT_QUEUE(&q, WdfIoQueueDispatchSequential);
    q.EvtIoDeviceControl = BlocLabEvtIoDeviceControl;          // +0x28 en x64
    return WdfIoQueueCreate(device, &q, WDF_NO_OBJECT_ATTRIBUTES, WDF_NO_HANDLE);
}

VOID BlocLabEvtIoDeviceControl(WDFQUEUE Queue, WDFREQUEST Request, size_t OutLen, size_t InLen,
ULONG Ioctl) {
    UNREFERENCED_PARAMETER(Queue); UNREFERENCED_PARAMETER(OutLen);
    if (Ioctl ≠ IOCTL_BLOCLAB_QUERY_VERSION) { WdfRequestComplete(Request,
STATUS_INVALID_DEVICE_REQUEST); return; }
    if (InLen ≠ 0)                           { WdfRequestComplete(Request,
STATUS_INVALID_PARAMETER);      return; }
    PBLOCLAB_INFO out = NULL;
    // valida automáticamente OutputBufferLength ≥ sizeof(BLOCLAB_INFO) (12 bytes):
    NTSTATUS st = WdfRequestRetrieveOutputBuffer(Request, sizeof(BLOCLAB_INFO), (PVOID*)&out,
NULL);
    if (!NT_SUCCESS(st)) { WdfRequestComplete(Request, st); return; }   // STATUS_BUFFER_TOO_SMALL
    RtlZeroMemory(out, sizeof(BLOCLAB_INFO));
    out→Size = sizeof(BLOCLAB_INFO); out→Version = 1; out→State = 1; // READY
    WdfRequestCompleteWithInformation(Request, STATUS_SUCCESS, sizeof(BLOCLAB_INFO));
}
```

Práctica controlada

1. Partí de una muestra KMDF de Microsoft/plantilla WDK y documentá el WDK/SDK, Visual Studio, arquitectura y build de Windows usados; solo completá versiones después de verificarlas localmente.
2. Implementá el contrato fijo de consulta y las comprobaciones de longitud/código; revisá SDDL, exposición del dispositivo y configuración INF con la lista de seguridad de Microsoft. Elegí los bits Access del código y justificalos.
3. Compilá y analizá el paquete con la lista de **puntos de observación estática**. La carga es opcional y solo en VM compatible con firma de prueba siguiendo WDK actual; no uses comandos de relajación en el host.
4. Si cargaste el driver, la app cliente prueba todos los casos de la tabla: respuesta correcta, buffer de salida corto, entrada no vacía, código no admitido y handle sin derecho de lectura. Resultado esperado: respuesta fija o error limpio; nunca acceso genérico a memoria.
5. Desinstalá el paquete propio con el procedimiento documentado para esa VM, confirmá que ya no está cargado/registrado y revertí el snapshot.

Alternativa sin carga: si no compilaste o no podés cargar el driver en una VM compatible, hacé solo análisis estático del código fuente y del binario compilado (o de una muestra KMDF de Microsoft): recorré los siete pasos del recorrido señalando en qué línea ocurre cada uno, completá la tabla de controles y marcá cada caso de la tabla de pruebas como “esperado según el código, no ejecutado”.

Entregable

Contrato IOCTL con el desglose del código, diagrama del recorrido, modelo de amenaza, SDDL y exposición del dispositivo, tabla de casos válidos/negativos (indicando cuáles se ejecutaron), log de build y verificación de limpieza/snapshot. Si no se cargó, marcá el ejercicio como revisión estática.

Repaso

<details>
<summary>¿Por qué el IOCTL devuelve una estructura fija?</summary>

Reduce superficie y permite validar longitudes y contenido de forma acotada.
</details>

<details>
<summary>¿Qué riesgo resuelve una SDDL restrictiva?</summary>

Limita qué identidades pueden abrir el dispositivo; no reemplaza la validación dentro del driver.
</details>

<details>
<summary>¿Qué controla el campo Access de un código IOCTL y quién lo aplica?</summary>

Qué derechos debe tener el handle para enviar ese código. Lo aplica el I/O manager antes de crear el IRP: si no se cumple, la solicitud no llega al driver.
</details>

<details>
<summary>Con <code>METHOD_BUFFERED</code>, ¿por qué el driver debe leer la entrada antes de escribir la salida?</summary>

Porque entrada y salida comparten un único buffer de sistema; escribir la salida primero puede pisar la entrada.
</details>

▼ **¿Qué hace `WdfRequestRetrieveOutputBuffer` si el buffer es menor que `MinimumRequiredSize`?**

> Devuelve `STATUS_BUFFER_TOO_SMALL` y no entrega el buffer.

▼ **Con `METHOD_OUT_DIRECT`, ¿dónde está el buffer de salida desde el punto de vista del driver?**

> En un MDL (`Irp→MdlAddress`) que describe las páginas bloqueadas del buffer de la app; en KMDF se obtiene con `WdfRequestRetrieveOutputBuffer` o `WdfRequestRetrieveOutputWdmMdl`.

▼ **¿Por qué un IOCTL `METHOD_BUFFERED` puede tener los mismos riesgos que uno `METHOD_NEITHER`?**

> Si la estructura de entrada trae punteros embebidos: esas direcciones son de usuario y el I/O manager no las valida.

▼ **¿Por qué se rechaza un IOCTL desconocido?**

> Porque una solicitud no definida no debe llegar a una operación implícita o permisiva.

▼ **No pudiste cargar el driver en la VM. ¿Qué entregás?**

> Una revisión estática: el recorrido señalado en el código, la tabla de controles y los casos marcados como esperados y no ejecutados.

Puerta optativa

Podés explicar el flujo KMDF desde `DeviceIoControl` hasta la finalización, desglosar un código IOCTL, distinguir las cuatro capas de control de acceso, señalar puntos de observación estática y dinámica, y demostrar (o justificar estáticamente) validación de buffers/códigos y respuesta sin datos no inicializados.

Lo que queda fuera

Escribir drivers con direct I/O o `METHOD_NEITHER`, DMA, la gestión de MDL propios y la firma de producción no se desarrollan. La revisión de `METHOD_NEITHER`, colas, cancelación y PnP/power sigue en la 6.4, como lectura de flujo. El laboratorio no demuestra que un driver real sea seguro: solo practica un contrato mínimo.

Fuentes

- Microsoft Learn, modelo y recorrido: Differences between WDM and WDF, Creating Framework Request Objects, Dispatching Methods for I/O Requests, IOCTL callback, WdfRequestRetrieveOutputBuffer, WdfRequestCompleteWithInformation.
- Microsoft Learn, códigos y acceso: Defining I/O Control Codes, Buffer Descriptions for I/O Control Codes, Methods for Accessing Data Buffers, Using Direct I/O, Accessing Data Buffers in WDF Drivers, KMDF device access control, WdfDeviceInitAssignSDDLString, IoValidateDeviceIoControlAccess, Driver security checklist.

- Microsoft Learn, observación y firma: Using the Framework's Event Logger (IFR), Wdfkd.dll extensions, Driver Verifier, Test signing.

Etiquetas

Núcleo conceptual: ninguno específico. **Conocimiento externo:** WDF, KMDF, recorrido de IOCTL, buffers y MDL por método, controles de acceso, herramientas de observación y firma. **Ejemplo ilustrativo:** contrato BlocLab, cálculo del código y tabla de casos (no compilados ni ejecutados en esta edición). **Laboratorio verificado:** no; requiere toolchain/VM y revisión de seguridad. La alternativa de análisis estático no requiere cargar nada.

Ampliación editorial basada en fuentes externas — Verificación de MinimumRequiredSize y EvtIoInCallerContext en KMDF

Según WdfRequestRetrieveOutputBuffer y Accessing Data Buffers in WDF Drivers en Microsoft Learn *(Hecho confirmado)*:

- **Validación de tamaño en `WdfRequestRetrieveInputBuffer` / `WdfRequestRetrieveOutputBuffer`:** El segundo parámetro (`RDX` en x64) es `MinimumRequiredLength` / `MinimumRequiredSize`. Si el buffer es menor, KMDF devuelve `STATUS_BUFFER_TOO_SMALL` (`0xC0000023`). Al revisar un driver en IDA/Ghidra (6.4), comprobá siempre que ese segundo argumento no sea `0` cuando luego se escribe una estructura de tamaño fijo.
- **Por qué `METHOD_NEITHER` requiere `EvtIoInCallerContext`:** Como las colas WDF pueden despachar solicitudes en un hilo arbitrario, las direcciones de usuario (`Type3InputBuffer` y `UserBuffer`) solo son válidas en el contexto del llamador. Por eso deben tomarse en `EvtIoInCallerContext` con `WdfRequestRetrieveUnsafeUserInputBuffer` y bloquearse con `WdfRequestProbeAndLockUserBufferForRead` / `ForWrite` antes de encolar la solicitud.

Debugging de kernel y bugchecks

> **Driver Verifier puede detener la VM**
>
> Microsoft advierte que Driver Verifier puede generar bugchecks. Usalo solo sobre el driver didáctico propio en una VM descartable, con snapshot previo y método de recuperación probado. No verifiques todos los drivers del host.

> **Propósito**
>
> Separar depuración user-mode de kernel, obtener un stack reproducible de un driver propio y explicar qué evidencia aporta un dump. Al terminar deberías poder recorrer un bugcheck con un método fijo —código, parámetros, contexto, stack, módulo, memoria— y escribir una conclusión que diga qué está probado y qué no.

Prerrequisitos

- *Obligatorios:* 6.1, driver propio revisado de 6.2, WinDbg y 2D.11 excepciones.
- *Recomendados:* 2C WinDbg/heap; test-target VM configurada según WDK vigente.

Flujo de depuración

WinDbg puede depurar el kernel local o un equipo de prueba conectado. En la VM, símbolos correspondientes al build permiten resolver funciones y tipos; sin ellos, direcciones/nombres pueden quedar incompletos. Ante bugcheck, guarda código, parámetros, dump y estado del snapshot. `!analyze -v` es un punto de partida, no una conclusión automática.

Driver Verifier comprueba reglas de drivers y puede provocar deliberadamente un bugcheck al detectar violaciones. Configurá solo el driver objetivo de prueba a través de su UI/documentación, mantén el rollback disponible y apágalo/revierte la VM al finalizar según el procedimiento Microsoft aplicable.

Símbolos

Sin símbolos correctos, el stack muestra direcciones o nombres aproximados y cualquier conclusión se debilita. La ruta de símbolos combina el servidor público de Microsoft (para el sistema) y la carpeta con el PDB **de tu build** del driver. Después de cambiarla se recargan con `.reload`. Anotá en el informe qué símbolos estaban disponibles; un frame que dice `BlocLab+0x1a2b` en lugar de un nombre de función es un límite, no un dato.

Qué contiene un dump y qué no

Windows ofrece cinco configuraciones de volcado de kernel, que difieren sobre todo en tamaño:

Tipo	Qué guarda	Para qué alcanza
Completo	Toda la memoria física, incluida parte de user mode	El análisis más amplio; es el más grande
Activo	Similar al completo para la mayoría de los fines, algo más chico	Casi todo lo del completo
De kernel	Memoria del kernel; en general omite user mode	Stacks, objetos y pool del kernel
Automático	Igual que el de kernel, con más flexibilidad en el archivo de paginación	Ídem kernel
Pequeño (minidump)	64 KB	Código, parámetros, stack básico y lista de módulos

Microsoft aclara que ningún dump da tanta información como depurar en vivo: es una **foto** de un instante. Si el dump es pequeño, declarar que no se pudo inspeccionar memoria que no está incluida.

Lectura guiada de un bugcheck

Un orden de trabajo que evita saltar a conclusiones:

Paso	Comando	Qué responde	Límite
1. Código y parámetros	`.bugcheck`, `!analyze -v`	Qué condición detectó el sistema y con qué argumentos	El "módulo sospechoso" es una heurística
2. Significado de los parámetros	Página del código en la referencia de bugchecks	Qué representa cada argumento para **ese** código	Cada código tiene su propia tabla
3. Contexto	`.trap` (si hay trap frame) o `.cxr`	Registros y stack en el punto del fallo	No siempre hay trap frame
4. Stack	`k`, `kv`	Ruta de llamadas	Depende de símbolos
5. Instrucción y memoria	`ln`, `u` sobre la dirección; `!pool`, `!address`, `!pte` sobre la memoria referenciada	Qué función accedió a qué y si era pool paginado	Interpretación técnica, no causa raíz automática
6. IRQL y módulos	`!irql`, `lm t n`	Nivel en el momento y versiones de los módulos	`!irql` refleja lo guardado por el depurador
7. Objetos del driver	`!drvobj`, `!devobj`, `!irp`; `!wdfkd.*` en KMDF (6.2)	Qué dispositivo y qué solicitud estaban en curso	Requiere saber qué objeto mirar

Para un 0xD1, por ejemplo, la documentación indica usar `ln` con el cuarto parámetro para ver la función que hizo el acceso, `!pool` sobre el primero para saber si era pool paginado y `dx KiBugCheckDriver` para leer el nombre del driver que el sistema registró como responsable. También advierte que, en la mayoría de los casos de ese código, el problema no es el IRQL sino la memoria accedida.

Ejemplo trabajado

(Esquema ficticio, no dump real.)

```
Bugcheck observado: DRIVER_VERIFIER_DETECTED_VIOLATION
!analyze -v: módulo sospechoso = BlocLab.sys
stack: nt!Verifier → Wdf01000!callback → BlocLab!EvtIoDeviceControl
Conclusión: revisar la longitud del buffer en el callback; confirmar parámetros y código fuente
```

El módulo señalado es evidencia para investigar; comprueba stack, parámetros, build/símbolos y si el driver estaba realmente activo en esa solicitud.

Informe de ese caso, separando observación de inferencia (ilustrativo):

Afirmación	Tipo	Evidencia
Driver Verifier detectó una violación	Observación	Código de bugcheck y parámetros
La violación ocurrió durante `EvtIoDeviceControl` de BlocLab	Observación con símbolos	Stack con PDB del build propio
La solicitud en curso era el IOCTL `0x80006000`	Observación	`!wdfkd` / `!irp` sobre la solicitud del frame
La causa es una validación de longitud ausente	Inferencia	Coincide con el parámetro y el código fuente; falta reproducir
Ningún otro driver intervino	No determinado	El dump pequeño no incluye toda la memoria

> **Conectar KDNET a una VM: los pasos que faltan**
>
> `bcdedit /dbgsettings net` genera una **clave** (`Key=w.x.y.z`) que hay que copiar en WinDbg; sin ella —o con un adaptador de red que `kdnet.sys` no soporta— WinDbg se queda en *Waiting to reconnect...*. Secuencia que funciona:
>
> ```
> :: 1. Hipervisor (VirtualBox/VMware): adaptador de red en Host-Only, tipo "Intel PRO/1000 MT
> Desktop (82540EM)".
> :: 2. En la VM (cmd como Administrador), apuntando a la IP Host-Only del HOST:
> bcdedit /debug on
> bcdedit /dbgsettings net hostip:192.168.56.1 port:50000
> :: 3. Copiá la clave que imprime (Key=1.2.3.4).
> :: 4. En el host:
> windbg -k net:port=50000,key=1.2.3.4
> :: 5. Reiniciá la VM: shutdown /r /t 0
> ```
>
> Para inspeccionar requests de KMDF, el nombre del módulo va **sin** `.sys` : `!wdfkd.wdflogdump bloclab` .

Práctica

En VM con snapshot, conecta el debugger a un target de prueba ya configurado y carga solo el driver propio permitido. Captura una solicitud válida, inspecciona lista de módulos/stack y unload. Driver Verifier es opcional y solo si el docente confirma el rollback; no provoques un fallo deliberado ni fuerces un bugcheck. Si aparece uno, conserva el dump, analiza offline y revierte.

1. Configurá la ruta de símbolos con el PDB de tu build y verificá con `lm` que el driver propio tiene símbolos.
2. Con el target vivo, ubicá tu driver con `!drvobj` y su dispositivo con `!devobj` ; si estás en el callback, seguí la solicitud con `!wdfkd` o `!irp` .

3. Si tenés un dump (propio o de un ejercicio provisto), recorré la tabla de lectura guiada paso por paso y completá la tabla de afirmaciones.

Entregable

Registro de build/VM, símbolos, tipo de dump, comandos de inspección usados, stack, tabla de afirmaciones (observación/inferencia/no determinado) y evidencia de que driver/verifier ya no están activos o snapshot revertido.

Repaso

▼ ¿`!analyze -v` da siempre la causa raíz?

> No. Resume el bugcheck y pistas; hay que correlacionar parámetros, stack, símbolos y estado.

▼ ¿Por qué no usar Driver Verifier sobre el host?

> Puede causar bugchecks y afectar estabilidad; se limita a un target de pruebas recuperable.

▼ ¿Qué no podés esperar de un minidump?

> La memoria completa: tiene 64 KB; alcanza para código, parámetros y stack básico, no para inspeccionar estructuras grandes.

▼ ¿Para qué sirve `.trap` o `.cxr` antes de `k`?

> Para poner el contexto en el punto del fallo, de modo que el stack corresponda a ese momento.

▼ En un 0xD1, ¿qué parámetro llevás a `ln` y cuál a `!pool`?

> El cuarto (dirección de la instrucción) a `ln`, para ver la función; el primero (memoria referenciada) a `!pool`, para ver si era pool paginado.

▼ Un frame dice `BlocLab+0x1a2b`. ¿Qué te falta?

> Símbolos correctos del build; sin ellos, el nombre de la función es una suposición.

Puerta optativa

Podés capturar un stack/dump del driver propio, configurar y declarar símbolos, recorrer un bugcheck con el orden código–parámetros–contexto–stack–memoria–objetos, explicar límites del tipo de dump y salir del ejercicio con target restaurado.

Lo que queda fuera

La configuración del enlace de depuración de kernel paso a paso (se sigue la documentación del WDK vigente), el análisis de hangs y deadlocks, la depuración de hipervisor y el análisis de dumps de terceros no se desarrollan.

Fuentes

- Microsoft Learn: Setting up kernel-mode debugging, Live kernel-mode debugging using WinDbg, Driver Verifier, Bug check analysis.
- Microsoft Learn, lectura: Symbol path, Varieties of Kernel-Mode Dump Files, !analyze, .bugcheck, !drvobj, !devobj, !irp, Bug Check 0xD1, Wdfkd.dll extensions.

Etiquetas

Núcleo conceptual: WinDbg de user mode. **Conocimiento externo:** depuración kernel, símbolos, tipos de dump, lectura de bugchecks y verifier. **Ejemplo ilustrativo:** bugcheck e informe ficticios. **Laboratorio verificado:** no; el target de kernel no se probó.

Revisión defensiva de un driver

> **Sin explotación**
>
> Esta revisión busca defectos de diseño y evidencia para corregirlos. No pruebes IOCTLs contra drivers de terceros ni intentes conseguir acceso privilegiado. Los patrones de riesgo de esta lección se usan para **leer** código y binarios propios o didácticos, no para construir pruebas contra un kernel cargado.

> **Propósito**
>
> Reconstruir la superficie de acceso de un driver desde nombre/interfaz y permisos hasta cada IOCTL y su validación, y leer el ciclo de vida de una solicitud (colas, propiedad, cancelación) para encontrar rutas donde la validación o la finalización fallan.

Prerrequisitos

- *Obligatorios:* 6.1 modelo E/S, 6.2 KMDF, 2D.1 objetos/ACL.
- *Recomendados:* 2D.2 memoria, 2D.11 excepciones, seguridad de 6.6.

Lista de lectura

1. ¿El driver crea un nombre de dispositivo solo cuando hace falta? ¿La interfaz usa ACL/SDDL limitada?
2. ¿Qué IOCTLs exporta, con qué **método** y qué permisos de acceso exige cada uno?
3. ¿Valida tamaño, versión, enum, flags, longitud de salida y estado antes de operar?
4. ¿Evita punteros del usuario, direcciones físicas, handles sin validar y salida no inicializada? Si usa `METHOD_NEITHER` o punteros embebidos, ¿aplica el patrón de la sección siguiente?
5. ¿Completa solicitudes en todas las rutas, una sola vez, y sincroniza estado compartido?
6. Si retiene solicitudes, ¿las marca como cancelables o comprueba la cancelación?

La **superficie de ataque** es el conjunto de entradas accesibles y efectos asociados; una interfaz amplia aumenta revisión necesaria, pero no basta para afirmar vulnerabilidad.

Patrón de revisión: `METHOD_NEITHER` y punteros de usuario

Con `METHOD_NEITHER`, el I/O manager entrega al driver las **direcciones de usuario** originales (`Type3InputBuffer`, `UserBuffer`) sin validarlas ni bloquearlas (ver la tabla de métodos en la 6.2). Lo mismo pasa con un **puntero embebido** dentro de una estructura que llegó por `METHOD_BUFFERED`: el buffer es de sistema, pero el puntero que trae adentro sigue siendo de usuario. Que un driver use cualquiera de los dos no es un defecto en sí, pero marca código que hay que leer con este patrón.

Dónde aparece. En el código: el bit de método `3` en las definiciones `CTL_CODE` (`METHOD_NEITHER`), accesos a `Type3InputBuffer` o `UserBuffer`, o en KMDF un `EvtIoInCallerContext` con `WdfRequestRetrieveUnsafeUserInputBuffer` / `…OutputBuffer`. En un binario: las mismas constantes en el `switch` del código de control (los dos bits bajos en `11`) y lecturas de esos campos del IRP o llamadas a esas funciones.

Por qué `ProbeForRead` no alcanza por sí solo. `ProbeForRead` / `ProbeForWrite` comprueban que el rango esté en la parte de usuario del espacio de direcciones y alineado, **en ese momento**. Microsoft documenta además que:

- con `Length = 0` no comprueba nada;
- deben llamarse dentro de `try/except`;
- **todo acceso posterior** también debe estar protegido, porque otro hilo de la aplicación puede liberar, sustituir o cambiar la protección del rango en cualquier momento, incluso durante o después del sondeo;
- el contenido puede cambiar mientras el driver lo usa.

Es decir: el sondeo valida una **dirección**, no una **decisión** basada en el contenido.

Preguntas de revisión (sobre código propio o didáctico):

Pregunta	Qué buscar	Señal de riesgo
¿Distingue llamadas desde user mode y kernel mode?	Uso de `RequestorMode` antes de sondear	Sondeo incondicional o ausencia de distinción
¿La longitud sondeada es la que después se usa?	Misma variable para sondear y copiar; cálculo de tamaños sin desbordamiento	Sondeo con un tamaño y copia con otro, o multiplicaciones sin control
¿Cada acceso al buffer de usuario está dentro de `try/except`?	Bloques de excepción alrededor de **todas** las lecturas y escrituras	Solo el sondeo protegido
¿Captura los datos una sola vez?	Copia a un buffer del kernel y validación sobre la copia	Validar un campo en memoria de usuario y volver a leerlo para usarlo (doble lectura)
¿Se usa el buffer fuera del hilo que llamó?	Bloqueo (`WdfRequestProbeAndLockUserBufferForRead` / `…ForWrite` o MDL propio) antes de encolar	Dirección de usuario guardada y usada desde otra cola, un DPC o un hilo del sistema
¿Hacen falta estos punteros?	Diseño alternativo con offsets relativos o buffers de longitud variable	Punteros embebidos en una interfaz que podría ser `METHOD_BUFFERED` puro

Lectura **ilustrativa** de un fragmento hipotético, para practicar el patrón (no es código de un driver real ni una receta):

```
// handler hipotético, METHOD_NEITHER
p = Type3InputBuffer
ProbeForRead(p, sizeof(REQ), 4)        // dentro de try/except: OK
if (p→Len > MAX) fail                   // valida en memoria de usuario…
memcpy(local, p→Data, p→Len)            //  …y vuelve a leer Len: doble lectura
                                        // memcpy fuera del try/except: acceso sin protección
```

Hallazgos que debería registrar el informe: la validación y el uso leen `Len` dos veces de memoria de usuario; la copia no está protegida por el manejador de excepciones. Corrección propuesta: capturar la estructura completa en un buffer del kernel dentro de `try/except`, validar la copia y usar solo la copia; o rediseñar el IOCTL como `METHOD_BUFFERED` sin punteros.

Leer el flujo de una solicitud: colas, propiedad y cancelación

Muchos defectos no están en la validación sino en **quién es dueño** de la solicitud y **cuántas veces** se completa. En WDF el modelo es explícito:

- **Propiedad.** Mientras la solicitud está en una cola, es del **framework**. Cuando la entrega a un callback, pasa a ser del **driver**, que debe hacer exactamente una de estas cosas: completarla, reencolarla, reenviarla a un destino de E/S o cancelarla. Después de eso ya no la posee y no puede tocarla (ni sus buffers).
- **Colas y despacho.** Cada cola tiene un modo de despacho (secuencial, paralelo o manual) y unos callbacks registrados; `WdfDeviceConfigureRequestDispatching` puede desviar tipos de solicitud a colas distintas. Al revisar, armá una tabla cola → modo → callbacks → tipos de solicitud.
- **Cancelación.** Si la operación se cancela, el framework cancela con `STATUS_CANCELLED` las solicitudes que **todavía no entregó** al driver. Una vez entregada, solo el driver puede cancelarla, y para enterarse debe haberla marcado con `WdfRequestMarkCancelable` / `…Ex` (registrando `EvtRequestCancel`) o consultar `WdfRequestIsCanceled` mientras la posee. Si la reencola en una cola con `EvtIoCanceledOnQueue`, el framework llama a ese callback en lugar de cancelarla, y el driver debe completarla.

Ejemplo de lectura **ilustrativo** de una solicitud retenida (no corresponde al driver de 6.2, que completa en el acto):

```
EvtIoDeviceControl(R)          // el driver ahora posee R
  WdfRequestMarkCancelableEx(R, EvtCancel)
  guarda R en la lista "pendientes"      // espera un evento del dispositivo
--- más tarde, una de dos rutas ---
(a) llega el evento:  WdfRequestUnmarkCancelable(R) → completa R con STATUS_SUCCESS
(b) la app cancela:   EvtCancel(R) → quita R de la lista → completa R con STATUS_CANCELLED
```

Preguntas de revisión: ¿hay alguna ruta en la que **ambas** ramas completen la misma solicitud? ¿Se respeta el resultado de `WdfRequestUnmarkCancelable` antes de completar? ¿La lista de pendientes está protegida contra acceso concurrente? ¿Una solicitud retenida mucho tiempo **sin** marcarse cancelable puede dejar colgado al proceso que la envió? Microsoft dedica una página a sincronizar cancelación y finalización; si el código no la sigue, es un hallazgo a documentar con archivo y línea.

> **Recuadro conceptual: PnP y energía**
>
> Un driver de función KMDF suele registrar callbacks de PnP y energía: `EvtDevicePrepareHardware` / `EvtDeviceReleaseHardware` (recursos del hardware) y `EvtDeviceD0Entry` / `EvtDeviceD0Exit` (entrada y salida del estado de trabajo D0). Las colas **administradas por energía** solo entregan solicitudes cuando el dispositivo está en D0; si no lo está, las solicitudes esperan. En la revisión basta con saber dos cosas: si la cola de IOCTL es administrada por energía (lo que explica solicitudes que "no avanzan") y si algún callback de energía libera estado que un callback de E/S todavía puede usar. El dispositivo software de la 6.2 apenas usa esta maquinaria; el diseño de PnP/energía queda fuera del curso.

Ejemplo trabajado

(Checklist ilustrativo para el driver de 6.2.)

```
Device interface: acceso solo a Administrators/SYSTEM
IOCTL 0x80006000 (BLOCLAB_QUERY_VERSION, contrato ilustrativo de 6.2)
  METHOD_BUFFERED, FILE_READ_DATA → input=0, output=12
  sin punteros embebidos; no hay METHOD_NEITHER
Otras solicitudes: rechazadas con STATUS_INVALID_DEVICE_REQUEST
Buffer: longitud comprobada; respuesta inicializada por completo
Cola: una cola por defecto; la solicitud se completa en el mismo callback
Cancelación: no aplica (no se retienen solicitudes)
Conclusión: diseño acotado en el fragmento revisado; falta validar build/configuración INF
```

El código `0x80006000` sale del contrato **ilustrativo** de la 6.2; en un driver real se toma de su header, no se supone.

Ampliación editorial basada en fuentes externas — Resolución de la tabla `WdfFunctions` (`WDFFUNCTIONS`) en drivers KMDF sin símbolos en IDA y Ghidra

Al analizar estáticamente un driver KMDF sin archivo `.pdb` , las llamadas a la API del framework (`WdfDriverCreate` , `WdfDeviceCreate` , `WdfIoQueueCreate` , `WdfRequestRetrieveInputBuffer` , etc.) **no aparecen en la tabla de importaciones (IAT)** (*Microsoft Learn: KMDF Driver Entry and Binding / `wdffuncenum.h` del WDK*) (**Hecho confirmado**):

1. **Mecanismo de enlace dinámico de KMDF:**

El binario solo importa `WdfVersionBind` desde `WdfLdr.sys` . Durante el arranque, el stub del compilador llena una estructura `WDF_BIND_INFO` que contiene la versión del framework (p. ej. `1.15` o `1.33`) y un puntero a un puntero global en `.data` : `WdfFunctions` (de tipo `const WDFFUNCTIONS*`), junto con `WdfDriverGlobals` (`PWDF_DRIVER_GLOBALS`).

2. **Patrón en ensamblador x64:**

Toda llamada a una función `Wdf*` se compila como una llamada indirecta indexada sobre esa tabla global, pasando siempre `WdfDriverGlobals` como primer argumento oculto en `RCX` (lo que desplaza un lugar todos los argumentos documentados de la función):

```
; Llamada en código C: WdfIoQueueCreate(device, &q, WDF_NO_OBJECT_ATTRIBUTES, WDF_NO_HANDLE);
mov     rax, cs:WdfFunctions_01015   ; Carga la base de la tabla de punteros KMDF
mov     rcx, cs:WdfDriverGlobals     ; 1.er arg oculto (RCX): PWDF_DRIVER_GLOBALS
mov     rdx, rbx                     ; 2.º arg (RDX): WDFDEVICE Device
lea     r8, [rsp+40h]                ; 3.er arg (R8) : PWDF_IO_QUEUE_CONFIG Config (+0x28 =
EvtIoDeviceControl)
xor     r9d, r9d                     ; 4.º arg (R9) : WDF_NO_OBJECT_ATTRIBUTES
mov     qword ptr [rsp+20h], 0       ; 5.º arg (stack): Queue (NULL)
call    qword ptr [rax+4C0h]         ; Offset +0x4C0 (índice 152 * 8) = pfnWdfIoQueueCreate
```

3. **Tabla de offsets de funciones críticas en `WDFFUNCTIONS` (`wdffuncenum.h` , KMDF 1.15–1.33):**

Función KMDF (`WDFFUNCENUM`)	Índice decimal (`TableIndex`)	Offset en x64 (`Índice * 8`)	Offset en x86 (`Índice * 4`)	Qué recibe en `RDX` / `R8` (x64, tras `DriverGlobals` en `RCX`)
`WdfDeviceInitAssignName`	`29`	`+0x0E8`	`+0x074`	`RDX` = `DeviceInit` , `R8` = `PCUNICODE_STRING` (`\Device\ ...`)
`WdfDeviceInitAssignSDDLString`	`31`	`+0x0F8`	`+0x07C`	`RDX` = `DeviceInit` , `R8` = `PCUNICODE_STRING` (cadena SDDL)
`WdfDeviceCreate`	`45`	`+0x168`	`+0x0B4`	`RDX` = `&DeviceInit` , `R8` = `Attributes` , `R9` = `&Device`
`WdfDeviceCreateSymbolicLink`	`46`	`+0x170`	`+0x0B8`	`RDX` = `Device` , `R8` = `PCUNICODE_STRING` (`\DosDevices\ ...`)
`WdfDeviceCreateDeviceInterface`	`47`	`+0x178`	`+0x0BC`	`RDX` = `Device` , `R8` = `const GUID* InterfaceClassGUID`
`WdfDriverCreate`	`118`	`+0x3B0`	`+0x1D8`	`RDX` = `DriverObject` , `R8` = `RegistryPath` , `R9` = `Attributes` , `[rsp+20h]` = `WDF_DRIVER_CONFIG*`
`WdfIoQueueCreate`	`152`	`+0x4C0`	`+0x260`	`RDX` = `Device` , `R8` = `WDF_IO_QUEUE_CONFIG*` (`+0x28` = `EvtIoDeviceControl`)
`WdfRequestComplete`	`263`	`+0x838`	`+0x41C`	`RDX` = `Request` , `R8D` = `NTSTATUS`
`WdfRequestCompleteWithInformation`	`265`	`+0x848`	`+0x424`	`RDX` = `Request` , `R8D` = `NTSTATUS` , `R9` = `ULONG_PTR Information`
`WdfRequestRetrieveInputBuffer`	`275`	`+0x898`	`+0x44C`	`RDX` = `Request` , `R8` = `MinimumRequiredSize` , `R9` = `&Buffer`
`WdfRequestRetrieveOutputBuffer`	`276`	`+0x8A0`	`+0x450`	`RDX` = `Request` , `R8` = `MinimumRequiredSize` , `R9` = `&Buffer`

4. **Tipado rápido en IDA 9.0+ y Ghidra:**

Si asignás el tipo `WDFFUNCTIONS*` a la variable global `WdfFunctions` (o importás `wdf.h` / `wdffuncenum.h`), el decompilador convierte instantáneamente `(*(void (**)( ... ))(WdfFunctions + 0x4C0))( ... )` en `WdfFunctions->pfnWdfIoQueueCreate( ... )` .

Práctica

Revisá el source de tu driver o una muestra didáctica de Microsoft:

1. Enumerá device interfaces, SDDL, definiciones de IOCTL (con método y acceso) y callbacks.
2. Trazá un IOCTL desde la entrada hasta la finalización.
3. Si hay `METHOD_NEITHER` o punteros embebidos, completá la tabla de preguntas del patrón para cada uno. Si no hay, decilo explícitamente.
4. Armá la tabla de colas (modo, callbacks, tipos de solicitud) y, para cada solicitud que el driver retenga, dibujá las rutas de finalización y de cancelación.
5. Verificá casos límite por pruebas unitarias en user mode, sin fuzzing contra un kernel cargado.

Entregable

Tabla entrada–control–validación–resultado; tabla de colas y rutas de finalización/cancelación; hallazgos con archivo/línea, severidad razonada y corrección propuesta. No publiques el driver o detalles sensibles sin autorización.

Repaso

▼ **¿Una DACL fuerte vuelve seguro un IOCTL?**

> No. Reduce quién puede llamarlo, pero el callback debe validar toda entrada y salida.

▼ **¿Por qué limitar IOCTLs?**

> Cada operación exportada amplía lo que un cliente puede pedir y debe tener contrato y permisos explícitos.

▼ **¿Qué implica "superficie de ataque"?**

> Entradas y operaciones accesibles desde una frontera; es una guía de revisión, no un veredicto.

▼ **¿Por qué `ProbeForRead` no vuelve seguro todo acceso posterior?**

> Porque solo comprueba rango y alineación en ese momento (y nada si la longitud es 0); otro hilo puede cambiar el contenido o la protección después. Cada acceso posterior debe estar protegido y los datos deben capturarse una vez.

▼ **¿Qué es una doble lectura en este contexto?**

> Validar un valor en memoria de usuario y volver a leerlo para usarlo: entre las dos lecturas el valor puede cambiar.

▼ **¿Cuándo deja el driver de poseer una solicitud WDF?**

> Cuando la completa, la reencola, la reenvía o la cancela; desde ese momento no puede acceder a ella ni a sus buffers.

▼ **¿Quién cancela una solicitud que todavía está en una cola del driver?**

> El framework, con `STATUS_CANCELLED`, sin entregarla. Una vez entregada, solo el driver puede cancelarla.

▼ **Una solicitud de IOCTL "queda colgada" y el dispositivo está en un estado de bajo consumo. ¿Qué revisás?**

> Si la cola es administrada por energía: esas colas solo entregan solicitudes en D0.

Puerta optativa

Podés rastrear una operación desde la interfaz hasta su callback y demostrar permisos, validación de longitudes y tratamiento de errores; aplicar el patrón de revisión a `METHOD_NEITHER` y punteros embebidos; y leer las rutas de propiedad, finalización y cancelación de una solicitud.

6.4.b — Fuzzing de IOCTLs en Kernel

La superficie de ataque primaria y más crítica de cualquier controlador de dispositivo en Windows reside en su interfaz de control de E/S: los códigos `IOCTL` despachados a través del manejador `IRP_MJ_DEVICE_CONTROL` (`EvtIoDeviceControl` en KMDF). Mientras que el fuzzing en espacio de usuario (Lección 2.24) prueba parsers de archivos, el **Kernel Fuzzing** somete al controlador a combinaciones anómalas de códigos de control, tamaños de buffer incongruentes y punteros no válidos para descubrir fallos de validación antes de que causen una vulnerabilidad en producción.

Arquitectura del Harness de Fuzzing en Modo Usuario

A diferencia de un ejecutable clásico que lee de `stdin` o de un archivo, un fuzzer de kernel ejecuta un **harness en espacio de usuario** que interactúa directamente con el objeto de dispositivo del driver:

1. **Apertura del Dispositivo:** Obtiene un handle con `CreateFileW(L"\\.\MiDriver", GENERIC_READ | GENERIC_WRITE, ...)` utilizando el enlace simbólico (*Symbolic Link*) expuesto por el driver.
2. **Bucle de Mutación de IOCTLs:** Genera o permuta códigos `CTL_CODE` iterando a través de:

- *DeviceType:* Códigos estándar o reservados (`0x8000`..`0xFFFF`).
- *Function:* Rango de índices de función del driver (`0x800`..`0xFFF`).
- *Method:* `METHOD_BUFFERED` (`0`), `METHOD_IN_DIRECT` (`1`), `METHOD_OUT_DIRECT` (`2`), `METHOD_NEITHER` (`3`).

3. **Manejo Estricto de Excepciones:** Como el envío de punteros inválidos hacia un driver con `METHOD_NEITHER` puede provocar un fallo en el propio llamador si no se captura, el harness encapsula `DeviceIoControl` dentro de un bloque `__try / __except`.

Esqueleto de Código C: Harness de Fuzzing de IOCTLs

```
// Archivo: plantillas_codigo/ioctl_fuzzer_harness.c
// Compilar con MSVC x64: cl /Zi /O2 ioctl_fuzzer_harness.c /Fe:IoctlFuzzer.exe

#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX_PAYLOAD_SIZE 4096

// Generador de payloads mutados con patrones límite
void GenerarPayloadMutado(BYTE* buffer, DWORD size) {
    static const DWORD patrones[] = { 0x00000000, 0xFFFFFFFF, 0x7FFFFFFF, 0x80000000, 0x41414141 };
    for (DWORD i = 0; i < size; i++) {
        buffer[i] = (BYTE)(rand() % 256);
    }
    // Sobrescribir los primeros 4 bytes con valores límite enteros
    if (size ≥ 4) {
        *(DWORD*)buffer = patrones[rand() % 5];
    }
}

int main(int argc, char* argv[]) {
    LPCWSTR deviceName = L"\\.\BlocLabDevice";
    HANDLE hDevice = CreateFileW(deviceName, GENERIC_READ | GENERIC_WRITE, 0, NULL, OPEN_EXISTING, 0,
NULL);
    if (hDevice == INVALID_HANDLE_VALUE) {
        printf("[-] Error abriendo dispositivo %ls (Error: %lu)\n", deviceName, GetLastError());
        return 1;
    }

    printf("[+] Iniciando fuzzing de IOCTLs sobre %ls ...\n", deviceName);
    srand((unsigned int)time(NULL));

    BYTE inBuffer[MAX_PAYLOAD_SIZE];
    BYTE outBuffer[MAX_PAYLOAD_SIZE];
    DWORD bytesReturned = 0;

    // Rango típico de IOCTLs definidos por el desarrollador (0x800 a 0x8FF)
    for (DWORD func = 0x800; func < 0x850; func++) {
        // Probar los 4 métodos de transferencia I/O
        for (DWORD method = 0; method < 4; method++) {
            DWORD ioctl = CTL_CODE(FILE_DEVICE_UNKNOWN, func, method, FILE_ANY_ACCESS);

            for (int iter = 0; iter < 100; iter++) {
                DWORD inSize = rand() % MAX_PAYLOAD_SIZE;
                DWORD outSize = rand() % MAX_PAYLOAD_SIZE;
                GenerarPayloadMutado(inBuffer, inSize);

                __try {
                    // Envío del IOCTL fuzzeado al driver de kernel
                    DeviceIoControl(
                        hDevice,
                        ioctl,
                        inBuffer,
                        inSize,
                        outBuffer,
                        outSize,
```

```
                        &bytesReturned,
                        NULL
                    );
                }
                __except (EXCEPTION_EXECUTE_HANDLER) {
                    // Capturar excepciones generadas en usermode si el driver usa METHOD_NEITHER
                    printf("[!] Excepción en usermode con IOCTL 0x%08X (InSize: %lu)\n", ioctl, inSize);
                }
            }
        }
    }

    CloseHandle(hDevice);
    printf("[+] Campaña de fuzzing de IOCTLs finalizada.\n");
    return 0;
}
```

Instrumentación Guiada y Captura de Crashes con KDNET

Para que el fuzzing sea eficiente y no un mero bombardeo a ciegas:

1. **Instrumentación (Intel PT / DynamoRIO):** Herramientas avanzadas como **WinAFL** (modo `-m` para targeting de módulos) o **kAFL / syzkaller** utilizan trazas de hardware de **Intel PT (Processor Trace)** para guiar la mutación hacia bloques básicos no explorados del driver en kernel.
2. **Detección y Captura de Crashes vía KDNET:**

- La VM de pruebas se ejecuta con depuración de kernel activa mediante KDNET conectada a una máquina anfitriona con WinDbg.
- Cuando el fuzzer desencadena una corrupción de memoria en el driver, el sistema no se reinicia a ciegas; WinDbg captura de inmediato el **Bugcheck**:
- `0x50` (`PAGE_FAULT_IN_NONPAGED_AREA`)**:** El driver desreferenció un puntero inválido proveniente de `inBuffer` sin validación previa.
- `0xD1` (`DRIVER_IRQL_NOT_LESS_OR_EQUAL`)**:** El driver intentó acceder a memoria paginada mientras se ejecutaba a `DISPATCH_LEVEL`.
- `0x7E` (`SYSTEM_THREAD_EXCEPTION_NOT_HANDLED`)**:** Excepción no capturada por un bloque `__try / __except` en el driver.
- En WinDbg, el comando `!analyze -v` desglosa el call stack exacto y señala la instrucción culpable en el desensamblado del `.sys`.

Fuentes

- Microsoft Learn: Driver security checklist, Controlling device access in KMDF, Using I/O control codes.
- Microsoft Learn, memoria de usuario: Methods for Accessing Data Buffers, Using Neither Buffered Nor Direct I/O, Errors in Referencing User-Space Addresses, ProbeForRead, Handling Exceptions, Accessing Data Buffers in WDF Drivers.
- Microsoft Learn, colas y cancelación: Request Ownership, Configuring Dispatch Mode for an I/O Queue, Canceling I/O Requests, Synchronizing Cancel and Completion Code.
- Microsoft Learn, PnP y energía: Supporting PnP and Power Management in Function Drivers, Power Management for I/O Queues.

Etiquetas

Núcleo conceptual: superficie/mitigaciones de user mode. **Conocimiento externo:** seguridad de drivers, métodos de buffer, sondeo de memoria de usuario, propiedad y cancelación de solicitudes WDF, PnP/energía (recuadro). **Ejemplo ilustrativo:** checklist, fragmento `METHOD_NEITHER` y flujo de cancelación hipotéticos. **Laboratorio verificado:** no; revisión de código fuente.

Ampliación editorial basada en fuentes externas — Localización estática de Dispatch Routines (WDM vs. KMDF) en IDA y Ghidra

Al abrir un driver `.sys` sin símbolos en IDA Pro o Ghidra, la ausencia de una función `main` clásica desorienta al analista; la clave operativa para encontrar el manejador de IOCTLs radica en rastrear cómo el punto de entrada `DriverEntry(PDRIVER_OBJECT DriverObject, PUNICODE_STRING RegistryPath)` —donde `DriverObject` se recibe en `RCX` (x64) o `[esp+4]` (x86)— inicializa sus tablas de despacho (Microsoft Learn: *DRIVER_OBJECT* y *WDF_IO_QUEUE_CONFIG*) *(Hecho confirmado)*:

- **Drivers WDM clásicos (`MajorFunction`):** El código escribe los punteros a sus rutinas directamente sobre el arreglo `DriverObject→MajorFunction` (offset `+0x70` en x64; `+0x38` en x86). Los desplazamientos clave en x64 son:
  - `DriverUnload`: `[rcx + 0x68]` (`+0x34` en x86).
  - `IRP_MJ_CREATE (0x00)`: `[rcx + 0x70]` (`0x70 + 0 * 8`).
  - `IRP_MJ_CLOSE (0x02)`: `[rcx + 0x80]` (`0x70 + 2 * 8`).
  - `IRP_MJ_DEVICE_CONTROL (0x0E = 14)`: **`[rcx + 0xE0]`** (`0x70 + 14 * 8` ; en x86 es `[reg + 0x70]`). En el desensamblado se reconoce típicamente por una instrucción como:

    ```
    mov qword ptr [rcx + 0xE0], sub_DispatchIoctl  ; Enlaza el procesador de DeviceControl
    ```

- **Drivers KMDF (`EvtIoDeviceControl`):** `DriverEntry` no puebla `MajorFunction` manualmente; en su lugar, inicializa la estructura `WDF_DRIVER_CONFIG` colocando el puntero a `EvtDriverDeviceAdd` en el offset **`+0x08`** (x64), enlaza el runtime con **`WdfVersionBind`** e invoca `WdfDriverCreate` (vía tabla global `WdfFunctions`). Posteriormente, dentro de `EvtDriverDeviceAdd` , se busca la inicialización de `WDF_IO_QUEUE_CONFIG` (96 bytes en x64), donde el puntero a **`EvtIoDeviceControl`** se almacena en el offset **`+0x28`** antes de invocar a `WdfIoQueueCreate` .

Minifilters y callbacks: observación del sistema de archivos

> **Alcance de lectura**
>
> No construyas un filtro para bloquear, cambiar o capturar archivos de terceros. La práctica usa documentación, una muestra didáctica y archivos descartables de la VM. No cargues, descargues, adjuntes ni separes filtros del sistema.

> **Propósito**
>
> Ubicar un minifilter en la pila de filesystem y describir pre/post callbacks, orden y límites de observación. Al terminar deberías poder leer la lista de filtros de una VM, ubicar cada uno por su altitud y explicar qué puede y qué no puede haber visto de una operación.

Prerrequisitos

- *Obligatorios:* 6.1 I/O y stacks, 6.3 WinDbg.
- *Recomendados:* 6.4 revisión de drivers.

Funcionamiento

Un minifilter se conecta a una o más pilas de filesystem y puede recibir callbacks antes y después de operaciones de I/O. El Filter Manager y el registro de operaciones ordenan la interacción con otros filtros. Según propósito puede observar, modificar o impedir operaciones; esta lección estudia la observación y el orden conceptual, no modifica contenido.

Una operación visible en un callback no necesariamente llega al disco ni representa una acción completa: puede fallar, cancelarse, ser reintentada o transformarse en otra operación. Correlacioná status, contexto, proceso y archivo de prueba.

Filter Manager, instancias y frames

El **Filter Manager** (`FltMgr.sys`) es un driver del sistema que se activa cuando se carga un minifilter y se adjunta a la pila de filesystem de cada volumen. Un minifilter **no** se adjunta directamente: se **registra** ante el Filter Manager con los tipos de operación que quiere ver. La adhesión de un minifilter a un volumen, a una altitud determinada, se llama **instancia**. Para convivir con filtros heredados (*legacy*), el Filter Manager puede tener varios **frames** en la pila, cada uno cubriendo un rango de altitudes.

Un minifilter puede recibir tres clases de operaciones: las basadas en IRP, las de **fast I/O** y las de callbacks del sistema de archivos. Una misma acción del usuario puede llegar por un camino u otro, así que “no vi el IRP” no significa “no hubo operación”.

Altitudes y grupos de carga

Cada minifilter tiene una **altitud**: un número (interpretado como decimal) que fija su posición en la pila. Microsoft asigna las altitudes y define **grupos de carga** con rangos por tipo de producto. Algunos ejemplos de la tabla oficial:

Grupo de carga	Rango de altitud	Propósito declarado
FSFilter Activity Monitor	360000–389999	Observar e informar sobre la E/S de archivos
FSFilter Anti-Virus	320000–329999	Detectar y desinfectar durante la E/S
FSFilter Replication	300000–309999	Replicar datos a servidores remotos
FSFilter Content Screener	260000–269999	Impedir la creación de ciertos archivos o contenidos
FSFilter Encryption	140000–149999	Cifrar y descifrar durante la E/S
FSFilter Virtualization	130000–139999	Virtualizar rutas de archivo

Para el análisis, la altitud es una **pista de propósito**: un filtro en el rango de antivirus probablemente pertenece a un producto de seguridad. Microsoft publica además una lista de altitudes asignadas con empresa y nombre de archivo, útil para identificar un filtro desconocido. Que la altitud caiga en un rango no prueba qué hace el filtro, pero un filtro sin correspondencia en la lista pública es algo a preguntar.

Orden de los callbacks

Cuando el Filter Manager recibe una operación, llama a los **preoperation callbacks** en orden de altitud **de mayor a menor**, y después, al completarse, a los **postoperation callbacks** en orden **inverso**, de menor a mayor. Un filtro alto ve la operación primero en la ida y último en la vuelta.

Qué puede decidir un preoperation callback (valores documentados):

Valor devuelto	Significado
`FLT_PREOP_SUCCESS_NO_CALLBACK`	Deja pasar la operación y no quiere su postoperation
`FLT_PREOP_SUCCESS_WITH_CALLBACK`	Deja pasar y quiere ver el resultado final en el postoperation
`FLT_PREOP_PENDING`	La deja pendiente para completarla o continuarla después
`FLT_PREOP_SYNCHRONIZE`	Pide que el postoperation corra en el mismo hilo que el pre
`FLT_PREOP_COMPLETE`	Termina la operación ahí, con un estado final propio

`FLT_PREOP_COMPLETE` es el caso en que un filtro **impide** o **responde** una operación sin que llegue a los filtros inferiores ni al sistema de archivos: por eso un filtro alto puede hacer que otro, más bajo, nunca vea algo.

IRQL y contexto. Un preoperation se llama a `PASSIVE_LEVEL` o `APC_LEVEL`, casi siempre en el hilo que originó la E/S. Un postoperation, como una rutina de completion, se llama a IRQL de hasta `DISPATCH_LEVEL` y en un hilo arbitrario, salvo excepciones documentadas (post-create a `PASSIVE_LEVEL` en el hilo original, operaciones sincronizadas y fast I/O). Esto explica por qué un filtro bien escrito toma en el pre la información que necesitará en el post.

Inspección de solo lectura

- **En la VM, con privilegios de administrador:** `fltmc filters` lista los minifilters cargados con su altitud y cantidad de instancias, y `fltmc instances` muestra en qué volúmenes está cada uno. Las otras órdenes de `fltmc` (cargar, descargar, adjuntar, separar) **no** se usan en este curso.
- **Con el depurador de kernel de la 6.3:** la extensión `!fltkd` ofrece `!filters`, `!frames`, `!instance` y `!volumes`, y `!cbd`, el equivalente de `!irp` para los datos de callback del Filter Manager.

Ejemplo trabajado

(Flujo esquemático ficticio.)

```
app solicita abrir C:\Lab\sample.txt
  → pre-operation callback (observa operación/flags)
  → filesystem y filtros inferiores
  → post-operation callback (status final/contexto)
  → app recibe resultado
```

El callback de pre-op no prueba que la apertura haya tenido éxito; el status final aparece después. Un driver distinto puede alterar el orden observado.

Ficha de inventario de una VM (ilustrativa; nombres y altitudes ficticios):

Filtro	Altitud	Grupo inferido	Instancias	Lectura
`labmon`	370100	Activity Monitor	C:, D:	Observa E/S en ambos volúmenes
`labav`	321500	Anti-Virus	C:	Ve la operación **después** de `labmon` en la ida y **antes** en la vuelta
`labcrypt`	141000	Encryption	D:	Si cifra, los filtros más altos ven datos en claro y los más bajos, cifrados

Lo que **no** se puede afirmar con esta ficha: qué hace cada filtro con las operaciones, si alguno completa operaciones en el pre, ni a qué producto pertenece sin contrastar la lista pública de altitudes y el inventario de software.

Práctica

Lee el diagrama y callbacks de una muestra oficial de minifilter sin instalarla. Opcionalmente, inspecciona un minifilter ya presente en una VM con herramientas administrativas de solo lectura y registra nombre/altitud/estado. No descargues ni cambies filtros del sistema. Si se usa una muestra cargada por el instructor, solo observa accesos a archivos creados para el lab y revierte snapshot.

1. En la muestra, enumerá qué operaciones registra, qué devuelve cada pre y si hay post.
2. En la VM, completá la ficha de inventario con `fltmc filters` y `fltmc instances`.
3. Ubicá cada filtro en su grupo por la altitud y contrastalo con la lista pública de altitudes asignadas.

Entregable

Diagrama de stack con altitudes, puntos pre/post, ficha de inventario, datos observados, límites y confirmación de que no se alteraron archivos ni filtros de la máquina.

Repaso

▼ **¿Pre-operation demuestra que la operación se completó?**

> No. La operación aún puede fallar o cancelarse; hay que leer resultado/post-op.

▼ **¿Un minifilter siempre modifica datos?**

> No. Puede observar, filtrar o modificar; depende de su implementación.

▼ **¿En qué orden se llaman los callbacks de varios minifilters?**

> Los pre, de mayor a menor altitud; los post, en orden inverso.

▼ **¿Qué significa `FLT_PREOP_COMPLETE` para los filtros inferiores?**

> Que la operación terminó en ese filtro: los inferiores y el sistema de archivos no la ven.

▼ **¿Por qué un postoperation no puede usar memoria paginada en general?**

> Porque puede correr hasta `DISPATCH_LEVEL` en un hilo arbitrario, salvo las excepciones documentadas.

▼ **¿Qué te dice la altitud de un filtro?**

> Su posición en la pila y, por el rango, el tipo de producto declarado; no prueba qué hace.

▼ **¿Por qué no retirar un filtro desconocido durante análisis?**

> Puede pertenecer al sistema o producto crítico; documenta y consulta al administrador para preservar estabilidad/evidencia.

> **Puerta optativa**
>
> Podés explicar el recorrido de una operación, ubicar filtros por altitud y grupo, distinguir callback previo de resultado final, explicar el efecto de cada valor de retorno del pre y documentar un inventario con alcance y límites.

6.5.b — Windows Filtering Platform (WFP): El "Minifilter" de la Red

Así como el **Filter Manager** (`FltMgr.sys`) gobierna la intercepción de archivos en disco, el subsistema de red de Windows implementa **WFP (Windows Filtering Platform)** para mediar, inspeccionar, autorizar y modificar el tráfico de red en modo kernel. WFP es la base técnica de los firewalls de próxima generación, soluciones EDR, proxies locales y sistemas de prevención de intrusiones corporativos.

Arquitectura Interna de WFP vs. Minifilters de Filesystem

A diferencia del modelo de altitudes de filesystem, WFP está integrado directamente en el stack TCP/IP (`tcpip.sys`) y en el driver del cliente de filtrado (`fwpkclnt.sys`). Opera mediante una jerarquía de capas (*Filtering Layers*), filtros, subcapas y módulos de llamada (*Callouts*):

1. **Capas de Filtrado Clave (*Layers*):**

- **`FWPM_LAYER_ALE_AUTH_CONNECT_V4` / `_V6`:** *Application Layer Enforcement* para conexiones salientes. Intercepta la llamada de red en el momento exacto en que un proceso invoca `connect()`, **antes** de que el paquete TCP SYN salga al cable. Provee el PID del proceso emisor, el path completo del ejecutable y las tuplas IP/puerto de origen y destino.
- **`FWPM_LAYER_ALE_AUTH_RECV_ACCEPT_V4` / `_V6`:** Intercepta intentos de conexión entrante (`listen()` / `accept()`), permitiendo bloquear puertos a nivel de proceso.
- **`FWPM_LAYER_INBOUND_TRANSPORT_V4` / `FWPM_LAYER_OUTBOUND_TRANSPORT_V4`:** Intercepción a nivel de paquete de transporte (TCP/UDP) post-reasignación. Provee acceso al descriptor de memoria de red `NET_BUFFER_LIST` (`NBL`).
- **`FWPM_LAYER_DATAGRAM_DATA_V4` / `_V6`:** Inspección de payloads UDP crudos.

2. **Callouts de Kernel (`FWPS_CALLOUT`):**

Un controlador de seguridad registra un Callout llamando a `FwpsCalloutRegister`. La estructura define tres punteros de función fundamentales:

- **`classifyFn`:** La función principal invocada por WFP cada vez que un paquete o conexión coincide con las reglas de filtrado.
- **`notifyFn`:** Notifica al driver cuando se añaden o eliminan reglas asociadas al Callout.
- **`flowDeleteFn`:** Notifica cuando un flujo de red contextual se termina, permitiendo liberar memoria de rastreo.

Anatomía de la Función de Clasificación (`classifyFn`)

```
void ClassifyCallback(
    const FWPS_INCOMING_VALUES0* inFixedValues,
    const FWPS_INCOMING_METADATA_VALUES0* inMetaValues,
    void* layerData,
    const void* classifyContext,
    const FWPS_FILTER0* filter,
    UINT64 flowContext,
    FWPS_CLASSIFY_OUT0* classifyOut
) {
    // 1. Obtener el PID del proceso desde los metadatos ALE
    UINT64 processId = 0;
    if (FWPS_IS_METADATA_FIELD_PRESENT(inMetaValues, FWPS_METADATA_FIELD_PROCESS_ID)) {
        processId = inMetaValues->processId;
    }

    // 2. Extraer dirección IP y puerto de destino (en FWPM_LAYER_ALE_AUTH_CONNECT_V4)
    UINT32 ipDest = inFixedValues-
>incomingValue[FWPS_FIELD_ALE_AUTH_CONNECT_V4_IP_REMOTE_ADDRESS].value.uint32;
    UINT16 puertoDest = inFixedValues-
>incomingValue[FWPS_FIELD_ALE_AUTH_CONNECT_V4_IP_REMOTE_PORT].value.uint16;

    // 3. Decisión defensiva de filtrado:
    // FWP_ACTION_PERMIT  : Permite el tráfico sin restricciones.
    // FWP_ACTION_BLOCK   : Descarta el paquete o rechaza la conexión TCP de inmediato.
    // FWP_ACTION_CONTINUE: Pasa la decisión a las subcapas siguientes.
    if (puertoDest == 4444) { // Detección de puerto sospechoso de reversa
        classifyOut->actionType = FWP_ACTION_BLOCK;
        classifyOut->flags |= FWPS_CLASSIFY_OUT_FLAG_ABSORB; // Absorbe el paquete para evitar RST
        return;
    }

    classifyOut->actionType = FWP_ACTION_PERMIT;
}
```

Herramientas de Inspección Defensiva de WFP

Un analista de malware o respuesta a incidentes debe auditar qué filtros y callouts están instalados en una máquina para verificar si una amenaza o software de seguridad está interceptando el tráfico de red:

1. **Inspección Nativa con `netsh`:**

```
   netsh wfp show state
```

Este comando genera un archivo `wfpstate.xml` en el directorio actual. Al parsearlo, se listan todos los proveedores (`providers`), filtros activos (`filters`) con sus condiciones booleanas, y los GUIDs de los callouts enlazados a drivers de terceros.

2. **Inspección con `wfptool` y WinDbg:**

- La herramienta de auditoría `wfptool.exe` permite listar filtros activos filtrando por PID del proceso o por capa (`ALE_AUTH_CONNECT_V4`).
- En WinDbg kernel mode:

```
    !wfp layers
    !wfp callouts
```

Permite inspeccionar los punteros a `classifyFn` en la memoria del kernel para identificar qué driver (`.sys`) maneja cada capa de red.

Fuentes

- Microsoft Learn: About File System Filter Drivers, Filter Manager concepts, Writing Preoperation Callback Routines, Writing Postoperation Callback Routines.
- Microsoft Learn, altitudes y herramientas: Load Order Groups and Altitudes for Minifilter Drivers, Allocated filter altitudes, Tools for Minifilter Development and Testing (Fltmc, !fltkd).

> **Etiquetas**
>
> **Núcleo conceptual:** monitoreo de procesos como contexto. **Conocimiento externo:** Filter Manager/minifilters, altitudes, valores de retorno e IRQL de callbacks. **Ejemplo ilustrativo:** flujo de archivo y ficha de inventario ficticia. **Laboratorio verificado:** no.

Ampliación editorial basada en fuentes externas — Estructuras FLT_REGISTRATION y callbacks de monitoreo en Kernel

Al auditar en IDA o Ghidra drivers defensivos, EDRs o módulos de filtrado de archivos sin símbolos, el reverser necesita identificar de inmediato qué operaciones y eventos del sistema están siendo interceptados (Microsoft Learn: *FLT_REGISTRATION* y *ObRegisterCallbacks*) *(Hecho confirmado)*:

1. Minifiltros de Filesystem (`FltRegisterFilter`): El segundo argumento (`RDX` en x64) apunta a la estructura estática `FLT_REGISTRATION` en `.rdata` / `.data`. Su campo **`OperationRegistration`** (offset **`+0x20`** en x64) apunta a un arreglo de estructuras `FLT_OPERATION_REGISTRATION` de 32 bytes cada una:

Offset (x64)	Campo	Tipo y Significado
`+0x00`	`MajorFunction`	`UCHAR` (ej. `0x00` = `IRP_MJ_CREATE`, `0x04` = `IRP_MJ_WRITE`; termina con `0x80` = `IRP_MJ_OPERATION_END`).
`+0x04`	`Flags`	`ULONG` (banderas de operación, ej. `FLTFL_OPERATION_REGISTRATION_SKIP_CACHED_IO`).
`+0x08`	`PreOperation`	`PFLT_PRE_OPERATION_CALLBACK` (puntero a la rutina previa a la E/S).
`+0x10`	`PostOperation`	`PFLT_POST_OPERATION_CALLBACK` (puntero a la rutina posterior a la E/S).
`+0x18`	`Reserved1`	`PVOID` (reservado para el sistema, completa los 32 bytes de la estructura).

2. Callbacks de Kernel no relacionados con Filesystem (Telemetría y Protección):

API de Registro	Estructura / Parámetro Clave	Propósito Defensivo / Intercepción
`PsSetCreateProcessNotifyRoutineEx`	`NotifyRoutine` (1.er arg, `RCX`)	Notifica creación y terminación de procesos en el sistema; permite denegar la creación retornando error en `CreateInfo→CreationStatus`.
`PsSetLoadImageNotifyRoutine`	`NotifyRoutine` (1.er arg, `RCX`)	Intercepta cada mapeo de ejecutables (`.exe`), bibliotecas (`.dll`) o módulos de kernel (`.sys`) en el espacio de memoria virtual.
`ObRegisterCallbacks`	`OB_CALLBACK_REGISTRATION` → arreglo `OB_OPERATION_REGISTRATION` (`+0x20`)	Intercepta apertura y duplicación de handles sobre `PsProcessType` y `PsThreadType`, despojando máscaras de acceso críticas como `PROCESS_VM_WRITE` o `PROCESS_TERMINATE` para blindar procesos.

Mitigaciones de Windows para drivers

> **Solo observación**
>
> La práctica consulta el estado vigente. No deshabilites Secure Boot, HVCI, WDAC, firma de drivers ni driver blocklist para que una muestra cargue. No apliques políticas ni actives modos de prueba en el equipo principal.

> **Propósito**
>
> Relacionar controles de integridad y política con el riesgo de drivers vulnerables, y evitar confundir "activo" con "cobertura total". Al terminar deberías poder leer el estado real de cada control en una VM, distinguir "configurado" de "en ejecución" y escribir qué protege y qué no.

Prerrequisitos

- *Obligatorios:* 6.1 arquitectura, 6.4 revisión.
- *Recomendados:* 2D.1 tokens, Etapa 0 snapshots y nociones de Secure Boot.

Qué protege cada control

- **Firma de driver**: ayuda a establecer origen/integridad aceptable bajo política de carga; no demuestra ausencia de bugs.
- **Secure Boot**: protege la cadena de arranque con verificación de componentes firmados; política y soporte dependen del hardware/firmware.
- **VBS/HVCI (Memory Integrity)**: usa virtualización para aislar decisiones de integridad de código del kernel y restringir código ejecutable en kernel.
- **WDAC/App Control**: permite definir qué software/drivers pueden ejecutarse según política.
- **Vulnerable driver blocklist**: bloquea un conjunto de drivers conocidos vulnerables; versión, habilitación y excepciones dependen de la versión/política.

Se complementan y pueden afectar compatibilidad. La configuración cambia entre Windows cliente/servidor y administración organizacional; registra build y fuente consultada.

Firma de drivers: qué exige hoy la política

Desde Windows 10 versión 1607, con Secure Boot activo, Windows no carga drivers de kernel **nuevos** que no estén firmados a través del portal de hardware de Microsoft. La documentación lista excepciones para drivers con firma cruzada (equipos actualizados desde versiones anteriores, Secure Boot desactivado y certificados emitidos antes de una fecha de corte). Para probar en clientes existe la firma por **atestación**, y para laboratorio, la firma de **prueba**, que requiere un modo de arranque específico.

Consecuencias para el análisis: "está firmado" dice **quién** y **bajo qué programa** se firmó; no dice que el código sea correcto ni benigno. Un driver vulnerable legítimamente firmado sigue siendo vulnerable: para eso existe la blocklist. Y un sistema en modo de firma de prueba tiene una política de carga distinta, que hay que anotar si la VM lo usa.

Integridad de memoria (HVCI) y VBS

La integridad de memoria, también llamada HVCI, ejecuta la integridad de código del kernel dentro del entorno aislado de VBS (con el hipervisor de Windows) y restringe asignaciones de memoria del kernel que podrían usarse para comprometerlo; también protege, entre otras cosas, el mapa de bits de CFG de los drivers. Microsoft advierte que hay drivers incompatibles y que activarla puede causar fallos de arranque, por eso en esta lección **solo se consulta**.

Cómo leer el estado, sin cambiarlo:

- `Get-CimInstance -ClassName Win32_DeviceGuard -Namespace root\Microsoft\Windows\DeviceGuard` (PowerShell elevado).
- `msinfo32`: las características de VBS aparecen al final del Resumen del sistema.

Campos de `Win32_DeviceGuard` que importan:

Campo	Qué dice	Valores clave
`VirtualizationBasedSecurityStatus`	Si VBS está habilitada y corriendo	0 no habilitada · 1 habilitada pero **no** corriendo · 2 habilitada y corriendo
`SecurityServicesConfigured`	Qué servicios están **configurados**	contiene 2 si la integridad de memoria está configurada
`SecurityServicesRunning`	Qué servicios están **corriendo**	contiene 2 si la integridad de memoria está corriendo
`CodeIntegrityPolicyEnforcementStatus`	Aplicación de la política de integridad de código	0 apagada · 1 auditoría · 2 aplicada
`AvailableSecurityProperties`	Capacidades de hardware disponibles	hipervisor, Secure Boot, protección DMA, MBEC/GMET…

La distinción entre **configurado** y **corriendo** es la que más errores evita: una política puede pedir HVCI y el sistema no poder ejecutarlo (por hardware, por un driver incompatible o, como documenta Microsoft para ciertas VM de Azure, por la opción de Secure Boot elegida).

Vulnerable driver blocklist

Según la documentación vigente, desde la actualización 2022 de Windows 11 la blocklist está habilitada por defecto y puede activarse o desactivarse desde Seguridad de Windows; también se aplica cuando están activos la integridad de memoria, Smart App Control o el modo S (salvo en Windows Server 2016). Se actualiza trimestralmente y llega además con las actualizaciones mensuales.

Límites que Microsoft declara: la lista **no garantiza** bloquear todo driver vulnerable, y la versión incluida en el sistema suele ser **menos completa** que la descargable, porque algunas entradas se retienen para no romper compatibilidad. Si se aplica la versión descargable como política de App Control, el registro *CodeIntegrity/Operational* registra el evento 3099 con el nombre e identificador de la política: es una forma de **verificar** qué política está cargada sin cambiar nada.

Secure Boot y App Control

Secure Boot verifica que los componentes de arranque estén firmados por autoridades confiables del firmware; su estado aparece en `msinfo32` ("Estado de arranque seguro"). Es requisito de varias de las otras protecciones y condición de la política de firma descrita arriba.

App Control for Business (antes WDAC) define qué código puede ejecutarse, incluidos drivers, y puede estar en modo **auditoría** (registra lo que bloquearía) o **aplicado**. Para el análisis, una política en auditoría no protege: solo deja evidencia.

Ejemplo trabajado

(Matriz ficticia para discusión, no estado de una máquina.)

Señal	Control relacionado	Qué no concluye
HVCI activo	integridad de código en kernel	que todo driver sea libre de fallos
paquete firmado	política de firma	que el código sea benigno
driver en blocklist	bloqueo por política	que todos los vulnerables estén listados

Lectura de una VM de laboratorio (ilustrativa):

Control	Estado leído	Fuente	Lectura prudente
Secure Boot	Activado	`msinfo32`	La política de firma estricta aplica a drivers nuevos
VBS	`VirtualizationBasedSecurityStatus = 2`	`Win32_DeviceGuard`	VBS habilitada y corriendo
Integridad de memoria	Configurada: sí (2); corriendo: sí (2)	`Win32_DeviceGuard`	HVCI efectiva en este arranque
Blocklist	Activada en Seguridad de Windows	Interfaz	Aplica la versión incluida en el sistema, no necesariamente la más reciente
App Control	Sin política propia; solo la del sistema	Registro *CodeIntegrity*	No hay lista de permitidos de la organización

Por qué los drivers x64 no parchean el kernel: PatchGuard (KPP) y el bugcheck 0x109

En Windows x64, **Kernel Patch Protection (PatchGuard / KPP)** verifica periódicamente la integridad de las estructuras críticas del kernel: el código y las secciones de solo lectura de `ntoskrnl.exe` / `hal.dll`, la **SSDT** (`KeServiceDescriptorTable`), la **IDT/GDT**, MSRs críticos (`IA32_LSTAR` → `KiSystemCall64`) y estructuras internas. Si cualquier driver modifica un byte de esas regiones, PatchGuard detiene el sistema de forma diferida con el bugcheck `0x109` **(CRITICAL_STRUCTURE_CORRUPTION)**. Esa es la razón arquitectónica por la que, en x64, los productos de seguridad y los drivers legítimos **abandonaron el SSDT/inline hooking** de la era de 32 bits y usan solo las interfaces de notificación documentadas de 6.5 (`FltRegisterFilter` , `ObRegisterCallbacks` , `PsSetCreateProcessNotifyRoutineEx`).

Práctica de solo lectura

En la VM, consulta Windows Security / System Information y la política administrada disponible. Registra build/edición, estado informado de Secure Boot/VBS/HVCI y fuente de la blocklist. Si la política está oculta por organización, anota "no disponible"; no cambies ajustes ni instales drivers para probarla.

1. Ejecutá la consulta de `Win32_DeviceGuard` y anotá los campos de la tabla, distinguiendo configurado de corriendo.
2. Anotá el estado de Secure Boot desde `msinfo32` y si la VM usa algún modo de firma de prueba.
3. Revisá en el Visor de eventos el registro *CodeIntegrity/Operational* solo para leer qué políticas están cargadas.

Entregable

Matriz control–estado–fuente–alcance–límite, con la distinción configurado/corriendo para VBS y HVCI. Incluí fecha/build y no publiques identificadores del equipo.

Repaso

▼ **¿Firma significa que el driver es seguro?**

> No. Indica una propiedad de identidad/integridad bajo una política de confianza, no corrección ni ausencia de comportamiento malicioso.

▼ **¿La blocklist cubre cualquier driver vulnerable?**

> No. Es una lista mantenida con criterios y actualizaciones; hay límites de versión y compatibilidad.

▼ **`SecurityServicesConfigured` contiene 2 pero `SecurityServicesRunning` no. ¿Qué significa?**

> Que la integridad de memoria está pedida por configuración pero no está corriendo en este arranque; no protege.

▼ **¿Qué exige la política de firma desde Windows 10 1607 con Secure Boot activo?**

> Que los drivers de kernel nuevos estén firmados a través del portal de Microsoft, con las excepciones documentadas.

▼ **¿Protege una política de App Control en modo auditoría?**

> No bloquea: solo registra lo que bloquearía.

▼ **¿Qué hacer si una prueba requiere desactivar controles?**

> No hacerlo en el equipo principal; si el entorno de laboratorio no está aprobado y documentado, abandonar la carga y continuar con análisis estático.

Puerta optativa

Podés explicar qué aporta firma, Secure Boot, HVCI/VBS, App Control y blocklist, leer su estado sin modificarlos, distinguir configurado de corriendo y declarar estado/fuente/límites.

Continuidad y Profundización

La arquitectura interna de VBS, el Secure Kernel en VTL1, HVCI por hardware y la auditoría en memoria viva de los vectores de notificación y telemetría de kernel (PspCreateProcessNotifyRoutine, ObRegisterCallbacks y ETWti) se desarrollan operativamente en la lección siguiente: 6.7 Seguridad profunda en kernel x64.

Fuentes

- Microsoft Learn: Driver security checklist, HVCI compatible code, Microsoft recommended driver block rules, Windows Defender Application Control.
- Microsoft Learn, estado y firma: Enable memory integrity (incluye Win32_DeviceGuard), Driver Signing Policy, The TESTSIGNING boot configuration option, Secure Boot.

Etiquetas

Núcleo conceptual: mitigaciones de user mode como contexto. **Conocimiento externo:** política de firma, VBS/HVCI y lectura de su estado, blocklist, Secure Boot y App Control. **Ejemplo ilustrativo:** matrices ficticias. **Laboratorio verificado:** no; consulta en solo lectura.

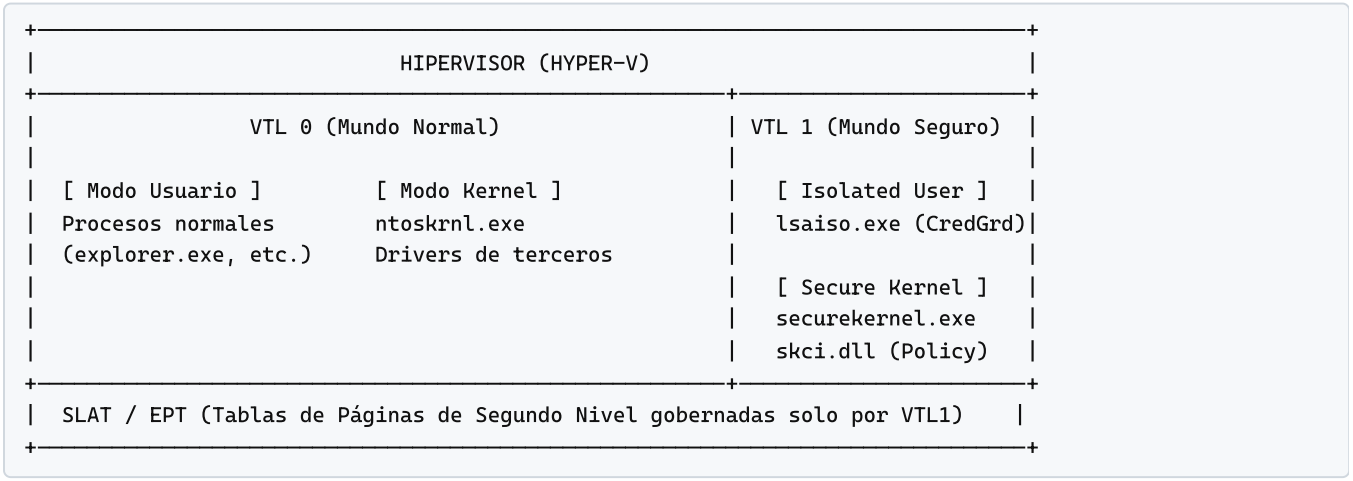
Seguridad Profunda en Kernel x64: VBS/HVCI (VTL1), Callbacks y Mecánicas BYOVD

> **Propósito**
>
> Esta lección profundiza la arquitectura de seguridad del Kernel en Windows 11 contemporáneo, eliminando la frontera de lo que antes se consideraba "fuera de alcance". Aquí se estudian los mecanismos internos que rigen la protección por hipervisor (**VBS / HVCI**), el aislamiento de **Credential Guard con trustlets IUM**, el flujo exacto de validación de imágenes (`NtCreateSection` $\to$ `VtlCall`), las estructuras en memoria que sustentan los vectores de notificación y telemetría de kernel (**PspCreateProcessNotifyRoutine**, **ObRegisterCallbacks**, **ETWti**, **CmCallbackListHead**, **InstrumentationCallback**), el vector **BYOVD** con inspección de **PiDDBCacheTable**, y el análisis forense de fallos por **Double Fetch** en controladores **KMDF**.

---

1. Arquitectura Interna de VBS y Secure Kernel (VTL0 vs. VTL1)

Desde Windows 10 versión 1709 y por defecto en Windows 11, el modelo de seguridad del sistema operativo se basa en **Virtualization-Based Security (VBS)**. El hipervisor de Windows (Hyper-V) divide el hardware en dos mundos o **Virtual Trust Levels (VTL)**:



Transiciones de Privilegio: `VtlCall` / `VtCall` y `VtlReturn`

- `VTL 0` **:** Contiene el kernel clásico (`ntoskrnl.exe`), drivers de terceros y aplicaciones de usuario.
- `VTL 1` **:** Contiene el **Secure Kernel** (`securekernel.exe`) y bibliotecas de integridad aisladas como `skci.dll` .
- Las comunicaciones entre mundos no ocurren mediante interrupciones de software clásicas, sino a través de la instrucción de hipervisor `vmcall` (en Intel) o `vmmcall` (en AMD), encapsulada en la rutina `VtlCall` (también referenciada como `VtCall`). Cuando VTL 1 concluye la operación, devuelve el control a VTL 0 mediante `VtlReturn` .

Integridad de Código por Hipervisor (HVCI) y $W \oplus X$

Bajo **HVCI (Hypervisor-Protected Code Integrity)**, las decisiones sobre qué páginas de memoria pueden ejecutarse en modo kernel se delegan a VTL 1. El hipervisor configura las tablas de páginas de segundo nivel (**SLAT / EPT**) de modo que ninguna página en VTL 0 pueda ser simultáneamente escribible y ejecutable ($W \oplus X$). Si un driver intenta asignar memoria en el pool ejecutable (`NonPagedPoolExecute`) o desproteger el bit de escritura de una página de código en `CR0` , el hipervisor intercepta el fallo de página anidado (*EPT Violation*) e interrumpe inmediatamente la ejecución antes de que el código no firmado pueda ejecutarse.

Credential Guard, LSAISO y Trustlets en Modo de Usuario Aislado (IUM)

En sistemas convencionales sin VBS, el proceso `lsass.exe` almacenaba hashes NTLM, credenciales en claro y tickets Kerberos directamente en su propio espacio de direcciones en modo usuario (VTL 0), haciéndolos vulnerables a lecturas de memoria con herramientas como Mimikatz si el atacante obtenía privilegios de `SeDebugPrivilege` o ejecutaba un driver en modo kernel.

Bajo **Credential Guard**:

1. El kernel crea un proceso especial en **Modo de Usuario Aislado (IUM - Isolated User Mode)** que reside dentro de **VTL 1**: `lsaiso.exe` (*LSA Isolated*).
2. El proceso `lsass.exe` en VTL 0 pasa a funcionar como un mero intermediario de RPC/ALPC: no contiene claves criptográficas ni hashes en claro.
3. Cada vez que se requiere autenticar un usuario o firmar un ticket Kerberos, `lsass.exe` envía una solicitud a través de llamadas ALPC canalizadas mediante una transición `VtlCall` / `VtCall` hacia `lsaiso.exe` en VTL 1.
4. El Secure Kernel en VTL 1 procesa la operación criptográfica usando claves protegidas por hardware (TPM) y devuelve únicamente el resultado de la autenticación a VTL 0.
5. Las tablas de páginas de VTL 1 no están mapeadas en el espacio de direcciones de VTL 0: ni siquiera un driver en modo kernel ejecutándose con máximos privilegios en VTL 0 puede leer la memoria de `lsaiso.exe` .

Flujo de Validación de Imágenes: `NtCreateSection` $\to$ `VtlCall` / `VtCall` $\to$ `skci.dll`

Cuando se carga un binario ejecutable o controlador en el sistema:

1. El llamador invoca `NtCreateSection` con la bandera `SEC_IMAGE` .
2. El gestor de memoria (`nt!MiCreateSection`) analiza las cabeceras PE y transfiere la validación de firma a la biblioteca de integridad de código del kernel (`ci.dll!CiValidateImageHeader`).
3. Si la política de HVCI está activa, `ci.dll` no toma la decisión de ejecución de manera local en VTL 0: emite una transición de hipervisor `VtlCall` hacia `skci.dll` (*Secure Kernel Code Integrity*) en VTL 1.
4. `skci.dll` valida los certificados digitales, la cadena de confianza y las reglas WDAC (*Windows Defender Application Control*), computando los hashes criptográficos de cada página física.
5. Si la validación es exitosa, el Secure Kernel programa las tablas de páginas SLAT/EPT del hipervisor marcando las páginas físicas correspondientes como ejecutables de solo lectura ($R \oplus X$). Si la validación fracasa, VTL 1 retorna un estado de error (`STATUS_IMAGE_CERT_REVOKED` o `STATUS_INVALID_IMAGE_HASH`) y `NtCreateSection` es abortada inmediatamente.

Código C: Gatillo de Mapeo con Validación Criptográfica en Kernel

```
// Invocación nativa para mapear una imagen PE sujeta a verificación de HVCI
HANDLE hFile = NULL, hSection = NULL;
OBJECT_ATTRIBUTES objAttr;
IO_STATUS_BLOCK ioStatus;
UNICODE_STRING uPath;

RtlInitUnicodeString(&uPath, L"\\??\\C:\\drivers\\vulnerable_or_untrusted.sys");
InitializeObjectAttributes(&objAttr, &uPath, OBJ_CASE_INSENSITIVE, NULL, NULL);

NtOpenFile(&hFile, FILE_EXECUTE | SYNCHRONIZE, &objAttr, &ioStatus,
           FILE_SHARE_READ, FILE_NON_DIRECTORY_FILE | FILE_SYNCHRONOUS_IO_NONALERT);

// Al invocar NtCreateSection con SEC_IMAGE (0x01000000), ntoskrnl transfiere el control a CI / VTL1
NTSTATUS status = NtCreateSection(
    &hSection,
    SECTION_ALL_ACCESS,
    NULL,
    NULL,
    PAGE_EXECUTE,
    SEC_IMAGE,   // 0x01000000: Gatillo de verificación de firma y validación SLAT/EPT
    hFile
);
// Si HVCI rechaza la firma en VTL1, status retornará STATUS_INVALID_IMAGE_HASH (0xC0000428)
```

Receta WinDbg para Rastrear el Pipeline `NtCreateSection` $\to$ `VtlCall`

```
$$ 1. Breakpoint en la entrada de creación de sección y análisis PE
bp nt!NtCreateSection "r @rcx; gc"
bp nt!MiCreateSection

$$ 2. Breakpoint en la biblioteca de integridad de código (VTL 0)
bp ci!CiValidateImageHeader "du poi(@rcx+0x30); gc"
bp ci!CiValidateImagePages

$$ 3. Breakpoint en la llamada de transición de hipervisor (VtlCall / VtCall hacia Secure Kernel)
$$ (Nota técnica: En ci.dll, la rutina nativa exportada que ensambla la transición VtlCall/VtCall hacia
VTL1 se denomina ci!IvtCallHypervisor, citada también como ci!VtCallHypervisor)
bp ci!IvtCallHypervisor
$$ Al alcanzar ci!IvtCallHypervisor, examinar los argumentos de la llamada:
$$ RCX = Código de comando VTL1 (ej. 0x17 = SECURE_IMAGE_VALIDATE_PAGES)
$$ RDX = Puntero al bloque de contexto con las PFNs físicas a verificar
r rcx, rdx

$$ 4. Breakpoint en el Secure Kernel (VTL 1) (si el depurador está conectado a VTL1)
bp skci!SkciValidateImagePages
```

kCFG (Kernel Control Flow Guard) Respaldado por VTL 1

En modo kernel, **kCFG** verifica cada llamada indirecta (`call rax`) comprobando si la dirección de destino está registrada en el bitmap de funciones válidas del kernel (`nt!_guard_dispatch_icall`). Bajo VBS, el mapa de bits de CFG se almacena en memoria de solo lectura para VTL 0 y solo puede ser actualizado por el Secure Kernel en VTL 1.

Precisión sobre códigos de fallo en CFG y CET (#CP)

Al auditar fallos de integridad de flujo de control en user-mode y kernel-mode (y como fe de erratas respecto de la mención en el Tomo I, Lección 5.2.b, p. 333), no confundas los códigos de estado de `ntstatus.h`: el código `0xC0000409` corresponde a `STATUS_STACK_BUFFER_OVERRUN` (invocado mediante `int 29h` / `__fastfail` con subcódigo en `RCX`, como `0x0A` = `FAST_FAIL_GUARD_ICALL_CHECK_FAILURE` en violaciones de CFG o `0x2A` / `0x39` en violaciones de retorno; en modo kernel desencadena el bugcheck `0x139 KERNEL_SECURITY_CHECK_FAILURE`). En cambio, el código `NTSTATUS` específico asignado a la excepción de hardware `#CP` **(Control Protection Exception, vector 21 de Intel CET / AMD Shadow Stack)** es `0xC00006A6` **(`STATUS_CONTROL_PROTECTION_EXCEPTION`)** (y en modo kernel el bugcheck `0x00000139` con parámetro `0x39` o `0x0000007F UNEXPECTED_KERNEL_MODE_TRAP` con trampa `0x15`).

---

2. Auditoría en Memoria de los Vectores de Telemetría y Notificación de Kernel

El reverser o analista defensivo debe dominar la inspección en **WinDbg** de los arreglos no exportados de notificación, registro y telemetría en memoria viva.

A. Notificaciones de Creación de Procesos (`PspCreateProcessNotifyRoutine`)

Windows mantiene una tabla interna de 64 punteros de tipo `EX_FAST_REF` que registran los callbacks suscritos con `PsSetCreateProcessNotifyRoutineEx`:

- En x64, los 4 bits inferiores de cada entrada corresponden al conteo de referencias concurrentes.
- Para obtener la dirección del bloque de datos se aplica una máscara de bits: `Puntero & ~0x0F`.
- En el bloque resultante (`_EX_CALLBACK_ROUTINE_BLOCK`), el offset `+0x08` contiene el puntero a la función del driver.

```
$$ Script WinDbg para auditar todos los slots de PspCreateProcessNotifyRoutine
r $t0 = nt!PspCreateProcessNotifyRoutine
.for (r $t1 = 0; @$t1 < 0x40; r $t1 = @$t1 + 1) {
    r $t2 = poi(@$t0 + @$t1 * 8) & 0xfffffffffffffff0
    .if (@$t2 != 0) {
        r $t3 = poi(@$t2 + 8)
        .printf "[Slot %02d] Callback en %p : ", @$t1, @$t3; ln @$t3
    }
}
```

B. Callbacks de Objetos de Proceso (`ObRegisterCallbacks`)

Los drivers de seguridad registran rutinas `PreOperation` y `PostOperation` sobre objetos `PsProcessType` para despojar permisos peligrosos (como `PROCESS_VM_WRITE` o `PROCESS_VM_OPERATION`):

- En Windows 10 y 11 x64, la cabeza de lista `CallbackListHead` se ubica en el offset `+0xC8` de la estructura `_OBJECT_TYPE` apuntada por `*PsProcessType`.

- Cada elemento de la lista apunta a una estructura `OB_CALLBACK_CONTEXT_BLOCK` donde `+0x28` es el puntero `PreOperation` y `+0x30` es `PostOperation`.

C. Callbacks de Registro: `nt!CmCallbackListHead` y `CmRegisterCallbackEx`

Los drivers de auditoría e integridad registran filtros de transacciones sobre el Registro mediante `CmRegisterCallbackEx`:

- Los callbacks activos se encadenan en la lista doblemente enlazada no exportada `nt!CmCallbackListHead`.
- Cada nodo corresponde a una estructura `_CM_CALLBACK_CONTEXT_BLOCK` donde el offset `+0x28` almacena el puntero `Function` (la rutina del driver) y `+0x30` contiene el puntero `Context`.

```
$$ Recorrer en WinDbg todos los callbacks de registro registrados
!list -t nt!_CM_CALLBACK_CONTEXT_BLOCK.List -x "dps @$extret+0x28 L1" nt!CmCallbackListHead
```

D. Sensor en Proceso: `KPROCESS→InstrumentationCallback`

Para evitar el monitoreo invasivo mediante inline hooking en modo usuario, Windows expone un mecanismo nativo de instrumentación en el despachador de syscalls:

- En la estructura `_KPROCESS` (`nt!_KPROCESS`), el campo `InstrumentationCallback` cambia de desplazamiento según la compilación: reside en `+0x3D8` **en Windows 10 22H2 (Build 19045)**, en `+0x3E8` **en Windows 11 22H2/23H2 (Builds 22621 y 22631)** y en `+0x2C8` **en Windows 11 24H2 (Build 26100)**. Almacena un puntero a la rutina de retorno en espacio de usuario.
- Se configura desde user mode mediante `NtSetInformationProcess` pasando la clase no documentada `ProcessInstrumentationCallback` (`0x28`).
- **Mecánica de intercepción:** Cuando este puntero está activo, cada vez que una llamada al sistema (`syscall`) o interrupción retorna desde el kernel al modo usuario, la CPU no regresa inmediatamente a la instrucción siguiente (`RIP`), sino que desvía la ejecución a la dirección configurada en `InstrumentationCallback`. Sensores defensivos y EDRs lo utilizan para inspeccionar el estado de los registros y el stack post-syscall.

Implementación en C: Registro de `InstrumentationCallback`

```
#include <windows.h>
#include <stdio.h>

#define ProcessInstrumentationCallback 0x28

typedef struct _PROCESS_INSTRUMENTATION_CALLBACK_INFORMATION {
    ULONG Version;      // 0 en x64
    ULONG Reserved;     // 0
    PVOID Callback;     // Puntero a la rutina en espacio de usuario
} PROCESS_INSTRUMENTATION_CALLBACK_INFORMATION;

typedef NTSTATUS (NTAPI *pfnNtSetInformationProcess)(
    HANDLE ProcessHandle,
    ULONG ProcessInformationClass,
    PVOID ProcessInformation,
    ULONG ProcessInformationLength
);

// Stub en ensamblador que recibe el control tras cada syscall
extern void InstrumentationCallbackEntry(void);

void InstalarInstrumentationCallback(void) {
    pfnNtSetInformationProcess NtSetInformationProcess =
        (pfnNtSetInformationProcess)GetProcAddress(GetModuleHandleA("ntdll.dll"),
"NtSetInformationProcess");
    if (!NtSetInformationProcess) return;

    PROCESS_INSTRUMENTATION_CALLBACK_INFORMATION info;
    info.Version = 0;
    info.Reserved = 0;
    info.Callback = (PVOID)(ULONG_PTR)InstrumentationCallbackEntry;

    NTSTATUS status = NtSetInformationProcess(
        GetCurrentProcess(),
        ProcessInstrumentationCallback,
        &info,
        sizeof(info)
    );
    if (status == 0) {
        printf("[+] InstrumentationCallback registrado correctamente.\n");
    }
}
```

Stub en Ensamblador x64 (`instrumentation_entry.asm`)

Al regresar de la syscall, el kernel coloca la dirección de retorno original (`RIP` en modo usuario) en el registro **`R10`** , y las banderas en **`R11`** :

```
.code
PUBLIC InstrumentationCallbackEntry
EXTRN AuditarRetornoSyscall:PROC

InstrumentationCallbackEntry PROC
    ; Preservar el contexto volátil completo del hilo
    pushfq
    push    rax
    push    rcx
    push    rdx
    push    r8
    push    r9
    push    r10
    push    r11

    ; Invocar función C pasando R10 (RIP original de retorno) como 1er argumento (RCX)
    sub     rsp, 20h
    mov     rcx, r10
    call    AuditarRetornoSyscall
    add     rsp, 20h

    ; Restaurar contexto
    pop     r11
    pop     r10
    pop     r9
    pop     r8
    pop     rdx
    pop     rcx
    pop     rax
    popfq

    ; Saltar a la dirección de retorno original que el kernel dejó en R10
    jmp     r10
InstrumentationCallbackEntry ENDP
END
```

Script WinDbg: Auditoría de `InstrumentationCallback` en Todos los Procesos

Un analista forense o investigador de evasión puede inspeccionar qué procesos del sistema tienen activo un sensor `InstrumentationCallback`:

```
$$ Script WinDbg: Recorrer todos los procesos y detectar callbacks activos
!for_each_process "r @$t0 = @#Process; .if (poi(@$t0+@@(#FIELD_OFFSET(nt!_KPROCESS,
InstrumentationCallback))) != 0) { .printf \"PID %p (%ma) -> InstrumentationCallback: %p\n\",
poi(@$t0+@@(#FIELD_OFFSET(nt!_EPROCESS, UniqueProcessId))), @$t0+@@(#FIELD_OFFSET(nt!_EPROCESS,
ImageFileName)), poi(@$t0+@@(#FIELD_OFFSET(nt!_KPROCESS, InstrumentationCallback))) }"
```

E. Proveedor de Telemetría de Kernel `ETWti` (*Microsoft-Windows-Threat-Intelligence*)

El subsistema ETWti alimenta a los EDRs con telemetría de bajo nivel:

- Su handle de registro global reside en `nt!EtwThreatIntProvRegHandle`.
- Apunta a una estructura `_ETW_REG_ENTRY`. En el offset `+0x20` en x64 moderno (`+0x10` en arquitecturas de 32 bits o builds heredadas) reside el puntero `GuidEntry` hacia `_ETW_GUID_ENTRY`.
- Dentro de `_ETW_GUID_ENTRY`, el campo `ProviderEnableInfo` define si el proveedor está activo (`EnableMask ≠ 0`) y a qué nivel de detalle registra eventos de kernel.

3. Mecánicas BYOVD (Bring Your Own Vulnerable Driver) y Detección Defensiva

El vector **BYOVD** (*Bring Your Own Vulnerable Driver*) es la técnica predilecta para eludir DSE (*Driver Signature Enforcement*) sin necesidad de vulnerar `ntoskrnl.exe`.

Mecánica de Abuso de Drivers Legítimos Firmados

1. El atacante selecciona un driver legítimo antiguo firmado con un certificado comercial válido (utilidades de placa madre, herramientas de overclocking o versiones antiguas de antivirus).
2. El driver se carga en el sistema mediante la API legítima `NtLoadDriver` o a través del Service Control Manager (`CreateServiceW` / `StartServiceW`). Dado que la firma digital es válida y no ha sido explícitamente bloqueada por la lista de revocación de Windows (WDAC Blocklist), DSE permite su carga.
3. El proceso atacante abre un handle hacia el objeto dispositivo del driver y despacha códigos IOCTL vulnerables que exponen primitivas de lectura/escritura arbitraria de memoria física (`MmMapIoSpace`), manipulación de registros de control (desactivación del bit SMEP en `CR4`) o copia no validada de memoria (`RtlCopyMemory`).
4. Con estas primitivas, el atacante modifica en memoria viva las estructuras del kernel (desactiva callbacks de EDR en `PspCreateProcessNotifyRoutine`, neutraliza `ETWti` o altera el token de seguridad en `_EPROCESS`).

Detección Defensiva: La Tabla de Caché `nt!PiDDBCacheTable`

Para llevar un registro forense de todos los controladores cargados, el kernel NT mantiene en la sección paginada (`PAGE`) una tabla genérica basada en un árbol balanceado AVL (`_RTL_AVL_TABLE`), protegida por el recurso ejecutivo `nt!PiDDBLock` (`_ERESOURCE`): **`nt!PiDDBCacheTable`**.

Estructura Interna de `_RTL_AVL_TABLE` (x64, en Win10 22H2 Build 19045 y Win11 23H2/24H2 Builds 22631/26100):

Estructura Interna de `_RTL_AVL_TABLE` (x64):

```
dt nt!_RTL_AVL_TABLE
   +0x00 BalancedRoot          : _RTL_BALANCED_LINKS (Árbol balanceado AVL)
   +0x20 OrderedPointer        : Ptr64 Void
   +0x28 WhichOrderedElement   : Uint4
   +0x30 NumberGenericTableElements : Uint4 (Cantidad total de drivers cacheados)
   +0x38 DepthOfTree           : Uint4
   +0x40 RestartKey            : Ptr64 _RTL_BALANCED_LINKS
   +0x48 DeleteCount           : Uint4
   +0x50 CompareRoutine        : Ptr64
   +0x58 AllocateRoutine       : Ptr64
   +0x60 FreeRoutine           : Ptr64
   +0x68 TableContext          : Ptr64 Void
```

Estructura de Entrada `_PiDDBCacheEntry`:

Cada entrada dentro del árbol AVL contiene los metadatos de un controlador:

- **`+0x00` `List` (`_LIST_ENTRY`):** Enlace doble en la lista de entradas.

- `+0x10` `DriverName` (`_UNICODE_STRING`): Nombre del archivo del driver (ej. `RTCore64.sys` , `gdrv.sys`).
- `+0x20` `TimeDateStamp` (`ULONG`): Marca de tiempo de compilación del encabezado PE del driver.
- `+0x24` `LoadStatus` (`NTSTATUS`): Estado de carga del controlador (`STATUS_SUCCESS = 0x00000000`).

Técnicas de Evasión y Auditoría Forense:

- **Técnica de evasión:** Herramientas de carga clandestina intentan localizar la tabla `nt!PiDDBCacheTable` , adquirir `nt!PiDDBLock` mediante `ExAcquireResourceExclusiveLite` y desvincular el nodo del driver vulnerable llamando a `RtlDeleteElementGenericTableAvl` para no dejar rastro en la memoria del kernel.
- **Auditoría Forense en WinDbg:** Un analista de incidentes puede volcar la tabla para comprobar si un driver vulnerable fue cargado, incluso si el archivo ya fue eliminado del disco. Notá dos detalles clave: `nt!PiDDBCacheTable` es la instancia misma de `_RTL_AVL_TABLE` (no un puntero, por lo que no se usa `poi()` sobre el símbolo), y el tipo interno `PiDDBCacheEntry` no se incluye en los símbolos públicos de `ntoskrnl.pdb` ; cada nodo del árbol AVL empieza con una cabecera `_RTL_BALANCED_LINKS` de `0x20` bytes seguida inmediatamente por los datos de la entrada (`LIST_ENTRY` en `+0x20` , `UNICODE_STRING DriverName` en `+0x30` y `ULONG TimeDateStamp` en `+0x40` desde la base del nodo):

```
$$ 1. Localizar la estructura _RTL_AVL_TABLE y su ERESOURCE lock en WinDbg
x nt!PiDDBCacheTable
x nt!PiDDBLock
dt nt!_RTL_AVL_TABLE nt!PiDDBCacheTable

$$ 2. Obtener la raíz real del árbol AVL (BalancedRoot.RightChild en +0x10)
r $t0 = poi(nt!PiDDBCacheTable + 0x10)
$$ 3. Mostrar el _RTL_BALANCED_LINKS (+0x00) y el DriverName (+0x20 + 0x10 = +0x30) y TimeDateStamp
(+0x40)
dt nt!_RTL_BALANCED_LINKS @$t0
.printf "Driver: %msu | TimeDateStamp: 0x%08x\n", @$t0 + 0x30, dwo(@$t0 + 0x40)
```

---

4. Condiciones de Carrera y Reversing de MDLs en Drivers KMDF

En drivers reales el análisis de seguridad debe verificar el manejo de **MDLs (Memory Descriptor Lists)** y la sincronización asíncrona.

El Peligro del *Double Fetch* en `METHOD_IN_DIRECT` / `METHOD_OUT_DIRECT`

Cuando un driver utiliza E/S directa (`METHOD_IN_DIRECT` o `METHOD_OUT_DIRECT`):

1. El I/O Manager bloquea las páginas físicas del buffer de usuario en memoria mediante una estructura MDL (`Irp->MdlAddress`).
2. El driver mapea esas páginas en el espacio virtual del sistema llamando a `MmGetSystemAddressForMdlSafe(Mdl, NormalPagePriority)` .
3. **La trampa de seguridad:** La memoria virtual devuelta por `MmGetSystemAddressForMdlSafe` comparte exactamente los mismos marcos de página física (PFNs) que el buffer en modo usuario. Si el driver lee un parámetro de tamaño o un puntero interno desde ese buffer dos veces (por ejemplo, una vez para validar y otra para copiar), un hilo atacante en espacio de usuario puede modificar el valor concurrentemente en la memoria compartida entre ambas lecturas (**Double Fetch**), desbordando buffers internos del kernel.

Reenvío Asíncrono a I/O Targets en KMDF

En drivers KMDF desensamblados sin símbolos, las llamadas indirectas a través de la tabla `WdfFunctions` que gestionan solicitudes asíncronas se identifican por sus índices fijos:

- **`WdfRequestSend` (Índice 285 / Offset `+0x8E8`):** Reenvía un paquete de E/S a un dispositivo inferior o destino de E/S.
- **`WdfRequestSetCompletionRoutine` (Índice 290 / Offset `+0x910`):** Registra la rutina de finalización que se ejecutará cuando el dispositivo inferior complete la operación. En el desensamblado x64:

```
; Registro de CompletionRoutine en KMDF (índice 290, offset +0x910 en x64)
mov rax, cs:WdfFunctions_01015     ; RAX = Base de la tabla WDFFUNCTIONS
mov rcx, cs:WdfDriverGlobals       ; 1.er arg oculto (RCX): PWDF_DRIVER_GLOBALS
mov rdx, r14                       ; 2.° arg (RDX): WDFREQUEST Request
lea r8, MiCompletionRoutine        ; 3.er arg (R8): PFN_WDF_REQUEST_COMPLETION_ROUTINE
xor r9d, r9d                       ; 4.° arg (R9): WDFCONTEXT CompletionContext = NULL
call qword ptr [rax + 910h]        ; Invocación indirecta a pfnWdfRequestSetCompletionRoutine
```

6.7.b — Detección de DKOM mediante Cross-View (VTL0 vs. VTL1 y PspCidTable)

Los atacantes avanzados y herramientas de evasión basadas en **BYOVD** (*Bring Your Own Vulnerable Driver*) intentan ocultar procesos maliciosos manipulando directamente las estructuras de datos del kernel (**DKOM - Direct Kernel Object Manipulation**). La técnica clásica consiste en desvincular el proceso de la lista doblemente enlazada `nt!_EPROCESS→ActiveProcessLinks`:

```
[Proceso Legítimo A] ⟷ [Proceso Oculto X] ⟷ [Proceso Legítimo B]
                                  |
             (Atacante reasigna Flink y Blink con BYOVD)
                                  ▼
[Proceso Legítimo A] ⟷⟷⟷
```

- `PspCidTable` es la estructura que el kernel consulta cada vez que despacha una llamada o valida un handle de proceso. Si un atacante desvincula un proceso de `PspCidTable`, el sistema operativo entra inmediatamente en Bugcheck por inconsistencia de scheduler.
- **Regla Cross-View:** Si un PID existe en `PspCidTable` pero no aparece al recorrer `ActiveProcessLinks`, **el proceso ha sido ocultado mediante DKOM**.

2. **Cruce VTL0 vs. VTL1 (Secure Kernel):**

Bajo Virtualization-Based Security (VBS), el **Secure Kernel** (`securekernel.exe`) en VTL 1 mantiene su propio inventario de páginas físicas y contextos de ejecución respaldado por las tablas SLAT/EPT del hipervisor. Ningún driver en VTL 0 (incluso con privilegios de kernel absolutos) puede alterar la memoria de VTL 1. Los sensores defensivos de última generación consultan al hipervisor para comparar los contextos de ejecución activos contra la lista visible en VTL 0.

3. **Telemetría de Kernel `ETWti`:**

El proveedor `Microsoft-Windows-Threat-Intelligence` emite eventos inmutables de creación de procesos (`PROCESS_CREATE`) generados en la ruta nativa `nt!PspInsertProcess`. Aunque el atacante desvincule el proceso a posteriori en memoria, la traza de auditoría de ETWti ya registró la creación con su PID legítimo.

Script Conceptual de WinDbg para Detección Cross-View

El siguiente script en WinDbg compara los procesos enumerados por la lista circular contra los registros de la tabla de identificadores `PspCidTable`:

```
$$ ═══════════════════════════════════════════════════════════════════
$$ Detección Cross-View de Procesos Ocultos por DKOM en WinDbg
$$ ═══════════════════════════════════════════════════════════════════

$$ 1. Recorrer la lista clásica ActiveProcessLinks (Vista VTL 0)
!process 0 0

$$ 2. Inspeccionar la tabla global de PIDs (PspCidTable)
r $t0 = poi(nt!PspCidTable)
.printf "[+] PspCidTable ubicada en %p\n", @$t0

$$ 3. Volcar los objetos de proceso referenciados por sus handles en PspCidTable
!handle 0 3 0 Process

$$ 4. Algoritmo de detección heurística en WinDbg:
$$ Para cada handle válido en PspCidTable, desreferenciar el puntero EPROCESS:
$$ Si EPROCESS→ActiveProcessLinks.Flink == &EPROCESS→ActiveProcessLinks
$$ o si Flink→Blink ≠ EPROCESS, se delata una desvinculación maliciosa (DKOM).
!for_each_process "r @$t1 = @#Process; .if (poi(@$t1 + @@(#FIELD_OFFSET(nt!_EPROCESS,
ActiveProcessLinks.Flink))) == (@$t1 + @@(#FIELD_OFFSET(nt!_EPROCESS, ActiveProcessLinks)))) { .printf
\"[ALERTA DKOM] Proceso oculto detectado: PID %p (%ma) en %p\n\", poi(@$t1 +
@@(#FIELD_OFFSET(nt!_EPROCESS, UniqueProcessId))), @$t1 + @@(#FIELD_OFFSET(nt!_EPROCESS,
ImageFileName)), @$t1 }"
```

5. Laboratorio Verificado

1. Inspeccioná la plantilla `plantillas_codigo/minifilter_obcallback_min.c`. Observá cómo interactúan las estructuras de `FLT_REGISTRATION` y `OB_CALLBACK_REGISTRATION`.

2. En una sesión de depuración con WinDbg KDNET en tu VM de kernel, ejecutá el script de auditoría completa:

```
$$><plantillas_codigo\auditar_callbacks.wdbg
```

3. Verificá que la lista de callbacks enumera los drivers de tu entorno y localiza el módulo exacto que atiende la creación de procesos en tu sistema.

Windows Filtering Platform (WFP): capas ALE, callouts y auditoría de red en kernel

> **Alcance de lectura y auditoría en VM descartable**
>
> No construyas ni instales un callout driver para interceptar, redirigir o bloquear tráfico de terceros. Esta lección enseña a reconocer las llamadas y estructuras de WFP al analizar un driver en IDA/Ghidra y a inventariar el estado de filtrado en una VM en modo lectura.

> **Propósito**
>
> Entender cómo el subsistema de red de Windows expone puntos de inspección en modo kernel a través de WFP, distinguir las capas de autorización de aplicación (ALE) de las capas de paquetes, reconstruir en estático qué registra un callout driver (`FwpsCalloutRegister` y su `classifyFn`) y auditar los filtros activos con `netsh wfp show state`.

Prerrequisitos

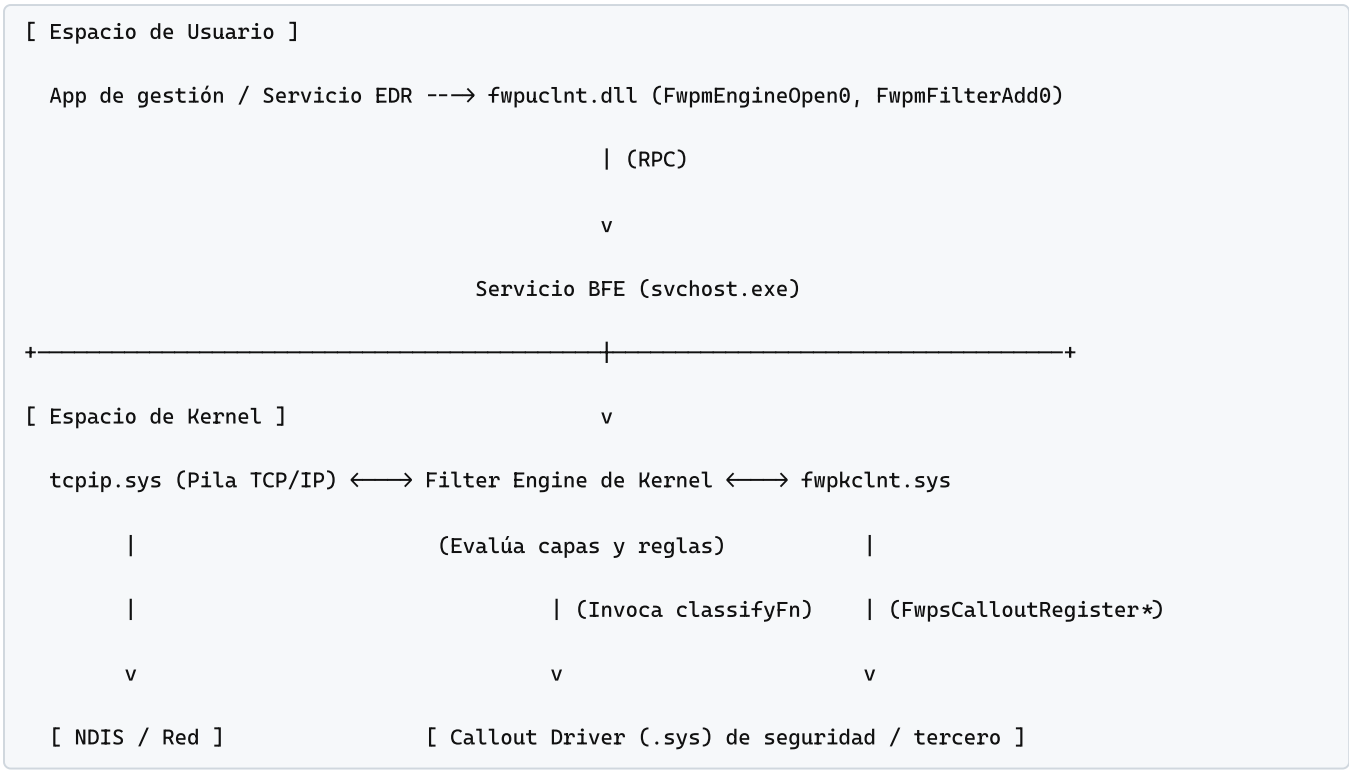
- *Obligatorios:* arquitectura del kernel e IRQL (6.1); revisión defensiva de drivers (6.4); minifilters (6.5).
- *Recomendados:* debugging de kernel con WinDbg (6.3); IPC y RPC (2D.10).

1. Arquitectura de WFP: BFE, motor de kernel y callouts

Así como el Filter Manager (`FltMgr.sys` , estudiado en 6.5) evita que los filtros de archivos se adhieran manualmente a la pila de volúmenes, **Windows Filtering Platform (WFP)** provee la infraestructura oficial para inspeccionar, autorizar o modificar el tráfico de red dentro de `tcpip.sys` sin recurrir a filtros NDIS intermedios ni a hooks sobre dispositivos `\Device\Tcp` .

WFP divide sus responsabilidades entre el modo usuario y el modo kernel:

- **Base Filtering Engine (BFE, servicio `BFE` en `svchost.exe`):** Componente de modo usuario que administra las políticas, proveedores, subcapas y reglas de filtrado, exponiendo las APIs de gestión `Fwpm*` (`fwpuclnt.dll`) sobre RPC.
- **Filter Engine en Modo Kernel (integrado en `tcpip.sys` y expuesto vía `fwpkclnt.sys`):** Evalúa los paquetes y eventos de conexión en cada capa de la pila de red a medida que ocurren, y despacha la ejecución hacia los módulos de llamada (*callouts*) registrados por drivers de terceros.
- **Callout Driver (`.sys`):** Un driver de kernel (WDM o KMDF) que importa `fwpkclnt.sys` , registra uno o más objetos callout identificados por un `GUID` mediante `FwpsCalloutRegister*` (API de ejecución `Fwps*`) y vincula esos GUIDs a reglas de filtrado en las capas del motor (API de gestión `Fwpm*`).



2. Capas de filtrado: ALE frente a transporte y paquetes

El motor de kernel de WFP está dividido en decenas de capas (*Filtering Layers*) identificadas por un identificador en tiempo de ejecución (`layerId`) y un `GUID` . Para el análisis de un driver importan sobre todo dos familias:

Familia y Capa (Constante `FWPM_LAYER_*`)	Cuándo se dispara	Qué metadatos recibe el driver	Uso defensivo típico
ALE Saliente: `FWPM_LAYER_ALE_AUTH_CONNECT_V4` / `_V6`	Cuando un proceso intenta establecer una conexión TCP saliente (antes del `SYN`) o enviar el primer datagrama UDP.	**Contexto de proceso:** PID (`processId`), ruta completa de la imagen (`processPath`), token/SID del usuario y tupla IP/puerto local y remoto.	Firewalls basados en host y sensores EDR que autorizan o bloquean conexiones por proceso.
ALE Entrante: `FWPM_LAYER_ALE_AUTH_RECV_ACCEPT_V4` / `_V6`	Cuando llega una solicitud de conexión TCP entrante o un datagrama UDP hacia un socket local en escucha.	PID del proceso servidor local, ruta del binario, SID e IP/puerto del emisor remoto.	Control de acceso a puertos abiertos por servicios locales.
Transporte: `FWPM_LAYER_INBOUND_TRANSPORT_V4` / `OUTBOUND_TRANSPORT_V4`	Por cada paquete TCP/UDP en tránsito en la capa de transporte, tras el reensamblado de fragmentos IP.	Estructura `NET_BUFFER_LIST` (NBL) con los bytes del paquete; **sin** contexto directo del hilo de usuario (puede correr a `DISPATCH_LEVEL`).	Inspección profunda de paquetes (DPI), contadores de tráfico o inyección/modificación de cabeceras.
Datos de Datagrama / Stream: `FWPM_LAYER_DATAGRAM_DATA_V4` / `FWPM_LAYER_STREAM_V4`	Al procesar cargas útiles UDP o flujos de bytes TCP ya reensamblados.	Buffers de flujo (`FWPS_STREAM_CALLOUT_IO_PACKET0`) o listas NBL.	Inspección de protocolos de aplicación y filtrado de contenido.

Consecuencia para revisar el código de un driver (IRQL y memoria): Las llamadas a un callout pueden ocurrir desde `PASSIVE_LEVEL` hasta `DISPATCH_LEVEL` (según la capa y la ruta de red). Por lo tanto, las rutinas `classifyFn` y `flowDeleteFn` deben respetar las reglas de `DISPATCH_LEVEL` estudiadas en 6.1: no esperar con intervalo no nulo ni tocar memoria paginada.

3. Anatomía binaria de `FWPS_CALLOUT` y la rutina `classifyFn`

Al abrir en IDA Pro o Ghidra un driver que importa `fwpkclnt.sys`, el punto de partida para encontrar su lógica de red es buscar las referencias cruzadas a `fwpkclnt!FwpsCalloutRegister0` / `FwpsCalloutRegister1` / `FwpsCalloutRegister2` / `FwpsCalloutRegister3`:

```
NTSTATUS FwpsCalloutRegister3(

    void                *deviceObject, // RCX (x64): PDEVICE_OBJECT asociado

    const FWPS_CALLOUT3 *callout,      // RDX (x64): Puntero a la estructura con el GUID y los 3
callbacks

    UINT32              *calloutId     // R8  (x64): Puntero de salida donde recibe el ID numérico

);
```

El segundo argumento (`RDX` en x64) apunta a la estructura `FWPS_CALLOUT` (48 bytes en x64), cuyo layout binario permite localizar de inmediato las tres funciones del driver sin necesidad de símbolos PDB:

Offset (x64)	Campo en `FWPS_CALLOUT` (v0..v3)	Tipo y propósito en el desensamblado
`+0x00`	`calloutKey`	`GUID` (16 bytes): Identificador único que enlaza este callout de código con el filtro administrativo registrado en BFE (`FwpmFilterAdd0`).
`+0x10`	`flags`	`UINT32` (4 bytes, más 4 bytes de padding de alineación en `+0x14`).
`+0x18`	`classifyFn`	`FWPS_CALLOUT_CLASSIFY_FN*` (8 bytes): **Puntero a la rutina principal de inspección** invocada cada vez que el tráfico coincide con las condiciones del filtro.
`+0x20`	`notifyFn`	`FWPS_CALLOUT_NOTIFY_FN*` (8 bytes): Puntero al callback notificado cuando se agrega o elimina un filtro asociado al callout.
`+0x28`	`flowDeleteFn`	`FWPS_CALLOUT_FLOW_DELETE_FN*` (8 bytes, opcional): Puntero invocado cuando termina un flujo al que el driver asoció contexto con `FwpsFlowAssociateContext0` .

Lectura de la decisión en `classifyFn`

En `FWPS_CALLOUT_CLASSIFY_FN1` / `2` / `3` en x64, la rutina recibe siete argumentos: `inFixedValues` (`RCX`), `inMetaValues` (`RDX`), `layerData` (`R8` , que en capas de paquetes apunta al `NET_BUFFER_LIST`), `classifyContext` (`R9`), `filter` (`[rsp+28h]`), `flowContext` (`[rsp+30h]`) y `classifyOut` (`[rsp+38h]` , puntero a `FWPS_CLASSIFY_OUT0`). La decisión final se escribe en el primer campo de `classifyOut` (`actionType` en offset `+0x00`):

- **`FWP_ACTION_BLOCK` (`0x00001001`):** Bloquea la conexión o descarta el paquete. Si además limpia el bit `FWPS_RIGHT_ACTION_WRITE` (`classifyOut→rights &= ~1`), impide que otro filtro de menor peso en la misma subcapa revierta el bloqueo (*hard block*).
- **`FWP_ACTION_PERMIT` (`0x00001002`):** Autoriza la operación en esa subcapa.
- **`FWP_ACTION_CONTINUE` (`0x00002003`):** No toma decisión y pasa la evaluación al siguiente filtro.
- **Bandera `FWPS_CLASSIFY_OUT_FLAG_ABSORB` (`0x00000001` en `classifyOut→flags`):** Indica que el callout retiene o clona el paquete (con `FwpsAllocateCloneNetBufferList0`) para reinjectarlo más tarde (con `FwpsInjectTransportSendAsync0` / `ReceiveAsync0`) o descartarlo silenciosamente.

4. Subcapas, pesos y arbitraje entre múltiples filtros

En una máquina real conviven el Firewall de Windows, un agente EDR y quizás un cliente VPN, todos en la misma capa ALE. WFP ordena la evaluación mediante **subcapas (*Sublayers*)** y **pesos (*Weights*)**:

1. Las subcapas se evalúan en orden de peso de mayor a menor; dentro de una subcapa, se evalúa primero el filtro de mayor peso.
2. La regla general de arbitraje de WFP es que **un bloqueo duro (`FWP_ACTION_BLOCK` que retira el derecho `FWPS_RIGHT_ACTION_WRITE`) prevalece sobre un permiso** de una subcapa posterior (*block overrides permit*), salvo excepciones explícitas por política de veto.

5. Auditoría defensiva en solo lectura: `netsh wfp` y WinDbg

Para inventariar qué filtros, proveedores y callouts están activos en una VM sin modificar el estado del sistema:

```
:: 1. En consola elevada de la VM: exportar el estado completo de BFE/WFP a XML

netsh wfp show state file=C:\Lab\wfpstate.xml



:: 2. Exportar únicamente la lista de filtros activos y sus IDs

netsh wfp show filters file=C:\Lab\wfpfilters.xml
```

En el archivo XML generado podés buscar los nodos `<callout>` y `<filter>`: cada uno registra su `displayData/name`, su `calloutKey` (GUID), la capa (`layerKey`) y el proveedor (`providerKey`). Si estás conectado con WinDbg al kernel de la VM (ver 6.3), podés correlacionar los punteros a `classifyFn` con el módulo `.sys` al que pertenecenresolviendo el símbolo más cercano con `ln <dir_classifyFn>`.

Ejemplo trabajado

(Ejemplo ilustrativo de revisión estática de un driver didáctico propio `BlocNetMon.sys`; los valores y direcciones son esquemáticos y no provienen de una corrida real.)

```
; Fragmento en BlocNetMon!RegistrarCalloutAle (x64)

lea     rax, BlocNetMonClassifyAleConnect

mov     [rsp+48h], rax                  ; callout.classifyFn   (+0x18 de FWPS_CALLOUT3)

lea     rax, BlocNetMonNotifyAle

mov     [rsp+50h], rax                  ; callout.notifyFn     (+0x20 de FWPS_CALLOUT3)

mov     qword ptr [rsp+58h], 0          ; callout.flowDeleteFn (+0x28 = NULL)

movups  xmm0, cs:GUID_BLOCLAB_ALE_V4

movups  [rsp+30h], xmm0                 ; callout.calloutKey   (+0x00)

lea     r8, g_CalloutId                 ; R8  = &calloutId

lea     rdx, [rsp+30h]                  ; RDX = &callout

mov     rcx, rbx                        ; RCX = g_DeviceObject

call    cs:__imp_FwpsCalloutRegister3
```

Al seguir `BlocNetMonClassifyAleConnect`, se observa que lee `inMetaValues→processId` y la IP/puerto remotos de `inFixedValues`, los registra en un buffer circular interno y escribe `0x1002` (`FWP_ACTION_PERMIT`) en `[r14]` (`classifyOut→actionType`) en todas las rutas. Conclusión acotada del análisis estático: el callout es puramente de observación sobre conexiones salientes IPv4 y nunca bloquea ni clona paquetes.

Práctica

En tu VM aislada, en modo de solo lectura:

1. Ejecutá `netsh wfp show state file=C:\Lab\wfpstate.xml` en una consola elevada y abrí el XML resultante.
2. Identificá dos proveedores registrados (`<provider>`) y dos callouts asociados a la capa `FWPM_LAYER_ALE_AUTH_CONNECT_V4` . Anotá su nombre descriptivo y su GUID.
3. En una muestra oficial de callout driver del WDK de Microsoft (por ejemplo, *Windows Filtering Platform Sample / Inspect*), ubicá estáticamente la llamada a `FwpsCalloutRegister*` y completá una tabla con el GUID, la dirección/nombre de `classifyFn` , `notifyFn` y `flowDeleteFn` , la capa a la que se asocia con `FwpmFilterAdd0` y qué valores de `actionType` puede escribir.
4. Verificá en el código de `classifyFn` si la rutina puede ejecutarse a `DISPATCH_LEVEL` y si realiza alguna operación prohibida en ese nivel.
5. Eliminá el archivo `wfpstate.xml` temporal al terminar.

Entregable

Tabla de callouts analizados (GUID, capa, punteros de `FWPS_CALLOUT` , acciones posibles en `classifyOut->actionType` , requisitos de IRQL) y extracto anonimizado de dos entradas de `wfpstate.xml` explicando qué componente del sistema las registró.

Repaso

▼ ¿Qué diferencia hay entre las funciones con prefijo `Fwpm*` y las que tienen prefijo `Fwps*` ?

> Las funciones `Fwpm*` (*management*) administran objetos de política en el Base Filtering Engine (proveedores, subcapas, filtros); las funciones `Fwps*` (*system/callout*, exportadas por `fwpkclnt.sys`) registran las funciones de callback del driver en el kernel e interactúan con los paquetes y flujos en tiempo real.

▼ ¿Por qué un sensor EDR prefiere las capas `FWPM_LAYER_ALE_AUTH_CONNECT_*` frente a una capa de transporte para auditar qué programa inicia una conexión?

> Porque las capas ALE se evalúan en el momento de autorizar el socket/conexión y entregan en `inMetaValues` el PID, la ruta del ejecutable y el token del proceso, datos que ya no están presentes en cada paquete individual de la capa de transporte.

▼ En un binario x64 sin símbolos que llama a `FwpsCalloutRegister3` , ¿en qué argumento y offsets encontrás las funciones del callout?

> En la estructura apuntada por el segundo argumento (`RDX`): `+0x00` contiene los 16 bytes del `calloutKey` (GUID), `+0x18` el puntero a `classifyFn` , `+0x20` el puntero a `notifyFn` y `+0x28` el puntero a `flowDeleteFn` .

▼ **¿Qué diferencia un bloqueo que conserva `FWPS_RIGHT_ACTION_WRITE` de uno que limpia ese bit?**

> Si `classifyOut→rights` conserva el bit `FWPS_RIGHT_ACTION_WRITE`, es un bloqueo blando que un filtro posterior de menor prioridad podría sobrescribir; al limpiar ese bit, el bloqueo es definitivo (*hard block*) y prevalece sobre permisos posteriores.

▼ **¿Por qué `classifyFn` no debe acceder a memoria paginada ni esperar un mutex estándar en capas de paquetes?**

> Porque el motor de filtrado de kernel puede invocar `classifyFn` a `DISPATCH_LEVEL`, donde un fallo de página o una espera bloqueante provocan un bugcheck inmediato (`0xD1` o `0x0A`).

Puerta optativa

Podés explicar la relación entre BFE, `tcpip.sys`, `fwpkclnt.sys` y un callout driver; distinguir capas ALE de capas de transporte; localizar `classifyFn`, `notifyFn` y `flowDeleteFn` desde la estructura pasada a `FwpsCalloutRegister*`; interpretar las acciones de `FWPS_CLASSIFY_OUT0` y auditar el estado de WFP con `netsh wfp show state`.

Lo que queda fuera

La modificación e inyección de paquetes (clonado y reinyección asíncrona de `NET_BUFFER_LIST`), los callouts de IPsec, la redirección de conexiones (`ALE_CONNECT_REDIRECT`) y el filtrado en el switch virtual de Hyper-V no se desarrollan en detalle.

Fuentes

- Microsoft Learn, arquitectura y capas: Windows Filtering Platform Architecture Overview, ALE Layers, Filter Arbitration.
- Microsoft Learn, DDI de callouts: Callout Driver Operations, FwpsCalloutRegister3, FWPS_CALLOUT3 structure, classifyFn3 callout function.
- Microsoft Learn, diagnóstico: Auditing and Diagnostic Tools for WFP (netsh wfp).
- **Fundamentos previos:** IRQL y pools en 6.1; modelo de minifilters de filesystem en 6.5.

Etiquetas

Núcleo conceptual: modelo de filtrado por capas comparado con 6.5. **Conocimiento externo:** arquitectura WFP, BFE, `fwpkclnt.sys`, capas ALE/transporte, layout de `FWPS_CALLOUT`, arbitraje de pesos y `netsh wfp`. **Recomendación editorial:** pauta de revisión estática de `FwpsCalloutRegister*` en IDA/Ghidra. **Ejemplo ilustrativo:** fragmento de registro de `BlocNetMon.sys`. **Laboratorio verificado:** no; requiere VM/toolchain (especificado, pendiente de corrida).

kCFG a nivel ISA: `_guard_dispatch_icall`, bitmap de CFG y contraste con CET/IBT

> **Análisis defensivo y lectura de código compilado**
>
> Esta lección estudia la instrumentación de llamadas indirectas y las protecciones de flujo de control para leer binarios de kernel en IDA/Ghidra y diagnosticar fallos en WinDbg. No se enseña a eludir CFG, kCFG ni CET.

> **Propósito**
>
> Comprender exactamente qué instrucciones emite el compilador en un driver compilado con `/guard:cf`, cómo valida `_guard_dispatch_icall` una dirección destino en el bitmap de CFG, qué metadatos guarda el directorio `Load Config` del PE y en qué se diferencia este mecanismo por software/VTL1 de las protecciones de hardware CET (Shadow Stack e Indirect Branch Tracking con `endbr64`).

Prerrequisitos

- *Obligatorios:* PE fino y Load Config (2.14); mitigaciones de vulnerabilidades (5.2); mitigaciones de drivers y VBS/HVCI (6.6, 6.7).
- *Recomendados:* llamadas y registros x64 (2.5); debugging de kernel (6.3).

1. Qué cambia en el ensamblador: de `call reg` a `_guard_dispatch_icall`

En un binario x64 compilado sin Control Flow Guard (CFG), una llamada indirecta a través de un puntero de función o de una tabla (como una `MajorFunction` de WDM o un callback de KMDF) se emite como `call rax` (2 bytes: `FF D0`). Cuando el driver se compila con `/guard:cf` (requisito obligatorio para compatibilidad con HVCI), el compilador MSVC reemplaza cada llamada indirecta por una secuencia que carga la dirección destino en `RAX` e invoca al despachador guardado en la sección de importación/configuración del módulo:

```
; --- Sin /guard:cf ---

mov     rax, [rbx + 28h]                    ; RAX = puntero al callback (p. ej. EvtIoDeviceControl)

mov     rcx, rdi                            ; 1.er argumento

call    rax                                 ; Salto indirecto sin validación previa



; --- Con /guard:cf (CFG / kCFG activo) ---

mov     rax, [rbx + 28h]                    ; RAX = dirección destino a verificar e invocar

mov     rcx, rdi                            ; RCX, RDX, R8, R9 llevan los argumentos normales

call    cs:__guard_dispatch_icall_fptr      ; Valida RAX en el bitmap de CFG y, si es válido, hace jmp
rax
```

En el directorio `IMAGE_LOAD_CONFIG_DIRECTORY64` del archivo PE (ver 2.14), dos punteros gobiernan esta instrumentación:

- **`GuardCFCheckFunctionPointer` (offset `+0x070` en Load Config x64):** Puntero a la celda que contiene la rutina de comprobación (`_guard_check_icall`, que valida el destino en `RCX` y retorna).
- **`GuardCFDispatchFunctionPointer` (offset `+0x078` en Load Config x64):** Puntero a la celda que contiene el despachador (`_guard_dispatch_icall`, que recibe el destino en `RAX`, preserva los registros de argumentos `RCX`, `RDX`, `R8` y `R9`, valida `RAX` y salta directamente al destino con `jmp rax`).

Si el sistema operativo o el entorno no soporta CFG, esos punteros en memoria apuntan a un stub trivial (`_guard_dispatch_icall_nop`) que ejecuta únicamente `jmp rax`. Cuando el cargador del kernel mapea el driver con kCFG activo, parchea ambas celdas para que apunten a `nt!_guard_check_icall` y `nt!_guard_dispatch_icall`.

2. Mecánica a nivel ISA del bitmap de CFG

Para validar en pocos ciclos de reloj si `RAX` es el inicio legítimo de una función exportada o tomada por dirección en tiempo de compilación, Windows mantiene un **bitmap jerárquico de CFG** (cuya base en kernel se almacena en `nt!MmCfgBitmapAddress` para el espacio de direcciones correspondiente):

1. **Alineación de funciones a 16 bytes:** El compilador y el enlazador alinean los puntos de entrada válidos de funciones llamadas indirectamente a múltiplos de 16 bytes (los 4 bits inferiores de la dirección valen `0`: `RAX & 0xF == 0`).
2. **Indexación del bitmap:** Cada bit del bitmap representa un bloque de memoria de código. La rutina `nt!_guard_dispatch_icall` toma los bits superiores de `RAX` (`RAX >> 9`) para indexar la entrada del directorio del bitmap y los bits intermedios (`(RAX >> 3) & 0x3F`) para seleccionar el bit exacto de 0 a 63 mediante la instrucción `bt` **(Bit Test)**.
3. **Caso alineado (camino rápido):** Si `RAX` está alineado a 16 bytes (`test al, 0Fh` da cero) y el bit correspondiente en el bitmap vale `1`, el destino es válido y `_guard_dispatch_icall` ejecuta `jmp rax`.
4. **Caso desalineado o bit en 0:** Si la dirección no está alineada a 16 bytes, el despachador comprueba si la política permite destinos válidos no alineados verificando el par de bits adyacente (`btr` / `bts`). Si el bit vale `0`, el destino no figura en la tabla `GuardCFFunctionTable` de ningún módulo cargado y la validación falla.

```
; Esquema conceptual ilustrativo de nt!_guard_dispatch_icall (x64)

; Entrada: RAX = dirección destino; RCX, RDX, R8, R9 = argumentos de la función destino

mov     r10, cs:MmCfgBitmapAddress            ; Base de la tabla del bitmap de CFG

mov     r11, rax

shr     r11, 9                                ; Índice de entrada en la tabla del bitmap

mov     r10, [r10 + r11*8]                    ; Puntero al QWORD de 64 bits del bitmap

mov     r11, rax

shr     r11, 3                                ; Desplazamiento de bit (granularidad de 8/16 bytes)

test    al, 0Fh                               ; ¿Está RAX alineado a 16 bytes?

jnz     short comprobar_no_alineado

bt      r10, r11                              ; Comprueba el bit (CF = bit del bitmap)

jnc     short fallo_cfg                       ; Si CF == 0, el destino NO es un inicio de función válido

jmp     rax                                   ; Destino válido → transfiere el control


comprobar_no_alineado:

btr     r11, 0                                ; Ajusta al bit par y verifica par de bits

bt      r10, r11

jnc     short fallo_cfg

or      r11, 1

bt      r10, r11

jnc     short fallo_cfg

jmp     rax


fallo_cfg:

mov     ecx, 0Ah                              ; RCX = 0x0A (FAST_FAIL_GUARD_ICALL_CHECK_FAILURE)

int     29h                                   ; Dispara __fastfail → Bugcheck 0x139 en kernel
```

Por qué kCFG depende de VBS/HVCI (VTL1)

En un sistema sin VBS, el bitmap de CFG reside en memoria del kernel en VTL0; si un atacante ya cuenta con una primitiva de escritura arbitraria en Ring 0, podría poner en `1` el bit correspondiente a su gadget antes de provocar la llamada indirecta. Cuando **HVCI** está activo (6.6 y 6.7), las páginas físicas que alojan el bitmap de kCFG y las celdas `GuardCFDispatchFunctionPointer` están protegidas en las tablas SLAT (EPT/NPT) como **solo**

lectura para VTL0: únicamente el Secure Kernel en VTL1 (`securekernel.exe` / `skci.dll`) puede poblar los bits del bitmap cuando valida y mapea un driver firmado.

3. Contraste arquitectónico: CFG/kCFG frente a CET (Shadow Stack e IBT)

Un error frecuente al analizar mitigaciones es confundir CFG/kCFG con **Control-flow Enforcement Technology (CET)** de Intel y AMD. Ambas protegen el flujo de control, pero operan en bordes y capas distintas:

Propiedad	CFG / kCFG (Software + VTL1)	CET Shadow Stack (Hardware)	CET IBT — Indirect Branch Tracking (Hardware)
Qué borde protege	**Forward-edge** (llamadas y saltos indirectos hacia adelante: `call reg` , `jmp reg`).	**Backward-edge** (direcciones de retorno hacia atrás: `ret`).	**Forward-edge** (saltos y llamadas indirectas hacia adelante).
Cómo valida el destino	La rutina `_guard_dispatch_icall` consulta un bitmap en memoria antes de saltar.	La CPU empuja la dirección de retorno en el stack de datos y en un *Shadow Stack* protegido por tabla de páginas; en `ret` compara ambos en microcódigo.	Tras un `call` / `jmp` indirecto, la máquina de estados de la CPU entra en `WAIT_FOR_ENDBRANCH` y exige que la primera instrucción del destino sea `endbr64` (`F3 0F 1E FA`).
Qué se ve en IDA / Ghidra	`call cs:__guard_dispatch_icall_fptr` y tabla `GuardCFFunctionTable` en Load Config.	El código de la función no cambia; se lee en las banderas de mitigación del proceso/kernel.	Un prólogo de 4 bytes `endbr64` (`F3 0F 1E FA`) al comienzo de cada función direccionable indirectamente.
Qué excepción o código genera al fallar	Ejecuta `int 29h` con `ECX = 0x0A` (`FAST_FAIL_GUARD_ICALL_CHECK_FAILURE`): en user-mode termina con `0xC0000409` **(`STATUS_STACK_BUFFER_OVERRUN`)** ; en kernel produce el bugcheck `0x139` **(`KERNEL_SECURITY_CHECK_FAILURE` , Arg1 = `0x0A`)**.	La CPU dispara la excepción de hardware `#CP` **(vector 21)** con código de error en pila: su `NTSTATUS` específico es `0xC00006A6` **(`STATUS_CONTROL_PROTECTION_EXCEPTION`)** (o terminación vía fastfail `0xC0000409` con subcódigo `0x2A` / `0x39`).	La CPU dispara la excepción de hardware `#CP` **(vector 21)** porque el destino indirecto no comenzó con `endbr64` .

En procesadores antiguos que no implementan CET IBT, los 4 bytes de `endbr64` (`F3 0F 1E FA`) pertenecen al espacio de NOPs multibyte reservados por Intel y AMD, por lo que se ejecutan como un simple `NOP` sin afectar la compatibilidad.

4. Inspección estática en PE y dinámica en WinDbg

Para verificar la tabla de destinos válidos de un driver propio o de sistema sin ejecutarlo, usá `dumpbin` del Visual Studio Developer Command Prompt o el visor de Load Config en PE-bear/IDA:

```
:: Volcar el directorio Load Config y la tabla completa de funciones válidas para CFG/kCFG

dumpbin /headers /loadconfig MiDriver.sys
```

En la salida de `dumpbin /loadconfig` verás `Guard CF address of check-function pointer`, `Guard CF address of dispatch-function pointer`, las banderas `Guard Flags` (por ejemplo, `0x00010500` = `CF Instrumented` + `CF Function Table Present`) y la lista numerada de RVAs bajo `Guard CF Function Table`. En una sesión de WinDbg de kernel (en tu VM de pruebas):

```
$$ Ver el desensamblado real de _guard_dispatch_icall en el build de tu VM

kd> uf nt!_guard_dispatch_icall



$$ Ver la cabecera y configuración de Load Config de un driver cargado

kd> !dh -l MiDriver
```

Ejemplo trabajado

(Escenario ilustrativo de lectura de un bugcheck causado por un puntero de callback corrupto en un driver de prueba propio; valores construidos.)

```
BugCheck 139, {a, ffff89812345f810, ffff89812345f768, 0}

KERNEL_SECURITY_CHECK_FAILURE (139)

Arg1: 000000000000000a, FAST_FAIL_GUARD_ICALL_CHECK_FAILURE

Arg2: ffff89812345f810, Dirección del trap frame (_KTRAP_FRAME)

Arg3: ffff89812345f768, Dirección del registro de excepción (_EXCEPTION_RECORD)
```

Recorrido de análisis siguiendo el método de 6.3:

1. **Parámetro 1 (`0x0A`):** Confirma que el bugcheck `0x139` provino de un fallo de validación de llamada indirecta en kCFG (`FAST_FAIL_GUARD_ICALL_CHECK_FAILURE`).
2. **Contexto (`.trap ffff89812345f810`):** `RIP` apunta a `nt!_guard_dispatch_icall+0x …` (la instrucción `int 29h`), y el registro `RAX` conserva la dirección destino inválida que se intentó llamar.
3. **Pila (`k`):** El frame inmediatamente anterior a `nt!_guard_dispatch_icall` señala la línea exacta de tu driver donde se ejecutó `call cs:__guard_dispatch_icall_fptr`, indicando qué estructura tenía el puntero de función corrompido o sin inicializar.

Práctica

En tu VM aislada, con el driver propio `BlocLab.sys` de 6.2 (o un binario propio compilado en C):

1. Compilá dos versiones del mismo binario propio: una sin CFG (`/guard:cf-`) y otra con CFG activo (`/guard:cf`).
2. Abrí ambas en IDA o Ghidra y compará el sitio de una llamada indirecta por puntero a función: anotá los bytes de la instrucción en cada versión.
3. Ejecutá `dumpbin /loadconfig` sobre la versión con `/guard:cf` y verificá que tu función llamada indirectamente aparece listada en `Guard CF Function Table`.

4. Si tenés WinDbg conectado al kernel de la VM, ejecutá `uf nt!_guard_dispatch_icall` y anotá en tu bitácora qué instrucciones realizan el desplazamiento de bits y la prueba `bt` en ese build concreto de Windows.

Entregable

Comparativa lado a lado del desensamblado con y sin `/guard:cf`, salida de `dumpbin /loadconfig` señalando la RVA de la función en `GuardCFFunctionTable` y cuadro comparativo entre kCFG, CET Shadow Stack y CET IBT con sus códigos de excepción.

Repaso

▼ **En un driver x64 compilado con `/guard:cf`, ¿en qué registro viaja la dirección destino cuando se llama a `__guard_dispatch_icall_fptr`?**

> En `RAX`, de modo que los cuatro registros de argumentos de la convención Microsoft x64 (`RCX`, `RDX`, `R8` y `R9`) lleguen intactos a la función destino tras el `jmp rax`.

▼ **¿Por qué kCFG necesita que VBS/HVCI esté activo para ser resistente frente a una escritura arbitraria en kernel?**

> Porque sin VBS el bitmap de CFG es escribible desde Ring 0 (VTL0). Con HVCI, las páginas del bitmap están marcadas como solo lectura en las tablas SLAT (EPT/NPT) y solo el Secure Kernel en VTL1 puede modificarlas.

▼ **¿Qué diferencia hay entre lo que protege kCFG y lo que protege el Shadow Stack de CET?**

> kCFG protege el borde hacia adelante (*forward-edge*: llamadas y saltos indirectos `call/jmp reg`) comprobando un bitmap; el Shadow Stack de CET protege el borde hacia atrás (*backward-edge*: instrucciones `ret`) comparando la dirección de retorno del stack de datos contra una pila sombra en hardware.

▼ **¿Qué función cumple la instrucción `endbr64` (`F3 0F 1E FA`)?**

> Es el marcador de aterrizaje de CET IBT (*Indirect Branch Tracking*): tras un salto o llamada indirecta, el procesador exige que la primera instrucción ejecutada en el destino sea `endbr64`; si no lo es, dispara una excepción de hardware `#CP`.

▼ **¿Qué diferencia hay entre los códigos `0xC0000409` y `0xC00006A6`?**

> `0xC0000409` es `STATUS_STACK_BUFFER_OVERRUN` (disparado por `int 29h` / `__fastfail` con un subcódigo en `RCX`, como `0x0A` en fallos de CFG/kCFG, que en kernel produce el bugcheck `0x139`). `0xC00006A6` es `STATUS_CONTROL_PROTECTION_EXCEPTION`, el código `NTSTATUS` específico de la excepción de hardware `#CP` (vector 21 de CET).

Puerta optativa

Podés leer una llamada indirecta instrumentada con `_guard_dispatch_icall`, explicar cómo se indexa el bitmap de CFG con `shr` y `bt`, extraer la tabla `GuardCFFunctionTable` del directorio Load Config, diagnosticar un bugcheck `0x139` (subcódigo `0x0A`) leyendo `RAX` en el trap frame y distinguir CFG/kCFG de CET Shadow Stack e IBT (`endbr64`).

Lo que queda fuera

El formateo de técnicas avanzadas de evasión de CFG en drivers comerciales, la manipulación de páginas de solo lectura respaldadas por HVCI sin llamadas VTL 1 documentadas y la implementación de analizadores binarios para tablas XFG se dejan fuera de alcance (requieren entornos de laboratorio y credenciales de prueba específicos).

6-10 · Fuzzing de Drivers en Modo Kernel: Superficie IOCTL, Snapshot Fuzzing y Cobertura por Hardware

Nivel: Avanzado (Ring 0 / Hypervisor-Assisted) | **Arquitectura:** Windows 10 / 11 x64 | **Laboratorio verificado:** no; requiere VM/toolchain

1. Anatomía de la Superficie de Ataque en Drivers WDM y KMDF

La inmensa mayoría de vulnerabilidades alcanzables localmente en drivers de terceros (antivirus, utilidades de overclocking, anticheats y periféricos susceptibles a ataques *Bring Your Own Vulnerable Driver* — BYOVD) se exponen a través de paquetes de petición de E/S (**IRP**, estructura `nt!_IRP`) despachados hacia `MajorFunction[IRP_MJ_DEVICE_CONTROL]` (índice `0x0E`) o su variante interna `IRP_MJ_INTERNAL_DEVICE_CONTROL` (`0x0F`).

Para que un proceso en Ring 3 (incluso con nivel de integridad *Low* o *Medium*) pueda enviar un IOCTL mediante la syscall `NtDeviceIoControlFile`, primero debe abrir un handle válido hacia el objeto de dispositivo (`_DEVICE_OBJECT`) invocando `NtCreateFile` sobre su enlace simbólico en el directorio de objetos `\GLOBAL??\`. Si el desarrollador omitió restringir el *Security Descriptor* (SDDL) mediante `IoCreateDeviceSecure` (WDK) o `WdfDeviceInitAssignSDDLString` (KMDF), cualquier usuario local podrá alcanzar la rutina de despacho.

1.1. Decodificación del Código de Control (CTL_CODE)

El parámetro `IoControlCode` (32 bits) almacenado en `IrpSp→Parameters.DeviceIoControl.IoControlCode` (donde `IrpSp` proviene de `IoGetCurrentIrpStackLocation(Irp)`, estructura `_IO_STACK_LOCATION`) codifica cuatro campos críticos que determinan cómo el *I/O Manager* (`ntoskrnl.exe`) mapea y valida los buffers de usuario antes de ceder la ejecución al driver:

```
// Definición estructural de un IOCTL (32 bits) en winioctl.h / wdm.h

// +------------------+------------+--------------+--------+

// |   DeviceType     |   Access   |   Function   | Method |

// |  Bits 31..16     | Bits 15..14|  Bits 13..2  |Bits 1.0|

// +------------------+------------+--------------+--------+


#define CTL_CODE(DeviceType, Function, Method, Access) ( \

    ((DeviceType) << 16) | ((Access) << 14) | ((Function) << 2) | (Method) \

)


#define METHOD_BUFFERED                 0

#define METHOD_IN_DIRECT                1

#define METHOD_OUT_DIRECT               2

#define METHOD_NEITHER                  3
```

Método (Bits 1..0)	Puntero de Entrada en el IRP	Puntero de Salida en el IRP	Superficie de Fallo Característica
`METHOD_BUFFERED (0)`	`Irp->AssociatedIrp.SystemBuffer`	`Irp->AssociatedIrp.SystemBuffer`	El I/O Manager aloja un único buffer en NonPagedPool del tamaño `max(InputBufferLength, OutputBufferLength)`. Propenso a *Pool Overflow* si no se valida `InputBufferLength` y a *Info Leak* de memoria de kernel no inicializada si `IoStatus.Information` excede los bytes realmente escritos.
`METHOD_IN_DIRECT (1)` / `METHOD_OUT_DIRECT (2)`	`Irp->AssociatedIrp.SystemBuffer`	`Irp->MdlAddress` (mapeado con `MmGetSystemAddressForMdlSafe`)	El buffer secundario se bloquea en páginas físicas mediante un `_MDL` (`MmProbeAndLockPages`). Sin embargo, las páginas físicas subyacentes siguen mapeadas en el espacio virtual de Ring 3: un hilo concurrente en modo usuario puede modificar el contenido en pleno procesamiento (condición **TOCTOU / Double Fetch**).
`METHOD_NEITHER (3)`	`IrpSp->Parameters.DeviceIoControl.Type3InputBuffer`	`Irp->UserBuffer`	El I/O Manager entrega punteros virtuales crudos de Ring 3 sin validar. Si el driver no ejecuta `ProbeForRead` / `ProbeForWrite` dentro de un bloque SEH (`__try / __except`) y copia los datos a un buffer local antes de usarlos, permite primitivas directas de *Arbitrary Kernel Read/Write (Write-What-Where)* y *Double Fetch*.

2. Amplificación Determinista de Fallos: Driver Verifier y Special Pool

Un problema fundamental al hacer fuzzing contra el kernel es la **corrupción silenciosa**: un desbordamiento de 4 bytes sobre un bloque del *Kernel Pool* (`ExAllocatePool2` / `ExAllocatePoolWithTag`) rara vez provoca un

BugCheck en la instrucción exacta que escribe fuera de límites; en su lugar, corrompe los metadatos del chunk adyacente (`_POOL_HEADER`) o punteros de un objeto vecino, provocando un fallo asíncrono minutos después en un hilo del sistema completamente distinto.

Para transformar cualquier lectura o escritura fuera de límites de un solo byte en un *BugCheck* inmediato y atribuible a la instrucción culpable, es obligatorio activar **Driver Verifier** con la característica **Special Pool** sobre el driver auditado.

2.1. Mecánica a Nivel de PTE del Special Pool

Cuando el *Special Pool* intercepta una llamada a `ExAllocatePoolWithTag` / `ExAllocatePool2` (para tamaños menores a una página, es decir, menos de `0xFE0` bytes), reserva un par de páginas virtuales contiguas en la región del Special Pool:

- **Página 1 (Asignación alineada al final):** El bloque solicitado se coloca pegado al límite superior de la primera página de 4 KB (`0x1000`), y el resto inferior de la página se rellena con un patrón aleatorio repetido capaz de detectar *underflows* al liberar el bloque.
- **Página 2 (Guard Page Inmediata):** La entrada de tabla de páginas (`_MMPTE`) de la página inmediatamente posterior tiene el bit `Valid = 0` (no mapeada físicamente). Cualquier *off-by-one* hacia adelante cruza la frontera de página y dispara instantáneamente una excepción `#PF` (Page Fault) en la CPU, que el gestor de memoria traduce en `DRIVER_PAGE_FAULT_BEYOND_END_OF_ALLOCATION (0xD6)` o `PAGE_FAULT_IN_NONPAGED_AREA (0x50)` apuntando exactamente al `RIP` de la instrucción defectuosa.

```
:: Activación recomendada de Driver Verifier para Fuzzing en la VM de pruebas

:: Bitmask 0x209BB activa:

::   0x00001 - Special Pool

::   0x00002 - Force IRQL checking

::   0x00008 - Pool tracking

::   0x00010 - I/O verification

::   0x00020 - Deadlock detection

::   0x00080 - DMA checking

::   0x00100 - Security checks

::   0x00800 - Miscellaneous checks

::   0x20000 - DDI compliance checking


verifier.exe /flags 0x209BB /driver target_driver.sys

shutdown /r /t 0
```

3. Arquitecturas Modernas de Fuzzing en Ring 0

A diferencia de una aplicación de consola en modo usuario, un driver de kernel mantiene estado global en el sistema operativo, interactúa con el hardware y ante el primer fallo provoca un *Blue Screen of Death* (`KeBugCheckEx`) que requeriría decenas de segundos para reiniciar la máquina virtual. Las herramientas modernas resuelven esto mediante dos paradigmas complementarios:

3.1. Fuzzing Basado en Snapshots y Emulación Determinista: WTF (What The Fuzz)

Desarrollado por Axel "0vercl0k" Souchet, **WTF** desacopla la ejecución del caso de prueba del sistema operativo vivo:

1. Se prepara un *harness* en la VM que abre el dispositivo del driver y se detiene justo antes de invocar `NtDeviceIoControlFile` (o en el punto de entrada de `IRP_MJ_DEVICE_CONTROL`).
2. Con WinDbg conectado por red (KDNET), se toma un volcado completo de memoria activa y estado de registros de la CPU mediante la extensión `!dmp` (generando un estado base determinista).
3. WTF carga dicho volcado sobre un backend de ejecución ultrarrápido (**Bochs** para emulación pura multiplataforma o **KVM / Hyper-V WHPX** para ejecución nativa asistida por virtualización).
4. En cada iteración (alcanzando miles de ejecuciones por segundo por núcleo), WTF inyecta el buffer mutado en la memoria del proceso/IRP, ejecuta hasta alcanzar un breakpoint de retorno (por ejemplo, `IofCompleteRequest`) o un hook en `nt!KeBugCheck2` / `nt!KiPageFault`, registra las nuevas aristas de código descubiertas y restaura únicamente las páginas de memoria física ensuciadas (*dirty pages*) en microsegundos.

3.2. Fuzzing Guiado por Cobertura de Hardware con Intel PT: kAFL y Nyx

Cuando el driver realiza operaciones complejas de sincronización multi-hilo, timers de kernel (DPCs) o interacción con subsistemas difíciles de contener en un micro-snapshot, se emplea **kAFL / Nyx** sobre KVM-PT:

- **Trazado sin instrumentación binaria:** El hipervisor configura los registros MSR de **Intel Processor Trace (Intel PT)** (`IA32_RTIT_CTL`) filtrando por el rango de direcciones virtuales (`CR3` o rangos de IP en Ring 0 correspondientes a `target_driver.sys`).
- **Decodificación en el Host:** La CPU graba paquetes comprimidos de saltos condicionales (paquetes `TNT` — Taken/Not-Taken) y saltos indirectos (`TIP` — Target IP) directamente en un buffer físico del hipervisor con menos de un 5% de sobrecarga.
- **Snapshots rápidos en Hypervisor (Nyx):** Un agente ligero en Ring 3 dentro de la VM se comunica mediante hiperllamadas (`vmcall`) con QEMU-PT/Nyx para solicitar un *fast-reload* de las páginas sucias del EPT tras cada serie de llamadas IOCTL o en cuanto el hook de `KeBugCheckEx` detecta un fallo.

4. Laboratorio Práctico: Desarrollo de Harness para WTF y Triaje de Pool Overflow

Declaración de Entorno: `Laboratorio verificado: no; requiere VM/toolchain`. Para reproducir este laboratorio se requiere una máquina Host con soporte de virtualización anidada o CPU compatible con Bochs/WHPX, SDK de Windows (WinDbg Preview), el repositorio oficial de WTF (0vercl0k) y una VM Guest Windows 11 x64 con Driver Verifier activo sobre el driver de pruebas (p. ej., `HEVD.sys` — HackSys Extreme Vulnerable Driver).

Paso 1: Captura del Estado Base (Target Harness en Guest VM)

El siguiente programa se ejecuta en la VM Guest. Reserva páginas de memoria bloqueadas en RAM física (`VirtualLock`) para garantizar que tanto el código del *harness* como el buffer de entrada estén presentes en las tablas de páginas al momento de volcar el estado con WinDbg:

```
// harness_ioctl_snapshot.c — Compilar en x64 Release sin optimizaciones agresivas

#include <windows.h>

#include <stdio.h>



#define DEVICE_SYMLINK L"\\\\.\\HackSysExtremeVulnerableDriver"

#define HEVD_IOCTL_BUFFER_OVERFLOW_NON_PAGED_POOL CTL_CODE(0x22, 0x803, METHOD_NEITHER, FILE_ANY_ACCESS)

#define MAX_FUZZ_SIZE 0x1000



#pragma optimize("", off)

int main(void) {

    HANDLE hDevice = CreateFileW(

        DEVICE_SYMLINK,

        GENERIC_READ | GENERIC_WRITE,

        FILE_SHARE_READ | FILE_SHARE_WRITE,

        NULL,

        OPEN_EXISTING,

        FILE_ATTRIBUTE_NORMAL,

        NULL

    );



    if (hDevice == INVALID_HANDLE_VALUE) {

        printf("[-] Error abriendo el dispositivo: %lu\n", GetLastError());

        return 1;

    }



    PVOID FuzzBuffer = VirtualAlloc(NULL, MAX_FUZZ_SIZE, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);

    VirtualLock(FuzzBuffer, MAX_FUZZ_SIZE);

    RtlFillMemory(FuzzBuffer, MAX_FUZZ_SIZE, 0x41);



    DWORD BytesReturned = 0;
```

```
    // Dispara interrupción en el depurador de kernel justo antes del syscall
    __debugbreak();

    DeviceIoControl(
        hDevice,
        HEVD_IOCTL_BUFFER_OVERFLOW_NON_PAGED_POOL,
        FuzzBuffer,
        MAX_FUZZ_SIZE,
        NULL,
        0,
        &BytesReturned,
        NULL
    );

    return 0;
}
#pragma optimize("", on)
```

Paso 2: Módulo de Inserción de Caso de Prueba y Hooks en WTF

En el motor de WTF, el script del objetivo intercepta el estado justo en el `int 3` (`__debugbreak()`), reemplaza el contenido de `FuzzBuffer` y el tamaño `nInBufferSize` pasado a `DeviceIoControl`, y coloca hooks de detención tanto al regresar a modo usuario como en las rutinas de pánico del kernel:

```
// target_hevd_wtf.cc - Fragmento de configuración de hooks en WTF

bool Init(const Options_t &Opts, const CpuState_t &State) {

    // 1. Detener la iteración y marcar CRASH si el kernel invoca KeBugCheck2 o ExRaiseAccessViolation

    if (!g_Backend->SetBreakpoint("nt!KeBugCheck2", [](Backend_t *Backend) {

        Backend->Stop(Crash_t( Backend->GetFormat("BugCheck_%llx", Backend->Rcx()) ));

    })) {

        return false;

    }



    // 2. Avanzar desde __debugbreak() hasta la llamada a DeviceIoControl e inyectar el testcase

    // En la convención x64 de Windows, al entrar en kernel32!DeviceIoControl:

    //   RCX = hDevice

    //   RDX = dwIoControlCode

    //   R8  = lpInBuffer

    //   R9  = nInBufferSize

    if (!g_Backend->SetBreakpoint("kernelbase!DeviceIoControl", [](Backend_t *Backend) {

        const uint8_t *TestcaseBuffer = nullptr;

        size_t TestcaseSize = 0;

        Backend->GetTestcase(TestcaseBuffer, TestcaseSize);



        if (TestcaseSize < sizeof(uint32_t) || TestcaseSize > 0x1000) {

            Backend->Stop(Ok_t());

            return;

        }



        Gva_t InBufferGva = Gva_t(Backend->R8());

        uint32_t PayloadLen = static_cast<uint32_t>(TestcaseSize);



        // Escribir el payload mutado directamente en la memoria virtual del proceso

        if (!Backend->VirtWriteDirty(InBufferGva, TestcaseBuffer, PayloadLen)) {
```

```
            Backend->Stop(Ok_t());
            return;
        }
        Backend->R9(PayloadLen);
    })) {
        return false;
    }

    return true;
}
```

Paso 3: Triaje del Crash en WinDbg con Special Pool

Al reproducir un caso de prueba que excede los `0x1F8` bytes del chunk del pool, Driver Verifier intercepta la escritura fuera de límites en la frontera de página del Special Pool:

```
kd> !analyze -v

DRIVER_PAGE_FAULT_BEYOND_END_OF_ALLOCATION (d6)

N bytes of memory was allocated and more than N bytes are being referenced.

This cannot be protected by try-except.

Arguments:

Arg1: ffffc98421647000, memory referenced

Arg2: 0000000000000001, value 0 = read operation, 1 = write operation

Arg3: fffff8056a411642, If non-zero, the instruction address which referenced the bad memory

Arg4: 0000000000000000



kd> !pool ffffc98421647000

Pool page ffffc98421647000 region is Special pool

*ffffc98421646e00 size:  200 data: ffffc98421646e08 (NonPaged) *Hack

        Pooltag Hack : HEVD Pool Allocation



kd> !pte ffffc98421647000

VA ffffc98421647000

PXE at FFFFF6FB7DBEDF98    PPE at FFFFF6FB7DBF3080    PDE at FFFFF6FB7E610858    PTE at FFFFF6FE4C210A38

contains 0A00000104523863  contains 0A00000104524863  contains 0A00000104525863  contains
0000000000000000

pfn 104523 ---DA--KWEV     pfn 104524 ---DA--KWEV     pfn 104525 ---DA--KWEV     not valid
```

5. Referencias y Fuentes Técnicas

- Souchet, A. (0vercl0k). (2021). *Building a new snapshot fuzzer: What The Fuzz (WTF)*. 0vercl0k.github.io/wtf
- Schumilo, S., Aschermann, C., Gawlik, R., Schinzel, S., & Holz, T. (2017). *kAFL: Hardware-Assisted Feedback Fuzzing for OS Kernels*. USENIX Security Symposium.
- Microsoft Learn. (2024). *Driver Verifier & Special Pool Internals*. Windows Hardware Developer Documentation.
- Yosifovich, P., Ionescu, A., Russinovich, M., & Solomon, D. A. (2017). *Windows Internals, Part 1: System architecture, processes, threads, memory management, and more* (7th ed.). Microsoft Press.

Etapa 2 · Rama optativa 2D — Windows internals

> **Rama optativa, fuera de la aprobación obligatoria**
>
> 2D es una **rama optativa asociada a la Etapa 2**, después de 2A–2C. Profundiza Windows internals, pero **no** forma parte del recorrido obligatorio ni es prerrequisito general: el recorrido base siguen siendo las Etapas 0–4 y la Etapa 5 agrupa otras ramas optativas. Aporta prerrequisitos selectivos para la Etapa 6 · Kernel y drivers, también optativa. Llegás acá desde el mapa de la Etapa 2 o desde las ramas optativas de la Etapa 5.

> **Marco defensivo y de laboratorio**
>
> Todo acá es para **entender, analizar, depurar y detectar**, sobre **programas propios o didácticos** en **VMs aisladas con snapshot**. Nada de rootkits, persistencia encubierta, evasión de protecciones, ocultamiento de tareas ni objetivos de terceros. Cada laboratorio que registre servicios, tareas, claves o consumidores WMI **termina con pasos de limpieza** y evidencia de que quedó limpio.

Prerrequisitos

- **Obligatorios:** Etapa 1 (memoria, PE), Etapa 2A–2B (assembly, debugger, IDA, PE fino). C básico (1.5).
- **Recomendados:** Etapa 2C (WinDbg/heap, scripting) y la rama de malware estática (5.1).

Las lecciones

Esta rama comprende las siguientes lecciones de profundización en la arquitectura interna de Windows (con una dedicación estimada de **55–70 h**, incluyendo el trabajo práctico de inspección de pipes, registro RPC, excepciones con `.exr` / `.ecxr`, sesiones ETW y el laboratorio integrador):

1. Objetos, handles, tokens, privilegios e integridad
2. Memoria y loader: PEB/TEB, VAD e image mapping
3. Registro de Windows: hives, permisos, WOW64 y análisis defensivo
4. Servicios: SCM, ciclo de vida, cuentas y auditoría
5. Task Scheduler: leer tareas y detectar cambios
6. COM: interfaces, IUnknown y vida de los objetos
7. COM: registro, activación y configuración
8. DCOM y RPC: activación remota y límites de confianza
9. WMI: proveedores, filtros y consumidores
10. IPC en Windows: named pipes, RPC y ALPC
11. Excepciones de Windows y debugging
12. ETW y telemetría defensiva
13. Integrador Windows internals

Extensión optativa: Hyper-V y virtualización

Separada del núcleo de 2D y **no requisito** de 2D ni de la Etapa 6. La práctica es de lectura conceptual, sin modificar Hyper-V ni enseñar evasión anti-VM.

Puerta de salida (optativa)

2D no tiene puerta obligatoria. Cada lección trae la suya. Si hacés el integrador (2D.13), el criterio es: analizar un programa propio, producir un **inventario con evidencia** y demostrar que **limpiaste** todos los cambios que hiciste en el sistema.

> **Metodología y fuentes de esta rama**
>
> Las lecciones de la rama 2D se apoyan en la documentación técnica oficial de Microsoft Learn y Windows SDK, acompañadas de trazas y esquemas ilustrativos de laboratorio. Los ejercicios de registro, servicios, tareas programadas e IPC incluyen siempre pasos de verificación y limpieza final para preservar intacto el entorno de prueba.

Etapa 6 · Kernel y drivers (optativa)

La Etapa 6 no es requisito para completar el curso base y no exige COM, tareas programadas ni completar toda 2D. Requiere C, assembly, memoria/sincronización, WinDbg y las lecciones 2D.1, 2D.2 y 2D.11. 2C es recomendado. Para ver prerrequisitos por lección, alcance y estimación, consultá el mapa de Etapa 6.

Alcance de las prácticas

Todas las prácticas de 2D y de la Etapa 6 están diseñadas para ejecutarse en una máquina virtual aislada con snapshot previo. En la Etapa 6, si la política de firma o arranque seguro de la VM no permite cargar un driver de prueba, el análisis puede completarse de forma estática en IDA/Ghidra. Nunca se modifican protecciones del sistema anfitrión.

Objetos, handles, tokens, privilegios e integridad

> **Rama optativa 2D**
>
> Lección de la rama optativa 2D, fuera del recorrido obligatorio. Defensiva: entender el contexto de seguridad para **analizar**, no para escalar privilegios.

> **Propósito**
>
> Entender cómo Windows modela el acceso: todo es un **objeto** al que se llega por **handle**, y lo que un proceso puede hacer lo define su **token** (SIDs, privilegios, nivel de integridad) contra la **ACL** del objeto. Saber leer ese contexto en Process Explorer/WinDbg.

Prerrequisitos

- *Obligatorios:* procesos e hilos (1.3), debugger (2.6).
- *Recomendados:* anti-análisis/PEB (2.15), WinDbg (2.19).

El modelo de objetos

En Windows casi todo recurso del kernel —procesos, hilos, archivos, claves de registro, eventos, secciones— es un **objeto** gestionado por el *Object Manager*. Un proceso no toca el objeto directamente: pide un **handle** (un índice en su tabla de handles privada) y opera a través de él. Cada handle guarda, además, los **permisos concedidos** (access mask) con los que se abrió. **Conocimiento externo** (Microsoft Learn).

Para el análisis, los handles abiertos de un proceso son una **radiografía de qué toca**: qué archivos, qué claves, qué objetos de sincronización (un **mutex** con nombre suele ser el "ya estoy corriendo" de un malware).

El token de acceso

Cuando un proceso quiere hacer algo sobre un objeto, Windows compara el **token** del proceso con la **ACL** del objeto. El token contiene:

- **SID de usuario y grupos:** quién sos y a qué grupos pertenecés.
- **Privilegios:** capacidades con nombre (`SeDebugPrivilege`, `SeBackupPrivilege`, `SeLoadDriverPrivilege`…) que el token puede tener **habilitadas** o solo presentes.
- **Nivel de integridad (IL):** Low / Medium / High / System. Con la política obligatoria predeterminada de Windows, un proceso de IL bajo normalmente no puede escribir en objetos de IL mayor (no es una regla universal para toda operación; también intervienen las etiquetas y políticas del objeto). Es una de las barreras usadas por sandboxes.

ACL: quién puede qué

Un objeto asegurable tiene un **descriptor de seguridad** con su **DACL** (lista de ACEs: a qué SID se permite/niega qué acceso) y su **SACL** (auditoría). El chequeo de acceso recorre la DACL con el token del que pide. Para el

reversing defensivo: una DACL **demasiado permisiva** en un servicio, una clave o un objeto con nombre es una debilidad a **reportar** (no a explotar).

Ejemplo trabajado

*(Ejemplo **ilustrativo**: vista típica de Process Explorer / `!token` en WinDbg; valores construidos.)*

```
bloc_lab.exe (PID 4312)   Integrity: Medium       ← editor de texto propio de laboratorio
  Token:
    User:  PC\lab  (S-1-5-21-…-1001)
    Grupos: BUILTIN\Users,  …
    Privilegios (habilitados en MAYÚSCULA):
       SeChangeNotifyPrivilege (ON)   SeShutdownPrivilege (off)
       SeDebugPrivilege (off)         ← presente pero deshabilitado
  Handles (extracto):
    0x0F8  File   C:\Users\lab\Documents\notas.txt              (el archivo que abriste)
    0x120  Key    HKCU\Software\BlocLab\Settings                (su propia config)
    0x134  Mutant \Sessions\1\BaseNamedObjects\BlocLab_SingleInstance
```

Lectura: proceso propio de integridad **Medium**, con `SeDebugPrivilege` **presente pero apagado** (no está usando debug de otros procesos ahora). Los handles muestran el documento que abriste, su **propia** clave de configuración y un **mutex** de instancia única. En un análisis real, el nombre del mutex o una ruta son **pistas** a corroborar, no evidencia por sí solos.

Los offsets de nt!_EPROCESS cambian entre builds: nunca los fijes

A diferencia de los primeros campos del PEB, la estructura `_EPROCESS` es opaca y el kernel reorganiza sus campos entre versiones de Windows. Un controlador que asuma offsets estáticos fijos fallará o provocará Bugchecks al actualizar el sistema operativo: mientras que entre Windows 10 2004–22H2 (Build 19045) y Windows 11 21H2–23H2 (Build 22631) los primeros campos se mantuvieron en `+0x440` / `+0x4B8`, en Windows 11 24H2 (Build 26100) Microsoft reordenó `_EPROCESS` desplazando `UniqueProcessId` a `+0x1D0` y `Token` a `+0x248` (además de compilaciones Insider previas de la rama 24H2 donde solo se habían corrido los campos superiores a `+0x8FA` / `+0xA58`). En x64:

Campo (`nt!_EPROCESS`)	Win10 1507–1809	Win10 2004–22H2 (Build 19045)	Win11 21H2–23H2 (Build 22631)	Win11 24H2 (Build 26100 GA / Pre-GA)
`UniqueProcessId`	`+0x2E0`	`+0x440`	`+0x440`	`+0x1D0` (`+0x440` pre-GA)
`ActiveProcessLinks`	`+0x2E8`	`+0x448`	`+0x448`	`+0x1D8` (`+0x448` pre-GA)
`Token` (`_EX_FAST_REF`)	`+0x358`	`+0x4B8`	`+0x4B8`	`+0x248` (`+0x4B8` pre-GA)
`ImageFileName` (`UCHAR[15]`)	`+0x450`	`+0x5A8`	`+0x5A8`	`+0x338` (`+0x5A8` pre-GA)
`Protection` (`_PS_PROTECTION`)	`+0x6CA`	`+0x87A`	`+0x87A`	`+0x5FA` (`+0x8FA` pre-GA)
`MitigationFlags` / `Flags2`	`+0x828`	`+0x9D0` / `+0x9D4`	`+0x9D8` / `+0x9DC`	`+0x750` / `+0x754` (`+0xA58` pre-GA)

Nunca asumas un offset fijo (y corregí en la Hoja 6 de la Separata la reestructuración de Build 26100 GA): verificá siempre el build con `vertarget` o corré `dt nt!_EPROCESS UniqueProcessId ActiveProcessLinks Token ImageFileName Protection MitigationFlags MitigationFlags2` contra los símbolos (`.pdb`) de `ntoskrnl.exe` de esa máquina, o utilizá la expresión dinámica `@@(#FIELD_OFFSET(nt!_EPROCESS, Campo))`.

Práctica segura

En tu VM, con un **programa propio** (o Process Explorer sobre uno benigno):

1. Con **Process Explorer**, abrí un proceso propio y mirá su cuenta y sus **handles**. Para los grupos y privilegios de tu sesión, ejecutá `whoami /all`; anotá cuáles figuran habilitados y cuáles no.
2. Identificá handles a un archivo, una clave y un objeto con nombre (evento/mutex). Si el proceso no tiene uno de esos objetos, anotalo: no hace falta crearlo para completar la observación.
3. Compará el token del proceso propio con el de la sesión solo si ambos pertenecen a la misma cuenta y nivel de integridad; no eleves privilegios para la práctica.
4. Mirá la **DACL** de un objeto propio (p. ej. `icacls` sobre un archivo tuyo) y explicá quién tiene qué.

Entregable

Una ficha del proceso: nivel de integridad, privilegios habilitados, 3–5 handles relevantes (con qué objeto) y una observación sobre la DACL de uno de esos objetos. Solo observación, sin modificar permisos.

Preguntas de repaso

▼ **¿Qué es un handle y qué guarda además del objeto?**

> Un índice en la **tabla de handles** privada del proceso que apunta a un objeto del kernel; guarda también la **access mask** (los permisos con que se abrió). El proceso opera por el handle, no sobre el objeto directo.

▼ **¿Qué define lo que un proceso puede hacer sobre un objeto?**

> La comparación de su **token** (SIDs, privilegios, nivel de integridad) contra la **DACL** del objeto. Si la DACL no concede el acceso pedido para ese SID, se rechaza.

▼ **¿Qué diferencia hay entre un privilegio "presente" y uno "habilitado"?**

> Presente = el token lo tiene en su lista; habilitado = está **activo** ahora. Muchos privilegios potentes vienen presentes pero deshabilitados y hay que habilitarlos para usarlos; en el análisis te fijás cuáles están **ON**.

▼ **¿Para qué sirve el nivel de integridad?**

> Es una barrera: un proceso de integridad **baja** no puede escribir en objetos de integridad **mayor**. Es la base de los sandboxes (p. ej. el del navegador).

▼ **¿Por qué un mutex con nombre es interesante al analizar, y qué cuidado hay que tener?**

> Suele ser el mecanismo de **instancia única** ("ya estoy corriendo"), así que es una **pista** útil. Pero su nombre **por sí solo no es un IOC**: un nombre genérico puede coincidir con software legítimo; sirve como indicio a **corroborar** con más contexto.

Puerta de salida (optativa)

Podés: (1) explicar el modelo objeto/handle/access-mask; (2) leer un token (SIDs, privilegios ON/off, integridad); (3) relacionar token + DACL en un chequeo de acceso; (4) sacar evidencia (handles, mutex) de un proceso propio sin modificarlo.

Fuentes

- **Conocimiento externo** (Microsoft Learn; páginas concretas): Handles and Objects, Process Explorer, Access Tokens, Privileges, Mandatory Integrity Control y Access Control Lists. No se citan versiones ni valores reales: el ejemplo es ilustrativo.

- **Fundamentos previos:** la lectura del **PEB** y el contexto de proceso aparecen en 2.15 Anti-análisis; el robo de token en kernel se menciona (de forma ofensiva, fuera de alcance) en los tutes 49–69 del Curso IDA, que acá **no** se usan como guía.

Etiquetas

Núcleo conceptual: solo el PEB/contexto de 2.15 (referencia). **Conocimiento externo:** todo el modelo de seguridad. **Ejemplo ilustrativo:** la vista de token/handles. **Laboratorio verificado:** ninguno en esta edición (no hay binario compilado con datos reales); la práctica se hace sobre procesos propios en tu VM.

Ampliación editorial basada en fuentes externas — Inspección de _EPROCESS, _EX_FAST_REF y máscaras de privilegios en _TOKEN con WinDbg

Al analizar privilegios y niveles de integridad desde el depurador de kernel, las herramientas administrativas de user-mode (como Process Explorer) no bastan; el analista debe inspeccionar directamente las estructuras del ejecutivo en memoria (Microsoft Learn: *!process*, *!token* y símbolos públicos de `ntoskrnl.exe`) *(Hecho confirmado)*:

```
kd> !process 0 0 BlocLab.exe                ; Obtiene la dirección de _EPROCESS
kd> dt nt!_EPROCESS <dir_eprocess> UniqueProcessId ActiveProcessLinks Token Protection
```

- **Puntero `Token` (`_EX_FAST_REF`):** En x64, los 4 bits menos significativos del campo `Token` almacenan un contador de referencias rápidas. Para obtener la dirección virtual real de la estructura `_TOKEN` se debe aplicar la máscara `Token & ~0xF` (poniendo en `0` el último dígito hexadecimal).
- **Máscaras de 64 bits en `_SEP_TOKEN_PRIVILEGES` (`dt nt!_TOKEN <dir_token> Privileges`):** En lugar de recorrer listas de privilegios, el kernel ubica en el offset `+0x40` de `_TOKEN` tres campos `QWORD` contiguos: `Present`, `Enabled` y `EnabledByDefault`. Cada privilegio corresponde a un bit fijo (p. ej., `SeDebugPrivilege` es el bit 20 = `0x100000`), permitiendo verificar privilegios activos mediante una única instrucción bit a bit (`bt` / `test`).
- **Nivel de protección (`PS_PROTECTION`):** Un byte que codifica `Type` (bits 0..2: `0` = ninguno, `1` = `ProtectedLight` / PPL, `2` = `Protected`) y `Signer` (bits 4..7: `WinTcb`, `Antimalware`, `Lsa`), determinando si el proceso puede ser accedido desde user-mode.

Memoria y loader: PEB/TEB, VAD e image mapping

> **Rama optativa 2D**
>
> Lección de la rama optativa 2D. Profundiza en cómo se **describe** la memoria del proceso; no repite el formato PE (1.4) ni el PE fino (2.14), se apoya en ellos.

> **Propósito**
>
> Saber de dónde sale la información de memoria de un proceso: el **PEB/TEB** (en user-mode), la **lista del loader** (módulos cargados) y las **VAD** (cómo el kernel describe cada región), y cómo se **mapea** una imagen. Para orientarte en un dump o en vivo sin perderte, y reconocer el contexto WOW64.

Prerrequisitos

- *Obligatorios:* memoria y stack (1.2), formato PE (1.4).
- *Recomendados:* PE fino (2.14), TLS (2.11), WinDbg (2.19).

PEB y TEB

El **PEB** (*Process Environment Block*) y el **TEB** (*Thread Environment Block*) son estructuras en **user-mode** que describen el proceso y cada hilo. Ya tocaste el PEB en anti-análisis (el campo `BeingDebugged`). Para memoria/loader, lo valioso del PEB:

- `Ldr` → tres listas doblemente enlazadas de **módulos cargados** (por orden de carga, de memoria y de inicialización): base, tamaño y nombre de cada DLL. Es de donde sale la lista de módulos que ves en el debugger.
- `ProcessParameters` → línea de comandos, directorio actual, variables de entorno: contexto de ejecución, útil en análisis.

El TEB guarda, entre otras cosas, el puntero a la cadena **SEH** en 32 bits (`FS:[0]`) y el TLS por hilo (lo viste en 2.11).

WOW64: proceso x86 en Windows de 64 bits

WOW64 permite ejecutar aplicaciones x86 en Windows x64; también hay variantes para ejecutar aplicaciones x86 y ARM de 32 bits en Windows ARM64. Para reversing importa porque el proceso conserva una vista de 32 bits, mientras el sistema operativo y algunas herramientas pueden mostrar el contexto nativo de 64 bits. Un proceso x86 no puede cargar una DLL x64 para ejecutarla, ni a la inversa; la comunicación entre ambos mundos requiere una frontera explícita, por ejemplo IPC o ciertos mecanismos de interoperabilidad.

Al comparar un proceso WOW64, anotá siempre la arquitectura del proceso y la del sistema. La redirección de archivos y Registro puede hacer que una ruta aparente no sea la que finalmente se consulta; no confundas esa redirección con una diferencia en el binario. En depuración, confirmá qué contexto de registros y qué vista de módulos estás examinando antes de interpretar direcciones o stacks. La práctica de esta lección sigue siendo de lectura: identificar arquitectura, módulos y regiones sin cambiar redirecciones del sistema.

Cruce de llamadas entre x86 y el sistema nativo

En Windows x64, WOW64 no convierte cada instrucción x86 a x64. El proceso usa su ntdll x86; cuando llega a una llamada de servicio, una secuencia de transición en user mode permite que WOW64 la intercepte. La transición entra en modo nativo, donde las thunks de Wow64.dll adaptan argumentos de 32 a 64 bits y encaminan la llamada al servicio nativo. Por eso una pila puede alternar contexto x86 y nativo, y una vista x86 del depurador no siempre muestra toda la transición.

Con WinDbg nativo y la extensión wow64exts, !wow64exts.sw cambia entre los contextos x86 y nativo, !wow64exts.k muestra una pila combinada y !wow64exts.info resume datos de PEB/TEB de WOW64. La implementación interna puede variar por arquitectura y build; estos comandos sirven para observarla, no son un contrato de aplicación.

La lista del loader

El **loader** de ntdll mantiene esas listas (`PEB_LDR_DATA` → `LDR_DATA_TABLE_ENTRY`). Que estén en listas enlazadas tiene una consecuencia para el análisis: un módulo se puede **desenlazar** de ellas para "esconderlo" de herramientas que solo recorren la lista. Por eso, al buscar módulos ocultos, se compara la lista del loader con lo que dicen las **VAD** del kernel. (Reconocer esto es defensivo; no se enseña a ocultar nada.)

VAD: cómo el kernel describe la memoria

Las **VAD** (*Virtual Address Descriptors*) son un árbol que el kernel mantiene por proceso: cada nodo describe un **rango** de direcciones virtuales (privado o mapeado, permisos, a qué archivo/sección corresponde). En WinDbg, `!vadump` lista las regiones de un proceso en user mode. La extensión `!vad` ofrece detalles de nodos VAD en depuración de kernel y requiere los argumentos/contexto adecuados; no es un sustituto directo de `!vadump`. Sirven para:

- Ver **qué es** cada región: imagen mapeada (`Image`), archivo mapeado (`Mapped`), o memoria privada (`Private`, típico del heap/stack).
- **Cruzar** con la lista del loader: si una VAD de tipo `Image` no aparece en la lista del loader, es **algo a investigar** —puede tener explicaciones legítimas—, no una prueba de que haya un módulo oculto.

Image mapping y secciones

Un `.exe` / `.dll` no se "copia" a memoria: se **mapea** vía un objeto **sección** (*section/file mapping*). Las páginas de código **suelen** ser de solo-lectura/ejecución y **compartidas** entre procesos (varias instancias comparten la misma imagen física); las de datos usan **copy-on-write** (se duplican al primer escribir). Ojo: los permisos exactos **dependen de las características de cada sección** del PE (una sección puede ser escribible) y pueden variar, así que no asumas "todo el código es RO". El loader aplica las **relocalizaciones** solo si la imagen no cargó en su base preferida (ASLR), y resuelve los imports (lo viste en PE/IAT, 1.4).

Ejemplo trabajado

*(Ejemplo **ilustrativo**: salida resumida de WinDbg; valores construidos.)*

```
0:000> lm                      (módulos cargados — fuente confiable para la lista)
    start    end      module
    00400000 0040c000 prog
    77000000 77180000 ntdll
    75d00000 75e00000 KERNEL32

0:000> !peb                    (extracto; la salida varía según versión/sesión)
    ProcessParameters.CommandLine: "prog.exe" --modo test

0:000> !vadump                 (extracto de regiones)
    BaseAddress  RegionSize  Protect         Type
    00400000     0000c000    EXECUTE_READ    Image    (prog.exe)
    00030000     00002000    READWRITE       Private  (heap/stack)
    00140000     00002000    READONLY        Mapped   (archivo de datos)
```

Lectura: `lm` da la lista de módulos; `prog.exe` aparece además como región `Image` en `!vadump`, el heap/stack es `Private` READWRITE y hay un archivo `Mapped` READONLY. La `CommandLine` del `!peb` da el contexto (ese comando puede variar según la versión de Windows y el tipo de sesión). Todo esto orienta un dump sin suponer.

Práctica segura

En tu VM, con WinDbg sobre un **proceso propio**:

1. Con `lm` listá los **módulos cargados**; con `!peb` mirá la **CommandLine** (su salida varía según versión/sesión, por eso los módulos salen de `lm`).
2. `!vadump`: identificá una región `Image`, una `Private` y, si hay, una `Mapped`. Decí qué es cada una.
3. Elegí una DLL de `lm` y confirmá que tiene una región `Image` en su base en `!vadump` (la coherencia módulos↔VAD).
4. Anotá base y tamaño de tu `.exe` y compará con lo que dice el PE en disco (2.14).
5. En un proceso x86 bajo WOW64, alterná el contexto con !wow64exts.sw y observá la pila combinada con !wow64exts.k; registra qué frames son x86 y cuáles nativos.

Entregable

Un mapa breve del proceso: lista de 3–5 módulos (base), 3 regiones de VAD clasificadas (Image/Private/Mapped) y la línea de comandos. Con una frase sobre la coherencia loader↔VAD.

Preguntas de repaso

▼ **¿Qué guarda el campo `Ldr` del PEB y por qué importa?**

> Las **listas de módulos cargados** (base, tamaño, nombre). Importa porque es la fuente de la lista de DLLs del proceso; y porque un módulo puede desenlazarse de ahí para esconderse, lo que se detecta cruzando con las VAD.

▼ **¿Qué describe una VAD?**

> Un **rango** de direcciones virtuales del proceso: su tipo (`Image` / `Mapped` / `Private`), permisos y a qué archivo/sección corresponde. El kernel las mantiene en un árbol por proceso; `!vadump` sirve para listar regiones en user mode y `!vad` para inspección detallada en el contexto de depuración de kernel.

▼ **¿Qué precaución tomás al analizar un proceso WOW64?**

> Anoto por separado la arquitectura del proceso y la del sistema, confirmo el contexto de registros y módulos, y tengo en cuenta la redirección de archivos/Registro. Un proceso de 32 bits no puede cargar una DLL de 64 bits para ejecutarla.

▼ **¿Cómo se investiga un posible módulo que no está en la lista del loader?**

> Comparando los módulos (`lm` / lista del loader) con las **regiones** `Image` (`!vadump` / VAD del kernel): una imagen mapeada que no figure en la lista es **algo a investigar** (puede tener causas legítimas), no una prueba de ocultamiento. Es reconocimiento defensivo, no ocultar nada.

▼ **¿Por qué el código de una DLL se comparte entre procesos y los datos no?**

> Porque el ejecutable se **mapea** como una imagen respaldada por una sección. Las páginas pueden compartirse mientras coinciden con el archivo; los permisos y el comportamiento ante escrituras dependen de cada sección. En una vista *copy-on-write*, la página modificada se vuelve privada para ese proceso. No todas las imágenes tienen permisos idénticos; verificá la región concreta con `!vadump` .

▼ **¿Cuándo aplica el loader las relocalizaciones?**

> Solo si la imagen **no** cargó en su base preferida (por ASLR u ocupación): entonces ajusta las direcciones absolutas con la tabla de relocalización.

Puerta de salida (optativa)

Podés: (1) listar módulos con `lm` y leer la CommandLine con `!peb` ; (2) clasificar regiones con `!vadump` (Image/Private/Mapped); (3) explicar la coherencia módulos↔VAD y qué amerita investigar; (4) explicar image mapping, código compartido y copy-on-write, y por qué los permisos de sección pueden variar; (5) describir qué cambia al analizar un proceso WOW64 y no confundir la vista x86 con la nativa.

Fuentes

- Microsoft Learn: Debugging WOW64, File System Redirector.
- Microsoft Learn: Running 32-bit Applications (WOW64), WOW64 Implementation Details.
- **Conocimiento externo** (páginas concretas): PEB (winternl), `lm` (list modules), `!vadump` y el formato PE (características de sección). *Windows Internals* como referencia general. No se citan versiones ni valores reales.

- **Fundamentos previos:** el PEB (`BeingDebugged`) en 2.15; PE, IAT y relocalizaciones en 1.4 y 2.14; TLS/SEH en 2.11.

Etiquetas

Núcleo conceptual: PEB y PE/loader de 1.4, 2.11, 2.14, 2.15 (referencia). **Conocimiento externo:** VAD, listas del loader, image mapping/COW, redirección y transición WOW64. **Ejemplo ilustrativo:** las salidas de `lm`, `!peb` y `!vadump`. **Laboratorio verificado:** ninguno en esta edición; la práctica se hace sobre procesos propios en tu VM.

Registro de Windows: hives, permisos, WOW64 y análisis defensivo

Rama optativa 2D

Lección de la rama optativa 2D. Defensiva: **observar y detectar** cambios en el registro; no crear persistencia encubierta.

Propósito

Entender cómo está organizado el registro (hives, claves, valores), quién puede escribir dónde (permisos), la redirección **WOW64** para apps de 32 bits, y cómo **observar** qué toca un programa propio (configuración y persistencia) con evidencia.

Prerrequisitos

- *Obligatorios:* objetos y ACL (2D.1), procesos (1.3).
- *Recomendados:* malware estático (5.1).

Hives y organización

El registro es una base jerárquica de **claves** (carpetas) y **valores** (datos tipados). Las cinco raíces predefinidas (`HKLM`, `HKCU`, `HKCR`, `HKU`, `HKCC`) no corresponden todas a archivos físicos uno a uno: los **hives** son unidades cargadas desde archivos como `SYSTEM`, `SOFTWARE` y el `NTUSER.DAT` de cada usuario; algunas raíces son alias o vistas. `HKCU` apunta al hive del usuario actual; `HKCR` combina las clases de máquina y usuario (`HKLM\Software\Classes` y `HKCU\Software\Classes`); `HKCC` representa la configuración de hardware actual. **Conocimiento externo** (Microsoft Learn).

Zonas que importan en el análisis defensivo:

- **Persistencia:** `…\CurrentVersion\Run` / `RunOnce` (en `HKLM` y `HKCU`), servicios (`HKLM\SYSTEM\CurrentControlSet\Services`), y otras claves de autoarranque.
- **Configuración de apps:** cada programa suele guardar lo suyo bajo `HKCU\Software\<Fabricante>\<App>`.

Permisos

Cada clave es un **objeto asegurable** con su DACL (como viste en 2D.1). Por eso `HKLM` (de máquina) normalmente requiere administrador y `HKCU` (del usuario) lo puede escribir el propio usuario. Una clave de autoarranque con permisos **demasiado abiertos** (escribible por usuarios sin privilegio) es una debilidad a **reportar**.

Redirección WOW64

Un proceso de **32 bits** en Windows de 64 bits usa la vista de 32 bits para determinadas claves. Por ejemplo, `HKLM\Software` está redirigida en las versiones actuales de Windows: una app x86 que acceda a

`HKLM\Software\BlocLab` puede aparecer en la vista de 32 bits que Regedit muestra bajo `HKLM\Software\WOW6432Node\BlocLab`. **No todas las claves se redirigen**; algunas son compartidas, y el resultado depende de la clave y de la versión de Windows. Para un caso real, consultá la tabla oficial de claves afectadas y no agregues `WOW6432Node` a mano en código nuevo.

Observar cambios (defensivo)

- **Diferencial: Regshot** saca un snapshot antes y después de ejecutar; te da la lista exacta de claves/valores creados o cambiados.
- **En vivo: Process Monitor** con filtro de operaciones de registro (`RegSetValue`, `RegCreateKey`) muestra quién tocó qué y cuándo, con el stack.

Ejemplo trabajado

*(Ejemplo **ilustrativo**: salida resumida de Regshot sobre una app de laboratorio; valores construidos.)*

```
Regshot — diff antes/después de correr bloc_lab.exe (x86) en la VM:
  Valores agregados:
    HKCU\Software\BlocLab\Settings\Theme = "dark"        ← su config propia (vista compartida)
    HKLM\Software\WOW6432Node\BlocLab\Build = "lab-1"    ← vista x86 de HKLM\Software (app 32-bit)
  (sin cambios en HKLM ni en claves Run)
```

Lectura: la app guardó su configuración de usuario en `HKCU` y, en este ejemplo, escribió también en la vista x86 de su clave de máquina `HKLM\Software`. El diff aparece resumido con la ruta física `WOW6432Node`; el acceso de la app fue a la vista lógica de 32 bits. **No** tocó `Run`: el ejemplo no muestra cambios de autoarranque. Es una ilustración de las vistas, no una regla aplicable a cualquier clave.

Práctica segura

En tu VM, con un **programa propio** que guarde configuración en el registro:

1. **Regshot:** snapshot → corré el programa → segundo snapshot → *compare*. Anotá qué claves/valores agregó.
2. Si es de 32 bits, confirmá la **redirección WOW64** (aparece bajo `WOW6432Node`).
3. En Regedit, abrí **Permisos** de la clave propia y observá su **DACL**. No cambies permisos. `icacls` trabaja con archivos y no es la herramienta para inspeccionar claves del registro.
4. Confirmá que la práctica no cambió claves de autoarranque. Si necesitás practicar la lectura de una clave `Run`, usá una captura o exportación de ejemplo; no la crees ni la modifiques en el equipo.

Entregable

Lista de claves/valores de configuración que tu programa propio creó o modificó (con su tipo), si hubo redirección WOW64, y una observación de permisos de una clave propia. Incluí evidencia de que no cambiaste claves de autoarranque.

Preguntas de repaso

▼ **¿Qué es un hive y cómo se relaciona con `HKCU` / `HKLM`?**

> Una unidad lógica de datos del registro que Windows carga y respalda, por ejemplo, con `NTUSER.DAT` o `SOFTWARE`. Las raíces predefinidas no son todas archivos ni hives uno a uno: `HKCU` apunta al usuario actual y `HKCR` es una vista combinada.

▼ **¿Por qué `HKLM` suele requerir administrador y `HKCU` no?**

> Por la **DACL** de cada clave: las de máquina (`HKLM`) conceden escritura a administradores; el hive del usuario (`HKCU`) lo puede escribir el propio usuario. Son objetos asegurables.

▼ **Una app de 32 bits dice escribir en `HKLM\Software\Foo` pero no la ves ahí. ¿Dónde está?**

> Si esa clave está entre las afectadas, la app x86 accede a la vista de 32 bits y Regedit de 64 bits suele mostrarla bajo `HKLM\Software\WOW6432Node\Foo`. Hay que confirmar el comportamiento para la clave concreta: WOW64 también tiene claves compartidas.

▼ **¿Cómo obtenés la lista exacta de lo que tocó un programa en el registro?**

> Con un **diferencial** (Regshot: snapshot antes/después) o en vivo con **Process Monitor** filtrando operaciones de registro. El diff te da claves/valores agregados o cambiados.

▼ **¿Qué hace "peligrosa" a una clave de autoarranque?**

> Que su **DACL** permita escribir a usuarios sin privilegio: cualquiera podría poner algo que se ejecute al inicio. Es una debilidad a **reportar**, no a aprovechar.

> **Puerta de salida (optativa)**
>
> Podés: (1) explicar hives y las vistas `HKCU` / `HKLM` / `HKCR` ; (2) relacionar permisos de clave con su DACL; (3) reconocer la redirección WOW64; (4) sacar con Regshot/Procmon la lista de cambios de un programa propio y dejar el sistema limpio.

Fuentes

- **Conocimiento externo** (Microsoft Learn): Registry Hives, Structure of the Registry, Registry Key Security and Access Rights, Registry Redirector (WOW64) y Registry Keys Affected by WOW64.
- **Fundamentos previos:** objetos asegurables y DACL en 2D.1; la persistencia por `Run` también aparece como IOC en la rama de malware.

Etiquetas

Núcleo conceptual: solo el marco de ACL (2D.1) y la noción de persistencia. **Conocimiento externo:** estructura del registro, permisos y WOW64. **Ejemplo ilustrativo:** el diff de Regshot. **Laboratorio verificado:** ninguno en esta edición; la práctica se hace sobre un programa propio en tu VM, con limpieza.

Servicios: SCM, ciclo de vida, cuentas y auditoría

> **Rama optativa 2D**
>
> Lección optativa y defensiva. La práctica usa únicamente un servicio propio en una VM descartable; no modifica servicios de Windows ni intenta abusar de permisos.

> **Propósito**
>
> Entender qué registra y controla el **Service Control Manager (SCM)**, seguir los estados de un servicio y leer su cuenta, ruta y permisos. Distinguir un servicio con ejecutable propio de uno hospedado como DLL, y terminar la práctica dejando la VM en el estado inicial.

Prerrequisitos

- *Obligatorios:* objetos, tokens y ACL (2D.1); registro de Windows (2D.3).
- *Recomendados:* procesos (1.3) y conceptos de depuración (2.19).

Qué controla el SCM

El **SCM** es el componente que mantiene el registro de servicios instalados, los inicia bajo demanda o al arrancar Windows, conserva sus estados y entrega solicitudes de control. Los programas administrativos consultan o cambian el servicio mediante las API del SCM; el registro contiene también datos asociados en `HKLM\SYSTEM\CurrentControlSet\Services\<nombre>`. Para este curso, consultá la configuración por las herramientas del SCM: **no edites a mano** las claves de un servicio.

Un servicio Win32 no es “una DLL que corre sola”. El programa de servicio contiene el punto de entrada y una función `ServiceMain`; el SCM se comunica con ese programa mediante el dispatcher y el handler de controles. Un ejecutable de consola cualquiera no cumple esa interfaz y no se convierte en servicio por registrarlo.

Tipo de servicio, tipo de inicio y estado

Son conceptos distintos:

- **Tipo:** `own` (proceso propio) o `share` (programa de servicio compartido). Los tipos `kernel` y `filesys` corresponden a drivers y quedan fuera de esta lección.
- **Tipo de inicio:** `demand` es inicio manual; `auto` inicia con Windows; `delayed-auto` lo inicia más tarde; `disabled` impide iniciarlo. `boot` y `system` son opciones de inicio para drivers, no para el servicio didáctico de esta práctica.
- **Estado actual:** `STOPPED`, `START_PENDING`, `RUNNING`, `STOP_PENDING` o `PAUSED`. El tipo de inicio describe cuándo puede arrancar; el estado describe qué está pasando ahora.

Al entrar en `ServiceMain`, el programa registra su handler de controles y comunica estados al SCM. Durante una inicialización larga informa que el inicio sigue pendiente; al detenerse, procesa la solicitud y notifica el estado final.

Ejecutables y servicios hospedados

Un servicio puede ejecutarse en un proceso propio o compartir un programa anfitrión. Windows usa `svchost.exe` para hospedar algunos de sus servicios; la organización de los grupos y procesos varía según la versión y la configuración. `svchost.exe` es un componente del sistema, no una plantilla para registrar servicios propios.

Algunos servicios hospedados cargan una DLL indicada por su configuración, que suele aparecer como `ServiceDll` bajo `Parameters`. Esto se analiza **en lectura**: el valor permite relacionar servicio, proceso anfitrión y módulo. No se cambia el valor ni se reemplaza la DLL. Para comprender el mecanismo, consultá la documentación de programas de servicio y el artículo de agrupación de `svchost`; no supongas que todos los servicios usan este modelo.

Cuenta y permisos

Cada servicio se ejecuta con un token asociado a una cuenta. `LocalSystem` tiene privilegios amplios; `LocalService` y `NetworkService` tienen perfiles distintos y más limitados. Un servicio bien configurado usa la cuenta de menor privilegio que necesite.

La DACL del **objeto servicio** controla quién puede consultarlo, iniciarlo, detenerlo o cambiar su configuración. Es distinta de la ACL del ejecutable y de sus carpetas. Para una auditoría defensiva, registrá si la ruta existe, quién puede modificar sus archivos y qué permisos tienen las cuentas sobre el objeto servicio. Un permiso de cambio de configuración tiene impacto alto porque permite alterar los datos que el SCM usa; documentá una asignación excesiva, pero no la pruebes ni la aproveches.

Ejemplo trabajado

*(Ejemplo **ilustrativo**, no observado en un Windows real. Los nombres y valores son de un servicio ficticio de laboratorio.)*

```
Nombre: ReversingLabSvc
Tipo: own-process (servicio Win32)
Tipo de inicio: demand (manual)
Estado: STOPPED
Cuenta: NT AUTHORITY\LocalService
ImagePath: C:\ReversingLab\SvcLab.exe
Objeto servicio: Administrators – control; Users – lectura
Carpeta del binario: solo administradores pueden modificarla
```

Lectura: es un servicio didáctico de proceso propio, detenido y de inicio manual. La cuenta elegida tiene menos privilegios que `LocalSystem`; la ruta y los permisos son coherentes con esa práctica. Esta ficha no demuestra que un servicio real sea seguro: cada hallazgo debe corroborarse con la configuración efectiva y el contexto del equipo.

Práctica segura: servicio propio y limpieza

Realizá el ejercicio solo en una VM descartable con snapshot. Realizá el ejercicio en una VM descartable con snapshot. Compilá el servicio didáctico desde el archivo `plantillas_codigo/svclab_min.c` (**Plantilla 7 del Kit de Código**), que implementa `StartServiceCtrlDispatcherW`, `ServiceMain` y `RegisterServiceCtrlHandlerExW` para Windows x86/x64:

```
cl /Zi /O1 /DUNICODE /D_UNICODE svclab_min.c advapi32.lib /Fe:C:\ReversingLab\SvcLab.exe
```

(O con MinGW: `gcc -O1 -g -municode svclab_min.c -ladvapi32 -o C:\ReversingLab\SvcLab.exe`).

Materiales y resultado esperado: VM/build registrados, snapshot recuperable, `sc.exe`, AccessChk de Sysinternals y el binario propio identificado por hash. El resultado esperado es registrar un servicio manual, verlo pasar a `RUNNING` y `STOPPED`, y confirmar que desaparece después de solicitar su eliminación. Guardá el build de Windows, hash, salidas de consulta y evidencia de limpieza; no se fija una versión de herramienta que no se haya probado.

Paso 1 — Preparar. Prepará `C:\ReversingLab` y colocá allí el binario propio. No uses una ruta de red ni un servicio de Windows.

Paso 2 — Registrar. En una consola elevada de la VM, registrá el servicio con inicio manual y una cuenta de menor privilegio:

```
   sc.exe create ReversingLabSvc binPath= "C:\ReversingLab\SvcLab.exe" start= demand obj= "NT
AUTHORITY\LocalService"
```

Paso 3 — Inspeccionar. Consultá su configuración y estado; auditá la DACL del objeto servicio sin modificarla:

```
   sc.exe qc ReversingLabSvc
   sc.exe query ReversingLabSvc
   accesschk.exe -v -c ReversingLabSvc
```

Paso 4 — Probar el ciclo de vida. Iniciá y detené solo `ReversingLabSvc`. Observá los cambios de estado y anotá la cuenta y la ruta reportadas.

Paso 5 — Limpiar. Detené el servicio si sigue activo y solicitá eliminar únicamente el servicio de laboratorio:

```
   sc.exe stop ReversingLabSvc
   sc.exe delete ReversingLabSvc
   sc.exe query ReversingLabSvc
```

`sc.exe delete` puede dejarlo marcado para borrar si todavía está activo o si hay una consola con un handle abierto. Cerrá `services.msc` y las herramientas que lo estén consultando; esperá a que desaparezca o revertí el snapshot. Confirmá que `sc.exe query ReversingLabSvc` ya no encuentra el servicio y recién entonces borrá `C:\ReversingLab`.

No cambies permisos, configuración ni cuenta de otros servicios. Si no tenés un binario propio que implemente el contrato de servicio, hacé el análisis como ejercicio de lectura con la ficha ilustrativa y dejá constancia de esa limitación.

Entregable

Una ficha con tipo, inicio, estado, cuenta, ruta, resultado de la inspección de permisos y evidencia de limpieza. Indicá si trabajaste con un servicio compilado o solo con el ejemplo ilustrativo. No incluyas datos personales de la VM.

Preguntas de repaso

▼ **¿Qué responsabilidad tiene el SCM?**

> Mantiene el registro de servicios, los inicia, conserva su estado y envía solicitudes de control.

▼ **¿En qué se diferencian el tipo de inicio y el estado?**

> El tipo de inicio determina cuándo puede comenzar; el estado indica si está detenido, iniciándose, activo, deteniéndose o pausado.

▼ **¿Un ejecutable de consola se vuelve servicio al registrarlo?**

> No. El programa debe implementar el dispatcher de servicios, `ServiceMain` y el handler de controles.

▼ **¿Qué diferencia hay entre el objeto servicio y los permisos del archivo ejecutable?**

> La DACL del objeto controla operaciones sobre el servicio; la ACL del archivo y sus carpetas controla quién puede leer, ejecutar o modificar esos archivos. Se auditan por separado.

▼ **¿Qué significa que `sc.exe delete` no lo haya quitado inmediatamente?**

> Puede estar marcado para eliminación porque sigue activo o porque otro programa conserva un handle. Cerrá las consolas que lo consultan, esperá y verificá su ausencia antes de borrar los archivos.

Puerta de salida (optativa)

Podés explicar SCM, `ServiceMain`, tipo de inicio y estado; leer cuenta, ruta y permisos; distinguir un proceso propio de un servicio hospedado; y demostrar que el servicio de prueba desapareció o declarar que solo completaste el ejercicio ilustrativo.

Fuentes

- **Conocimiento externo:** Service Control Manager, Service Programs, ServiceMain, Service User Accounts y Service Security and Access Rights.
- **Herramientas:** `sc.exe create`, `sc.exe delete` y AccessChk.
- **Hospedaje:** Service host grouping in Windows. Los detalles de agrupación dependen de la versión de Windows.
- **Fundamentos previos:** ACL y token en 2D.1; representación del registro en 2D.3.

Etiquetas

Núcleo conceptual: marco de token/ACL (2D.1) y referencia al registro (2D.3). **Conocimiento externo:** SCM, estados, cuentas y servicios hospedados. **Ejemplo ilustrativo:** ficha de `ReversingLabSvc`. **Laboratorio verificado:** no; requiere un binario propio compilado como servicio.

Task Scheduler: leer tareas y detectar cambios

> **Rama optativa 2D**
>
> Lección optativa de análisis defensivo. La práctica inspecciona tareas existentes y exporta su definición; no crea, modifica, ejecuta ni oculta tareas.

> **Propósito**
>
> Entender de qué partes se compone una tarea programada y reconstruir, a partir de sus metadatos y XML, **qué se ejecuta, cuándo y bajo qué identidad**. Aprender a separar una configuración poco habitual de una conclusión de compromiso.

Prerrequisitos

- *Obligatorios:* registro y contexto de seguridad (2D.1, 2D.3).
- *Recomendados:* servicios y SCM (2D.4); procesos (1.3).

Partes de una tarea

Una tarea registrada combina una **acción** con uno o más **triggers** y ajustes. La definición también describe el **principal** (la identidad y el nivel con que se ejecuta) y datos de registro como nombre, autor, descripción y ruta lógica (URI). El XML permite revisar estos elementos juntos.

- **Trigger:** condición temporal o de evento que solicita la ejecución, por ejemplo, un horario o el inicio de sesión.
- **Action:** lo que Task Scheduler ejecuta y sus argumentos. Revisá ruta, firma y relación con el software instalado.
- **Principal / contexto:** cuenta, tipo de inicio de sesión y nivel de ejecución. `RunLevel` no concede privilegios por sí mismo; importa junto con la identidad, sus derechos y la configuración de la tarea.
- **Settings:** habilitación, restricciones, límites y propiedad `Hidden`.
- **RegistrationInfo:** autor, descripción, fecha, URI y otros datos administrativos.

Visibilidad y auditoría

`Hidden=true` indica que la tarea no se muestra normalmente en la interfaz; no la vuelve invisible para un administrador ni demuestra que sea maliciosa. Una tarea puede ser legítima y poco conocida. Evaluá el conjunto: acción y argumentos, firma y ubicación del ejecutable, trigger, principal, nivel, autor, permisos y coherencia con el software instalado.

La revisión es **de solo lectura**. Podés usar el Programador de tareas para abrir la pestaña de propiedades y exportar el XML, o consultar una tarea específica con herramientas de Windows. No deshabilites ni borres una tarea desconocida durante el análisis; preservá los datos y pedí revisión al administrador responsable.

Ejemplo trabajado

*(Fragmento **ficticio e incompleto**, solo para aprender a leer campos; no es un XML para registrar.)*

```
<RegistrationInfo>
  <Author>Reversing Lab</Author>
  <Description>Comprobación local de estado</Description>
</RegistrationInfo>
<Triggers>
  <TimeTrigger><Enabled>false</Enabled></TimeTrigger>
</Triggers>
<Principals>
  <Principal id="Author">
    <UserId>PC-LAB\alumno</UserId>
    <RunLevel>LeastPrivilege</RunLevel>
  </Principal>
</Principals>
<Settings><Hidden>false</Hidden></Settings>
<Actions Context="Author">
  <Exec>
    <Command>C:\ReversingLab\status.exe</Command>
    <Arguments>──once</Arguments>
  </Exec>
</Actions>
```

Lectura: la acción apunta a un ejecutable local y lleva el argumento `--once`; el trigger está deshabilitado en este fragmento; el principal es un usuario de laboratorio con nivel `LeastPrivilege`; y `Hidden` es falso. Como el ejemplo está incompleto y ficticio, no permite registrar ni evaluar una tarea real.

Práctica segura: inspección de solo lectura

En una VM de laboratorio o en un equipo donde tengas autorización, elegí una tarea conocida y no crítica del sistema o de una aplicación instalada. No la ejecutes ni cambies su configuración.

1. Abrí sus propiedades en Task Scheduler y anotá nombre/URI, autor, descripción y estado habilitado/deshabilitado.
2. Revisá trigger, acción y argumentos. Confirmá si el archivo existe y consultá su firma sin abrirlo ni ejecutarlo.
3. Anotá principal, tipo de inicio de sesión, nivel de ejecución y si `Hidden` figura en las propiedades o en el XML.
4. Exportá el XML a una carpeta temporal del laboratorio y revisá los mismos campos. Si vas a compartir el informe, quitá nombres de usuario, rutas personales y otros datos privados.
5. Compará la tarea con la documentación del producto o con otra referencia confiable. Si algo no coincide, describilo como **anomalía para investigar**, no como prueba de malware.
6. Eliminá solo la copia exportada si ya no la necesitás. No borres ni deshabilites la tarea analizada.

Entregable

Una ficha con URI/nombre, autor, trigger, acción y argumentos, principal, nivel, valor `Hidden`, procedencia conocida y fuente usada para la comparación. Cerrá con una conclusión calibrada: normal, no explicado o requiere revisión; indicá qué evidencia falta.

Preguntas de repaso

▼ **¿Qué diferencia hay entre trigger y acción?**

> El trigger define **cuándo** se solicita ejecutar la tarea; la acción define **qué** programa o comando ejecuta Task Scheduler.

▼ **¿Qué datos del principal conviene leer?**

> La identidad, el tipo de inicio de sesión y el nivel de ejecución; deben interpretarse junto con los derechos de esa cuenta.

▼ **¿`Hidden=true` significa que la tarea es invisible o maliciosa?**

> No. Cambia su visibilidad normal en la interfaz, pero no la oculta a un administrador ni demuestra intención maliciosa.

▼ **¿Qué hacés si encontrás una acción o ruta que no reconocés?**

> Registrás los datos, conservás la evidencia y contrastás con una fuente confiable o con el administrador; no ejecutás, deshabilitás ni borrás la tarea durante el análisis.

Puerta de salida (optativa)

Podés explicar trigger, acción, principal y ajustes; leer esos campos en XML; describir el efecto limitado de `Hidden`; y fundamentar una anomalía sin convertirla en una acusación no corroborada.

Fuentes

- **Conocimiento externo:** About the Task Scheduler, Tasks, Security Contexts for Running Tasks, Task Registration Information y Task Scheduler schema.
- **Propiedad de visibilidad:** TaskSettings.Hidden.
- **Consulta:** `schtasks.exe`.
- **Fundamentos previos:** permisos del registro en 2D.3, contexto de tokens en 2D.1 y servicios en 2D.4.

Etiquetas

Núcleo conceptual: marco de lectura de configuración y persistencia defensiva. **Conocimiento externo:** Task Scheduler, XML, contextos, triggers y acciones. **Ejemplo ilustrativo:** fragmento ficticio no registrable. **Laboratorio verificado:** no; la práctica propuesta es de inspección de solo lectura.

COM: interfaces, IUnknown y vida de los objetos

> **Rama optativa 2D**
>
> Lección conceptual y de análisis. El ejemplo es un componente propio; no se registra ni modifica software ajeno.

> **Propósito**
>
> Reconocer cómo un cliente pide una interfaz COM, cómo interpretar el resultado y cómo seguir la llamada hasta la implementación sin confundir una interfaz con una clase C++ concreta. Al terminar deberías poder decir, frente a una llamada indirecta, **qué contrato** se está usando, **en qué apartamento** vive el objeto y si el puntero que tenés delante es el objeto o un **proxy**.

Prerrequisitos

- *Obligatorios:* punteros, estructuras y convención de llamada (1.5, 2.5); objetos y handles (2D.1).
- *Recomendados:* C++ profundo (2.18); memoria y loader (2D.2); procesos e hilos (1.3).

Conceptos

- **CLSID** identifica una clase COM; **IID** identifica un contrato de interfaz. Ambos son GUID, pero responden preguntas distintas.
- `IUnknown` es la base de las interfaces COM: `QueryInterface`, `AddRef` y `Release`. La consulta devuelve `S_OK` si el objeto ofrece el IID o `E_NOINTERFACE` si no.
- **HRESULT** comunica éxito o fallo. No lo interpretes como un booleano: se usa `SUCCEEDED(hr)` / `FAILED(hr)`; algunos resultados no negativos (como `S_FALSE`) indican éxito informativo.
- Una **vtable** contiene punteros a métodos en orden contractual. En una interfaz derivada, los métodos de `IUnknown` preceden a los propios.
- El **reference counting** administra la vida del objeto. Una referencia obtenida debe equilibrarse con `Release`; después de la última liberación, ese puntero no se reutiliza.
- Una llamada puede atravesar una **proxy/stub** si el objeto vive en otro apartamento o proceso. El puntero visible al cliente no tiene por qué apuntar directamente al código servidor.

Inicialización por hilo: `CoInitializeEx` y `CoUninitialize`

COM no se "enciende" una vez por proceso: **cada hilo** que usa interfaces COM llama a `CoInitializeEx` y, con eso, declara su modelo de concurrencia y queda asociado a un **apartamento**. El primer parámetro es reservado (`NULL`); el segundo lleva exactamente uno de `COINIT_APARTMENTTHREADED` o `COINIT_MULTITHREADED`, más opciones como `COINIT_DISABLE_OLE1DDE`.

Lo que conviene registrar al analizar un binario:

Resultado	Significado documentado	Qué implica para el análisis
`S_OK`	COM se inicializó en ese hilo.	Debe haber un `CoUninitialize` equilibrado antes de que el hilo termine.
`S_FALSE`	COM ya estaba inicializado en ese hilo con el mismo modelo.	Es éxito; **también** debe equilibrarse con `CoUninitialize`.
`RPC_E_CHANGED_MODE`	Un llamado previo fijó un modelo incompatible para ese hilo.	Es fallo: el hilo **sigue** en el modelo anterior y no corresponde un `CoUninitialize` por este llamado.

Microsoft indica además que no se llame a `CoInitialize(Ex)` ni a `CoUninitialize` desde `DllMain`. Cuando veas una DLL que lo hace, anotalo como hallazgo de calidad, no como intención maliciosa.

Apartamentos: STA, MTA y por qué existe el marshaling

Un **apartamento** agrupa objetos COM y los hilos que pueden llamarlos **directamente**. Cada objeto vive en exactamente un apartamento; un hilo de otro apartamento que quiera llamarlo debe pasar por un **proxy**.

- **STA (single-threaded apartment)**: un único hilo. Las llamadas que llegan desde otros apartamentos se entregan a través de la **cola de mensajes** de ese hilo, así que se serializan y solo avanzan cuando el hilo bombea mensajes. Si un hilo STA se bloquea en una espera sin bombear (por ejemplo, `WaitForSingleObject`), las llamadas entrantes no se entregan y aparece un interbloqueo. La documentación recomienda `CoWaitForMultipleHandles` o `MsgWaitForMultipleObjects` en esos hilos.
- **MTA (multithreaded apartment)**: uno o más hilos; a lo sumo **una** MTA por proceso. Cualquier hilo de la MTA puede llamar directamente a los objetos de la MTA, y **el objeto** debe sincronizarse por su cuenta (secciones críticas, mutex, etc.).
- Un proceso puede tener **cero o más STA** y **cero o una MTA**. También existe el apartamento **neutral** (NA), que aparece en la próxima lección a través de `ThreadingModel=Neutral`.

Marshaling es empaquetar la llamada (parámetros, punteros de interfaz) para que cruce un límite de apartamento, de proceso o de equipo. Existe porque las reglas anteriores solo se cumplen si nadie copia un puntero de interfaz "crudo" a un hilo de otro apartamento. Para pasar una interfaz entre apartamentos del mismo proceso se usan mecanismos explícitos —la Global Interface Table o, en código moderno (Windows 8.1+), `RoGetAgileReference`— que devuelven un puntero directo si se resuelve en el mismo apartamento o un proxy si no.

> **Lectura defensiva del modelo de hilos**
>
> En un análisis, el modelo de hilos sale de la evidencia: el flag pasado a `CoInitializeEx` (si lo ves en un argumento constante), el `ThreadingModel` del servidor (2D.7) y la presencia de un bucle de mensajes. Si solo tenés una de las tres, escribí "probable STA/MTA" y por qué.

Proxies y stubs: cuando el puntero no es el objeto

Un **proxy** vive en el espacio del llamador y, para el cliente, *es* el objeto: implementa la misma interfaz, empaqueta los parámetros en un búfer y los entrega al canal (RPC). Del otro lado, un **stub** desempaqueta y llama a la implementación real. El proxy es un objeto compuesto: un *proxy manager* (que implementa el

`IUnknown` controlador) y un *interface proxy* por cada interfaz expuesta a ese cliente. El código de esos interface proxies puede estar en una DLL distinta, que el registro asocia al IID mediante `HKCR\Interface\{IID}\ProxyStubClsid32`.

Cómo reconocerlo en el depurador, sin suponer nombres:

1. Tomá el puntero de interfaz `p` y leé su primer campo: es la dirección de la vtable.
2. Resolvé a qué **módulo** pertenecen la vtable y sus primeras entradas (`lm` / `ln` en WinDbg, o el mapa de memoria de tu depurador).
3. Si apuntan al **módulo servidor**, probablemente es un puntero directo. Si apuntan a módulos del runtime COM/RPC o a una DLL de proxy/stub, es un proxy. Los nombres concretos de esos módulos y funciones dependen de la versión de Windows y de los símbolos disponibles: registrá lo que veas, no lo que esperabas ver.
4. Confirmalo con el contexto: ¿el objeto se creó en otro proceso (`LocalServer32`) o en otro apartamento? ¿El hilo cliente es STA y el servidor `Free`?

Consecuencia práctica: poner un breakpoint en la entrada de la vtable de un proxy **no** te lleva a la implementación; te lleva al código de marshaling. La implementación se sigue en el **proceso o apartamento del servidor**.

Marshaling a medida y el *free-threaded marshaler*

El marshaling estándar no es la única opción. Un objeto puede implementar `IMarshal` y decidir por su cuenta cómo se empaqueta una referencia a sí mismo (*custom marshaling*). Para el analista, esto significa que el proxy que aparece del otro lado puede no ser el estándar: puede ser un objeto escrito por el autor del componente, con su propio protocolo.

El caso más común es el **free-threaded marshaler** (FTM), que se obtiene con `CoCreateFreeThreadedMarshaler` y se **agrega** al objeto (ver "Agregación y *tear-offs*", más abajo). Cuando alguien marshalea una interfaz de ese objeto, el FTM mira el contexto de destino:

- si el destino es **otro apartamento del mismo proceso** (`MSHCTX_INPROC`), copia el puntero real: el otro apartamento recibe un **puntero directo**, sin proxy;
- si el destino es **otro proceso u otro equipo**, delega en el marshaling estándar: ahí **sí** hay proxy y stub.

Tres límites que conviene escribir en el informe:

1. El FTM **no elimina** el marshaling entre procesos; solo evita el proxy dentro del proceso.
2. Al eliminar el proxy, también elimina la sincronización que el apartamento le daba al objeto: **el objeto tiene que manejar su propia concurrencia**. Microsoft lo describe como una violación calculada de las reglas de COM, con riesgo de comportamiento indefinido si el objeto guarda punteros directos a objetos que no usan el FTM o referencias a proxies de otros apartamentos.
3. En código nuevo (Windows 8.1+), Microsoft recomienda `RoGetAgileReference` en lugar del FTM.

Señales en un binario: una llamada a `CoCreateFreeThreadedMarshaler` en la construcción del objeto y un `QueryInterface` que responde a `IID_IMarshal` delegando en otro objeto. Si las ves, "no hay proxy entre apartamentos" deja de ser una anomalía y pasa a ser el comportamiento esperado; lo que hay que revisar es si el objeto se sincroniza bien.

Reglas de `QueryInterface` que podés usar como evidencia

Microsoft fija tres reglas para implementar `QueryInterface`:

- **Identidad**: para un mismo objeto, `QueryInterface(IID_IUnknown)` devuelve **siempre el mismo valor de puntero**. Por eso, para saber si dos interfaces pertenecen al mismo objeto, se comparan los resultados de pedir `IID_IUnknown` a cada una, no los punteros de interfaz originales (que suelen diferir).
- **Conjunto estático**: si un IID devolvió `S_OK` una vez, no puede devolver `E_NOINTERFACE` después en el mismo objeto, y viceversa.
- **Reflexiva, simétrica y transitiva**: desde cualquier interfaz se puede pedir la misma interfaz, volver a la anterior y llegar a cualquier otra que el objeto ofrezca.

Uso en reversing: si observás un objeto que ofrece un IID "a veces", o dos `IUnknown` distintos para lo que creías un solo objeto, no es una curiosidad. Puede ser un bug del componente, que en realidad son objetos distintos o que estás mirando un proxy. Registralo y explicá cuál hipótesis sostenés y con qué evidencia.

Agregación y *tear-offs*: un objeto, varias implementaciones

La regla de identidad habla del **objeto COM**, no de una clase C++. Dos técnicas hacen que un objeto COM esté formado por varias piezas de código, y las dos se ven raras si no las conocés.

Agregación. Un objeto **externo** expone interfaces de un objeto **interno** como si fueran propias. Para que la identidad se mantenga:

- al crear el interno, el externo pasa su propio `IUnknown` como `pUnkOuter` en `CoCreateInstance` y **debe pedir** `IID_IUnknown`; si pide otra interfaz, la creación falla (un objeto que no admite agregación devuelve `CLASS_E_NOAGGREGATION`);
- el interno tiene **dos** `IUnknown`: uno propio (*no delegante*), que controla su vida y que solo conoce el externo, y el de sus demás interfaces, que **delega** `QueryInterface`, `AddRef` y `Release` en el `IUnknown` **controlador** del externo;
- el cliente siempre ve un solo objeto: pedir `IID_IUnknown` desde cualquier interfaz devuelve el `IUnknown` del externo.

Cómo se ve en un binario: un `CoCreateInstance` (o un constructor) que recibe un `pUnkOuter` no nulo junto con `IID_IUnknown`, y métodos `QueryInterface` / `AddRef` / `Release` que solo hacen un salto indirecto a través de un puntero guardado (el controlador). Si seguís la vtable de una interfaz agregada, sus primeros tres slots apuntan a código del **interno** que solo reenvía al controlador del **externo**, y el resto, a la implementación del interno, que puede estar en otra DLL.

Tear-offs. Algunas bibliotecas (ATL, por ejemplo, con `COM_INTERFACE_ENTRY_TEAR_OFF`) crean la implementación de una interfaz poco usada **recién cuando alguien la pide**, como un objeto auxiliar que delega `IUnknown` en el principal. Consecuencias: dos `QueryInterface` del mismo IID pueden devolver **punteros de interfaz distintos** (dos tear-offs), sin violar la identidad, porque ambos devuelven el mismo `IUnknown`; y la memoria de esa interfaz puede no existir antes del primer pedido.

Para el informe: si los punteros de interfaz difieren pero `IID_IUnknown` coincide, anotá "compatible con agregación o tear-off" y buscá el patrón de delegación antes de afirmar cuál es.

Monikers y la tabla de objetos en ejecución (ROT)

Un **moniker** es un objeto (interfaz `IMoniker`) que **nombra** a otro objeto y sabe cómo obtenerlo: a eso se le llama *binding*. Hay monikers de archivo, de ítem, de clase, compuestos y otros; OLE los usa para conectar y activar objetos en el mismo equipo o en la red, por ejemplo en los vínculos de documentos compuestos.

La **ROT** (*Running Object Table*, interfaz `IRunningObjectTable`, obtenida con `GetRunningObjectTable`) es una tabla por estación de trabajo con pares (moniker, objeto en ejecución). Cuando un cliente resuelve un moniker, se consulta la ROT para conectarse a la instancia que ya corre en vez de crear otra. Un objeto se **registra** en la ROT al cargarse y debe **revocar** su registro al terminar.

Qué aporta al análisis:

- Un cliente que obtiene un objeto con `CoGetObject`, `BindMoniker` / `IMoniker::BindToObject` o consultando la ROT **no** pasa necesariamente por `CoCreateInstance`: si buscás solo activaciones, te lo perdés.
- Un servidor que llama a `IRunningObjectTable::Register` está publicando un objeto para que otros procesos lo encuentren por nombre. Anotá qué moniker usa y con qué flags; es parte de su superficie.
- Enumerar la ROT (`EnumRunning`) es una operación de lectura; conectarse a un objeto encontrado ahí **no** lo es, porque ejecuta código del servidor. En la práctica de esta lección solo se anota lo que el **binario propio** registra o consulta.

> **Recuadro conceptual: COM+**
>
> COM+ (Component Services) agrupa componentes COM en **aplicaciones COM+** que se administran como una unidad: alcance de despliegue, configuración y seguridad común, atributos que el autor del componente no escribió (transacciones, sincronización), y procesos anfitriones administrados (`DllHost.exe`) que cargan las DLL bajo demanda. Su configuración vive en un **catálogo** propio, no solo en las claves de 2D.7. Si en un análisis ves un `DllHost.exe` con una DLL de componente o referencias a contextos COM+, sabé que existe esta capa; esta lección **no** la desarrolla y no conviene sacar conclusiones de seguridad sin consultar la configuración del catálogo.

IID, orden de métodos, slot y convención de llamada

El **slot** de vtable se deduce así: `slot = offset / tamaño_de_puntero`. El nombre del método **no** sale del offset; sale de combinar cuatro datos:

1. **IID** usado para obtener el puntero (lo ves como argumento de `QueryInterface` o `CoCreateInstance`).
2. **Definición de la interfaz** (IDL, header, type library o PDB) con el **orden** de los métodos, incluida la herencia (`IUnknown` aporta los slots 0–2).
3. **Arquitectura**: 4 bytes por slot en x86, 8 bytes en x64.
4. **Convención de llamada**: en x86, los métodos COM usan `__stdcall` y `this` va en la pila como primer argumento; en x64 hay una sola convención y `this` viaja en `RCX`, seguido de `RDX`, `R8` y `R9`.

Slot	Método (`IStatus : IUnknown`)	Offset x86	Offset x64
0	`QueryInterface`	`+0x00`	`+0x00`
1	`AddRef`	`+0x04`	`+0x08`
2	`Release`	`+0x08`	`+0x10`
3	`GetState`	`+0x0C`	`+0x18`

Un `call [rax+18h]` en x64 es "slot 3"; solo es `IStatus::GetState` si el puntero vino de un `QueryInterface` / `CoCreateInstance` con `IID_IStatus` (o de una interfaz que deriva de ella) **y** la definición confirma el orden.

Ejemplo trabajado (x64, MSVC)

*(Ejemplo ilustrativo de un componente propio `BlocStatus` compilado para **x64**. Las instrucciones muestran el patrón que genera MSVC para llamadas COM; las posiciones de pila y los resultados no son una captura de un binario real.)*

Contrato, tal como lo declararía el IDL propio:

```
interface IStatus : IUnknown        // IID_IStatus
{
    HRESULT GetState([out] ULONG* state);   // slot 3
};
```

Llamadas del cliente, en el orden que se ve en el desensamblado. Primero, inicialización y creación:

```
; hilo cliente
xor   ecx, ecx                  ; pvReserved = NULL
mov   edx, 2                    ; COINIT_APARTMENTTHREADED  → hilo STA
call  CoInitializeEx            ; anotar HRESULT en eax

lea   rcx, CLSID_BlocStatus     ; rclsid
xor   edx, edx                  ; pUnkOuter = NULL
mov   r8d, 1                    ; CLSCTX_INPROC_SERVER
lea   r9,  IID_IStatus          ; riid
lea   rax, [rsp+40h]
mov   [rsp+20h], rax            ; ppv (5.° argumento, en la pila tras el shadow space)
call  CoCreateInstance
```

Después, las llamadas a través de la vtable y el cierre:

```
mov   rcx, [rsp+40h]              ; this = p (IStatus*)
mov   rax, [rcx]                  ; rax = vptr
lea   rdx, [rsp+48h]              ; &state
call  qword ptr [rax+18h]         ; slot 3 = 0x18 / 8 → IStatus::GetState

mov   rcx, [rsp+40h]
mov   rax, [rcx]
lea   rdx, IID_IResettable
lea   r8,  [rsp+50h]
call  qword ptr [rax]             ; slot 0 → QueryInterface

mov   rcx, [rsp+40h]
mov   rax, [rcx]
call  qword ptr [rax+10h]         ; slot 2 → Release

call  CoUninitialize              ; equilibra el CoInitializeEx anterior
```

Ficha de lectura, separando qué sale del código y qué se infiere:

Dato	Fuente	Tipo
El hilo pide STA (`2` = `COINIT_APARTMENTTHREADED`).	Constante en `EDX` + definición del enum.	Observable en el binario.
`[rax+18h]` es el slot 3.	Aritmética x64: `0x18 / 8`.	Observable.
El slot 3 es `GetState`.	IID pasado en `R9` + IDL propio.	Inferencia respaldada por el contrato.
`E_NOINTERFACE` para `IID_IResettable`.	Valor de `EAX` tras la llamada al slot 0.	Se obtiene **solo** ejecutando en el laboratorio; acá es ilustrativo.
`p` apunta directo a la implementación.	Vtable dentro de `BlocCom.dll`, servidor in-proc compatible con STA.	Inferencia; se confirma resolviendo el módulo de la vtable.

Variante x86, para contraste: el mismo `GetState` se vería como `push` de `&state`, `push` de `p` (el `this`), `mov eax,[p]` y `call [eax+0Ch]`, porque cada slot ocupa 4 bytes y `__stdcall` pasa `this` en la pila. No mezcles las dos tablas al leer un binario.

Práctica

Con un componente COM **propio** o un binario didáctico ya incluido en una VM:

1. Anotá la arquitectura del cliente (x86/x64), CLSID e IIDs documentados, y cuál interfaz obtiene el cliente.
2. Ubicá `CoInitializeEx` en cada hilo que use COM: flag pasado, HRESULT y dónde está el `CoUninitialize` correspondiente. Si no lo encontrás, decilo.
3. En el depurador, seguí una llamada a `QueryInterface`; registrá IID solicitado, HRESULT y destino de retorno.
4. Identificá la dirección de la vtable, el **módulo** al que pertenece y si el puntero es directo o un proxy. Relacioná un método con su prototipo usando la tabla slot/offset de tu arquitectura. No invoques métodos desconocidos.
5. Pedí `IID_IUnknown` desde dos interfaces del mismo objeto y compará los punteros (regla de identidad). Si los punteros de interfaz difieren, buscá el patrón de agregación o tear-off.
6. Buscá en el binario `CoCreateFreeThreadedMarshaler`, respuestas a `IID_IMarshal`, `pUnkOuter` no nulos y llamadas a la ROT. Anotá cada hallazgo como observación y su consecuencia como inferencia.

7. Dibujá el ciclo de referencias: creación, consultas, `AddRef` / `Release` y liberación final. Si no hay símbolos, marcá las inferencias.

Entregable

Diagrama cliente → (proxy/stub si corresponde) → interfaz → implementación; tabla CLSID/IID/HRESULT; tabla slot/offset/método con la arquitectura declarada; apartamento de cada hilo y evidencia de una llamada. Separá lo observado de lo inferido; no afirmes reference count exacto si el binario no lo expone.

Repaso

▼ **¿CLSID e IID significan lo mismo?**

> No. CLSID identifica la clase; IID, una interfaz/contrato que el objeto puede ofrecer.

▼ **¿Qué comunica `E_NOINTERFACE`?**

> Que el objeto consultado no implementa el IID solicitado; no prueba que la clase COM no exista.

▼ **¿Por qué una vtable no basta para nombrar métodos?**

> Los slots solo son direcciones. Hace falta el IID, el contrato con su orden, la arquitectura (tamaño de slot) y la convención de llamada para asociarlos con nombres y prototipos.

▼ **En x64, ¿qué slot es `call [rax+18h]` y dónde estaría ese mismo slot en x86?**

> Slot 3 (`0x18 / 8`). En x86 el slot 3 está en `+0x0C` (`3 × 4`).

▼ **¿Qué riesgo hay después de `Release`?**

> Si se liberó la última referencia, el objeto pudo destruirse; reutilizar el puntero sería usar una referencia inválida.

▼ **`CoInitializeEx` devuelve `S_FALSE`. ¿Hace falta `CoUninitialize`?**

> Sí. `S_FALSE` es éxito (COM ya estaba inicializado en ese hilo) y debe equilibrarse igual. `RPC_E_CHANGED_MODE`, en cambio, es fallo y no se equilibra.

▼ **¿Por qué un hilo STA que espera con `WaitForSingleObject` puede colgar una llamada COM?**

> Porque las llamadas entrantes a objetos de una STA se entregan por la cola de mensajes de ese hilo; si no bombea mensajes, nunca llegan.

▼ **¿Cómo comprobás que dos punteros de interfaz pertenecen al mismo objeto?**

> Pidiendo `IID_IUnknown` a cada uno y comparando los resultados: la regla de identidad exige el mismo valor.

▼ **Un breakpoint en la entrada de vtable cae en código del runtime COM/RPC y no en tu DLL. ¿Qué estás viendo?**

> Probablemente un proxy: la llamada se empaqueta y la implementación corre en otro apartamento o proceso.

▼ **Un objeto agrega el free-threaded marshaler. ¿Hay proxy cuando se pasa a otro apartamento del mismo proceso? ¿Y a otro proceso?**

> Dentro del proceso, no: el otro apartamento recibe el puntero directo y el objeto debe sincronizarse solo. Hacia otro proceso, sí: el FTM delega en el marshaling estándar.

▼ **Dos `QueryInterface` del mismo IID devuelven punteros distintos, pero `IID_IUnknown` coincide. ¿Se rompió la identidad?**

> No necesariamente. Es compatible con tear-offs o agregación; la identidad se juzga por el `IUnknown`, no por cada puntero de interfaz.

▼ **¿Por qué al crear un objeto agregado hay que pedir `IID_IUnknown`?**

> Porque el externo necesita el `IUnknown` no delegante del interno para controlar su vida; las demás interfaces del interno delegan en el externo.

▼ **¿Por qué buscar solo `CoCreateInstance` puede no alcanzar para encontrar cómo un cliente obtiene un objeto?**

> Porque puede obtenerlo resolviendo un moniker o consultando la ROT, que conecta con una instancia ya en ejecución.

Puerta optativa

Podés distinguir CLSID/IID, interpretar HRESULT (incluidos los de `CoInitializeEx`), explicar qué cambia entre STA y MTA, reconocer un proxy por el módulo de su vtable, explicar cuándo el free-threaded marshaler evita un proxy y cuándo no, reconocer agregación y tear-offs sin confundirlos con una violación de identidad, aplicar la regla de identidad y nombrar un slot de vtable con el contrato y la arquitectura correctos.

Lo que queda fuera

La implementación de `IMarshal` propio, el formato de una referencia marshaleada (OBJREF), el detalle interno de los proxies estándar, los contextos COM+ y el catálogo COM+ (solo como recuadro), la escritura de monikers propios y WinRT no se desarrollan. El apartamento neutral solo se menciona. La lección prepara para leer evidencia COM; no es una cobertura completa de COM.

Fuentes

- Microsoft Learn: IUnknown, QueryInterface, reglas de QueryInterface, vida y reference counting, HRESULT.
- Microsoft Learn, hilos y apartamentos: Initializing the COM Library, CoInitializeEx, Processes, Threads, and Apartments, Accessing Interfaces Across Apartments.
- Microsoft Learn, proxies y convención: Proxy, ProxyStubClsid32, x64 calling convention.
- Microsoft Learn, marshaling a medida y composición: In-Process Server Threading Issues, CoCreateFreeThreadedMarshaler, IMarshal, Aggregation, ATL COM_INTERFACE_ENTRY macros (tear-offs).
- Microsoft Learn, monikers y COM+: Monikers, IRunningObjectTable, COM+ Application Overview.
- **Fundamentos previos:** complemento externo a la ruta de C++ y vtables de 2.18.

Etiquetas

Núcleo conceptual: conceptos generales de C++ y vtables. **Conocimiento externo:** COM, apartamentos, marshaling estándar y a medida, free-threaded marshaler, agregación, tear-offs, monikers/ROT, COM+ (recuadro), reglas de `QueryInterface` y convención x64. **Ejemplo ilustrativo:** desensamblado y resultados de `BlocStatus` (patrón MSVC x64, no capturado de un binario). **Laboratorio verificado:** no; usar solo un componente propio/didáctico.

COM: registro, activación y configuración

> **Rama optativa 2D**
>
> El ejercicio inspecciona una clase propia o una instalación conocida. No cambia el registro de componentes ajenos, no ejecuta `DllRegisterServer` / `regsvr32` sobre binarios que no son tuyos y no configura activación automática.

> **Propósito**
>
> Reconstruir cómo COM pasa de un CLSID a un servidor local o dentro del proceso y cómo usar metadatos para confirmar el contrato. El objetivo es poder responder, con evidencia, **qué archivo** atiende una clase, **en qué proceso y apartamento** corre y **por qué** fue ese y no otro.

Prerrequisitos

- *Obligatorios:* 2D.3 Registro, 2D.6 COM.
- *Recomendados:* PE e imports (1.4); ACL y token (2D.1).

El recorrido de una activación

La activación parte de una solicitud de clase y contexto. Con `CoCreateInstance(rclsid, pUnkOuter, dwClsContext, riid, ppv)` el cliente dice **qué clase** quiere, **qué tipos de servidor acepta** (`CLSCTX_INPROC_SERVER`, `CLSCTX_LOCAL_SERVER`, `CLSCTX_REMOTE_SERVER`, combinables) y **qué interfaz** espera recibir. Conceptualmente, la cadena es:

1. **CLSID + contexto de activación.** Si el hilo tiene un *activation context* activo (manifiesto), COM busca primero ahí qué DLL aloja el CLSID. Si no lo encuentra, usa el registro.
2. **Registro efectivo.** `HKEY_CLASSES_ROOT` es una vista combinada de `HKLM\Software\Classes` (máquina) y `HKCU\Software\Classes` (usuario); para una misma subclave, la entrada del usuario tiene prioridad. Además, la vista de 32 o 64 bits del proceso cambia qué subárbol se lee (ver más abajo).
3. **Servidor.** Según `dwClsContext` y lo registrado, COM elige un servidor en proceso (`InprocServer32`), uno local (`LocalServer32`, o un servicio vía `LocalService` del AppID) o uno remoto (2D.8).
4. **Class factory.** COM obtiene el *class object* de la clase: en una DLL, llamando a su export `DllGetClassObject`; en un EXE, esperando a que el servidor registre su class object al arrancar. Ese objeto implementa normalmente `IClassFactory`.
5. **Instancia e interfaz.** `IClassFactory::CreateInstance` crea el objeto y devuelve la interfaz pedida por `riid`. Si el servidor está en otro apartamento o proceso, el cliente recibe un **proxy** (2D.6).

`CoCreateInstance` equivale a `CoGetClassObject` + `IClassFactory::CreateInstance` + liberar la factory. En un binario podés ver cualquiera de las dos formas.

`InprocServer32` y `LocalServer32`

	`InprocServer32`	`LocalServer32`
Qué señala	Ruta de una **DLL** que se carga **en el proceso cliente**.	Línea de comando de un **EXE** que corre en **otro proceso**.
Cómo entrega la factory	Export `DllGetClassObject`.	El EXE registra su class object al arrancar; si no lo hace a tiempo (por defecto, 60 s), la activación expira.
Arquitectura	La DLL debe coincidir con la del cliente (un proceso x64 no carga una DLL x86).	Puede tener otra arquitectura; las llamadas viajan por proxy/stub.
Detalles útiles	Declara `ThreadingModel`.	COM agrega `-Embedding` a la línea de comando. El valor `ServerExecutable` evita ambigüedad de ruta. Si el AppID define `LocalService`, se usa el servicio y `LocalServer32` se ignora.
Riesgo de lectura	Una ruta sin comillas o relativa es ambigua; verificá qué archivo se resolvería.	Igual; la documentación advierte sobre rutas sin comillas en `CreateProcess`.

Una DLL también puede correr fuera de proceso dentro de un **surrogate** si su AppID define `DllSurrogate`. Por eso "es una DLL" no implica automáticamente "corre dentro del cliente".

La ubicación por sí sola no determina si algo es confiable: verificá ruta, firma, propietario, permisos de la carpeta y compatibilidad de arquitectura.

`ThreadingModel`: valores y límites

Solo aplica a servidores **en proceso**. Declara en qué apartamento (2D.6) puede vivir el objeto; COM lo usa para decidir dónde crearlo.

Valor	Apartamento declarado	Si el cliente está en un apartamento incompatible
(ausente)	Solo el **STA principal** (el primero inicializado).	Se crea en el STA principal y el cliente recibe un proxy.
`Apartment`	STA.	Si el cliente está en la MTA, COM crea o usa un **host STA**; el cliente recibe un proxy.
`Both`	STA o MTA.	Se crea en el mismo apartamento del cliente.
`Free`	MTA.	Si el cliente es STA, se crea en la MTA (o en una *host MTA* creada por COM); proxy.
`Neutral`	Apartamento neutral.	Se crea en el apartamento neutral.

Límites que conviene escribir en el informe:

- Es una **declaración del autor**, no una prueba: `Both` o `Free` no demuestran que el código sea seguro entre hilos.
- Debe ser el mismo valor para todos los objetos de ese servidor.
- Explica **por qué aparece un proxy** aun en una activación in-proc: no siempre hay un salto de proceso detrás.

Registro por usuario, por máquina y vistas de 32/64 bits

- **Usuario vs. máquina.** Un CLSID puede estar en `HKCU\Software\Classes\CLSID`, en `HKLM\Software\Classes\CLSID` o en ambos. La vista `HKCR` da prioridad al usuario. La documentación aclara que, si una aplicación corre con derechos de administrador y UAC deshabilitado, el runtime COM ignora la configuración por usuario. Un proceso que atiende a clientes distintos del usuario interactivo (un servicio, por ejemplo) necesita abrir explícitamente la vista de ese usuario con `RegOpenUserClassesRoot`.
- **WOW64.** En Windows 7 y posteriores de 64 bits, `HKLM\Software\Classes\CLSID`, `HKCU\Software\Classes\CLSID` e `…\Interface` están **redirigidas**: un proceso de 32 bits ve otra copia (bajo `Wow6432Node`). En cambio `…\Classes\AppID` está **compartida**, y `Wow6432Node\Typelib` es un enlace a la clave compartida. Consecuencia: el mismo CLSID puede resolver a archivos distintos según la arquitectura del cliente.

Antes de afirmar "el CLSID no está registrado", anotá **qué vista** consultaste (usuario/máquina, 32/64) y con qué herramienta. Regedit de 64 bits muestra `Wow6432Node` como una rama más; una API con `KEY_WOW64_32KEY` o `KEY_WOW64_64KEY` elige la vista explícitamente.

AppID, TypeLib e IDL: qué prueba cada uno

Metadato	Qué aporta	Qué **no** demuestra
AppID (`HKLM\Software\Classes\AppID\{GUID}`, enlazado desde el valor `AppID` del CLSID)	Configuración común de los objetos DCOM de un mismo ejecutable: `LaunchPermission`, `AccessPermission`, `RunAs`, `LocalService`, `DllSurrogate`, `AuthenticationLevel`, `RemoteServerName`, entre otros.	Que esos valores sean los efectivos: si el proceso llama a `CoInitializeSecurity`, sus parámetros reemplazan a los del registro (2D.8). Tampoco prueba que la clase se active remotamente.
TypeLib (`.tlb` o recurso dentro de la DLL; referenciada por `TypeLib` o por `tlbid`)	Descripción binaria de interfaces, métodos, parámetros y tipos; base del marshaling de automatización.	Que la implementación cumpla el contrato, que todos los métodos estén implementados o que no haya métodos no documentados.
IDL (fuente que MIDL compila a headers, typelib y código proxy/stub)	El contrato tal como lo escribió el autor: herencia, orden de métodos, atributos `[in]` / `[out]`.	Que el binario distribuido se haya compilado de **ese** IDL o de esa versión.
`Interface\{IID}` con `ProxyStubClsid32`	Qué proxy/stub se usa para marshalear ese IID.	Qué objeto implementa la interfaz.

Regla práctica: los metadatos te dan **hipótesis de nombres y contratos**; la implementación y la ejecución en laboratorio las confirman o las refutan.

Activación sin registro (registration-free COM)

Una aplicación puede declarar sus clases COM en un **manifiesto** en lugar del registro. En un manifiesto de ensamblado, el elemento `file` (la DLL) contiene uno o más `comClass` con atributos como `clsid`, `threadingModel`, `progid` y `tlbid`, y puede incluir `typelib` y `comInterfaceProxyStub`. Cuando el hilo tiene ese **contexto de activación** activo, `CoCreateInstance` y `CLSIDFromProgID` lo consultan **antes** que el registro; si el contexto no contiene la información, COM recurre al registro como siempre. Los contextos de activación son por hilo, y COM los propaga al hilo anfitrión al cargar la DLL.

Para el análisis, esto significa que **una búsqueda en el registro puede dar vacío y la activación igual funcionar**. Dónde mirar, en modo lectura:

- manifiesto de aplicación embebido como recurso del EXE o archivo `.manifest` junto al ejecutable;
- manifiestos de ensamblados privados (archivo junto a la DLL o recurso de la DLL);
- llamadas a las APIs de contextos de activación en el cliente.

Ejemplo trabajado: observación frente a inferencia

*(Ejemplo **ilustrativo**: rutas ficticias de un componente propio; las cadenas entre llaves son marcadores didácticos, no GUID utilizables. No es salida de una VM.)*

```
Cliente: BlocCliente.exe (PE32+, x64)
Llamada vista en el desensamblado:
  CoCreateInstance(CLSID {A … 1}, NULL, CLSCTX_INPROC_SERVER|CLSCTX_LOCAL_SERVER, IID {D … 4}, &p)

Vista 64 bits:
  HKCU\Software\Classes\CLSID\{A … 1}   (no existe)
  HKLM\Software\Classes\CLSID\{A … 1}
    AppID           {B … 2}
    InprocServer32  (Default)       C:\ReversingLab\x64\BlocCom.dll
                    ThreadingModel  Both
    TypeLib         {C … 3}
HKLM\Software\Classes\AppID\{B … 2}
    (sin LaunchPermission / AccessPermission / RunAs)
Vista 32 bits (Wow6432Node):
  HKLM\Software\Classes\Wow6432Node\CLSID\{A … 1}
    LocalServer32   "C:\ReversingLab\x86\BlocSrv.exe"
Manifiesto de BlocCliente.exe: sin comClass
```

Afirmación	Tipo	Por qué
El cliente acepta servidor in-proc **o** local.	Observación	Flags de `dwClsContext` en el código.
En la vista de 64 bits, el CLSID resuelve a `BlocCom.dll` (x64).	Observación	Clave leída en esa vista.
No hay registro por usuario ni registro sin manifiesto que lo pise.	Observación	`HKCU` vacío y manifiesto sin `comClass`.
Un cliente **x86** del mismo CLSID terminaría en `BlocSrv.exe`, fuera de proceso.	Inferencia	Depende de la vista de 32 bits; habría que confirmarlo con un cliente x86.
Como el servidor x64 es `Both`, el objeto se crea en el apartamento del cliente y no hay proxy.	Inferencia	Se confirma resolviendo el módulo de la vtable (2D.6).
El AppID no define permisos propios, así que regirían los valores por defecto del equipo.	Inferencia con límite	Solo si el servidor no llama a `CoInitializeSecurity`; además, para una activación in-proc los permisos de lanzamiento no intervienen.
La typelib `{C … 3}` describe `IID {D … 4}`.	Hipótesis	Falta abrir la typelib y comparar; tampoco probaría la implementación.

Práctica: trazado de solo lectura

1. Elegí un componente propio ya registrado o una aplicación de laboratorio conocida.
2. Anotá la arquitectura del cliente y los flags `CLSCTX_*` que usa. Si no los ves, decilo.
3. Revisá si el cliente tiene manifiesto con `comClass` (recurso o archivo). Registrá el resultado aunque sea "no tiene".
4. Seguí `CLSID → servidor → AppID → TypeLib` con Regedit en modo lectura, en la vista de **usuario y de máquina** y en la de **32 y 64 bits** que correspondan.
5. Anotá ruta, firma, arquitectura y `ThreadingModel` del servidor. No ejecutes un `DllRegisterServer` desconocido.
6. Clasificá cada dato como observación, inferencia o hipótesis, como en la tabla del ejemplo.
7. Si el componente de práctica no existe, analizá el ejemplo ilustrativo y declará que no verificaste activación en vivo.

Entregable

Tabla de CLSID, contexto pedido, tipo de servidor, ruta, arquitectura, `ThreadingModel`, AppID y valores relevantes, typelib, vista de registro consultada, existencia de manifiesto y evidencias. Redactá anomalías como preguntas a investigar, no como diagnóstico automático.

Repaso

▼ **¿Qué diferencia hay entre `InprocServer32` y `LocalServer32`?**

> El primero apunta a un servidor DLL cargado en el proceso cliente (debe coincidir en arquitectura); el segundo, a un servidor ejecutable fuera del proceso, que puede tener otra arquitectura y se comunica por proxy/stub.

▼ **¿Qué agrega `ThreadingModel`?**

> Declara el modelo de apartamento compatible para el servidor en proceso y determina dónde lo crea COM; no describe por sí solo toda la sincronización del código.

▼ **Un servidor in-proc tiene `ThreadingModel=Apartment` y el cliente está en la MTA. ¿Qué recibe el cliente?**

> Un proxy: COM crea el objeto en un host STA y marshalea las llamadas.

▼ **¿Qué aporta AppID?**

> Agrupa opciones/configuración de uno o varios objetos distribuidos (permisos, identidad, surrogate, servicio) y permite relacionar la clase con ajustes DCOM. No garantiza que sean los valores efectivos si el proceso llama a `CoInitializeSecurity`.

▼ **¿Un TLB demuestra el comportamiento interno?**

> No. Describe contratos y tipos; el comportamiento se confirma siguiendo la implementación.

▼ **Buscaste el CLSID en el registro y no aparece, pero la aplicación lo crea. ¿Qué dos explicaciones revisás primero?**

> Que estés mirando la vista equivocada (usuario/máquina o 32/64 bits) o que la clase se declare en un manifiesto y se resuelva por contexto de activación sin registro.

Puerta optativa

Podés seguir una activación desde CLSID hasta servidor y metadatos, distinguir servidor in-process/local, explicar el efecto de `ThreadingModel`, declarar la vista de registro consultada y reconocer cuándo el registro no alcanza por la existencia de un manifiesto.

Lo que queda fuera

Monikers, ROT, servidores de servicio en detalle, catálogo COM+ y la creación de manifiestos o de registros propios no se desarrollan. La lección enseña a **leer** una configuración, no a registrar componentes.

Fuentes

- Microsoft Learn, activación: CoCreateInstance, CoGetClassObject, IClassFactory, DllGetClassObject, COM Clients and Servers.
- Microsoft Learn, registro: COM Registry Keys, InprocServer32, LocalServer32, AppID Key, Interface Key, Merged View of HKEY_CLASSES_ROOT, Registry Keys Affected by WOW64.
- Microsoft Learn, metadatos y sin registro: Generating a type library with MIDL, Creating Registration-Free COM Objects, Assembly Manifests, Activation Contexts.
- **Fundamentos previos:** Registro/WOW64 en 2D.3; COM base en 2D.6.

Etiquetas

Núcleo conceptual: registro y permisos tratados en 2D.3. **Conocimiento externo:** activación, `ThreadingModel`, vistas de registro, metadatos y COM sin registro. **Ejemplo ilustrativo:** claves, rutas y conclusiones del componente ficticio. **Laboratorio verificado:** no.

DCOM y RPC: activación remota y límites de confianza

> **Laboratorio aislado**
>
> No habilites acceso remoto ni abras puertos para esta lección. No cambies DCOMCNFG, permisos, identidades, restricciones de equipo ni políticas. Usá dos VMs ya configuradas para un ejercicio autorizado; si la práctica requeriría debilitar una protección, limitate al análisis del diagrama y de la configuración existente.

> **Propósito**
>
> Entender DCOM como activación/llamada de objetos COM a través de RPC y separar identidad autenticada, autorización, permisos de lanzamiento/activación y permisos de acceso. Al terminar deberías poder leer un error de DCOM como **un dato con varias explicaciones posibles** y documentarlo sin proponer que se baje la seguridad.

Prerrequisitos

- *Obligatorios:* 2D.6 COM, 2D.7 activación, red básica (1.3).
- *Recomendados:* tokens e impersonación (2D.1); Windows Firewall y seguridad de cuenta; no exige completar toda 2D.

Modelo

RPC permite invocar operaciones entre procesos locales o remotos. DCOM añade al modelo COM activación distribuida, proxies/stubs y negociación de seguridad. Una interfaz remota no se convierte en acceso libre por conocer su CLSID: intervienen autenticación, nivel de protección, identidad del servidor, permisos de lanzamiento/activación, permisos de acceso, firewall y políticas locales/dominio.

La autenticación de paquetes puede aportar integridad; privacidad agrega cifrado. Las actualizaciones de hardening DCOM elevaron requisitos de autenticación para activación. El detalle efectivo depende de la versión, actualizaciones y políticas administradas: documentá el estado, no copies recetas antiguas que bajan el nivel.

Dos momentos distintos: activar y llamar

Conviene separar siempre dos operaciones, porque las controlan **permisos distintos** y fallan por **motivos distintos**.

1. Activación remota. El cliente pide un objeto en otro equipo (por ejemplo, `CoCreateInstanceEx` con un `COSERVERINFO`, o un `RemoteServerName` configurado en el AppID). El pedido llega al servicio de activación del equipo servidor, que:

- identifica al cliente (con el token de proceso o, si el cliente usa *cloaking*, el de hilo);
- comprueba los **permisos de lanzamiento y activación**: si el servidor no está corriendo, hace falta permiso para **lanzarlo**; si ya corre, permiso para **activar** un objeto en él;

- si corresponde, lanza el servidor con la **identidad** configurada (usuario que lanza, usuario interactivo, una cuenta concreta o un servicio);
- devuelve al cliente una referencia marshaleada, que en el cliente se convierte en un **proxy**.

2. Llamada a un método. Con el proxy ya en mano, cada llamada viaja por RPC al proceso servidor. Ahí se evalúan los **permisos de acceso** (quién puede llamar a un servidor que ya está corriendo), con el nivel de autenticación negociado, y el servidor puede **impersonar** al cliente dentro del nivel que el cliente permitió.

Consecuencia: un cliente puede **activar** con éxito y luego fallar en la **primera llamada**, o al revés. El informe tiene que decir en qué momento ocurrió el fallo.

Cómo encuentra la llamada su destino: endpoints y puertos dinámicos

Muchos servidores RPC de Windows, entre ellos los servidores DCOM, no escuchan en un puerto fijo. Usan **endpoints dinámicos**: al arrancar, el servidor obtiene un puerto del rango dinámico y registra sus interfaces en el **mapeador de endpoints** (*Endpoint Mapper*, servicio RPCSS). El cliente primero pregunta al mapeador del equipo remoto —que en TCP escucha en el puerto **135**— dónde está la interfaz, y después se conecta al puerto que le devuelven. Desde Windows Vista y Server 2008, el rango dinámico predeterminado de TCP/IP es **49152–65535**, salvo que una política o configuración lo cambie.

En DCOM hay un detalle más: la referencia de interfaz que el servidor devuelve al activar contiene **direcciones IP del servidor** tal como él las ve. Microsoft advierte que por eso DCOM no funciona a través de firewalls que traducen direcciones (NAT).

Qué cambia para la evidencia:

- En el escenario típico, la activación usa el mapeador en el 135 y las llamadas posteriores van al **puerto dinámico** del proceso servidor. Un firewall que deja pasar el 135 pero no ese puerto produce exactamente el patrón “activa bien, falla la primera llamada”, sin que haya un problema de **permisos**.
- Un `0x800706BA` (servidor RPC no disponible) puede venir del mapeador inaccesible, del puerto dinámico bloqueado, de una dirección no alcanzable dentro de la referencia o de un servidor que terminó.
- En el laboratorio, de solo lectura: el puerto y el PID que escuchan en la VM servidor (`netstat -ano`), las reglas de firewall existentes y la dirección que la VM cliente usó. No se abren puertos ni se fija un rango para que “funcione”.

Autenticación, autorización y permisos

- **Autenticación**: establece *quién* es el interlocutor (con NTLM, Kerberos u otro proveedor). No otorga nada por sí misma.
- **Autorización**: decide *qué puede hacer* esa identidad. En DCOM se expresa con listas de control de acceso.

Desde Windows XP SP2 / Server 2003 SP1, los derechos se separan por **distancia** (local o remoto):

ACL	Derechos	Qué controla	Dónde vive
Lanzamiento y activación	Local Launch, Remote Launch, Local Activation, Remote Activation	Quién puede **arrancar** el servidor y quién puede **obtener** un objeto (un puntero de interfaz).	`LaunchPermission` del AppID; si no existe, `DefaultLaunchPermission`.
Acceso	Local Access, Remote Access	Quién puede **llamar** a un servidor que ya está en ejecución.	`AccessPermission` del AppID (solo si el proceso **no** llama a `CoInitializeSecurity`), el descriptor que pase `CoInitializeSecurity` o el valor por defecto del equipo.
Restricciones de equipo	Las mismas, como mínimo obligatorio	Un control adicional que se aplica a **todo** servidor COM del equipo, además del propio de cada aplicación.	`MachineLaunchRestriction` / `MachineAccessRestriction` o la política de grupo equivalente.

Leé estas ACL como capas: una solicitud tiene que pasar **la restricción del equipo y** la de la aplicación. Que una ACL de aplicación sea permisiva no alcanza si la restricción del equipo lo impide, y viceversa.

Nivel de autenticación frente a nivel de impersonación

Son dos perillas distintas que se negocian en el mismo "blanket" de seguridad:

	Nivel de **autenticación** (`RPC_C_AUTHN_LEVEL_*`)	Nivel de **impersonación** (`RPC_C_IMP_LEVEL_*`)
Pregunta que responde	¿Cuánta protección tiene la comunicación?	¿Cuánto puede hacer el servidor **en nombre** del cliente?
Valores	None, Default, Connect, Call, Packet, Packet Integrity, Packet Privacy	Anonymous, Identify (el predeterminado del sistema), Impersonate, Delegate
Quién decide	Cliente y servidor proponen; en el handshake se usa **el más alto** de los dos.	Lo fija el **cliente** en su proxy.
Matices documentados	La mayoría de los proveedores promueve Call a Packet; Default nunca elige None.	Anonymous solo funciona en transporte local (en otros se promueve a Identify); Delegate requiere configuración de dominio específica.

Un nivel de autenticación alto no autoriza más operaciones, y un nivel de impersonación bajo no debilita el cifrado: son ejes independientes.

Servicio de autenticación: Kerberos, NTLM, Negotiate y el SPN

El *security blanket* —el conjunto de valores de seguridad que COM negocia para cada proxy— incluye, además de los niveles de la sección anterior, el **servicio de autenticación**: el paquete que de verdad autentica. Los más comunes en DCOM son **NTLM** (`RPC_C_AUTHN_WINNT`), **Kerberos** (`RPC_C_AUTHN_GSS_KERBEROS`) y **Negotiate/Snego** (`RPC_C_AUTHN_GSS_NEGOTIATE`), que no autentica por sí mismo sino que elige, entre los paquetes que admiten cliente y servidor, uno que funcione para ambos.

Qué paquete termina usándose **depende del contexto y de la configuración**: las listas que cada lado declaró (explícitamente con `CoInitializeSecurity` / `CoSetProxyBlanket` o por defecto), si los equipos y cuentas

están en un dominio y se confían, el nombre con el que el cliente llegó al servidor, la existencia del SPN correcto y las políticas que restringen paquetes. No hay una regla universal del tipo "primero Kerberos y si no, NTLM" que puedas aplicar sin mirar esa configuración.

El **SPN** (*Service Principal Name*) es el identificador único de una instancia de servicio que Kerberos usa para asociarla con la cuenta con la que esa instancia inicia sesión. El SPN se registra en el objeto de esa cuenta en Active Directory, un mismo SPN puede estar en **una sola** cuenta, y si la cuenta del servicio cambia, los SPN deben registrarse de nuevo. El cliente **compone** el SPN a partir del nombre con el que localizó el servicio y lo presenta al autenticar; por eso, con Kerberos, el nombre de principal del servidor tiene que ser el correcto o la llamada falla.

Para la evidencia:

- Anotá **con qué nombre** se llegó al servidor (nombre corto, FQDN, alias o IP), **con qué cuenta** corre el servidor y si el binario fija un servicio de autenticación o un nombre de principal en `CoInitializeSecurity` / `CoSetProxyBlanket`.
- Si el código usa Negotiate, `CoQueryProxyBlanket` informa **Negotiate** y no el paquete que eligió: no deduzcas "Kerberos" o "NTLM" de esa consulta.
- Si había auditoría de inicio de sesión habilitada en el servidor, el registro de seguridad suele indicar el paquete usado; si no la había, decí que no se pudo determinar.

> **Recuadro conceptual: delegación de Kerberos**
>
> El nivel de impersonación **Delegate** permite que el servidor use la identidad del cliente hacia **otros** equipos, y solo funciona con Kerberos y configuración de dominio específica. Microsoft distingue, a grandes rasgos, tres modalidades: **sin restricciones** (la cuenta del servidor puede delegar hacia cualquier servicio), **restringida** (la cuenta del servidor solo puede delegar hacia una lista de servicios; se configura sobre la cuenta del servicio de entrada) y **restringida basada en recursos** (la decide el dueño del servicio de destino, sobre la cuenta de ese recurso). Señales que vale la pena **revisar**, sin cambiarlas: código que pide `RPC_C_IMP_LEVEL_DELEGATE`, cuentas de servicio marcadas como confiables para delegación, servicios con listas de delegación o con delegación basada en recursos, y cuentas sensibles que deberían estar marcadas como "no delegables". Esta lección no enseña a configurar ni a probar delegación: el inventario lo hace o lo autoriza quien administra el directorio.

`CoInitializeSecurity` y `CoSetProxyBlanket`

- `CoInitializeSecurity` fija la seguridad **por proceso**: nivel de autenticación, nivel de impersonación, servicios de autenticación y, del lado servidor, quién puede acceder. Si el proceso la llama, **se ignoran** los valores de seguridad del registro para ese proceso (salvo que se le pase un AppID, en cuyo caso usa solo el registro). Algunas opciones, como cambiar el nivel de impersonación por defecto del proceso, solo se pueden fijar así.
- `CoSetProxyBlanket` ajusta la seguridad de **un proxy concreto** (una interfaz remota), sobrescribiendo los valores del proceso para las llamadas por ese proxy. Internamente equivale a pedir `IClientSecurity` al proxy y llamar a `SetBlanket`.

Para el análisis: si ves `CoInitializeSecurity` en el binario, lo que diga DCOMCNFG sobre **acceso** de esa aplicación puede no ser lo efectivo. Si ves `CoSetProxyBlanket`, el nivel usado en esa interfaz puede diferir del

resto. Registrá los argumentos constantes que veas y marcá como inferencia los que dependen de valores en tiempo de ejecución.

Ejemplo trabajado

(Diagrama hipotético, sin tráfico capturado ni HRESULT observado.)

```
VM-Cliente: CoCreateInstanceEx(CLSID propio, servidor VM-Servicio)
  → [ACTIVACIÓN] RPC hacia el servicio de activación de VM-Servicio
VM-Servicio: autentica al cliente
  → restricción de equipo (launch/activation)  ¿pasa?
  → LaunchPermission del AppID (o el valor por defecto)  ¿pasa?
  → lanza BlocSrv.exe con la identidad configurada (si no estaba en ejecución)
  → devuelve referencia marshaleada → proxy para IID_IStatus en VM-Cliente
```

```
VM-Cliente: p→GetState(&state)
  → [LLAMADA] RPC al proceso BlocSrv.exe, con el nivel de autenticación negociado
VM-Servicio: restricción de equipo (access) + permisos de acceso del proceso  ¿pasa?
  → stub → BlocStatus::GetState; respuesta HRESULT + estructura de estado
```

Si la activación falla, las causas pueden estar en identidad, permisos, endpoint, DNS/red o hardening. Un error aislado no identifica cuál. El análisis correcto conserva HRESULT, evento y contexto y compara con la documentación de la versión.

Cómo documentar un error sin atribuirle una causa única

Dato	Ejemplo de registro	Por qué importa
Versión y build de **ambos** equipos	Edición, número de build, actualizaciones instaladas	El hardening DCOM cambió con actualizaciones; el mismo código puede comportarse distinto.
Momento del fallo	¿Activación o llamada? ¿Primera llamada o posterior?	Separa permisos de lanzamiento/activación de permisos de acceso.
HRESULT exacto	Valor hexadecimal, no solo "acceso denegado"	`0x80070005` (`E_ACCESSDENIED`) puede venir de lanzamiento, activación o acceso; `0x800706BA` (servidor RPC no disponible) puede ser red, nombre, endpoint o firewall; `0x80080005` (`CO_E_SERVER_EXEC_FAILURE`) puede ser identidad, ruta o un fallo propio del servidor.
Identidades	Cuenta del cliente, identidad configurada del servidor	Distingue quién pidió de quién ejecuta.
Red y nombre	Nombre usado para llegar al servidor, puerto y PID que escuchan, reglas de firewall existentes	Separa fallos de transporte (mapeador, puerto dinámico, NAT) de fallos de autorización.
Autenticación	Servicio pedido en código (si es constante), paquete observado en auditoría (si existía), SPN esperado	Evita suponer Kerberos o NTLM sin evidencia.
Configuración leída (sin cambiarla)	AppID, `LaunchPermission` / `AccessPermission` presentes o ausentes, si el binario llama a `CoInitializeSecurity`	Indica qué capa podría estar decidiendo.
Eventos	Entradas de origen DCOM en el registro *System* de **ambos** equipos, con fecha y hora	Suelen nombrar CLSID/AppID y la identidad, pero no siempre la capa exacta.
Hipótesis y descartes	"Compatible con X; no descartado Y porque…"	Evita que el informe convierta una correlación en causa.

Práctica

Solo con dos VMs en una red virtual host-only, ya provistas y aprobadas por quien administra el laboratorio:

1. Revisá sin modificar DCOM Config (identidad del servidor, permisos de lanzamiento/acceso, autenticación), las restricciones de equipo y el firewall. Anotá versión/build de ambas VMs.
2. En el binario cliente y servidor, buscá llamadas a `CoInitializeSecurity` y `CoSetProxyBlanket` y registrá los niveles constantes que veas.
3. Si hay un componente propio de prueba ya preparado, registrá una llamada inocua de consulta, separando el resultado de la **activación** y el de la **llamada**, y la identidad visible en ambos lados.
4. Si no está preparado o la activación requiere cambiar la seguridad, no la habilites: reconstruí el flujo a partir de documentación y del diagrama del ejemplo, completando la tabla de documentación de errores como ejercicio.

5. Apagá las VMs o revertí ambos snapshots; no reutilices credenciales de producción ni expongas la red virtual al host/LAN.

Entregable

Diagrama cliente–RPC–servidor con los dos momentos (activación y llamada) y las capas de control; versión/actualizaciones observadas; identidad; permisos; niveles de autenticación e impersonación; HRESULT/eventos y limitaciones. No incluyas credenciales, direcciones públicas ni cambios de seguridad.

Repaso

▼ **¿Qué añade DCOM a una llamada COM local?**

> Activación y comunicación distribuidas a través de RPC, con proxies/stubs y controles de seguridad remotos.

▼ **¿Autenticación equivale a autorización?**

> No. La autenticación establece quién es el interlocutor; la autorización decide qué puede hacer.

▼ **¿Qué permiso falta si la activación funciona pero la primera llamada al método devuelve acceso denegado?**

> Un candidato fuerte es un permiso de **acceso** (de la aplicación o la restricción de equipo), porque el de lanzamiento/activación ya se cumplió. Pero si el error es de transporte (por ejemplo, `0x800706BA`), también encaja un firewall que bloquea el puerto dinámico del servidor. Se distingue con el HRESULT exacto, eventos y configuración antes de afirmarlo.

▼ **¿Por qué un firewall que permite el puerto 135 puede no alcanzar para DCOM?**

> Porque el 135 es el del mapeador de endpoints; las llamadas al objeto van al puerto dinámico que registró el proceso servidor.

▼ **El código usa Negotiate y `CoQueryProxyBlanket` devuelve Negotiate. ¿Se autenticó con Kerberos?**

> No se puede saber por esa consulta: informa Negotiate, no el paquete elegido. Hace falta otra evidencia, como la auditoría de inicio de sesión del servidor.

▼ **¿Qué es un SPN y por qué importa el nombre con el que se llega al servidor?**

> Es el identificador de una instancia de servicio que Kerberos asocia a la cuenta del servicio. El cliente lo compone a partir del nombre que usó; si no corresponde a un SPN registrado en la cuenta correcta, Kerberos no puede autenticar ese servicio.

▼ **¿En qué se diferencian el nivel de autenticación y el de impersonación?**

> El de autenticación fija la protección de la comunicación (y se usa el más alto propuesto); el de impersonación, que fija el cliente, limita qué puede hacer el servidor en su nombre.

▼ **El registro de una aplicación dice una cosa y la aplicación se comporta como si dijera otra. ¿Qué buscás en el binario?**

> Una llamada a `CoInitializeSecurity` (que hace ignorar los valores del registro para ese proceso) o a `CoSetProxyBlanket` (que cambia la seguridad de un proxy).

▼ **¿Qué hacés si la práctica requiere desactivar hardening?**

> No la ejecutás. Analizás la configuración existente o usás un laboratorio actualizado que funcione con las protecciones vigentes.

Puerta optativa

Podés explicar las capas COM/DCOM/RPC, separar activación de llamada, autenticación de autorización y permisos de lanzamiento de permisos de acceso, distinguir nivel de autenticación de nivel de impersonación y documentar un error sin atribuirle una causa única ni recomendar bajar protecciones.

Lo que queda fuera

El formato del protocolo DCOM en la red (OBJREF, resolución OXID), la configuración de puertos o rangos, el funcionamiento interno de Kerberos y NTLM, la configuración de delegación (solo como recuadro), *cloaking* y COM+ no se desarrollan. Tampoco se enseña a configurar DCOM ni el firewall: la lección es de lectura y diagnóstico.

Fuentes

- Microsoft Learn: COM Clients and Servers, Locating a Remote Object, Activation Security, Authentication Level, Impersonation Levels.
- Microsoft Learn, configuración: Process-wide security / DCOMCNFG, Process-wide security with CoInitializeSecurity, CoSetProxyBlanket, LaunchPermission, AccessPermission, DCOM Security Enhancements (XP SP2 / 2003 SP1).
- Microsoft Learn, red y autenticación: Security Blanket Negotiation, COM authentication service constants, Snego, Service Principal Names, How clients compose a service's SPN, Kerberos constrained delegation overview, RPC dynamic ports and firewalls, Default dynamic port range, Service overview and network port requirements.
- Microsoft Support: DCOM hardening KB5004442.

- **Fundamentos previos:** contratos COM en 2D.6; activación en 2D.7.

Etiquetas

Núcleo conceptual: ninguno específico de DCOM. **Conocimiento externo:** COM/RPC, mapeador de endpoints y puertos dinámicos, permisos, niveles de seguridad, servicios de autenticación, SPN, delegación (recuadro) y hardening. **Ejemplo ilustrativo:** flujo hipotético y tabla de registro de errores. **Laboratorio verificado:** no; la práctica depende de dos VMs preconfiguradas.

WMI: proveedores, filtros y consumidores

> **Alcance de la práctica**
>
> La práctica principal inspecciona WMI y usa consultas temporales. No crea suscripciones permanentes, no ejecuta las acciones que encuentre y no borra ni "repara" objetos durante el triage. Si un docente prepara una demostración permanente, debe usar solo una VM aislada y un consumidor benigno, con inventario previo y limpieza verificada.

> **Propósito**
>
> Reconocer el modelo WMI/CIM y entender por qué un filtro unido a un consumidor permanente merece revisión, sin concluir malware solo por encontrar una suscripción. Al terminar deberías poder reconstruir una suscripción completa —qué evento la dispara, qué hace, quién la creó y con qué permisos— y dejar constancia de lo que **no** pudiste determinar.

Prerrequisitos

- *Obligatorios:* Registro (2D.3) y contexto de seguridad (2D.1).
- *Recomendados:* COM (2D.6), Task Scheduler (2D.5), servicios (2D.4).

Modelo

WMI expone clases/instancias a través de namespaces y proveedores. Un cliente consulta o se suscribe a eventos. La suscripción **temporal** vive mientras el cliente está conectado; una permanente permanece en el repositorio hasta cancelarse y relaciona `__EventFilter`, una instancia de `__EventConsumer` y `__FilterToConsumerBinding`. Los proveedores y consumidores tienen configuración y seguridad propias.

Para revisar una suscripción, lee namespace, filtro, consumidor, binding, propietario/CreatorSID, permisos y destino. Los consumidores estándar pueden tener efectos diferentes; no ejecutes scripts ni comandos encontrados. Una suscripción inesperada es un hallazgo que requiere contexto, no una prueba automática.

Temporal frente a permanente

	Consumidor **temporal**	Consumidor **permanente**
Cómo nace	Un programa o script llama a `ExecNotificationQuery` (o su variante asíncrona) y recibe eventos mientras corre.	Se guardan en el repositorio un filtro, un consumidor lógico y un binding.
Cuánto dura	Hasta que el cliente termina, WMI se detiene o el sistema se reinicia.	Hasta que alguien cancela la registración; sobrevive a reinicios.
Quién lo ejecuta	El propio cliente.	WMI carga el consumidor físico cuando hace falta.
Para el analista	Es lo que usás en la práctica para observar.	Es lo que inventariás y documentás, sin tocar.

La documentación recomienda llamadas **semisíncronas** antes que asíncronas para consumidores temporales, porque en el modo asíncrono un llamador no autenticado puede entregar datos al *sink*.

Namespaces: dónde vive la suscripción y de dónde vienen los eventos

WMI organiza clases en **namespaces** (`root\cimv2`, `root\subscription`, `root\default` y muchos otros que agregan productos y roles). Para este tema importan dos ideas:

- `root\cimv2` aloja las clases del proveedor Win32 y eventos intrínsecos como `__InstanceCreationEvent` sobre esas clases.
- Microsoft **recomienda** compilar las suscripciones permanentes en `root\subscription`, para tener un solo lugar donde buscarlas. Es una recomendación, no una garantía: los tres objetos pueden existir en **cualquier** namespace que tenga las clases necesarias. Un inventario que solo mira `root\subscription` puede quedar incompleto.

Hay que distinguir **dos namespaces** por suscripción:

1. El namespace donde **están guardados** el filtro, el consumidor y el binding (por ejemplo, `root\subscription`).
2. El valor de la propiedad `EventNamespace` del filtro: el namespace de los **eventos** que el filtro consulta (por ejemplo, `root\cimv2`). Con eso se arman suscripciones *cross-namespace*: el objeto vive en un lugar y escucha eventos de otro.

Si en la ficha anotás solo uno de los dos, la reconstrucción queda ambigua.

Las piezas de una suscripción permanente

```
__EventFilter                 (Name, Query WQL, QueryLanguage, EventNamespace,
   ^                           EventAccess, CreatorSID)
   | Filter
__FilterToConsumerBinding     (Filter, Consumer, MaintainSecurityContext, CreatorSID)
   | Consumer
   v
<consumidor lógico>           instancia de una clase derivada de __EventConsumer
                              (p. ej. NTEventLogEventConsumer, LogFileEventConsumer,
                               CommandLineEventConsumer, ActiveScriptEventConsumer…)
   | clase del consumidor
   v
__EventConsumerProviderRegistration  → __Win32Provider (proveedor de consumidor, COM)
   v
consumidor físico (objeto COM que realmente ejecuta la acción)
```

- **Filtro**: *qué* evento interesa (consulta WQL de eventos).
- **Consumidor lógico**: *qué hacer* (escribir en el registro de eventos, en un archivo, lanzar un proceso, ejecutar un script, enviar correo…). Sus propiedades describen la acción, no el evento.
- **Binding**: la asociación que dice qué filtro alimenta a qué consumidor. Un filtro puede tener varios consumidores y un consumidor varios filtros.
- **Proveedor de consumidor**: el objeto COM que WMI localiza, mediante `__EventConsumerProviderRegistration`, para ejecutar al consumidor físico de esa clase. Por eso un consumidor de una clase no estándar te lleva a revisar también su registro COM (2D.7).

Entre los consumidores estándar, `CommandLineEventConsumer` lanza un proceso **en el contexto de sistema local**, y `ActiveScriptEventConsumer` ejecuta un script. Microsoft los señala como sensibles. Su presencia aumenta la prioridad de revisión; no define la intención.

Leer la consulta del filtro: WQL de eventos

La propiedad `Query` del filtro está escrita en **WQL**. Microsoft la describe como un subconjunto de SQL con cambios semánticos: las consultas de datos van contra **una clase por vez**, su lista de palabras clave no incluye `JOIN` ni sentencias de modificación, y las consultas de **eventos** agregan cláusulas propias. No la leas como SQL general.

Qué clase de evento se pide.

- **Intrínsecos:** reflejan un cambio en el modelo de datos de WMI: creación, modificación o borrado de instancias, clases o namespaces (`__InstanceCreationEvent` , `__InstanceModificationEvent` , `__InstanceDeletionEvent` , `__InstanceOperationEvent` …). La instancia afectada viaja en `TargetInstance` , y el operador `ISA` restringe a una clase y sus subclases: `WHERE TargetInstance ISA 'Win32_Service'` .
- **Extrínsecos:** sucesos que un proveedor define como clase propia derivada de `__ExtrinsicEvent` , sin relación directa con instancias. Por ejemplo, el proveedor del registro define `RegistryKeyChangeEvent` .
- **De temporizador:** clases de reloj (`__TimerEvent` y relacionadas) que disparan en momentos o intervalos.

Dos usos distintos de `WITHIN` . La misma palabra cumple dos funciones que no son intercambiables:

Forma	Dónde va	Qué hace	Qué te dice como analista
`WITHIN n` (sondeo)	Inmediatamente **antes** de `WHERE`	Fija el **intervalo de sondeo** que WMI usa para detectar eventos intrínsecos cuando no hay un proveedor de eventos para esa clase. Admite fracciones de segundo, pero valores demasiado chicos pueden rechazarse; Microsoft recomienda intervalos mayores a 5 minutos para la mayoría de los consumidores.	El costo y la latencia de detección. Un sondeo muy frecuente sobre una clase grande es una carga constante; la notificación puede llegar hasta `n` segundos tarde.
`GROUP WITHIN n` (agrupación)	**Después** de `WHERE` , tras `GROUP`	Junta los eventos similares que llegan durante `n` segundos desde el primero y entrega **un** evento agregado (`__AggregateEvent`). Admite `BY propiedades` y `HAVING NumberOfEvents …` .	Que el consumidor reacciona a **ráfagas o umbrales**, no a cada evento.

Ejemplos de lectura (ilustrativos):

```
SELECT * FROM __InstanceCreationEvent WITHIN 30
  WHERE TargetInstance ISA 'Win32_Process'
  → sondea cada 30 s la aparición de procesos; reacciona a cada uno

SELECT * FROM __InstanceCreationEvent
  WHERE TargetInstance ISA 'Win32_NTLogEvent'
  GROUP WITHIN 600 BY TargetInstance.SourceName
  HAVING NumberOfEvents > 25
  → agrupa 10 minutos por origen; solo entrega si hubo más de 25
```

Microsoft indica además que crear consumidores **permanentes** con consultas de sondeo requiere privilegios de administrador, y que cuando la consulta de un consumidor permanente falla se registra un evento de origen *WinMgmt* con id 10. Ese evento es evidencia útil de un filtro roto.

Seguridad: CreatorSID, EventAccess y ACL del namespace

`CreatorSID`**.** WMI guarda en cada instancia el SID de quien la creó. Para que una suscripción permanente funcione, el filtro, el consumidor y el binding deben tener **el mismo** `CreatorSID`, y el creador debe ser una cuenta de dominio con privilegios de administrador local o el grupo Administradores local. Para los consumidores sensibles, WMI verifica que ese SID pertenezca a Administradores. Qué revisar:

- ¿Coinciden los tres `CreatorSID`? Si no, la suscripción probablemente no funciona; queda como hallazgo a explicar, no a reparar.
- ¿A qué cuenta o grupo corresponde el SID? Convertí el arreglo de bytes a SID de texto y resolvelo; si no resuelve (cuenta borrada, otro dominio), anotalo.

`EventAccess`**.** Es un descriptor de seguridad SDDL en el filtro que limita **qué identidades pueden publicar** eventos hacia él (derecho `WBEM_RIGHT_PUBLISH`). El valor por defecto es `NULL`, que permite a todos. Que esté vacío no es anómalo en sí; que esté restringido indica una configuración deliberada.

ACL del namespace. Cada namespace tiene un descriptor de seguridad que WMI compara con el token de quien se conecta: derechos como *Enable Account*, *Execute Methods*, *Full Write*, *Partial Write*, *Provider Write*, *Remote Enable*, *Read Security* y *Edit Security*. Revisá quién tiene escritura en el namespace donde vive la suscripción, comparalo con los permisos por defecto que documenta Microsoft y anotá las diferencias. La auditoría de namespaces existe (SACL), pero no viene habilitada; si no estaba activa **antes** del incidente, no esperes encontrar eventos de quién creó la suscripción.

Leer un archivo MOF sin compilarlo

MOF (*Managed Object Format*) es el lenguaje con el que se describen clases e instancias CIM. Un proveedor suele distribuirse como un MOF (que define sus clases) más una DLL (que entrega los datos), y el MOF se **compila** al repositorio con `mofcomp`. Para el triage importa porque una suscripción permanente también puede escribirse como MOF.

Qué mirar en un `.mof` encontrado durante una revisión, **solo leyéndolo como texto**:

- `#pragma namespace("\\\\.\\root\\subscription")`: en qué namespace se crearían los objetos.
- Bloques `instance of __EventFilter as $FLT { … }`, `instance of <ClaseConsumidor> as $CONS { … }` e `instance of __FilterToConsumerBinding { Filter = $FLT; Consumer = $CONS; }`: es exactamente la terna filtro–consumidor–binding de esta lección.
- `#pragma autorecover`: agrega el archivo a la lista que WMI **recompila automáticamente** si reconstruye un repositorio dañado (la lista vive en `HKLM\SOFTWARE\Microsoft\WBEM\CIMOM\autorecover mofs`).

Dos límites de interpretación:

- Un MOF en disco **no prueba** que esos objetos existan en el repositorio: pudo no haberse compilado nunca. Y al revés, una suscripción puede existir en el repositorio sin ningún MOF en disco.
- **Nunca** ejecutes `mofcomp` sobre un archivo encontrado durante una revisión: compilarlo **crearía** los objetos que describe.

Recuadro conceptual: proveedores WMI

Un **proveedor** es un componente COM que entrega instancias, métodos o eventos de ciertas clases. Se registra con una instancia de `__Win32Provider`, cuyo `CLSID` apunta a su servidor COM (el recorrido de 2D.7 se aplica igual). La propiedad `HostingModel` define dónde y con qué cuenta corre: dentro de un proceso anfitrión compartido `WmiPrvSE.exe` como *NetworkService* (el predeterminado desde Vista cuando no se especifica), *LocalService* o *LocalSystem*; como servidor propio (*SelfHost*), o como proveedor desacoplado dentro de otra aplicación. Microsoft desaconseja *LocalSystem* por su privilegio y documenta eventos de registro cuando un proveedor usa ese modelo. Para la revisión basta con saber leer `__Win32Provider`, su CLSID y su `HostingModel`; **escribir** proveedores queda fuera del curso.

WMI remoto: el modelo, sin habilitar nada

Un cliente puede conectarse a WMI de **otro** equipo indicando el nombre del equipo en la ruta del namespace (`\\EQUIPO\root\cimv2` en `IWbemLocator::ConnectServer`, o `-ComputerName` en PowerShell). Esa conexión clásica viaja por **DCOM**, así que todo lo de 2D.8 aplica: firewall, permisos y autenticación DCOM, más el derecho **Remote Enable** sobre el namespace y las restricciones de UAC para cuentas remotas. La alternativa es **WinRM**, la implementación de Microsoft del protocolo WS-Management (SOAP sobre HTTP/HTTPS), que expone datos de WMI a través de su propio servicio y con su propia configuración; las sesiones CIM de PowerShell pueden usarlo.

Para la evidencia: una suscripción o una consulta pudo haberse creado **desde otro equipo**. Por eso un `CreatorSID` de cuenta de dominio, sin actividad local que lo explique, es una pista para preguntar al administrador por herramientas de gestión remota, no una conclusión. Esta lección **no** habilita WMI remoto, WinRM ni reglas de firewall.

Criterios para documentar sin concluir de más

Una suscripción **existe**: eso es una observación. Que sea **maliciosa** es una conclusión que necesita más evidencia. Antes de clasificar, respondé:

- **Origen plausible:** ¿la instaló un producto conocido (agente de gestión, monitoreo, antivirus)? Nombres, rutas y proveedores de consumidor suelen delatar al producto; confirmalo con el inventario de software o con el administrador.
- **Coherencia:** ¿coinciden los `CreatorSID`? ¿El consumidor apunta a un archivo que existe, firmado, en una ruta protegida? Una ruta escribible por usuarios en un `CommandLineEventConsumer` es un riesgo aunque el producto sea legítimo.
- **Disparador:** ¿qué hace la consulta? Un temporizador, el arranque, el inicio de sesión o la creación de un proceso tienen implicancias distintas.
- **Temporalidad:** ¿hay registros de cambio, tickets o eventos que fechen la creación? Si no, decí que no se pudo fechar.
- **Alcance:** ¿la misma suscripción aparece en otros equipos de la flota? Si es así, sugiere despliegue centralizado (legítimo o no).

Conclusiones calibradas posibles: “esperada y documentada”, “legítima con riesgo de configuración”, “sin origen identificado: escalar”, “inconsistente/no funcional”. Ninguna de estas habilita a borrarla durante el triage.

Ejemplo trabajado

(Vista conceptual ficticia de un laboratorio, sin consulta activa. Nombres, rutas y SID son marcadores.)

```
Namespace de la suscripción: ROOT\subscription
  __EventFilter.Name           = "BlocLab-Filtro"
  __EventFilter.Query          = SELECT * FROM __InstanceCreationEvent WITHIN 30
                                 WHERE TargetInstance ISA 'Win32_Process'
                                 AND TargetInstance.Name = 'BlocLab.exe'
  __EventFilter.EventNamespace = root\cimv2
  __EventFilter.EventAccess    = (vacío)
  Consumidor: NTEventLogEventConsumer "BlocLab-Log"
  Binding: Filtro "BlocLab-Filtro" → Consumidor "BlocLab-Log"
  CreatorSID (filtro, consumidor, binding): S-1-5-32-544 (los tres iguales)
```

Lectura: la suscripción vive en `root\subscription`, pero escucha eventos de `root\cimv2` (cross-namespace). El filtro sondea cada 30 segundos (`WITHIN 30`) la creación de un proceso llamado `BlocLab.exe` y el consumidor escribe una entrada en el registro de eventos: una acción de bajo impacto. Los tres `CreatorSID` coinciden con el SID del grupo Administradores, que es una forma documentada de crear suscripciones. `EventAccess` vacío significa que cualquier proveedor puede publicar al filtro.

La relación filtro–binding–consumidor permite reconstruir el flujo. Si falta un elemento o los SIDs no coinciden, la suscripción puede estar incompleta o requerir explicación; no la repares ni la borres durante triage.

Ficha de evidencia

Campo	Valor (ejemplo)	Fuente / tipo
Equipo, versión/build, fecha y hora de la revisión	VM-Lab-01, *(completar)*	Observación
Herramienta y cuenta usadas para inspeccionar	*(completar)*; sesión elevada sí/no	Observación
Namespace de la suscripción	`ROOT\subscription`	Observación
`EventNamespace` del filtro	`root\cimv2`	Observación
Filtro: nombre, consulta, lenguaje	`BlocLab-Filtro`, WQL arriba	Observación
Lectura de la consulta	Evento intrínseco de creación; sondeo `WITHIN 30`; sin `GROUP`	Interpretación de la WQL
Consumidor: clase, nombre y propiedades de acción	`NTEventLogEventConsumer`, `BlocLab-Log`	Observación (no ejecutado)
Proveedor de consumidor	Estándar de WMI	Inferencia por clase; confirmar registración
Binding y `MaintainSecurityContext`	Filtro → consumidor; *(completar)*	Observación
`CreatorSID` de los tres objetos y resolución	Iguales; Administradores	Observación + resolución
`EventAccess`	Vacío (`NULL`, todos pueden publicar)	Observación
ACL del namespace (escritura)	*(completar)*; diferencias con los valores documentados	Observación
Archivos referenciados (ruta, firma, hash, ACL)	No aplica para este consumidor	Observación
Origen plausible y evidencia	Laboratorio del curso, ticket *(completar)*	Contexto
Conclusión calibrada	Esperada y documentada	Juicio del analista
Lo que no se pudo determinar	Fecha de creación (sin auditoría previa)	Límite declarado

Práctica: inspección y evidencia

1. En una VM de laboratorio, inspeccioná primero una clase/proveedor WMI y una consulta temporal de lectura sobre datos conocidos (por ejemplo, instancias de `Win32_Service`).
2. Listá los namespaces existentes y decidí cuáles revisar; no te limites a `root\subscription`. Con PowerShell, de solo lectura: `Get-CimInstance -Namespace root -ClassName __NAMESPACE` para los namespaces y `Get-CimInstance -Namespace root/subscription -ClassName __EventFilter` (y luego `__EventConsumer` y `__FilterToConsumerBinding`) para cada namespace elegido.
3. Para cada suscripción, completá la ficha: namespace, `EventNamespace`, filtro, consumidor, binding, `CreatorSID` convertido a SID de texto, `EventAccess` y ACL del namespace. Sin ejecutar sus acciones.

4. Contrastá cuenta creadora, ACL del namespace, query y destino. Para cada consulta, anotá tipo de evento, si usa sondeo (`WITHIN`) o agrupación (`GROUP WITHIN`) y con qué intervalos. Redactá rutas personales antes de compartir evidencia.
5. Si en la VM hay archivos `.mof` de laboratorio, leelos como texto y compará lo que describen con lo que está en el repositorio. No los compiles.
6. No cambies ni elimines suscripciones encontradas. Si el laboratorio incluye una permanente creada previamente por el docente, identificá su ticket/ID y seguí la hoja de limpieza del laboratorio; verificá que los tres objetos vinculados ya no aparecen.

Entregable

Ficha de evidencia de una consulta temporal o de una suscripción auditada, con todos los campos de la tabla, la conclusión calibrada y la sección de límites. Si no había una suscripción autorizada de prueba, indicá que no realizaste el paso de eliminación.

Repaso

<details>
<summary>¿Qué mantiene viva una suscripción permanente?</summary>

> La inscripción persistente de filtro, consumidor y binding en el repositorio WMI; permanece hasta que se cancela.
</details>

<details>
<summary>¿Por qué hay que revisar el binding?</summary>

> Une qué evento se busca con qué consumidor recibirá el evento.
</details>

<details>
<summary>¿Qué diferencia hay entre el namespace de la suscripción y `EventNamespace`?</summary>

> El primero es donde están guardados filtro, consumidor y binding; `EventNamespace` indica de qué namespace vienen los eventos que el filtro consulta.
</details>

<details>
<summary>¿Alcanza con revisar `root\subscription`?</summary>

> No. Es la ubicación recomendada, pero una suscripción puede crearse en otros namespaces.
</details>

<details>
<summary>Los `CreatorSID` del filtro y del binding no coinciden. ¿Qué concluís?</summary>

> Que la suscripción probablemente no funciona tal como está y que hay algo que explicar; no la reparás ni la borrás durante el triage.
</details>

<details>
<summary>¿Qué controla `EventAccess`?</summary>

> Qué identidades pueden publicar eventos hacia ese filtro; vacío (`NULL`) permite a todos.
</details>

▼ **¿Qué diferencia hay entre `WITHIN 30` antes del `WHERE` y `GROUP WITHIN 30` después?**

> El primero fija un intervalo de **sondeo** para detectar eventos intrínsecos; el segundo **agrupa** eventos durante 30 s y entrega uno agregado (`__AggregateEvent`). No son intercambiables.

▼ **¿Qué distingue un evento intrínseco de uno extrínseco?**

> El intrínseco refleja un cambio en el modelo de datos de WMI (instancias, clases, namespaces); el extrínseco es una clase de evento definida por un proveedor para un suceso que no es un cambio de instancia.

▼ **Encontrás un `.mof` con un filtro, un consumidor y un binding. ¿Prueba que la suscripción existe?**

> No. Hay que comprobarlo en el repositorio; el archivo pudo no compilarse nunca. Y no se compila durante la revisión, porque crearía los objetos.

▼ **¿Qué aporta `#pragma autorecover` en un MOF?**

> Que WMI lo recompile automáticamente si reconstruye el repositorio; por eso conviene anotarlo junto con la entrada de registro correspondiente.

▼ **¿Por qué un `CreatorSID` de dominio sin actividad local no prueba nada por sí solo?**

> Porque WMI admite conexiones remotas (por DCOM o WinRM): la suscripción pudo crearse con una herramienta de gestión remota. Es una pregunta para el administrador.

▼ **¿Encontrar `__EventFilter` prueba una infección?**

> No. Hace falta relacionarlo con consumidor, binding, identidad, autorización y producto que lo instaló.

▼ **¿Qué no se debe hacer con un consumidor desconocido?**

> No ejecutarlo, borrarlo ni repararlo durante la revisión; preservá datos y escalá al administrador.

Puerta optativa

Podés distinguir consulta temporal y suscripción permanente, ubicar ambos namespaces de una suscripción, leer la WQL del filtro (tipo de evento, sondeo y agrupación), reconstruir filtro–binding–consumidor–proveedor, leer un MOF sin compilarlo, revisar `CreatorSID`, `EventAccess` y ACL del namespace, y describir evidencia con una conclusión calibrada sin ejecutar la acción configurada.

Lo que queda fuera

La escritura de proveedores (solo como recuadro), la sintaxis completa de MOF y de WQL (consultas de esquema, `ASSOCIATORS OF`, `REFERENCES OF`), el formato interno del repositorio, la configuración de WMI remoto o WinRM, y la habilitación de trazas o auditoría (que son cambios de configuración y no corresponden al triage) no se desarrollan.

Fuentes

- Microsoft Learn: WMI architecture, Monitoring Events, Receiving Events at All Times, Receiving Events Securely, Maintaining WMI Security.
- Microsoft Learn, clases y namespaces: __EventFilter, __EventConsumer, __FilterToConsumerBinding, __EventConsumerProviderRegistration, Standard Consumer Classes, Cross-Namespace Permanent Event Subscriptions, Access to WMI Namespaces.
- Microsoft Learn, WQL y MOF: WQL (SQL for WMI), Determining the Type of Event to Receive, WITHIN Clause, GROUP Clause, HAVING Clause, ISA Operator for Event Queries, Managed Object Format (MOF), pragma autorecover.
- Microsoft Learn, proveedores y remoto: __Win32Provider, Provider Hosting and Security, Connecting to WMI on a Remote Computer, About Windows Remote Management.
- **Fundamentos previos:** servicios y tareas en 2D.4 y 2D.5; DCOM en 2D.8.

Etiquetas

Núcleo conceptual: análisis defensivo y persistencia como concepto. **Conocimiento externo:** arquitectura, namespaces, suscripciones, WQL de eventos, MOF, proveedores (recuadro), WMI remoto y seguridad de eventos WMI. **Ejemplo ilustrativo:** suscripción y ficha del laboratorio ficticio. **Laboratorio verificado:** no; inspección de solo lectura.

IPC en Windows: named pipes, RPC y ALPC

> **Rama optativa 2D**
>
> Se analizan mecanismos en programas propios o didácticos. No se buscan endpoints de terceros para enviarles datos ni se enseña a abusarlos.

> **Propósito**
>
> Reconocer IPC por evidencia estática/dinámica y explicar qué controles separan al cliente del servidor. Al terminar deberías poder decir, para un canal propio, **quién puede abrirlo**, **quién puede hacerse pasar por servidor**, **con qué identidad actúa** el servidor al atender y **qué valida** antes de procesar un mensaje.

Prerrequisitos

- *Obligatorios:* procesos e hilos (1.3); objetos, handles, tokens y ACL (2D.1).
- *Recomendados:* COM/RPC (2D.6, 2D.8).

Diferencias

- **Named pipe:** canal nombrado, local o de red según configuración; el nombre no es secreto. El servidor decide acceso con ACL y puede autenticar/impersonar clientes; revisá también framing, longitud y validación de mensajes.

Para reconstruir la estructura de los mensajes más allá del mecanismo IPC, seguí 4.5 · Reconstruir formatos y protocolos propios.

- **RPC:** contrato de operaciones que puede comunicarse local o remotamente; el runtime gestiona transporte/marshalling según binding. Analizá interfaz, endpoint, identidad y autorización.
- **ALPC:** mecanismo local de bajo nivel usado por componentes de Windows. En reversing suele reconocerse por módulos, estructuras y trazas; la documentación pública del contrato es limitada y varía por versión. Se cubre conceptualmente, sin construir clientes de endpoints del sistema.

	Named pipe	RPC	ALPC
Qué transporta	Bytes o mensajes	Llamadas a operaciones de una interfaz (IDL)	Mensajes locales del sistema
Dónde se ve	Nombre `\\.\pipe\…`, handles de tipo archivo	UUID y versión de interfaz, secuencia de protocolo, endpoint	Módulos y trazas; poca documentación pública
Control de acceso principal	Descriptor de seguridad del pipe	Autenticación + callback de seguridad + chequeos en cada operación	Interno del sistema
Lo que lees en un análisis	Creación, apertura, ACL, modo, impersonación	Registro de interfaz, flags, protocolos, autenticación	Que existe, quién lo usa y qué evento registra

Named pipes en detalle

Nombres y alcance. Un servidor crea `\\.\pipe\nombre` con `CreateNamedPipe`; el nombre no distingue mayúsculas y cualquiera que lo conozca puede **intentar** abrirlo. Un cliente de otro equipo puede llegar por la red con `\\servidor\pipe\nombre`, salvo que el servidor cree la instancia con `PIPE_REJECT_REMOTE_CLIENTS`, que rechaza automáticamente las conexiones remotas.

Descriptor de seguridad. Si el servidor pasa `NULL`, el pipe recibe un descriptor por defecto que, según la documentación, da control total a LocalSystem, a los administradores y al creador, y **lectura** a Everyone y a la cuenta anónima. Un servidor que atiende solo a su sesión puede restringir el acceso con el *logon SID*; Microsoft lo recomienda para impedir accesos desde otras sesiones o equipos.

Quién puede ser servidor. Para crear una instancia hace falta el derecho `FILE_CREATE_PIPE_INSTANCE` sobre el pipe. La documentación advierte que `FILE_GENERIC_WRITE` incluye ese derecho (comparte valor con `FILE_APPEND_DATA`), así que una ACL que da "escritura genérica" también deja **crear instancias**. Si el servidor real crea la primera con `FILE_FLAG_FIRST_PIPE_INSTANCE`, la llamada falla con `ERROR_ACCESS_DENIED` cuando el nombre ya existe: es la forma documentada de detectar que otro proceso ocupó el nombre antes.

Modo. Un pipe de tipo **mensaje** conserva los límites de cada escritura; uno de tipo **byte** no. El lado cliente arranca en modo byte aunque el servidor sea de mensajes. Para el análisis, esto define cómo hay que leer el protocolo: con mensajes, cada `WriteFile` es una unidad; con bytes, el protocolo necesita su propio *framing* (longitud, delimitador) y es ahí donde se validan tamaños.

Impersonación. El servidor puede llamar a `ImpersonateNamedPipeClient` para actuar con el token del cliente y volver a su propia identidad con `RevertToSelf`. Por defecto la impersonación es al nivel `SecurityImpersonation`, pero el **cliente** puede limitarla al abrir el pipe con `CreateFile` y la bandera `SECURITY_SQOS_PRESENT` (por ejemplo, a nivel identificación). Para el análisis importan tres preguntas: ¿el servidor impersona? ¿revierte en todas las rutas, incluidos los errores? ¿qué nivel pidió el cliente?

Qué revisar en un pipe propio	Evidencia	Señal a explicar
Nombre y proceso creador	Handle en el proceso servidor, argumentos de `CreateNamedPipe`	Nombre predecible usado por un servicio privilegiado
Descriptor de seguridad	DACL pasada al crear (o `NULL`)	Descriptor por defecto o ACL con `FILE_GENERIC_WRITE` amplio
Primera instancia	`FILE_FLAG_FIRST_PIPE_INSTANCE` en la primera creación	Sin la bandera, otro proceso pudo ocupar el nombre antes
Clientes remotos	`PIPE_REJECT_REMOTE_CLIENTS`	Aceptación remota sin necesidad
Modo y framing	Tipo byte/mensaje, prefijo de longitud	Longitud leída del mensaje sin tope
Impersonación	`ImpersonateNamedPipeClient` / `RevertToSelf`	Rutas de error sin `RevertToSelf`

RPC en detalle

Una interfaz RPC se define en **IDL** con un UUID y una versión; MIDL genera los stubs de cliente y servidor. El servidor registra la interfaz y elige **secuencias de protocolo**: `ncalrpc` (local), `ncacn_np` (sobre named pipes) o `ncacn_ip_tcp` (TCP, normalmente con endpoint dinámico resuelto por el mapeador de 2D.8).

Tres lecturas de seguridad que Microsoft documenta y que cambian conclusiones:

- **Los endpoints no aíslan interfaces.** En un proceso con varios servidores RPC, *todas* las interfaces son invocables por *todos* los endpoints del proceso. Que tu componente registre solo `ncalrpc` no impide que lo llamen por un pipe o un socket que abrió otro componente del mismo proceso. Por eso un descriptor de seguridad sobre un endpoint no es un control efectivo; la recomendación es un **callback de seguridad** que verifique protocolo e identidad en cada llamada.
- **Registrar autenticación no la exige.** `RpcServerRegisterAuthInfo` agrega una **opción** de autenticación, no un requisito: un cliente no autenticado que podía conectarse sigue pudiendo, aunque luego fallen funciones como `RpcImpersonateClient`.
- **Flags de registro.** Con `RpcServerRegisterIf2` / `Ex` / `3`, `RPC_IF_ALLOW_LOCAL_ONLY` rechaza clientes remotos (con las excepciones que detalla la documentación para `ncacn_np`), y `RPC_IF_ALLOW_SECURE_ONLY` exige un nivel de autenticación mayor que *none*, **sin** garantizar que el usuario sea privilegiado: puede ser una cuenta de invitado.

Microsoft remarca que `ncalrpc` no es "seguro por ser local": en un servidor de terminales, cientos de usuarios pueden llegar a un servicio local.

Anatomía Interna de Stubs RPC y Decodificación de NDR (Format Strings)

Para el analista de ingeniería inversa que audita un binario sin símbolos ni archivos `.idl`, reconstruir la firma exacta de las funciones expuestas por un servidor RPC requiere inspeccionar las estructuras internas que MIDL compila en la sección `.data` o `.rdata`:

```
RPC_SERVER_INTERFACE (puntero exportado o registrado con RpcServerRegisterIf2):
   +0x00: Length: 0x60 (96 bytes en x64)
   +0x04: InterfaceId.SyntaxGUID (UUID de 16 bytes de la interfaz)
   +0x14: InterfaceId.SyntaxVersion (Major/Minor version)
   +0x50: InterpreterInfo (Ptr64 a MIDL_SERVER_INFO)

MIDL_SERVER_INFO:
   +0x00: pStubDesc (Ptr64 a MIDL_STUB_DESC)
   +0x08: DispatchTable (Ptr64 a arreglo de punteros a funciones C del servidor)
   +0x10: ProcFormatString (Ptr64 a tabla de formatos de procedimiento NDR)
   +0x18: FmtStringOffset (Ptr64 a tabla de índices a ProcFormatString)
```

Decodificación de `ProcFormatString` y `TypeFormatString`

El compilador MIDL genera secuencias de bytes compactas que codifican los parámetros de cada método mediante la máquina de tipos de **NDR (Network Data Representation)**:

1. **Cabecera de Procedimiento:** En `ProcFormatString`, cada rutina arranca con una cabecera de 6 bytes en modo `-Oicf` que define el tamaño del marco en la pila (`stack_size`) y el número de parámetros.
2. **Descriptores de Parámetros (6 bytes por parámetro):**

- **Flags de Dirección:** Los primeros 2 bytes indican si el parámetro es entrante o saliente (`0x0048` = `[in]`, `0x0158` = `[out]`, `0x0050` = `[in, out]`).
- **Offset en el Stack:** Los siguientes 2 bytes indican la posición relativa en el marco de la llamada.
- **Descriptor de Tipo:** Los 2 bytes finales contienen el tipo básico (`FC_LONG = 0x08`, `FC_HYPER = 0x0b`, `FC_CHAR = 0x02`) o un offset relativo hacia `TypeFormatString` (almacenado en `MIDL_STUB_DESC+0x28`).

3. **Tipos Complejos en `TypeFormatString`:**

- `0x11` (`FC_RP`): Puntero de referencia (no puede ser nulo).
- `0x12` (`FC_UP`): Puntero único (puede ser nulo).
- `0x21` (`FC_C_CSTRING`): Cadena ASCII conformante terminada en nulo (`char*`).
- `0x22` (`FC_C_WSTRING`): Cadena UTF-16 conformante terminada en nulo (`wchar_t*`).
- `0x1A` (`FC_BOGUS_STRUCT`): Estructura compleja con alineación manual y punteros anidados.

El laboratorio `plantillas_codigo/rpc_com_server_lab.c` y su definición `rpc_com_server_lab.idl` permiten compilar un servidor local sobre `ncalrpc` y verificar la disposición exacta de estas tablas en memoria.

ALPC

ALPC es el mecanismo de mensajes locales que usan muchos componentes de Windows. Su contrato no está documentado para uso general y cambia entre versiones, así que el curso solo pide **reconocerlo**: qué proceso lo usa, que existe una clase de eventos ETW para ALPC (ver 2D.12) y que, si un análisis llega a ALPC, la conclusión debe apoyarse en la documentación del componente, no en suposiciones sobre el formato de los mensajes.

Ejemplo trabajado

(Ejemplo ilustrativo de una app propia; los valores no son una captura.)

```
BlocClient → \\.\pipe\BlocLab.Status → BlocService
Mensaje: {version=1, request=GET_STATUS}
Respuesta: {status=READY}
Seguridad: ACL permite al usuario de la sesión; no acepta datos arbitrarios
```

El pipe sugiere el transporte, pero hay que confirmar el proceso propietario, ACL, modo de lectura y formato del protocolo. Un nombre predecible no implica vulnerabilidad por sí solo.

Ficha del análisis, separando observación de inferencia:

Dato	Valor (ilustrativo)	Tipo
Creador del pipe	`BlocService.exe`, cuenta de servicio de laboratorio	Observación (handle y proceso)
Flags de creación	`PIPE_ACCESS_DUPLEX`, `FILE_FLAG_FIRST_PIPE_INSTANCE`, `PIPE_TYPE_MESSAGE`, `PIPE_REJECT_REMOTE_CLIENTS`	Observación (argumentos en el código o el depurador)
DACL	Lectura/escritura al *logon SID* de la sesión; control total a SYSTEM	Observación (`GetSecurityInfo` en el lab)
Impersonación	`ImpersonateNamedPipeClient` antes de leer la configuración del usuario; `RevertToSelf` en salida normal y en error	Observación en código propio
Validación	Mensaje de tamaño fijo; versión distinta de 1 se rechaza	Observación en código propio
"Ningún otro proceso puede hacerse pasar por servidor"	—	Inferencia: la sostiene la primera instancia con bandera, no el nombre

Ampliación editorial basada en fuentes externas — Inspección práctica de puertos y conexiones ALPC en user-mode y WinDbg (`!alpc`)

Aunque ALPC (*Advanced Local Procedure Call*) es una interfaz interna de `ntdll.dll`, el analista necesita saber reconocer sus llamadas en un binario y auditar qué procesos están conectados a un puerto local durante una sesión de depuración (*Windows Internals 7th Ed., Cap. 3 / extensión `!alpc` de WinDbg*) (**Hecho confirmado**):

1. **Las tres primitivas de `ntdll.dll` y el espacio de nombres `\RPC Control\` :**

- **Servidor:** Llama a `ntdll!NtAlpcCreatePort(&PortHandle, &ObjectAttributes, &PortInformation)` publicando un objeto de tipo `ALPC Port` en el directorio del Object Manager (casi siempre bajo `\RPC Control\<NombreDelPuerto>` o `\Sessions\<N>\BaseNamedObjects\ ...`).
- **Cliente:** Se conecta con `ntdll!NtAlpcConnectPort` (o `NtAlpcConnectPortEx`) especificando el `UNICODE_STRING` del puerto destino.
- **Intercambio de mensajes:** Ambos lados usan `ntdll!NtAlpcSendWaitReceivePort(PortHandle, Flags, SendMessage, SendMessageAttributes, ReceiveMessage, &BufferLength, ReceiveMessageAttributes, Timeout)` .

2. **Cabecera `PORT_MESSAGE` en memoria (`+0x00..+0x27` , 40 bytes en x64 / 24 bytes en x86):**

Todo mensaje enviado o recibido por ALPC comienza con una cabecera fija `PORT_MESSAGE` seguida inmediatamente por los datos de la aplicación:

```
typedef struct _PORT_MESSAGE_X64 {
USHORT DataLength;     // +0x00 (2B): Bytes útiles del payload después de esta cabecera
USHORT TotalLength;    // +0x02 (2B): sizeof(PORT_MESSAGE) (0x28 en x64) + DataLength
USHORT Type;           // +0x04 (2B): LPC_REQUEST (1), LPC_REPLY (2), LPC_CONNECTION_REQUEST (10)
USHORT DataInfoOffset; // +0x06 (2B)
PVOID  ClientIdUniqueProcess; // +0x08 (8B): PID del emisor (CLIENT_ID.UniqueProcess)
PVOID  ClientIdUniqueThread;  // +0x10 (8B): TID del emisor (CLIENT_ID.UniqueThread)
ULONG  MessageId;      // +0x18 (4B): ID correlativo del mensaje
ULONG  Reserved;       // +0x1C (4B)
ULONG_PTR ClientViewSize; // +0x20 (8B): Total cabecera = 0x28 (40 bytes en x64)
} PORT_MESSAGE_X64;
```

3. **Comandos de inspección en solo lectura (Sysinternals y WinDbg de kernel):**

```
# En user-mode (consola elevada en la VM): listar todos los puertos ALPC abiertos y su proceso
dueño
handle.exe -a -p <PID_o_nombre> "ALPC Port"

# En WinDbg conectado al kernel de la VM (extensión !alpc):
kd> !alpc /lpp <dir_EPROCESS>    ; Lista todos los puertos ALPC creados y conectados por ese
proceso
kd> !alpc /p <dir_PortObject>    ; Muestra el proceso servidor, la cola de mensajes y todos los
clientes conectados
kd> !alpc /m <dir_Message>       ; Decodifica un mensaje ALPC en vuelo y quién lo envió
```

Práctica

Con un cliente/servidor propios ya preparados:

1. Listá los pipes visibles en la VM solo para ubicar el tuyo (en PowerShell, `Get-ChildItem \\.\pipe\` muestra nombres) y no abras pipes ajenos.
2. Inspeccioná en Process Explorer/Procmon la apertura del endpoint y seguí una sola solicitud inocua. Capturá nombre, PID, operación, respuesta y ACL observada.
3. Completá la tabla "Qué revisar en un pipe propio" con evidencia del código o del depurador.
4. Para RPC, usá el propio contrato local: anotá UUID/versión, secuencias de protocolo registradas, flags de registro y si hay callback de seguridad.
5. Para ALPC, describí una traza/documentación existente, sin enviar mensajes.

Entregable

Diagrama de dos procesos, mecanismo IPC, endpoint, identidad, controles, esquema del mensaje y evidencia. Incluí la tabla de revisión del pipe (o del registro RPC) con observación/inferencia. Ocultá datos personales; no registres payloads sensibles.

Repaso

▼ **¿Un named pipe siempre es local?**

> No. Puede ser local o accesible por red según cómo se publique y la configuración del sistema/servidor; `PIPE_REJECT_REMOTE_CLIENTS` rechaza clientes remotos.

▼ **¿Qué diferencia RPC de un pipe?**

> RPC expresa una llamada mediante una interfaz; un pipe es un canal de bytes/mensajes. RPC puede usar distintos transportes.

▼ **¿Qué derecho permite crear instancias de un pipe y por qué importa una ACL con escritura genérica?**

> `FILE_CREATE_PIPE_INSTANCE`. `FILE_GENERIC_WRITE` lo incluye, así que esa ACL también permite crear instancias, es decir, actuar como servidor.

▼ **¿Para qué sirve `FILE_FLAG_FIRST_PIPE_INSTANCE`?**

> Hace que la creación falle si el nombre ya existe, así el servidor detecta que otro proceso ocupó el pipe antes.

▼ **¿Quién controla hasta dónde puede impersonar un servidor de pipe?**

> El cliente, con `SECURITY_SQOS_PRESENT` y el nivel elegido al abrir el pipe; por defecto es `SecurityImpersonation`.

▼ **Un servidor RPC registra solo `ncalrpc`. ¿Queda accesible solo localmente?**

> No necesariamente: en un proceso con otros servidores RPC, la interfaz es invocable por cualquier endpoint del proceso. Se controla con un callback de seguridad.

▼ **¿`RpcServerRegisterAuthInfo` impide que se conecten clientes no autenticados?**

> No. Agrega una opción de autenticación, no un requisito; hacen falta chequeos propios.

▼ **¿Por qué ALPC se trata como concepto?**

> Es una interfaz interna de bajo nivel con contratos menos estables; el curso enseña reconocimiento, no clientes de endpoints del sistema.

Puerta optativa

Podés distinguir pipe, RPC y ALPC, seguir un intercambio propio, revisar ACL, primera instancia, clientes remotos, modo e impersonación de un pipe, explicar por qué en RPC el control debe estar en el callback de seguridad y no en el endpoint, e inspeccionar interfaces RPC y puertos ALPC en modo lectura.

Lo que queda fuera

El protocolo SMB de los pipes remotos en la red, la especificación completa del motor NDR64, la construcción de clientes para puertos ALPC del sistema, mailslots y otros mecanismos IPC legados no se desarrollan. No se enseña a enviar datos a pipes ni a endpoints RPC o ALPC ajenos.

Fuentes

- Microsoft Learn: Interprocess Communications, Named Pipes, RPC, ALPC ETW class.
- Microsoft Learn, pipes: CreateNamedPipe, Named Pipe Security and Access Rights, Impersonating a Named Pipe Client, Named Pipe Type, Read, and Wait Modes.
- Microsoft Learn, RPC seguro: Writing a Secure RPC Client or Server, Be Wary of Other RPC Endpoints Running in the Same Process, RpcServerRegisterAuthInfo doesn't deny unauthorized users, Interface Registration Flags, Protocol Sequence Constants.
- **Fundamentos previos:** call stack y procesos en 1.3; DCOM/RPC en 2D.8.

Etiquetas

Núcleo conceptual: procesos e IPC general. **Conocimiento externo:** IPC Windows, seguridad de named pipes y RPC. **Ejemplo ilustrativo:** protocolo y ficha BlocLab. **Laboratorio verificado:** no; ejemplo propio propuesto.

Ampliación editorial basada en fuentes externas — Reconstrucción estática de interfaces RPC (RPC_SERVER_INTERFACE y MIDL_SERVER_INFO)

Cuando un binario registra un servidor RPC mediante `RpcServerRegisterIf` / `RpcServerRegisterIf2` / `RpcServerRegisterIfEx`, IDA Pro o Ghidra no resuelven por defecto la relación entre números de procedimiento (opcodes RPC) y funciones C. El primer argumento (`IfSpec`, registro `RCX` en x64) apunta a la estructura `RPC_SERVER_INTERFACE` en `.rdata` o `.data` generada por el compilador MIDL (`rpcndr.h` del Windows SDK) *(Hecho confirmado)*:

Offset (x64)	Campo en `RPC_SERVER_INTERFACE`	Tipo y Significado en IDA / Ghidra
`+0x04`	`InterfaceId`	`RPC_SYNTAX_IDENTIFIER` (20 bytes: `GUID` de 16 bytes de la interfaz RPC + `MajorVersion` de 2B + `MinorVersion` de 2B).
`+0x30`	`DispatchTable`	`PRPC_DISPATCH_TABLE`: cantidad de funciones (`DispatchTableCount`) y puntero al despachador NDR (`NdrServerCall2` / `NdrServerCallAll`).
`+0x50`	`InterpreterInfo`	`const void*` que apunta a `MIDL_SERVER_INFO`: metadatos de decompilación del servidor.

Siguiendo el puntero `InterpreterInfo` hacia la estructura `MIDL_SERVER_INFO`:

- En `+0x00` (`pStubDesc`) reside el descriptor de tipos y sintaxis NDR.
- En `+0x08` (`DispatchTable`) reside el **arreglo directo de punteros a las funciones C reales del servidor** (`ServerRoutineTable[0..N-1]`). Navegar a esta dirección revela inmediatamente la lista completa de procedimientos remotos expuestos por el binario.

Automatización práctica: En análisis defensivo o auditoría de servicios de Windows, herramientas de código abierto como `RpcView` o un script de **IDAPython** que automatice la navegación `InterfaceId → InterpreterInfo → DispatchTable` permiten extraer y tipar todas las rutinas del servidor RPC en cuestión de segundos.

---

5. Inspección Práctica de Puertos ALPC con `handle64.exe` y `!alpc` en WinDbg

Declaración de Entorno

`Laboratorio verificado: no; requiere VM/toolchain`. Requiere Sysinternals `handle64.exe` ejecutándose con privilegios administrativos y una sesión de depuración de kernel en WinDbg.

5.1. Enumeración en Modo Usuario con `handle64.exe`

Desde una consola elevada, la utilidad `handle64.exe` de Sysinternals permite filtrar la tabla de handles de cualquier proceso (por ejemplo, el servicio de programación de tareas o `lsass.exe`) restringiendo la salida al

tipo de objeto del ejecutivo `ALPC Port` mediante los modificadores `-a` (volcar todos los tipos de objetos) y `-p`:

```
C:\Tools\Sysinternals> handle64.exe -a -p lsass.exe | findstr /i "ALPC"
   228: ALPC Port      \RPC Control\lsapolicylookup
   254: ALPC Port      \RPC Control\lsasspirpc
   2bc: ALPC Port      \RPC Control\protected_storage
```

Para mapear el proceso que actúa como cliente de dicho endpoint, se busca el handle de tipo `ALPC Port` en procesos cliente que apunte al mismo nombre simbólico en el directorio `\RPC Control` de la tabla de objetos del Object Manager.

5.2. Inspección de Estructuras Internas (`_KALPC_PORT`) en WinDbg con `!alpc`

En una sesión viva de depuración de kernel (KD), la extensión `!alpc` de WinDbg interpreta los grafos de conexión entre puertos servidor y puertos de comunicación cliente:

```
kd> !alpc /lpl
Scanning connection ports ...
  Port: ffffb609148e2c90  Owner: ffffb60912c95080 (lsass.exe)  Name: \RPC Control\lsasspirpc
  Port: ffffb60915a1a080  Owner: ffffb609121a80c0 (services.exe) Name: \RPC Control\spoolerpc

kd> !alpc /p ffffb609148e2c90
Port @ ffffb609148e2c90
  Type                      : ConnectionPort
  OwnerProcess              : ffffb60912c95080 (lsass.exe)
  ConnectionPort            : ffffb609148e2c90
  ServerCommunicationPort   : 0000000000000000
  ClientCommunicationPort   : 0000000000000000
  Flags                     : 0x00000000
  State                     : 0x00000001
  SequenceNo                : 0x00000001
  PortContext               : 0000000000000000
  HasCompletionList         : No
  ConnectionRefused         : No
  Disconnected              : No
  Closed                    : No
  FlushOnClose              : Yes
  MainQueueLength           : 0x00000000
  PendingQueueLength        : 0x00000000
  LargeMessageQueueLength   : 0x00000000

kd> !alpc /c ffffb609148e2c90
Connections for port ffffb609148e2c90
  CommunicationInfo @ ffffb609182143a0
    ConnectionPort          : ffffb609148e2c90 (lsasspirpc)
    ServerCommunicationPort : ffffb60918215920 [lsass.exe]
    ClientCommunicationPort : ffffb60918215410 [svchost.exe]
```

Referencias y Fuentes Técnicas

- Garnier, C. (2008). *Windows Vista / Server 2008 ALPC Internals*. SyScan Conference.
- Russinovich, M., Solomon, D., & Ionescu, A. (2012). *Windows Internals, Part 2* (6th ed.). Microsoft Press.
- Microsoft Learn. (2023). *ALPC Reference and Port Message Structures*.

Excepciones de Windows y debugging

> **Rama optativa 2D**
>
> La práctica usa una excepción esperada en un programa propio. No busca provocar corrupción ni explotar un fallo.

> **Propósito**
>
> Entender cómo el depurador reconstruye el stack y cómo Windows transfiere el control cuando una función falla, distinguiendo el modelo histórico x86 del unwind x64. Al terminar deberías poder explicar **quién** vio una excepción, **en qué orden** y **por qué** una excepción de primera oportunidad no significa que el programa haya fallado.

Prerrequisitos

- *Obligatorios:* stack y llamadas (1.2, 2.5); 2D.2 memoria/loader.
- *Recomendados:* WinDbg/TTD (2.19); C++ profundo (2.18); taxonomía anti-análisis (2.15).

Modelos

En x86, SEH usa registros/estructuras de excepción y el runtime del compilador puede agregar handlers de lenguaje; los detalles dependen del compilador y de mitigaciones como SafeSEH/SEHOP. En x64, el unwinder consulta metadatos de funciones: `.pdata` indexa entradas `RUNTIME_FUNCTION` y `.xdata` describe cómo deshacer prólogos y recuperar registros; algunas funciones hoja no requieren entrada.

WinDbg puede mostrar el evento de excepción, contexto y stack. En TTD, una traza de user mode permite avanzar o retroceder por esa ejecución, pero no reemplaza símbolos, documentación ni comprensión del estado. La salida cambia con arquitectura, build y símbolos disponibles.

Qué guarda el sistema cuando ocurre una excepción

Cuando el procesador o el software genera una excepción, Windows guarda dos estructuras:

- el **registro de contexto** (`CONTEXT`): el estado de la máquina del hilo en ese momento, dependiente de la arquitectura;
- el **registro de excepción** (`EXCEPTION_RECORD`): el **código** de excepción, si es **continuable**, un puntero a otro registro (excepciones anidadas), la **dirección** donde ocurrió y un arreglo de **parámetros** adicionales.

Para el análisis, el par contexto + registro es la evidencia primaria: la dirección y el código dicen qué pasó; el contexto dice con qué registros. En WinDbg, `.exr` muestra un registro de excepción y `.ecxr` cambia el contexto al de la excepción guardada, para que `k` muestre el stack **de ese momento** y no el del depurador.

Orden de despacho en user mode

Microsoft documenta este orden para buscar quién maneja una excepción de user mode:

1. **Depurador, primera oportunidad.** Si el proceso está siendo depurado, el sistema notifica al depurador.
2. **Vectored exception handlers (VEH).** Se registran con `AddVectoredExceptionHandler`, no dependen del marco de llamada, se aplican a todo el proceso y se llaman en el orden en que se agregaron, **después** de la primera oportunidad del depurador y **antes** de que comience el desenrollado.
3. **Handlers basados en marcos.** El sistema recorre los marcos del hilo desde el actual hacia atrás: en x86, la cadena SEH; en x64, la información de `.pdata` / `.xdata`. Aquí entran `__try/__except` y los handlers de lenguaje como el de C++.
4. **Depurador, segunda oportunidad.** Si nadie la manejó y hay depurador, se lo notifica de nuevo.
5. **Manejo por defecto.** Sin depurador o si este no la maneja, el sistema aplica la acción por defecto, que para la mayoría de las excepciones es terminar el proceso.

En kernel mode el sistema busca handlers en la pila del kernel; si no hay ninguno, el sistema se detiene (bugcheck, ver 6.3).

Un filtro de `__try/__except` decide con su valor de retorno si el bloque maneja la excepción, si se sigue buscando o si se continúa la ejecución. Cuando un handler acepta la excepción, el sistema **desenrolla** los marcos intermedios y ejecuta los bloques `__finally`/destructores que correspondan antes de llegar al handler.

Primera y segunda oportunidad en el depurador

Para WinDbg, cada tipo de excepción tiene un **estado de corte**: cortar en la primera oportunidad (`sxe`), cortar solo en la segunda (`sxd`), solo informar (`sxn`) o ignorar (`sxi`). Por defecto, por ejemplo, una violación de acceso corta en la primera oportunidad, mientras que la excepción de C++ (`eh`) corta solo en la segunda. `.lastevent` muestra el último evento.

Consecuencias para leer evidencia:

- Una **excepción de primera oportunidad** es normal en muchos programas: excepciones C++ lanzadas y atrapadas, sondeos de memoria protegidos por `__try`, mecanismos de runtime. **No** significa un fallo.
- Una **segunda oportunidad** sí significa que nadie la manejó.
- Un **VEH** registrado por el programa ve todas las excepciones del proceso. Si al analizar un binario encontrás `AddVectoredExceptionHandler`, anotá qué hace el handler: puede ser diagnóstico legítimo (registro de fallos) o una técnica anti-depuración que usa excepciones a propósito (2.15). La llamada sola no decide cuál.

En binarios MSVC, las excepciones de C++ se lanzan como una excepción de Windows con un código propio del runtime (habitualmente `0xE06D7363`, conocimiento externo de reversing); reconocerlo evita confundir un `throw` controlado con un fallo.

Ejemplo trabajado

(Secuencia ilustrativa de un programa C++ propio que lanza y captura una excepción documentada.)

```
app!RunCase → app!ParseOwnInput → throw/catch controlado
WinDbg: evento de excepción → handler del runtime → app!RunCase
x64: .pdata señala RuntimeFunction; .xdata describe unwind del prólogo
```

Al recorrer el stack, identificá qué frames son la ruta de llamada y cuál maneja el evento. La mera presencia de `.pdata` no demuestra que una excepción haya ocurrido; es metadata disponible para unwind.

Cronología de lectura (ilustrativa) del mismo caso, en el orden en que aparece cada dato:

Paso	Qué se ve	Herramienta	Interpretación prudente
1	Evento de primera oportunidad, código de excepción C++	Mensaje de WinDbg / `.lastevent`	Se lanzó una excepción; todavía no se sabe si alguien la maneja
2	Registro de excepción con dirección dentro del runtime	`.exr -1`	El `throw` pasa por el runtime de C++
3	Stack de la excepción hasta `app!ParseOwnInput`	`.ecxr` + `k`	Ruta de llamada al momento del `throw`
4	El programa continúa y no hay segunda oportunidad	Ejecución	Un `catch` la manejó
5	Ejecución reanudada en `app!RunCase`	Breakpoint en el `catch`	Coincide con el handler esperado

Ampliación editorial basada en fuentes externas — Verificación de marcos de pila y detección de *Call Stack Spoofing* mediante `.pdata` y validación de sitios de `CALL`

En la lección 2.15 vimos que las técnicas de *Call Stack Spoofing* fabrican marcos de pila sintéticos para ocultar el origen real de una llamada al sistema, y en 2D.11 vimos el layout binario de `RUNTIME_FUNCTION` (`.pdata`) y `UNWIND_INFO` (`.xdata`) (*Microsoft Learn: x64 stack usage y RtlVirtualUnwind*) (**Hecho confirmado**). Un analista o motor defensivo verifica la autenticidad de una pila de llamadas en x64 aplicando tres comprobaciones encadenadas sobre cada marco:

1. **Respaldo de módulo (`RtlLookupFunctionEntry`):**

Para cada `ControlPc` (`RIP` actual o dirección de retorno en la pila), debe existir una entrada `RUNTIME_FUNCTION` válida dentro de la sección `.pdata` de un módulo PE firmado/respaldado en disco (`Image` en el VAD, no `MEM_PRIVATE`). Las únicas funciones x64 sin `.pdata` son las *funciones hoja* (*leaf functions*), que por definición de la ABI **no pueden modificar `RSP` ni llamar a ninguna otra función** (solo pueden estar en el frame `0` del tope de la pila, nunca en un frame intermedio).

2. **Coherencia del tamaño de marco (`UNWIND_CODE`):**

La distancia entre el `RSP` de un marco y el `RSP` del marco padre debe coincidir exactamente con la suma de los códigos `UWOP_ALLOC_SMALL` (`OpInfo * 8 + 8`), `UWOP_ALLOC_LARGE` y `UWOP_PUSH_NONVOL` (`8` bytes por registro no volátil salvado) declarados en `UNWIND_INFO` .

3. **Validación de la instrucción previa a `ReturnAddress` (detección de *Gadgets*):**

Cuando `call destino` se ejecuta, empuja en la pila la dirección de la instrucción inmediatamente siguiente (`ReturnAddress`). Por lo tanto, los **2 a 7 bytes anteriores a `ReturnAddress`** en memoria ejecutable deben decodificarse como una instrucción `CALL` legítima:

Bytes previos a `ReturnAddress`	Instrucción x64	Interpretación en auditoría de pila
`E8 xx xx xx xx` (`ReturnAddress - 5`)	`call rel32`	Llamada directa normal. Se verifica que el destino calculado (`ReturnAddress + rel32`) coincida con el inicio de la función hija.
`FF 15 xx xx xx xx` (`ReturnAddress - 6`)	`call qword ptr [rip+disp32]`	Llamada indirecta vía IAT o puntero global (normal en llamadas a DLLs).
`FF D0` .. `FF D7` o `41 FF D0` .. `D7` (`-2` / `-3`)	`call rax` .. `r15`	Llamada indirecta por registro (vtable, callback o `GetProcAddress`).
`FF 23` / `FF 26` / `FF 27` (`jmp [rbx]` / `[rsi]` / `[rdi]`)	`jmp qword ptr [reg]`	**Anomalía crítica (*Gadget de Call Stack Spoofing*):** la dirección en la pila apunta después de un `JMP` , no de un `CALL` , o el `call rel32` apunta a una función distinta de la que está en el frame hijo.

Inspección rápida en WinDbg ante una pila sospechosa:

```
0:000> kn                                ; Lista los números de frame, Child-SP y Call Site
0:000> !fnent <Call_Site>                ; Verifica la entrada RUNTIME_FUNCTION y el tamaño del
marco en .xdata
0:000> ub <Call_Site> L1                 ; Desensambla la instrucción inmediatamente ANTERIOR al
retorno (debe ser CALL)
```

Anatomía Binaria de UNWIND_INFO y Decodificación de UNWIND_CODE en x64

En arquitecturas x64, la estructura `UNWIND_INFO` almacenada en la sección `.xdata` gobierna cómo el kernel y los depuradores desenrollan la pila hacia atrás. Cada registro de desenrollado tiene la siguiente estructura binaria en memoria:

```
Estructura UNWIND_INFO (.xdata):
   +0x00: UBYTE Version : 3;      // Versión de unwind (típicamente 1 o 2)
   +0x00: UBYTE Flags   : 5;      // 0x01=EHANDLER (__except), 0x02=UHANDLER (__finally), 0x04=CHAININFO
   +0x01: UBYTE SizeOfProlog;     // Longitud del prólogo de la función en bytes
   +0x02: UBYTE CountOfCodes;     // Número de entradas en el arreglo UnwindCode[]
   +0x03: UBYTE FrameRegister : 4;// Registro de marco (0=ninguno, 5=RBP, etc.)
   +0x03: UBYTE FrameOffset   : 4;// Offset del registro de marco escalado (*16)
   +0x04: UNWIND_CODE UnwindCode[CountOfCodes]; // Arreglo de operaciones de desenrollado
```

Decodificación de Slots de `UNWIND_CODE` (2 bytes por slot)

Cada elemento de `UnwindCode` ocupa exactamente 16 bits (2 bytes) y describe una operación que el prólogo ejecutó y que el desenrollador debe revertir:

- `CodeOffset` **(8 bits bajos):** Offset en bytes desde el inicio de la función donde se realizó la operación.
- `UnwindOp` **(4 bits):** Operación de reversión de pila:
- `0` = `UWOP_PUSH_NONVOL` : El prólogo hizo un `push` de un registro no volátil. Operación inversa: pop del registro indicado en `OpInfo` .
- `1` = `UWOP_ALLOC_LARGE` : Asignó un tamaño grande en stack (ocupa 2 o 3 slots sucesivos de `UNWIND_CODE` para especificar el tamaño).

- `2` = `UWOP_ALLOC_SMALL`: Asignó entre 8 y 128 bytes en la pila. Operación inversa: sumar `(OpInfo * 8 + 8)` a `RSP`.
- `3` = `UWOP_SET_FPREG`: Estableció el puntero de marco de pila. Operación inversa: restaurar `RSP` desde `FrameRegister`.
- `4` = `UWOP_SAVE_NONVOL`: Guardó un registro no volátil en la pila usando `mov` en lugar de `push` (ocupa 2 slots para codificar el offset).
- `6` = `UWOP_EPILOG`: Define los epílogos de la función (en `UNWIND_INFO` versión 2, desde Windows 8.1 / VS 2013).
- `10` = `UWOP_PUSH_MACHFRAME`: Empujó un marco de interrupción o trampa de máquina de hardware (ocupa 1 slot; empuja `SS`, `RSP`, `EFLAGS`, `CS` y `RIP`).
- `OpInfo` **(4 bits altos):** Índice del registro x64 involucrado (`0`=RAX, `1`=RCX, `2`=RDX, `3`=RBX, `4`=RSP, `5`=RBP, `6`=RSI, `7`=RDI, `8..15`=R8..R15).

Verificación Forense frente a Call Stack Spoofing

Cuando una herramienta de seguridad (EDR) o un analista en WinDbg valida una pila con `!fnent`, el kernel simula la ejecución inversa de cada `UNWIND_CODE`. Si un atacante utilizó *Call Stack Spoofing* fabricando una dirección de retorno falsa pero no ajustó `RSP` para coincidir exactamente con la sumatoria de los `UWOP_ALLOC_SMALL` / `UWOP_PUSH_NONVOL` de la función invocadora, o si la instrucción previa a la dirección de retorno no es un opcode `CALL` legítimo (`E8 rel32` o `FF /2`), el desenrollado falla con inconsistencia de marco y delata la intrusión.

Práctica

1. Compilá o usá un programa propio mínimo que lance/capture una excepción prevista, sin leer datos externos.
2. En WinDbg, anotá arquitectura, evento, excepción y `k` / `kv` antes y después del handler. Usá `.exr -1` y `.ecxr` para leer el registro y el contexto de la excepción.
3. Cambiá **solo para tu sesión** el estado de corte de la excepción C++ (`sxe eh`) para verla en primera oportunidad, y comparalo con el comportamiento por defecto.
4. En x64, inspeccioná la entrada de función/unwind con símbolos y comandos documentados para tu build; en x86, describe el frame SEH observado sin modificar la cadena.
5. Opcional: agregá a tu programa un VEH que solo registre el código de excepción y verificá en qué momento se llama respecto del depurador y del `catch`.
6. Opcional: grabá el mismo caso de user mode con TTD y navega a la excepción. Guardá el trace solo mientras lo necesites y borrá la copia al finalizar.

Entregable

Diagrama de control antes/después de la excepción, arquitectura, stack, handler y límites de símbolos; la cronología con primera/segunda oportunidad y, si lo hiciste, el momento en que actúa el VEH. Separá información observada de interpretación del compilador.

Repaso

▼ **¿Qué función cumplen `.pdata` y `.xdata` en x64?**

> `.pdata` localiza metadata de funciones; `.xdata` describe información de unwind/handlers necesaria para recuperar estado y recorrer el stack.

▼ **¿x86 SEH y x64 unwind son el mismo mecanismo?**

> No. Tienen representación y reglas distintas; el compilador y las mitigaciones también afectan lo observable.

▼ **¿Un frame de stack prueba que esa función lanzó la excepción?**

> No. Es una pista del camino actual; se confirma con instrucción, contexto y flujo de control.

▼ **¿En qué orden se buscan handlers en user mode?**

> Depurador (primera oportunidad), handlers vectorizados, handlers basados en marcos, depurador (segunda oportunidad) y manejo por defecto.

▼ **¿Una excepción de primera oportunidad indica que el programa falló?**

> No. Muchas se lanzan y se manejan normalmente; solo la segunda oportunidad indica que nadie la manejó.

▼ **¿Para qué sirve `.ecxr`?**

> Pone el contexto del depurador en el registro de contexto de la excepción, para que el stack corresponda al momento del fallo.

▼ **Encontrás `AddVectoredExceptionHandler` en un binario. ¿Qué concluís?**

> Que el programa ve todas las excepciones del proceso antes que los handlers por marco. Hay que leer qué hace el handler: puede ser diagnóstico o anti-depuración.

> **Puerta optativa**
>
> Podés describir una excepción propia en WinDbg, leer registro y contexto, explicar el orden de despacho (incluidos VEH y primera/segunda oportunidad), leer el stack y explicar la diferencia general entre SEH x86 y unwind x64.

Lo que queda fuera

Las tablas y descriptores de excepciones tipadas de C++ (`throw` / `catch` y `FuncInfo` de MSVC), los manejadores de lenguaje en detalle, SafeSEH/SEHOP y la depuración post mortem configurada en el sistema no se desarrollan en esta lección (el despacho de CFG/kCFG a nivel ISA se estudia en 6.9). No se cambia ninguna configuración global de depuración.

Fuentes

- Microsoft Learn: x64 exception handling, Structured Exception Handling, Debugging a user-mode process using WinDbg.
- Microsoft Learn, despacho y depurador: Exception Dispatching, Vectored Exception Handling, try-except statement, Controlling Exceptions and Events, .exr, .ecxr.
- **Fundamentos previos:** stack en 1.2, unwind C++ en 2.18, WinDbg en 2.19, anti-análisis en 2.15.

Etiquetas

Núcleo conceptual: excepciones y stack como conceptos generales. **Conocimiento externo:** SEH/unwind, despacho de excepciones, VEH y control de excepciones en WinDbg; el código de excepción C++ de MSVC como dato de reversing. **Ejemplo ilustrativo:** flujo y cronología de excepción. **Laboratorio verificado:** no.

Ampliación editorial basada en fuentes externas — Layout binario de RUNTIME_FUNCTION (.pdata) y UNWIND_INFO (.xdata) en x64

A diferencia de x86 (donde la lista SEH se enlaza en la pila mediante `FS:[0]`), en Windows x64 el desenrollado de pila y el despacho de excepciones se rigen por tablas estáticas en el PE sin tocar la pila en el prólogo (Microsoft Learn: *x64 exception handling*) *(Hecho confirmado)*:

Estructura y Sección	Offset / Tamaño	Campos Binarios y Significado
`RUNTIME_FUNCTION` (Sección `.pdata`)	`12 bytes` fijos por función	• `+0x00`: `DWORD BeginAddress` (RVA de inicio de la función). • `+0x04`: `DWORD EndAddress` (RVA de fin de la función). • `+0x08`: `DWORD UnwindInfoAddress` (RVA que apunta a la cabecera en `.xdata`).
`UNWIND_INFO` (Sección `.xdata`)	`4 bytes` cabecera + array variable	• `+0x00`: `Version : 3` (bits 0..2, vale 1 o 2) y `Flags : 5` (bits 3..7: `0x01 = UNW_FLAG_EHANDLER`, `0x02 = UNW_FLAG_UHANDLER`, `0x04 = UNW_FLAG_CHAININFO`). • `+0x01`: `UBYTE SizeOfProlog` (longitud en bytes del prólogo). • `+0x02`: `UBYTE CountOfCodes` (cantidad de slots de 2 bytes en `UnwindCode[]`). • `+0x03`: `UBYTE FrameRegisterAndOffset` (registro base y desplazamiento de marco). • `+0x04`: Arreglo `UNWIND_CODE[]` (códigos como `UWOP_PUSH_NONVOL=0`, `UWOP_ALLOC_SMALL=2` indicando qué registros se salvaron y cuánto creció `RSP`). • **Tras** `UnwindCode[]` (alineado a 4B): Si `Flags & 0x03`, sigue la **RVA de 32 bits de** `ExceptionHandler` (`__C_specific_handler`, `__CxxFrameHandler3` o `__CxxFrameHandler4`) y los datos de excepción (`SCOPE_TABLE` / `FuncInfo`).

Inspección en vivo con WinDbg: El comando `!fnent <dirección_de_función>` decodifica en vivo tanto la entrada `RUNTIME_FUNCTION` como todos los códigos `UNWIND_CODE` de `.xdata`, siendo fundamental para validar si una dirección de retorno en la pila fue falsificada (Call Stack Spoofing).

ETW y telemetría defensiva

> **Propósito**
>
> Comprender cómo se genera, habilita y consume telemetría ETW, dominar la sesión del sistema **NT Kernel Logger**, analizar la **anatomía binaria del formato ETL**, y auditar de forma reproducible las técnicas de evasión y **manipulación de ETW (ETW Patching y Blinding)** tanto en modo usuario como en modo kernel.

Prerrequisitos

- *Obligatorios:* procesos/hilos (1.3); WinDbg y TTD.
- *Recomendados:* 2D.10 IPC; conceptos de excepciones en 2D.11; registro de Windows (2D.3).

---

1. El Modelo de Arquitectura ETW

ETW separa tres roles clave: **provider** (emite eventos), **controller** (inicia sesión, habilita providers y define buffers/archivo) y **consumer** (procesa eventos en vivo o desde un archivo ETL).

ETW permite habilitar o deshabilitar providers **en caliente**, sin reiniciar el programa. Un provider registrado que nadie habilitó no emite eventos: por eso "el programa no generó nada" y "nadie estaba escuchando" se ven idénticos desde afuera.

Tipos de Providers y qué implican para decodificar

Tipo	Cómo emite	Dónde está la información para decodificar	Sesiones simultáneas
MOF (clásico)	`RegisterTraceGuids` / `TraceEvent`	Clases MOF en WMI	Una sola
WPP	`RegisterTraceGuids` / `TraceEvent`	Archivos TMF (normalmente en el PDB del binario)	Una sola
Manifest-based	`EventRegister` / `EventWrite`	Manifiesto XML registrado en el sistema	Hasta ocho
TraceLogging	`TraceLoggingRegister` / `TraceLoggingWrite`	El propio evento binario (autodescriptivo)	Hasta ocho

---

2. La Sesión de Kernel: `NT Kernel Logger`

A diferencia de los proveedores de modo usuario, el kernel de Windows expone una sesión de rastreo reservada de alto rendimiento denominada `NT Kernel Logger`.

Identificador Global y Banderas de Activación

- **GUID de Sesión:** `SystemTraceControlGuid = {9e814aad-3204-11d2-9a82-006008a86939}`.
- La sesión no se suscribe a proveedores por GUID individual, sino a través del campo **`EnableFlags`** de la estructura `EVENT_TRACE_PROPERTIES`:

Bandera de Kernel (`EnableFlags`)	Valor Hex (Constante)	Telemetría Capturada
`EVENT_TRACE_FLAG_PROCESS`	`0x00000001` (bit 0 = 1)	Creación y terminación de procesos (PID, PPID, SID, ImageFileName).
`EVENT_TRACE_FLAG_THREAD`	`0x00000002` (bit 1 = 2)	Creación, destrucción y conmutación de contexto de hilos (*Context Switch*).
`EVENT_TRACE_FLAG_IMAGE_LOAD`	`0x00000004` (bit 2 = 4)	Carga de módulos ejecutables (`.exe` , `.dll` , `.sys`) con dirección base y tamaño.
`EVENT_TRACE_FLAG_DISK_IO`	`0x00000100` (bit 8 = 256)	Operaciones de lectura y escritura física en disco.
`EVENT_TRACE_FLAG_NETWORK_TCPIP`	`0x00010000` (bit 16)	Conexiones de red TCP/UDP (send, receive, connect, disconnect).
`EVENT_TRACE_FLAG_REGISTRY`	`0x00020000` (bit 17)	Creación, modificación y borrado de claves y valores del Registro.

(Nota técnica sobre EnableFlags: Valores hexadecimales canónicos definidos en `evntrace.h` del Windows SDK. Nótese que `EVENT_TRACE_FLAG_DISK_IO` corresponde estrictamente al bit 8 = `0x00000100` / 256 decimal; no debe confundirse con cero ni con valores truncados).

Control Práctico de la Sesión con `logman`

```
:: Iniciar la captura de kernel a un archivo ETL
logman start "NT Kernel Logger" -p "Windows Kernel Trace" (process,thread,img,net) -ets -o
kernel_trace.etl

:: Consultar estado y estadísticas de buffers y eventos perdidos
logman query "NT Kernel Logger" -ets

:: Detener la sesión
logman stop "NT Kernel Logger" -ets
```

---

3. Anatomía y Formato Binario del Archivo ETL (`.etl`)

Un archivo ETL generado por ETW es una secuencia binaria estructurada de bloques de 64 KB organizados de la siguiente manera:

A. Cabecera de Buffer (`WMI_BUFFER_HEADER`)

Cada bloque físico de 64 KB dentro del archivo `.etl` comienza con una cabecera de 32 a 48 bytes:

- **`BufferSize`:** Tamaño del bloque (típicamente `0x10000` = 64 KB).
- **`SavedOffset`:** Desplazamiento hasta el último byte de evento escrito en el buffer.

- `CurrentOffset`: Posición del cursor de escritura.
- `ReferenceTime`: Timestamp de inicio del bloque en unidades de temporizador de alta resolución (`RDTSC` o `KeQueryPerformanceCounter`).

B. El Primer Registro: `TRACE_LOGFILE_HEADER`

El primer evento del archivo describe los metadatos globales de la captura:

- `BufferSize` y `VersionDetail`: Versión del sistema operativo que generó la traza.
- `StartTime` y `EndTime`: Tiempos de inicio y cierre en formato `FILETIME` de 64 bits.
- `TimerResolution`: Resolución del temporizador del kernel (en unidades de 100 ns).
- `NumberOfProcessors` y `CpuSpeedInMHz`: Hardware subyacente.
- `PointerSize`: `4` para arquitecturas x86, `8` para x64.
- `LogFileMode`: Configuración de almacenamiento (`EVENT_TRACE_FILE_MODE_SEQUENTIAL`, `CIRCULAR`, etc.).

C. Registros de Eventos (`EVENT_RECORD` / `EVENT_HEADER`)

Cada evento individual dentro de los buffers contiene una cabecera estandarizada de 80 bytes:

1. `Size` (`USHORT`): Longitud total del registro del evento en bytes.
2. `HeaderType` y `Flags`: Indican si el evento es de 32 o 64 bits y qué temporizador se usó.
3. `ProviderId` (`GUID`): Identificador de 16 bytes del proveedor emisor.
4. `EventDescriptor` **(16 bytes):**

- `Id` (Identificador del evento dentro del proveedor).
- `Version` (Revisión del esquema).
- `Channel` (Canal de destino: Admin, Operational, Analytic, Debug).
- `Level` (Severidad: Crítico, Error, Warning, Info).
- `Opcode` (Operación: Start, Stop, Info).
- `Task` y `Keyword` (Máscara de 64 bits para filtrado).

5. `ProcessId` y `ThreadId`: PID y TID del hilo que emitió el evento.
6. `TimeStamp`: Marca temporal de 64 bits.
7. `UserData` / `UserDataLength`: Payload binario con los argumentos específicos del evento.

---

4. Técnicas de Manipulación y Cegado de ETW (ETW Tampering / Blinding)

En ciberseguridad defensiva e ingeniería inversa de malware, el análisis de telemetría debe contemplar cómo los atacantes neutralizan la generación de eventos:

A. Parcheo en Modo Usuario (`ntdll!EtwEventWrite`)

Cuando un proceso en modo usuario desea impedir que componentes de seguridad reciban eventos (como telemetría de inyección de código o AMSI), recurre a **ETW Patching**:

1. El proceso localiza la función exportada `ntdll!EtwEventWrite`.
2. Modifica los permisos de la página de memoria usando `VirtualProtect` a `PAGE_EXECUTE_READWRITE`.
3. Sobrescribe el prólogo de la función con una instrucción de retorno anticipado:
    - En x64: `xor eax, eax; ret` (codificado como `31 C0 C3` por ensambladores GAS/NASM o `33 C0 C3` por MSVC/MASM) o simplemente `ret` (`C3`).
4. Al retornar `0` (`STATUS_SUCCESS`), la aplicación cree que el evento se registró con éxito, pero ningún byte de telemetría llega al despachador de ETW.

Detección Defensiva: Un monitor de integridad lee la imagen limpia de `ntdll.dll` directamente desde el disco (`\SystemRoot\System32\ntdll.dll`) y compara los primeros 16 bytes de `EtwEventWrite` contra los bytes en memoria. Cualquier discordancia (`0xC3`, `0x31 0xC0` o `0x33 0xC0` en los primeros bytes) delata un parcheo activo de ETW.

B. Cegado en Modo Kernel (ETWti Blinding)

A nivel de kernel, los controladores de seguridad se alimentan del proveedor `Microsoft-Windows-Threat-Intelligence` (Lección 6.7). Para cegar este sensor:

- Un atacante con primitiva de escritura en kernel (vector BYOVD) localiza el handle de registro apuntado por `nt!EtwThreatIntProvRegHandle`.
- En la estructura `_ETW_GUID_ENTRY`, manipula el campo `ProviderEnableInfo`, estableciendo la máscara de habilitación a cero (`EnableMask = 0`).
- Al quedar deshabilitado el proveedor a nivel de hipervisor/kernel, el sensor EDR deja de recibir eventos de inyección de hilos y acceso a memoria sin generar alertas visibles en user mode.

---

5. Repaso

▼ **¿Qué rol cumple el controller?**

> Define y controla una sesión ETW, habilita providers y establece cómo registrar eventos.

▼ **¿Qué diferencia provider y consumer?**

> El provider produce eventos; el consumer los lee e interpreta desde una sesión o archivo.

▼ **¿Una traza sin eventos prueba que no ocurrió actividad?**

> No. El provider puede no estar habilitado, no emitir ese evento, perder datos o haber sido parcheado (`EtwEventWrite`).

▼ **¿Qué es la sesión NT Kernel Logger?**

> Es la sesión del sistema reservada para telemetría de bajo nivel del kernel (procesos, hilos, DLLs, disco, red), controlada por `SystemTraceControlGuid`.

▼ **¿Cómo detectás en memoria un parcheo de ETW en user mode?**

> Comparando los primeros bytes de `ntdll!EtwEventWrite` en memoria contra los bytes del archivo original en disco; si empieza con `0xC3` o `0x31 0xC0`, fue alterado.

Fuentes

- Microsoft Learn: About Event Tracing, Configuring NT Kernel Logger, EVENT_TRACE_PROPERTIES structure.
- Microsoft Learn: WMI_BUFFER_HEADER, EVENT_RECORD structure.
- Repositorio y Laboratorio: Plantilla de auditoría en WinDbg en `plantillas_codigo/auditar_callbacks.wdbg`.

Ampliación editorial basada en fuentes externas — Extracción estática del Provider GUID (EventRegister) y detección de ETW Patching

Al analizar un binario en IDA o Ghidra que interactúa con ETW, existen dos patrones opuestos: la telemetría legítima instrumentada por el desarrollador y el intento de malware de cegar la monitorización del sistema (Microsoft Learn: *EventRegister* y *EtwEventWrite*) *(Hecho confirmado)*:

1. **Localización del Provider GUID en aplicaciones legítimas o instrumentadas:**
   Al buscar referencias cruzadas a `advapi32!EventRegister` (o `ntdll!EtwEventRegister`), la firma en x64 recibe:

   ```
   lea rcx, [rip + GUID_Provider] ; 1.er arg (RCX): Puntero a los 16 bytes del GUID en .rdata
   xor edx, edx                   ; 2.° arg (RDX): EnableCallback (opcional)
   xor r8d, r8d                   ; 3.er arg (R8) : CallbackContext
   lea r9, [rip + g_RegHandle]    ; 4.° arg (R9) : Puntero global donde se guarda el REGHANDLE
   call cs:EventRegister
   ```

   El valor guardado a través de `R9` (`REGHANDLE`) es el manejador global que **luego usarán todas las llamadas subsiguientes a `EventWrite` / `EtwEventWrite`** para emitir eventos. Leer los 16 bytes apuntados por `RCX` permite cruzar de inmediato el GUID con `logman query providers` en la VM para conocer la identidad y finalidad del proveedor.
2. **Reconocimiento defensivo de *ETW Patching* en User-Mode:**
   Ciertas muestras evasivas intentan silenciar la telemetría del propio proceso (como los eventos del runtime de `.NET CLR` o `AMSI`) resolviendo dinámicamente `ntdll!EtwEventWrite` (o `NtTraceEvent`), cambiando la protección de su página de memoria a `PAGE_EXECUTE_READWRITE` y sobrescribiendo su primer byte con `ret` (`0xC3`) o `xor eax, eax ; ret` (`33 C0 C3`). En el depurador, el analista lo detecta comparando los primeros bytes de `ntdll!EtwEventWrite` en memoria frente a la copia intacta de `ntdll.dll` en disco o monitoreando llamadas a `VirtualProtect` sobre rangos de memoria de `ntdll`.

---

5. ETW Threat Intelligence (`Microsoft-Windows-Threat-Intelligence` / ETWti)

Introducido para alimentar directamente a Microsoft Defender for Endpoint (MDE) y sensores EDR de terceros certificados por el programa MVI (*Microsoft Virus Initiative*), el proveedor **`Microsoft-Windows-Threat-Intelligence` (ETWti)** —identificado por el GUID `{f4e1897c-bb5d-5668-f1d8-040f4d8dd344}`— instrumenta directamente las syscalls de gestión de memoria virtual, mapeo de secciones y encolado de APCs dentro del propio ejecutivo `ntoskrnl.exe`.

5.1. Restricción de Consumo mediante PPL (`PsProtectedSignerAntimalware-Light`)

A diferencia de los proveedores ETW convencionales accesibles desde cualquier sesión de usuario, el registro de una sesión de traza contra el GUID de ETWti está restringido a nivel del kernel mediante una verificación de token protegido:

- Únicamente procesos que ejecutan bajo el nivel **Protected Process Light (PPL)** con el firmante antimalware (`PsProtectedSignerAntimalware-Light` o superior) pueden suscribirse a este proveedor vía `EnableTraceEx2`.
- Esta barrera impide que un binario no privilegiado o un atacante en Ring 3 intercepte los eventos generados por las rutinas de inyección de código de otros procesos.

5.2. Punteros en Kernel y Registro en WinDbg (`nt!EtwThreatIntProvRegHandle`)

Nota de entorno: `Laboratorio verificado: no; requiere VM/toolchain`. En una sesión de depuración con `ntoskrnl.pdb`, el puntero al registro del proveedor se almacena en el símbolo global **`nt!EtwThreatIntProvRegHandle`** (de tipo `_ETW_REG_ENTRY*`). Desde este objeto se inspecciona la estructura `_ETW_GUID_ENTRY` que controla qué palabras clave (keywords) de monitorización están habilitadas:

```
kd> x nt!EtwThreatIntProvRegHandle
ffffb609`120489a0 nt!EtwThreatIntProvRegHandle = 0xffffb609`102e3a10

kd> dt nt!_ETW_REG_ENTRY 0xffffb609`102e3a10 GuidEntry
   +0x020 GuidEntry : 0xffffb609`102f6840 _ETW_GUID_ENTRY

kd> dt nt!_ETW_GUID_ENTRY 0xffffb609`102f6840 Guid ProviderEnableInfo
   +0x018 Guid               : {f4e1897c-bb5d-5668-f1d8-040f4d8dd344}
   +0x060 ProviderEnableInfo : _TRACE_ENABLE_INFO
      +0x000 IsEnabled          : 0x1
      +0x004 Level              : 0xff ''
      +0x008 MatchAnyKeyword    : 0x80000000`ffffffff
```

Si un rootkit o driver en modo kernel sobrescribe el puntero `nt!EtwThreatIntProvRegHandle` con cero o modifica `ProviderEnableInfo.IsEnabled = 0`, ciega la telemetría de kernel que reciben los agentes EDR sin disparar inmediatamente un Bugcheck de PatchGuard (aunque kernels modernos protegen este handle mediante monitorización periódica de integridad).

Referencias y Fuentes Técnicas

- Baines, J., & Sealighter Project. (2021). *Telemetry Source Analysis: Microsoft-Windows-Threat-Intelligence (ETWti)*.
- Microsoft Learn. (2023). *Event Tracing for Windows (ETW) Architecture and Kernel Providers*.

- Russinovich, M., Solomon, D., & Ionescu, A. (2017). *Windows Internals, Part 1* (7th ed.). Microsoft Press.

Integrador Windows internals

Optativo y acotado

Usá una aplicación propia o didáctica, sin malware ni software ajeno. El integrador no crea persistencia: no instala un servicio, no registra tareas ni consumidores WMI y no cambia permisos.

Propósito

Unir conceptos de objetos, memoria, registro, COM, IPC, excepciones y telemetría en un informe reproducible que distinga evidencia de hipótesis.

Prerrequisitos

- *Obligatorios:* 2D.1, 2D.2 y 2D.3, más las lecciones de los mecanismos que use tu caso (por ejemplo 2D.6 y 2D.7 si usás COM, 2D.10 si usás IPC, 2D.12 si capturás ETW).
- *Recomendados:* 2D completa y 2C; no requiere 2D.8/DCOM ni modificar configuraciones de seguridad.

Caso de laboratorio

Elegí una aplicación didáctica con un flujo inocuo y observable, idealmente propia. Definí de antemano qué datos se recopilan y qué sistemas quedan fuera. Modelo sugerido: leer una clave de configuración propia, consultar una interfaz COM propia, intercambiar un mensaje de estado por IPC y emitir un evento ETW propio. No agregar autoarranque ni ejecución automática.

Fichas que reúne el informe

Cada lección de 2D dejó una herramienta de registro. El integrador no inventa otra: reúne las que correspondan a tu caso. Las marcadas como obligatorias son el mínimo; el resto se agrega si tu programa usa ese mecanismo.

Lección	Ficha o tabla	En el integrador
2D.1	Token, privilegios e integridad del proceso	Obligatoria
2D.2	Módulos y regiones relevantes	Obligatoria
2D.3	Clave propia leída y vista consultada (usuario/máquina, 32/64)	Obligatoria
2D.6	Apartamento de cada hilo, tabla slot/offset/método, puntero directo o proxy	Si usás COM
2D.7	Observación/inferencia de la activación (servidor, `ThreadingModel`, manifiesto)	Si usás COM
2D.9	Ficha de evidencia WMI	Solo si inspeccionás suscripciones **existentes**; el integrador no crea ninguna
2D.10	Revisión del pipe (ACL, primera instancia, remotos, modo, impersonación)	Si usás IPC
2D.11	Cronología de excepción con primera/segunda oportunidad	Si tu caso incluye una excepción controlada
2D.12	Ficha de la sesión ETW (nivel, keywords, pérdidas, decodificación)	Obligatoria si capturás ETW

DCOM (2D.8), servicios (2D.4) y tareas (2D.5) quedan fuera del integrador: exigirían habilitar acceso remoto o crear persistencia.

Ejemplo trabajado

(Escenario didáctico diseñado para ilustrar el método sobre una aplicación propia.)

BlocLab es una aplicación propia que lee su clave `HKCU\Software\BlocLab`, consulta una interfaz COM local y devuelve `READY` por un named pipe. El recorrido de análisis puede documentarse así:

```
Paso 1 — proceso: BlocLab.exe, PID 4120; token estándar del usuario de laboratorio.
Paso 2 — configuración: valor Mode="demo" leído desde HKCU\Software\BlocLab.
Paso 3 — COM: el cliente solicita IID_IFakeStatus al CLSID propio {11111111-2222-3333-4444-
555555555555};
         hilo cliente STA; servidor in-proc ThreadingModel=Both → puntero directo (vtable en el módulo
propio).
Paso 4 — IPC: el cliente abre \\.\pipe\BlocLabStatus y recibe READY;
         pipe creado con FILE_FLAG_FIRST_PIPE_INSTANCE y PIPE_REJECT_REMOTE_CLIENTS; ACL al logon SID.
Paso 5 — telemetría: el provider BlocLab emite StatusQuery con resultado READY;
         sesión propia, nivel informativo, 0 eventos perdidos.
Conclusión acotada: esta ejecución coincide con el flujo documentado por el programa.
Alternativa: una capa proxy podría intermediar COM; el evento ETW por sí solo no identifica al lector
del pipe.
Limpieza: detener app, cerrar sesión ETW, borrar traza temporal y comparar/revertir snapshot.
```

La cadena proceso → configuración → interfaz → canal → evento es una **hipótesis de recorrido** que se valida correlacionando observaciones. La coincidencia temporal entre COM y ETW no demuestra causalidad; conserva timestamps y anota qué herramienta produjo cada dato.

Práctica

1. Registra VM/build, programa, hash del binario propio y snapshot base.
2. Capturá solo una ejecución; observá token, módulo/segmento relevante, acceso a configuración y flujo IPC/COM que esté documentado. Completá las fichas de la tabla de arriba que correspondan.
3. Si añadís ETW, limita la traza al provider propio, detené la sesión y evita conservar payloads sensibles.
4. No ejecutes acciones desconocidas, no cambies servicio/tarea/consumidor ni DACL de componentes del sistema.
5. Cierra procesos, elimina traces/capturas temporales y comprueba las claves/archivos creados por tu programa. Revierte snapshot si el estado no coincide con el inicial.

Entregable

Informe con alcance, mapa de componentes, evidencias fechadas, tabla observación/interpretación/alternativas, límites, hashes del material propio y checklist de limpieza firmado por quien realizó el lab. Como anexos, las fichas de 2D que aplicaron (al menos las obligatorias), cada una con su fecha y la herramienta que produjo cada dato.

Repaso

▼ **¿Qué convierte una observación en una conclusión sólida?**

> Repetibilidad, fuente/contexto, evidencia correlacionada y consideración de explicaciones alternativas.

▼ **¿Por qué el integrador evita autoarranque?**

> No hace falta para demostrar los conceptos y agrega cambios persistentes difíciles de limpiar y revisar.

▼ **¿Por qué el integrador no incluye DCOM ni suscripciones WMI nuevas?**

> Porque demostrarlos exigiría habilitar acceso remoto o crear persistencia, y el integrador no cambia la seguridad ni deja configuraciones permanentes.

▼ **¿Cuándo se considera terminado?**

> Cuando el informe es reproducible, sus límites están declarados y cada cambio temporal se eliminó o la VM se revirtió.

Puerta optativa 2D

Entregás un análisis acotado, basado en evidencia, y demostración de limpieza. No se aprueba por cantidad de herramientas usadas.

Fuentes

- Documentación primaria enlazada en 2D.1, 2D.2, 2D.3, 2D.6, 2D.7, 2D.9, 2D.10, 2D.11, 2D.12, 2D.14 y 2D.15.
- **Fundamentos previos (Tomo I):** método de análisis en 4.1 · Método de análisis (Tomo I, p. 282–292) y estructura de evidencia en 4.3 · El informe reproducible (Tomo I, p. 301–305).

> **Síntesis de estudio de esta lección**
>
> **Núcleo conceptual:** método de análisis (4.1) e informe reproducible (4.3). **Conocimiento externo:** integración de subsistemas de Windows internals. **Ejemplo ilustrativo:** aplicación de práctica `BlocLab`.

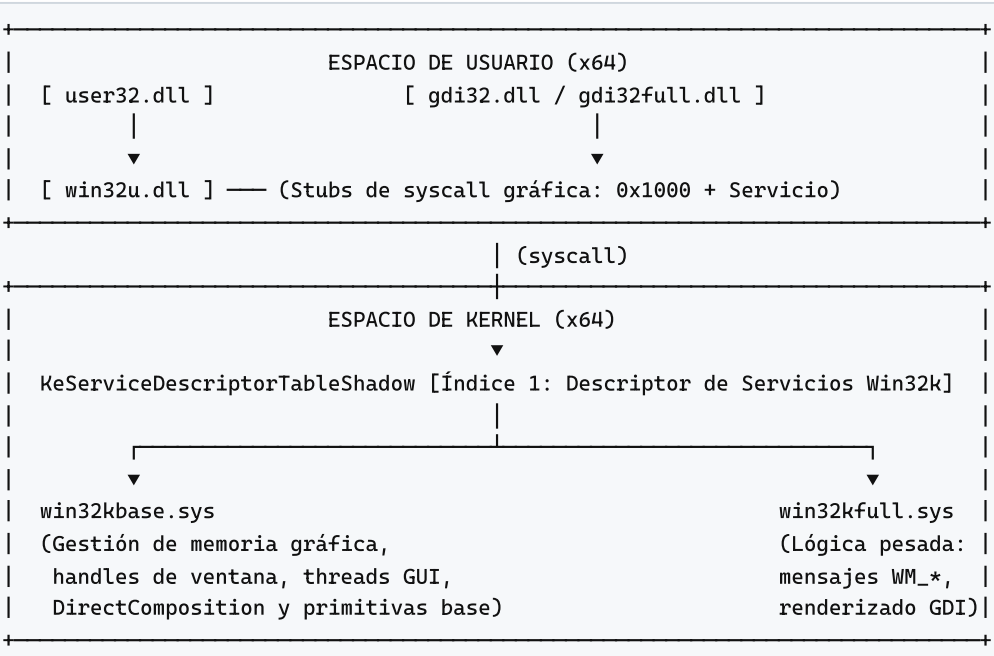
El Subsistema Gráfico en Modo Kernel (win32k) y la Tabla de Servicios Sombra

> **Propósito y Relevancia**
>
> En la arquitectura de Windows, el modo kernel no solo alberga los servicios de sistema estándar del ejecutivo y del gestor de memoria (`ntoskrnl.exe`). También aloja el subsistema de ventanas, interfaz gráfica y gestión de dispositivos de entrada (**GDI y USER**). Este subsistema está implementado en controladores de sistema especiales (`win32kbase.sys` , `win32kfull.sys` y el driver legado `win32k.sys`). Con más de 1.200 llamadas al sistema que históricamente han constituido la mayor superficie de ataques de escalada local de privilegios (LPE) en Windows, su análisis es obligatorio para cualquier investigador de seguridad.

---

1. Descomposición de Win32k y Arquitectura Modular

En versiones antiguas de Windows (hasta Windows 7 y 8.1), toda la lógica gráfica residía en un único driver monolítico de gran tamaño: `win32k.sys`. Desde Windows 10, Microsoft refactorizó el subsistema en módulos especializados para reducir el acoplamiento y mejorar la telemetría:



El Módulo de Transición en Modo Usuario: `win32u.dll`

Históricamente, las funciones de `user32.dll` (como `CreateWindowExW`) invocaban directamente la instrucción `syscall`. En Windows 10 y 11 modernos, Microsoft centralizó todos los stubs de llamadas al sistema gráficas dentro de una DLL de bajo nivel llamada **`win32u.dll`**. Cualquier llamada de alto nivel en `user32.dll` o `gdi32.dll` pasa primero por un stub `NtUser*` o `NtGdi*` dentro de `win32u.dll`.

---

2. Despacho por la Tabla Sombra: `KeServiceDescriptorTableShadow`

En la Lección 2.15 vimos cómo las llamadas al sistema nativas (`NtOpenProcess` , `NtAllocateVirtualMemory`) se despachan mediante la tabla global exportada `KeServiceDescriptorTable` . Sin embargo, los servicios gráficos utilizan una tabla extendida no exportada: `KeServiceDescriptorTableShadow` .

Estructura de `KSERVICE_TABLE_DESCRIPTOR` y Navegación en WinDbg

La tabla sombra contiene un arreglo de dos descriptores de servicio (`KSERVICE_TABLE_DESCRIPTOR`), cada uno de 32 bytes (en x64):

Offset (x64)	Campo	Tipo	Significado y Función
`+0x00`	`Base`	`PULONG_PTR`	Puntero a la tabla de direcciones relativas de las rutinas de servicio (Service Table Base).
`+0x08`	`Count`	`PULONG`	Puntero al arreglo de contadores de invocaciones de cada syscall (Service Counter Table Base).
`+0x10`	`Limit`	`ULONG`	Cantidad total de llamadas al sistema registradas en la tabla (ej. ~ `0x1C0` para `ntoskrnl` , ~ `0x500` para `win32k`).
`+0x18`	`Number`	`PUCHAR`	Puntero al arreglo que contiene el número de bytes de argumentos pasados en el stack por cada llamada.

- **Entrada 0 (`KeServiceDescriptorTableShadow[0]` , offset `+0x00`):** Descriptor idéntico a `KeServiceDescriptorTable` para los servicios nativos del ejecutivo de `ntoskrnl.exe` .
- **Entrada 1 (`KeServiceDescriptorTableShadow[1]` , offset `+0x20`):** Descriptor reservado para el subsistema gráfico despachado por `win32kbase.sys` y `win32kfull.sys` .

```
$$ Inspeccionar los descriptores de KeServiceDescriptorTableShadow en WinDbg
kd> dq nt!KeServiceDescriptorTableShadow L8
$$ El primer QWORD en +0x20 es el puntero Base de win32k:
kd> dps poi(nt!KeServiceDescriptorTableShadow + 0x20) L20
```

Conversión de Números de Servicio (SSN Gráfico)

Las syscalls de Win32k poseen el bit 12 activo (índice de tabla 1, base `0x1000`). Como los números de servicio (SSN) tanto de `ntoskrnl` como de `win32k` cambian entre builds de Windows, nunca deben asumirse como constantes fijas entre versiones. Cuando una instrucción `syscall` se ejecuta con `EAX = 0x1000 + N` :

1. El despachador `nt!KiSystemCall64` extrae el índice de tabla (`(EAX >> 12) & 1`) y comprueba si el número de servicio pertenece al rango gráfico (`0x1000+`).
2. Obtiene el puntero al `_KTHREAD` actual desde `GS:[0x188]` (`KPCR.Prcb.CurrentThread`) y verifica si el hilo ya fue promovido a **GUI Thread** comprobando que el campo `_KTHREAD→Win32Thread` no sea nulo (ubicado en el offset `+0x1C8` de `nt!_KTHREAD` en Windows 10 22H2 Build 19045, Windows 11 23H2 Build 22631 y Windows 11 24H2 Build 26100).
3. Si el hilo aún es estándar (no GUI), el kernel invoca `nt!PsConvertToGuiThread` : valida que el proceso no tenga activa la política Win32k Lockdown, asigna el stack de kernel extendido requerido por el subsistema

gráfico, inicializa `Win32Thread` y cambia el índice de tabla de servicios del hilo hacia `KeServiceDescriptorTableShadow`.

4. El kernel toma el descriptor de índice 1 de `KeServiceDescriptorTableShadow` (offset `+0x20`) y calcula la dirección destino sumando a `Base` el desplazamiento relativo con signo almacenado en `Base[N] >> 4`.

---

3. El Fenómeno Crítico: Inversión de Control con `KeUserModeCallback`

La razón principal por la cual Win32k ha sido la mayor fuente histórica de vulnerabilidades de kernel (Use-After-Free y elevación de privilegios) es su necesidad de interactuar con el espacio de usuario mientras procesa llamadas de kernel:

Mecánica de Inversión

Cuando una función en modo kernel dentro de `win32kfull.sys` necesita notificar a una ventana (por ejemplo, para ejecutar un hook de ventana global o despachar un mensaje `WM_NCCREATE`), el kernel no puede ejecutar la función de ventana directamente en modo kernel.

1. El driver de kernel invoca **`nt!KeUserModeCallback`**.
2. Esta función guarda el estado actual del stack de kernel, prepara un frame de retorno hacia el modo usuario y salta a la rutina `ntdll!KiUserCallbackDispatcher` en el proceso.
3. El código de usuario procesa el mensaje (ejecutando callbacks como `user32!__ClientLoadLibrary`).
4. Al concluir, el modo usuario invoca la syscall nativa `NtCallbackReturn` para devolver el control al driver de kernel en el punto exacto donde se suspendió.

El Riesgo de Seguridad (Race Conditions y Reentrancia)

Durante el tiempo en que el control está en espacio de usuario a través de `KeUserModeCallback`, el proceso puede ejecutar código malicioso que destruya o libere la ventana o el objeto GDI (`DestroyWindow`) sobre el cual el kernel estaba operando. Cuando el control regresa a `win32kfull.sys`, si el driver asume que el objeto sigue existiendo en el puntero original sin revalidar su estado de bloqueo, se produce un fallo clásico de **Use-After-Free (UAF)** en espacio de kernel.

---

4. Mitigación de Superficie de Ataque: Win32k System Call Lockdown

Dado que el subsistema gráfico en modo kernel (`win32k.sys`, `win32kfull.sys` y `win32kbase.sys`) expone más de 1.100 syscalls en la `KeServiceDescriptorTableShadow` y ha concentrado históricamente el mayor volumen de vulnerabilidades de elevación de privilegios (EoP) utilizadas para escapar de sandboxes de navegadores, Windows 10/11 incorpora la política de mitigación de proceso **Win32k System Call Disable Policy** (conocida como *Win32k Lockdown*).

Procesos altamente expuestos a contenido no confiable (como los procesos de renderizado de Google Chrome, Microsoft Edge y sandboxes de visualización) activan esta mitigación para auto-despojarse permanentemente del derecho de invocar llamadas a la tabla sombra. Si un exploit intenta invocar cualquier syscall gráfica (`0x1000+`), el despachador de kernel intercepta la transición y devuelve acceso denegado.

4.1. Mecanismo Interno en `KiConvertToGuiThread` y `_EPROCESS`

Cuando un proceso habilita esta política mediante `SetProcessMitigationPolicy(ProcessSystemCallDisablePolicy, ...)` o al ser creado por un broker mediante el atributo `PROC_THREAD_ATTRIBUTE_MITIGATION_POLICY` con la bandera `PROCESS_CREATION_MITIGATION_POLICY_WIN32K_SYSTEM_CALL_DISABLE_ALWAYS_ON`, el kernel activa el bit **`DisableWin32kSystemCalls`** (bit `0x04`) dentro del campo `MitigationFlags` de la estructura `nt!_EPROCESS`.

El bloqueo no ocurre parcheando la tabla SSDT Shadow, sino cortando de raíz la transición del hilo en `PsConvertToGuiThread` / `KiConvertToGuiThread`:

1. Un hilo no-GUI (cuyo `KTHREAD→ServiceTable` apunta a `KeServiceDescriptorTable`) intenta invocar cualquier syscall del rango `0x1000..0x1FFF` (por ejemplo, `NtUserCreateWindowEx` o `NtGdiBitBlt`).
2. El despachador de syscalls `KiSystemServiceRepeat` detecta que el índice pertenece a la tabla #1 (Win32k), la cual está vacía en la tabla estándar, y llama a `KiConvertToGuiThread` → `PsConvertToGuiThread`.
3. Al inicio de `PsConvertToGuiThread`, el kernel inspecciona `Process->MitigationFlagsValues.DisableWin32kSystemCalls`. Si el bit vale `1`, aborta inmediatamente la asignación del `_W32THREAD`, impide que `KTHREAD→ServiceTable` apunte a `KeServiceDescriptorTableShadow` y devuelve `STATUS_ACCESS_DENIED (0xC0000022)` (o genera un fallo de auditoría ETW si se configuró en modo *Audit*).

```
// Activación programática de Win32k Lockdown desde el propio proceso en Ring 3
#include <windows.h>

BOOL EnableWin32kLockdown(void) {
    PROCESS_MITIGATION_SYSTEM_CALL_DISABLE_POLICY policy = { 0 };
    policy.DisallowWin32kSystemCalls = 1;

    return SetProcessMitigationPolicy(
        ProcessSystemCallDisablePolicy,
        &policy,
        sizeof(policy)
    );
}
```

4.2. Verificación en WinDbg de la Mitigación en Procesos Sandbox

Nota de entorno: `Laboratorio verificado: no; requiere VM/toolchain`. En procesos de renderizado de navegadores basados en Chromium (`msedge.exe --type=renderer` o `chrome.exe`), la verificación del bit en la estructura `_EPROCESS` muestra:

```
kd> dt nt!_EPROCESS <EPROCESS_RENDERER> MitigationFlagsValues.DisableWin32kSystemCalls
   +0x9d0 MitigationFlagsValues :
      +0x000 DisableWin32kSystemCalls : 0y1
```

Referencias y Fuentes Técnicas

- Forshaw, J. (2015). *Windows 10 Mitigation Improvements: Win32k System Call Filtering*. Google Project Zero.
- Microsoft Learn. (2023). *SetProcessMitigationPolicy function* (ProcessSystemCallDisablePolicy).
- Russinovich, M., Solomon, D., & Ionescu, A. (2017). *Windows Internals, Part 1* (7th ed.). Microsoft Press.

Reversing de Arranque Temprano, UEFI y Cadena de Carga

> **Propósito y Alcance**
>
> El análisis del sistema operativo no comienza cuando el kernel ya está en ejecución. Para comprender las amenazas más persistentes (Bootkits UEFI, técnicas de desactivación temprana de DSE, y el despliegue del hipervisor VBS antes de `ntoskrnl.exe`), el analista de ingeniería inversa debe dominar el proceso de arranque temprano de Windows en plataformas de 64 bits con firmware **UEFI**. Esta lección disecciona la cadena de cargadores EFI, el layout de estructuras del firmware, el formato de los módulos DXE y la estructura de transición fundamental: el bloque `LOADER_PARAMETER_BLOCK`.

---

1. El Formato PE32+ de las Aplicaciones EFI

Los archivos ejecutables del cargador de arranque de Windows (como `bootmgfw.efi`, `winload.efi` y `hvloader.efi`) no son formatos binarios propietarios exóticos: son archivos estándar **PE32+ (PE de 64 bits)** con características específicas en sus cabeceras:

1. `Subsystem` **en OptionalHeader:**

- `IMAGE_SUBSYSTEM_EFI_APPLICATION` (`10` / `0x000A`): Aplicaciones que se ejecutan directamente bajo el entorno UEFI antes de que el sistema operativo tome el control.
- `IMAGE_SUBSYSTEM_EFI_BOOT_SERVICE_DRIVER` (`11` / `0x000B`): Controladores DXE que extienden los servicios de arranque de la placa madre.

2. **Punto de Entrada EFI:**

La función de inicio de un ejecutable EFI recibe siempre dos parámetros estandarizados en la convención x64 de Microsoft:

```
EFI_STATUS EFIAPI EfiMain(
    IN EFI_HANDLE ImageHandle,       // RCX: Handle del binario cargado en memoria
    IN EFI_SYSTEM_TABLE *SystemTable // RDX: Puntero a la tabla global de servicios UEFI
);
```

Layout Binario de `EFI_SYSTEM_TABLE` en x64

En un ejecutable UEFI desensamblado, toda interacción con memoria, consola y protocolos se realiza a través de la tabla `SystemTable` pasada en `RDX`. En arquitecturas de 64 bits, su disposición en memoria es fija:

Offset (x64)	Campo	Tipo	Propósito
`+0x00`	`Hdr`	`EFI_TABLE_HEADER`	Cabecera estándar de 24 bytes (Firma mágica `0x5453595320494249` = `"IBI SYST"`, revisión, tamaño y CRC32).
`+0x18`	`FirmwareVendor`	`CHAR16*`	Puntero a cadena Unicode UTF-16 con el nombre del fabricante del BIOS/UEFI.
`+0x20`	`FirmwareRevision`	`UINT32`	Versión interna del firmware.
`+0x24`	`_Padding`	`UINT32`	Relleno de alineación a 8 bytes en arquitecturas de 64 bits.
`+0x28`	`ConsoleInHandle`	`EFI_HANDLE`	Handle del dispositivo de entrada estándar (teclado).
`+0x30`	`ConIn`	`EFI_SIMPLE_TEXT_INPUT_PROTOCOL*`	Puntero al protocolo de lectura de caracteres del teclado.
`+0x38`	`ConsoleOutHandle`	`EFI_HANDLE`	Handle de la salida de texto (pantalla).
`+0x40`	`ConOut`	`EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL*`	Puntero al protocolo de impresión de texto en pantalla.
`+0x48`	`StandardErrorHandle`	`EFI_HANDLE`	Handle del dispositivo de error.
`+0x50`	`StdErr`	`EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL*`	Puntero a la salida de errores en pantalla.
`+0x58`	`RuntimeServices`	`EFI_RUNTIME_SERVICES*`	Puntero a la tabla de servicios en tiempo de ejecución (`gRT`), que persisten tras arrancar el SO (variables NVRAM).
`+0x60`	`BootServices`	`EFI_BOOT_SERVICES*`	Puntero a la tabla de servicios de arranque (`gBS`), activa solo hasta que se invoca `ExitBootServices` .

El porqué del Padding de 4 bytes (`+0x24`) y Desensamblado en x64

En la especificación UEFI y en la ABI x64 de Microsoft, los punteros de 64 bits (`CHAR16*` , `EFI_HANDLE` , punteros a tablas de servicios) exigen **alineación natural obligatoria a 8 bytes** (`offset % 8 == 0`):

1. `Hdr` mide 24 bytes (`0x18`), ocupando `+0x00` a `+0x18` .
2. `FirmwareVendor` es un puntero (8 bytes), ocupando `+0x18` a `+0x20` .
3. `FirmwareRevision` es un entero de 32 bits `UINT32` (4 bytes), ocupando `+0x20` a `+0x24` .
4. El siguiente miembro declarado es `EFI_HANDLE ConsoleInHandle` (un puntero de 8 bytes). Si el compilador lo ubicara inmediatamente en `+0x24` , rompería la alineación de 8 bytes (ya que `0x24` no es múltiplo de 8).
5. Por lo tanto, el compilador C inserta automáticamente **4 bytes de relleno inactivo** (`_Padding` en `+0x24` a `+0x28`).
6. A partir de `ConsoleInHandle` en `+0x28` , se suceden 7 punteros de 64 bits consecutivos ($7 \times 8 = 56 = \text{0x38}$ bytes): `0x28 + 0x38 = 0x60` .
7. Gracias a este padding riguroso, el puntero crítico **`BootServices`** se ubica indefectiblemente en el offset **`+0x60`** .

Desensamblado en x64: Cómo el Firmware o Bootkit Recupera `BootServices`

```
; ═══════════════════════════════════════════════════════════════
; Prólogo estándar de una aplicación EFI (bootmgfw.efi / winload.efi / malware UEFI)
; Convención Microsoft x64: RCX = ImageHandle, RDX = SystemTable (EFI_SYSTEM_TABLE*)
; ═══════════════════════════════════════════════════════════════
EfiMain PROC
    mov     qword ptr [rsp + 10h], rdx  ; Salvar SystemTable en shadow space
    mov     qword ptr [rsp + 08h], rcx  ; Salvar ImageHandle
    sub     rsp, 28h

    ; Desreferenciar BootServices desde SystemTable (+0x60)
    mov     rax, qword ptr [rdx + 60h]  ; RAX = SystemTable→BootServices (¡offset exacto +0x60!)
    mov     qword ptr [gBS], rax        ; Almacenar en la variable global gBS

    ; Demostración de invocación de protocolo vía BootServices:
    ; LocateProtocol está en el offset +0x140 dentro de EFI_BOOT_SERVICES
    mov     rax, qword ptr [gBS]
    lea     r8, [pProtocol]             ; Argumento 3: Puntero de salida
    xor     edx, edx                    ; Argumento 2: Registration = NULL
    lea     rcx, [ProtocolGuid]         ; Argumento 1: Protocol GUID
    call    qword ptr [rax + 140h]      ; Invocar gBS→LocateProtocol()

    add     rsp, 28h
    ret
EfiMain ENDP
```

---

2. La Cadena de Carga de Windows en Sistemas x64 / UEFI

El proceso de arranque moderno desde el encendido de la máquina hasta la primera instrucción del kernel sigue una secuencia estricta de componentes encadenados y firmados bajo **Secure Boot**:

```
[ Encendido / Reset ]
         |
         ▼
[ Firmware UEFI (SEC → PEI → DXE → BDS) ]
         | (Carga desde partición ESP FAT32: \EFI\Microsoft\Boot\bootmgfw.efi)
         ▼
[ Windows Boot Manager (bootmgfw.efi) ]
         | 1. Parsea la base de datos BCD (\EFI\Microsoft\Boot\BCD)
         | 2. Si BitLocker está activo: desbloquea la clave TPM mediante llamadas EFI
         | 3. Carga y verifica la firma digital del cargador del SO
         ▼
[ Windows OS Loader (winload.efi) ]
         | 1. Configura el mapa preliminar de tablas de páginas de 4 niveles (PML4)
         | 2. Carga la colmena de registro SYSTEM (HKLM\SYSTEM) desde el disco
         | 3. Carga ntoskrnl.exe, hal.dll y drivers con StartType = 0 (SERVICE_BOOT_START)
         | 4. Si VBS está habilitado: invoca a hvloader.efi para iniciar Hyper-V
         | 5. Llama a EFI_BOOT_SERVICES→ExitBootServices() (Termina el firmware)
         ▼
[ Kernel EntryPoint: ntoskrnl.exe!KiSystemStartup(LOADER_PARAMETER_BLOCK* pLpb) ]
```

---

3. La Estructura de Transición: `LOADER_PARAMETER_BLOCK`

Cuando `winload.efi` invoca la función de arranque del kernel (`nt!KiSystemStartup`), el hardware ya no tiene acceso a los servicios de UEFI (se ha llamado a `ExitBootServices`). La única vía por la cual el cargador transfiere información vital al kernel es mediante un puntero único pasado en el registro `RCX`: la estructura `LOADER_PARAMETER_BLOCK`.

Offsets Críticos de `LOADER_PARAMETER_BLOCK` en x64 (Win10 22H2 Build 19045 / Win11 23H2 Build 22631 / Win11 24H2 Build 26100):

Los primeros 16 bytes (`+0x000..+0x00F`) contienen `OsMajorVersion`, `OsMinorVersion`, `Size` y `OsLoaderSecurityVersion` (4 `ULONG`s). A partir de `+0x010` se ubican las cabezas de lista (notá que en Windows 10/11 modernos se agregaron listas de drivers tempranos y TPM respecto de Windows 7, desplazando `Extension` a `+0x0F0`):

- **`+0x010` `LoadOrderListHead` (`_LIST_ENTRY`):** Lista doblemente enlazada de estructuras `_KLDR_DATA_TABLE_ENTRY` que enumera todos los módulos y drivers cargados en memoria por `winload.efi` (incluyendo `ntoskrnl.exe`, `hal.dll`, `kd.dll` y drivers de arranque).
- **`+0x020` `MemoryDescriptorListHead` (`_LIST_ENTRY`):** Lista de descriptores `_MEMORY_ALLOCATION_DESCRIPTOR` con el mapa de memoria física del sistema clasificado por tipos (memoria utilizable, memoria reservada por firmware, memoria ACPI).
- **`+0x030` `BootDriverListHead` (`_LIST_ENTRY`):** Lista ordenada de nodos `_BOOT_DRIVER_LIST_ENTRY` que deben ser inicializados en el arranque (seguida en `+0x040` por `EarlyLaunchListHead`, en `+0x050` por `CoreDriverListHead`, en `+0x060` por `CoreExtensionsDriverListHead` y en `+0x070` por `TpmCoreDriverListHead`).
- **`+0x0F0` `Extension` (`Ptr64 _LOADER_PARAMETER_EXTENSION`):** Puntero a la extensión del bloque de arranque (en Windows 7 residía en `+0x098`; en Windows 10 22H2 y Windows 11 23H2/24H2 reside en `+0x0F0`). Contiene las estructuras de configuración de **VBS/HVCI**, el estado de inicialización de **Hyper-V** y los metadatos de integridad de código (*Code Integrity*).

Inspección en WinDbg durante el Arranque Temprano

Si estás depurando una máquina virtual con WinDbg y configuras un breakpoint inicial (`sxe ibp` o `break` antes del kernel):

```
$$ Inspeccionar el bloque de parámetros pasado a KiSystemStartup en RCX
dt nt!_LOADER_PARAMETER_BLOCK @rcx
$$ Volcar la lista de módulos cargados en el arranque
!list -t nt!_KLDR_DATA_TABLE_ENTRY.InLoadOrderLinks -x "dps @$extret+0x30 L2" (poi(@rcx+0x10))
```

---

4. Metodología de Análisis de Módulos EFI y Bootkits en IDA / Ghidra

Los bootkits modernos (como BlackLotus o Bootkm) buscan infectar la cadena de arranque antes de que `ntoskrnl.exe` active los mecanismos de protección de PatchGuard o las políticas de EDR.

Pasos de Reversing para un Binario `.efi`:

1. **Carga en el Desensamblador:** Seleccionar el formato PE. En Ghidra o IDA Pro, utilizar los tipos de datos de UEFI (importando los headers de EDK2 o utilizando el plugin *EFISeek* / *Ghidra-EFI*).
2. **Localización de `SystemTable`:**

En el prólogo de `EfiMain`, identificar la instrucción que salva el registro `RDX` en una variable global. Ese puntero es `gST` (`EFI_SYSTEM_TABLE`). En el offset `+0x60` de `gST` reside el puntero a `gBS` (`EFI_BOOT_SERVICES`).

3. **Identificación de Protocolos Hijackeados:**

Los bootkits suelen instalar ganchos sobre funciones críticas de `EFI_BOOT_SERVICES` antes de que se cargue `winload.efi`:

- Ganchos sobre `LocateProtocol` o `OpenProtocol`: interceptan la carga de protocolos para devolver punteros a rutinas modificadas.
- Ganchos sobre `ExitBootServices`: capturan el momento exacto en que `winload.efi` va a transferir el control al kernel. En este punto, el bootkit parchea en memoria viva las rutinas de validación de firmas de `winload.efi` o inyecta código en `ntoskrnl!KiSystemStartup` para deshabilitar DSE antes de que el kernel arranque.

4. **Verificación Defensiva:** En sistemas corporativos, la integridad de esta cadena se audita mediante **Measured Boot** (el chip TPM registra los hashes SHA-256 de cada componente de arranque en los registros PCR 0 a 7, permitiendo a la red verificar remotamente que ningún binario EFI fue alterado).

Extensión optativa: Hyper-V, particiones, VMBus y señales de virtualización

Extensión separada y de lectura conceptual

No integra el núcleo obligatorio de 2D, no es prerrequisito de la Etapa 6 ni forma parte de la aprobación del curso. El objetivo es comprender la arquitectura del hipervisor de Windows y los límites de observación de las señales de entorno virtual; no implementar técnicas de evasión anti-VM ni modificar la configuración de Hyper-V del equipo anfitrión.

Propósito

Distinguir el modelo de particiones y dispositivos sintéticos de Hyper-V (root/child partition, VMBus, VSP/VSC) de las estructuras de hardware del procesador (VMCS/VMCB y EPT/NPT), y evaluar con rigor técnico qué demuestra y qué no demuestra una señal de virtualización hallada al analizar un binario en Windows 10/11.

Prerrequisitos

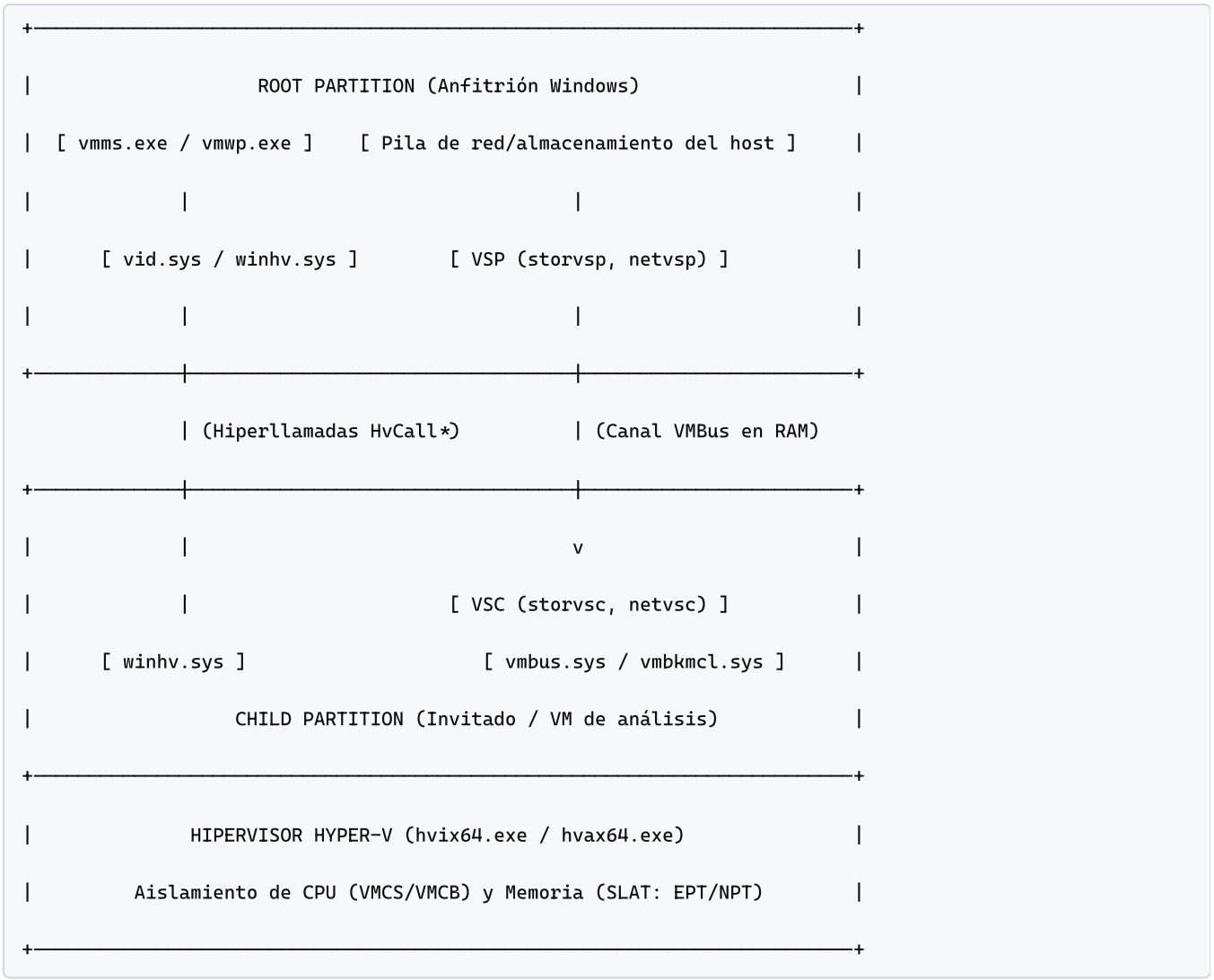
- *Obligatorios:* procesos y memoria (1.2, 1.3); taxonomía anti-análisis (2.15).
- *Recomendados:* objetos y tokens (2D.1); arranque UEFI (2D.15); mitigaciones VBS/HVCI (6.6, 6.7).

1. Arquitectura de particiones: Root y Child Partitions

Hyper-V es un hipervisor de tipo 1 (*bare-metal*) basado en particiones. Cuando Hyper-V o Virtualization-Based Security (VBS) están activos, el hipervisor (`hvix64.exe` en procesadores Intel o `hvax64.exe` en procesadores AMD, cargado por `hvloader.efi` durante la fase de arranque descrita en 2D.15) se sitúa directamente sobre el hardware en el modo de privilegio de virtualización del procesador (VMX root operation en Intel; SVM host mode en AMD).

Una **partición** es la unidad lógica de aislamiento soportada por el hipervisor: posee su propio espacio de direcciones físicas de invitado, sus procesadores virtuales y sus políticas de temporización e interrupciones.

- **Root Partition (Partición Raíz o Padre):** Es la única partición que tiene acceso directo a los dispositivos físicos de entrada/salida y administra el ciclo de vida de las demás particiones. Ejecuta una instancia completa del sistema operativo anfitrión (Windows 10/11 o Windows Server) y aloja la pila de virtualización en modo usuario (como el servicio `vmms.exe` y un proceso trabajador `vmwp.exe` por cada máquina virtual activa) junto con el controlador de infraestructura de virtualización (`vid.sys`).
- **Child Partitions (Particiones Hijas):** Ejecutan los sistemas operativos invitados. Una partición hija no tiene acceso directo al procesador físico ni a los controladores de hardware reales; ve procesadores virtuales (VPs), un espacio de memoria física de invitado (GPA, *Guest Physical Address*) y dispositivos emulados o sintéticos.



2. Dispositivos sintéticos: VMBus, VSP y VSC

Emular hardware clásico (como un controlador IDE o una placa de red DEC 21140) exige interceptar accesos a puertos de E/S o memoria mapeada, provocando constantes salidas al hipervisor (*VM Exits*). Para evitar ese costo, Hyper-V utiliza un modelo de dispositivos ilustrados o **sintéticos** sobre un bus compartido en memoria llamado **VMBus**:

- **VMBus (`vmbus.sys`):** Es un mecanismo de comunicación entre particiones basado en buffers circulares (*ring buffers*) en páginas de memoria compartida entre la partición hija y la partición raíz, notificados mediante eventos sintéticos del hipervisor (SynIC, *Synthetic Interrupt Controller*).
- **VSC (Virtualization Service Client):** Es el driver que corre dentro del kernel de la **partición hija** (invitado). Por ejemplo, `storvsc.sys` para almacenamiento, `netvsc.sys` para red o `vmbusr.sys` para componentes de modo usuario.
- **VSP (Virtualization Service Provider):** Es el componente que corre en la **partición raíz** (anfitrión), recibe las solicitudes que el VSC colocó en el ring buffer de VMBus y las traduce en operaciones sobre el hardware físico o los archivos `.vhdx` del host (por ejemplo, `storvsp.sys` o `vmswitch.sys`).

En reversing defensivo, distinguir entre VMBus y el hipervisor evita errores de atribución: VMBus no es el hipervisor ni una estructura del procesador; es un canal de transporte entre particiones para dispositivos sintéticos.

3. Estructuras de hardware de la CPU: VMCS/VMCB y SLAT (EPT/NPT)

Por debajo del modelo de software de Hyper-V, el aislamiento real descansa en extensiones de arquitectura del procesador:

Concepto	Intel VT-x (`hvix64.exe`)	AMD-V / SVM (`hvax64.exe`)	Función técnica
Estructura de control de CPU virtual	**VMCS** (*Virtual Machine Control Structure*, Intel SDM Vol. 3C Cap. 24)	**VMCB** (*Virtual Machine Control Block*, AMD APM Vol. 2 Cap. 15)	Región de memoria por procesador virtual que guarda el estado del guest, el estado del host y qué eventos (`CPUID` , `RDMSR` , acceso a `CR3` / `CR4` , excepciones) fuerzan una transición al hipervisor (*VM Exit*).
Traducción de direcciones de segundo nivel (SLAT)	**EPT** (*Extended Page Tables*, Intel SDM Vol. 3C Cap. 28)	**NPT / RVI** (*Nested Page Tables*, AMD APM Vol. 2 §15.25)	Segundo juego de tablas de páginas en hardware que traduce Direcciones Físicas de Invitado (GPA) a Direcciones Físicas del Sistema (SPA/HPA), con sus propios bits de permiso Lectura/Escritura/Ejecución.
Instrucción de hiperllamada	`VMCALL` (opcode `0F 01 C1`)	`VMMCALL` (opcode `0F 01 D9`)	Transfiere explícitamente el control desde el kernel de una partición (vía la página de código de hiperllamada mapeada por `winhv.sys`) hacia el hipervisor.

Gracias a SLAT (EPT/NPT), el hipervisor puede quitar el permiso de ejecución o escritura a una página física de la partición aun cuando las tablas de páginas del kernel invitado (`CR3`) la marquen como escribible y ejecutable. Ese mismo mecanismo es el que utiliza HVCI entre VTL0 y VTL1 (ver 6.7 y 2D.17).

4. Señales de virtualización y el falso positivo de la Root Partition

La especificación pública de Microsoft (*Hypervisor Top-Level Functional Specification*, TLFS) define cómo un sistema operativo o una aplicación descubre la presencia y capacidades de un hipervisor compatible con la interfaz de Microsoft mediante la instrucción `CPUID` :

Hoja `CPUID` (`EAX`)	Registros y bits devueltos	Qué informa según la especificación TLFS	Por qué NO demuestra que sea una VM de análisis
`0x00000001`	`ECX` bit 31 (`HypervisorPresent`) = `1`	Indica que hay un hipervisor ejecutándose debajo del sistema operativo actual.	En cualquier equipo físico moderno con Windows 10/11 que tenga activo **VBS, HVCI (Integridad de memoria), Credential Guard, WSL2 o el rol Hyper-V**, el sistema operativo anfitrión corre dentro de la **Root Partition** y este bit vale `1` .
`0x40000000`	`EAX` = máxima hoja; `EBX:ECX:EDX` = `"Microsoft Hv"`	Firma de interfaz del hipervisor de Microsoft.	Aparece idéntico tanto en una VM invitada (Child Partition) como en una máquina física con VBS/HVCI (Root Partition).
`0x40000003`	`EAX:EBX` = máscara de privilegios de la partición (`HV_PARTITION_PRIVILEGE_MASK`)	El bit 0 de `EBX` (`CreatePartitions`) y flags como `AccessPartitionReferenceCounter` indican qué derechos posee la partición actual.	Permite distinguir la Root Partition (donde `CreatePartitions` vale `1`) de una Child Partition (donde vale `0`), aunque un usuario estándar no administra particiones.

Otras señales observables en modo usuario o kernel —como la presencia de dispositivos bajo `HKLM\SYSTEM\CurrentControlSet\Enum\VMBUS`, servicios de integración (`vmicheartbeat`, `vmickvpexchange`) o cadenas ACPI/SMBIOS (`Virtual Machine`)— son consultadas de forma completamente legítima por instaladores, motores gráficos, herramientas de telemetría de rendimiento y agentes de soporte técnico.

Ejemplo trabajado

(Escenario ilustrativo de lectura estática y contextual; no proviene de una muestra real ejecutada.)

Al analizar un binario de diagnóstico propio (`DiagEntorno.exe`) en IDA/Ghidra, encontrás dos bloques consecutivos:

```
Bloque A (0x140001180):

   mov eax, 1

   cpuid

   bt  ecx, 1Fh                ; ¿ECX bit 31 (HypervisorPresent) == 1?

   jnc entorno_fisico_sin_vbs

   mov eax, 40000000h

   cpuid                       ; Lee EBX="Micr", ECX="osof", EDX="t Hv"



Bloque B (0x140001210):

   RegOpenKeyExW(HKLM, L"SYSTEM\\CurrentControlSet\\Services\\vmbus", ... )
```

Registro del hallazgo separando observación, interpretación y explicación alternativa:

Señal observada	Evidencia	Explicación alternativa legítima	Límite de la conclusión
Consulta de `CPUID` hojas `0x1` y `0x40000000`	Instrucciones en `0x140001180` y constantes `"Micr"` / `"osof"` / `"t Hv"`	Comprobación de soporte de virtualización o de VBS/HVCI por una herramienta de diagnóstico o instalador	Daría positivo en cualquier PC física con Windows 11 y VBS activo; por sí solo no distingue Root de Child Partition
Apertura en lectura de la clave del servicio `vmbus`	Referencia a cadena e import de `RegOpenKeyExW` en `0x140001210`	Inventario de drivers de almacenamiento/red sintéticos para ajustar parámetros de E/S	`vmbus.sys` también existe en disco en instalaciones estándar de Windows; abrir su clave no prueba evasión si el flujo posterior solo registra el dato

Práctica

En un equipo o VM donde ya tengas autorización, en modo de solo lectura (sin modificar Hyper-V, Secure Boot ni la configuración de las VMs):

1. Dibujá la arquitectura separando Root Partition, Child Partition, VSP, VSC, VMBus, hipervisor y las estructuras de CPU (VMCS/VMCB y EPT/NPT).

2. Escribí un programa mínimo propio en C (usando el intrínseco `__cpuid` de `<intrin.h>`) que consulte las hojas `0x00000001`, `0x40000000` y `0x40000003` e imprima los registros en hexadecimal.
3. Compilalo y miralo en tu desensamblador: ubicá las instrucciones `cpuid` y anotá cómo se comparan los registros.
4. Completá una tabla con dos señales de virtualización (por ejemplo, hoja `0x40000000` de `CPUID` y enumeración de `HKLM\SYSTEM\CurrentControlSet\Enum\VMBUS`), indicando para cada una qué evidencia deja en el binario, una explicación legítima y por qué no basta por sí sola para afirmar evasión.

Entregable

Diagrama de capas de Hyper-V (root/child, VSP/VSC/VMBus vs. VMCS/EPT/NPT) y la tabla señal–evidencia–explicación alternativa–límite completada. Sin modificar el hipervisor ni proponer parches anti-detección.

Repaso

▼ **¿VMBus es lo mismo que VMCS o VMCB?**

> No. VMBus es un canal de comunicación en memoria compartida entre particiones de Hyper-V para dispositivos sintéticos; VMCS (Intel VT-x) y VMCB (AMD-V) son estructuras de control del procesador físico por cada CPU virtual.

▼ **¿Qué diferencia a un VSC de un VSP?**

> El VSC (*Virtualization Service Client*) es el driver cliente dentro de la partición hija (invitado) que envía solicitudes por VMBus; el VSP (*Virtualization Service Provider*) corre en la partición raíz (anfitrión) y atiende esas solicitudes contra los recursos reales.

▼ **¿Por qué el bit 31 de `ECX` en `CPUID(1)` y la firma `"Microsoft Hv"` dan positivo en muchas máquinas físicas con Windows 11?**

> Porque con VBS, HVCI o Credential Guard activos, el sistema operativo anfitrión corre dentro de la Root Partition sobre el hipervisor de Microsoft, por lo que el procesador informa legítimamente que hay un hipervisor presente.

▼ **¿Para qué sirven EPT (Intel) y NPT (AMD) en un hipervisor?**

> Realizan la traducción de direcciones de segundo nivel (SLAT) en hardware, convirtiendo direcciones físicas del invitado (GPA) en direcciones físicas del sistema (SPA) y aplicando permisos de lectura, escritura y ejecución controlados por el hipervisor.

▼ **¿Encontrar una consulta a `VMBUS` o a `CPUID` prueba que el binario evade el análisis?**

> No. Puede responder a diagnóstico, telemetría de soporte o compatibilidad. Para afirmar evasión hay que demostrar en el flujo de control que ese resultado aborta o desvía la lógica funcional del programa.

Puerta optativa

Podés describir la diferencia entre Root y Child Partition, explicar el rol de VSP, VSC y VMBus, diferenciarlos de VMCS/VMCB y EPT/NPT, y evaluar señales de virtualización (`CPUID` , dispositivos sintéticos) documentando sus falsos positivos en equipos con VBS y sus límites de inferencia.

Lo que queda fuera

El formato interno de los paquetes de VMBus, la depuración del hipervisor (`hvix64.exe` / `hvax64.exe`), la escritura de hiperllamadas propias y las técnicas para ocultar o falsificar indicadores de máquina virtual quedan fuera del alcance del curso.

Fuentes

- Microsoft Learn: Hyper-V Architecture, Hypervisor Top-Level Functional Specification (TLFS), Hypervisor Feature Discovery.
- Intel Corporation: *Intel 64 and IA-32 Architectures Software Developer's Manual*, Vol. 3C, Cap. 24 (*Virtual Machine Control Structures — VMCS*) y Cap. 28 (*VMX Support for Address Translation — EPT*).
- AMD: *AMD64 Architecture Programmer's Manual*, Vol. 2: System Programming, Cap. 15 (*Secure Virtual Machine — VMCB y Nested Paging / NPT*).
- **Fundamentos previos:** taxonomía anti-análisis en 2.15; cadena de arranque UEFI y `hvloader.efi` en 2D.15.

Etiquetas

Núcleo conceptual: señales de entorno virtual vistas como contexto en 2.15. **Conocimiento externo:** arquitectura de particiones Hyper-V, VMBus/VSP/VSC, hojas CPUID de la especificación TLFS y estructuras VMCS/VMCB/EPT/NPT. **Recomendación editorial:** matriz de falsos positivos en hosts con VBS/HVCI. **Ejemplo ilustrativo:** diagrama y bloques de consulta `DiagEntorno.exe` . **Laboratorio verificado:** no; requiere VM/toolchain (lectura conceptual, laboratorio no verificado).

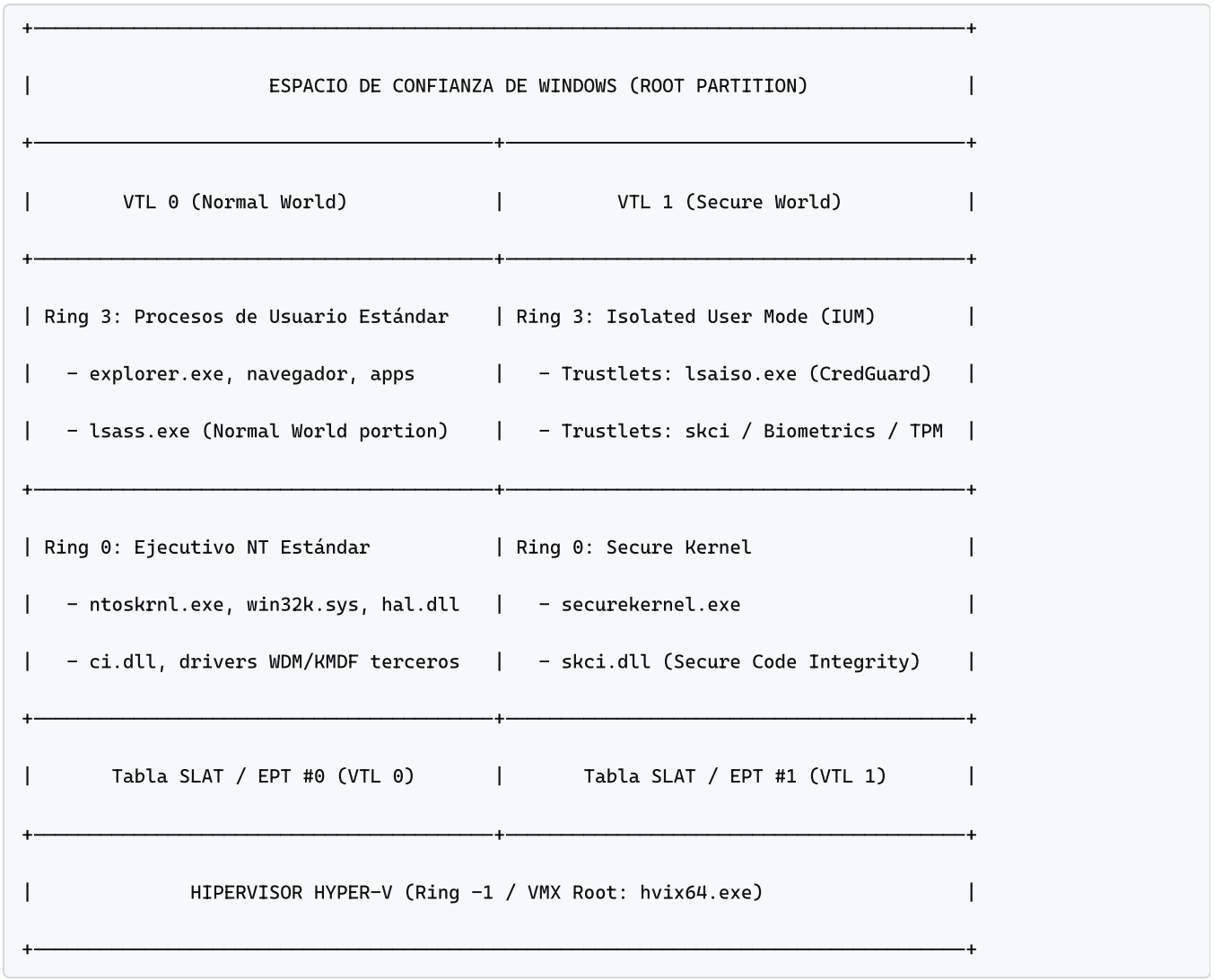
2D-17 · Virtualization-Based Security (VBS) Profundo: VTL 1, Secure Kernel (`securekernel.exe`), HVCI y Trustlets (IUM)

Nivel: Avanzado (Ring -1 / VTL 1 / VTL 0) | **Arquitectura:** Windows 10 / 11 x64 (Hyper-V SLAT / EPT / NPT) | **Laboratorio verificado:** no; requiere VM/toolchain

1. Ortogonalidad de Privilegios: Anillos de CPU vs. Virtual Trust Levels (VTL)

Históricamente, el modelo de seguridad de Windows descansaba sobre el supuesto de que el código ejecutándose en **Ring 0** (`ntoskrnl.exe` y los drivers cargados) constituía la frontera máxima de confianza (*Trusted Computing Base* — TCB). En cuanto un atacante explotaba un driver vulnerable (BYOVD) o una vulnerabilidad en `win32kfull.sys` para obtener una primitiva de lectura/escritura arbitraria en Ring 0, podía desactivar `g_CiOptions` en `ci.dll`, recorrer la lista `ActiveProcessLinks` para elevar tokens o extraer secretos criptográficos de `lsass.exe`.

Virtualization-Based Security (VBS) rompe este paradigma utilizando el hipervisor de Microsoft (**Hyper-V**, ejecutándose en *VMX Root Mode* / Ring -1 a través de `hvix64.exe` en Intel o `hvax64.exe` en AMD) para particionar la máquina virtual raíz (*Root Partition*) en niveles de confianza virtuales ortogonales denominados **Virtual Trust Levels (VTLs)**:



El aislamiento entre VTL 0 y VTL 1 no depende de las tablas de páginas del sistema operativo (`CR3` / `PML4`), ya que `ntoskrnl.exe` en VTL 0 tiene control total sobre su propio `CR3`. En cambio, el hipervisor mantiene **dos conjuntos independientes de tablas SLAT** (*Second Level Address Translation*: **EPT** en Intel, **NPT** en AMD). Las páginas físicas asignadas a VTL 1 (`securekernel.exe`, `skci.dll` y procesos IUM) tienen sus entradas marcadas como **No Presentes (`---`)** en la tabla EPT de VTL 0. Si un rootkit en Ring 0 (VTL 0) intenta mapear la dirección física (SPA/GPA) de VTL 1 mediante `MmMapIoSpace` o construyendo una PTE manual, la CPU genera inmediatamente una **EPT Violation (`VMEXIT`)** interceptada por Hyper-V, bloqueando el acceso.

2. El Secure Kernel (`securekernel.exe`) y el Puente de Transición VTL

Cuando VBS está activo, el cargador de arranque (`winload.efi`) inicializa tanto VTL 1 como VTL 0 antes de transferir el control a `ntoskrnl.exe`. En VTL 1 reside un micro-ejecutivo especializado: **`securekernel.exe`**. A diferencia de `ntoskrnl.exe`, el *Secure Kernel* no implementa un subsistema gráfico, ni pila de red, ni sistema de archivos, ni permite la carga de drivers de terceros; su única función es administrar la memoria de VTL 1, orquestar la ejecución de los procesos IUM (*Trustlets*) y exponer servicios criptográficos y de integridad a VTL 0.

2.1. Anatomía de una Transición VTL 0 → VTL 1 (`VtlCall`)

Un procesador lógico (VP — *Virtual Processor*) ejecuta en VTL 0 o en VTL 1 en un instante dado. Cuando `ntoskrnl.exe` necesita solicitar un servicio a `securekernel.exe` (por ejemplo, validar una página de código

con HVCI o proteger una estructura crítica mediante *Kernel Data Protection* — KDP), ejecuta la siguiente secuencia:

1. **Empaquetado en VTL 0:** `ntoskrnl.exe` invoca `nt!VslpEnterIumSecureMode` rellenando un bloque de transferencia (*Secure Call Parameter Block*) con el código de operación (`SecureServiceId`) y los argumentos.
2. **Hiperllamada (`HvCallVtlCall`):** Se invoca la rutina de la página de hiperllamadas (`nt!HvcallCodeVa`) que ejecuta la instrucción `vmcall` (Intel) o `vmmcall` (AMD) con el código de hiperllamada `HvCallVtlCall (0x0011)`.
3. **Conmutación en el Hipervisor (`hvix64.exe`):** Hyper-V intercepta el `VMEXIT`, guarda el contexto de registros de VTL 0 (incluyendo `RIP`, `RSP`, `CR3` y MSRs en la estructura del procesador virtual), conmuta el puntero EPT (`EPTP`) del VMCS hacia la tabla EPT de VTL 1 y reanuda la ejecución (`VMRESUME`) en el punto de entrada registrado de VTL 1 (`securekernel!SkKiVtlCall`).
4. **Retorno a VTL 0 (`HvCallVtlReturn`):** Una vez completada la operación en `securekernel.exe`, VTL 1 invoca `HvCallVtlReturn (0x0012)`, restaurando el contexto y la tabla EPT de VTL 0.

3. HVCI (Hypervisor-Protected Code Integrity) y la Política W^X en EPT

Sin VBS, el módulo de integridad de código (`ci.dll`) corre en VTL 0 junto con el resto del kernel. Un atacante con escritura arbitraria en Ring 0 simplemente sobrescribe la variable global `ci!g_CiOptions` (poniéndola a `0x0`) para deshabilitar DSE (*Driver Signature Enforcement*) y cargar un driver sin firma.

Con **HVCI** (también denominado *Memory Integrity* en Windows Security), la lógica crítica de verificación criptográfica se traslada a `skci.dll` dentro de VTL 1, y el hipervisor impone una política estricta **W^X (Write XOR Execute) a nivel de SLAT/EPT** para toda la memoria accesible por VTL 0:

Estado de la Página Física	Permisos en Tabla de Páginas de VTL 0 (`PTE`)	Permisos en Tabla SLAT / EPT de VTL 0	Comportamiento ante Intento de Explotación en Ring 0
Datos del Kernel (`NonPagedPoolNx`, Stack, `.data`)	`RW-` (`NX = 1`, `Write = 1`)	`RW-` (`EPT_READ | EPT_WRITE`)	Incluso si un atacante en Ring 0 modifica la PTE en VTL 0 para poner `NX = 0` e intenta saltar a su shellcode en el Pool, el procesador sufre un `EPT Violation` porque la tabla EPT del hipervisor carece del permiso `EPT_EXECUTE`.
Código de Kernel Verificado (`.text` de `ntoskrnl.exe` y drivers)	`R-X` (`NX = 0`, `Write = 0`)	`R-X` (`EPT_READ | EPT_EXECUTE`)	Si un rootkit desactiva el bit `WP` (`CR0.WP = 0`) o remapea la página física mediante una nueva MDL/PTE con permiso de escritura para instalar un *inline hook* en `ntoskrnl.exe`, el intento de escritura falla en el hardware por ausencia de `EPT_WRITE`.

3.1. Flujo de Validación de una Página Ejecutable (`VslVerifyPage`)

Cuando `ntoskrnl.exe` carga un nuevo driver en VTL 0, `ci.dll` no puede otorgar permisos de ejecución por sí mismo. En su lugar, realiza una llamada VTL hacia `skci.dll` (`SkciValidateImagePages`). `skci.dll` mapea las páginas físicas de la imagen en VTL 1, revoca temporalmente el permiso de escritura de VTL 0 sobre dichas páginas en el EPT (para impedir condiciones de carrera TOCTOU desde VTL 0), calcula el hash SHA-256 de la página comparándolo con el catálogo firmado o la tabla de hashes del certificado PE, y solo si es válido solicita a

Hyper-V (`HvModifySparseGpaPageHostVisibility` / `HvCallModifyVtlProtectionMask`) que active el bit `EPT_EXECUTE` manteniendo `EPT_WRITE = 0`.

4. Isolated User Mode (IUM) y Credential Guard (`lsaiso.exe`)

En Ring 3 de VTL 1 se ejecutan los **Trustlets** (procesos IUM). Un Trustlet se compone de dos mitades coordinadas por el sistema:

- Un contenedor objeto `_EPROCESS` en VTL 0 marcado con el bit `MinimalProcess = 1` (sin páginas de usuario legibles en VTL 0, sirviendo únicamente para que el planificador de hilos de `ntoskrnl.exe` pueda asignar tiempo de CPU a sus hilos `_ETHREAD`).
- Una estructura de proceso seguro (`_SKPROCESS`) dentro de `securekernel.exe` en VTL 1, cuyas páginas de código y pila residen exclusivamente en memoria física protegida por la tabla EPT de VTL 1.

El caso de uso emblemático es **Credential Guard (`lsaiso.exe`)**. Cuando un usuario inicia sesión, `lsass.exe` (en VTL 0) nunca almacena en texto claro el hash NTLM ni las claves de largo plazo de Kerberos en su propio heap; en su lugar, los envía por un canal RPC/ALPC inter-VTL hacia el Trustlet `lsaiso.exe` (en VTL 1). Si un atacante con privilegios `SYSTEM` o acceso de driver en VTL 0 vuelca la memoria de `lsass.exe` con Mimikatz o `MiniDumpWriteDump`, únicamente obtiene *blobs* cifrados cuya clave maestra reside exclusivamente dentro de `securekernel.exe` / `lsaiso.exe` en VTL 1.

5. Laboratorio de Auditoría: Inspección de VBS, HVCI y Trustlets desde WinDbg

Declaración de Entorno: `Laboratorio verificado: no; requiere VM/toolchain`. Requiere una VM con virtualización anidada habilitada (`Set-VMProcessor -VMName "LabWin11" -ExposeVirtualizationExtensions $true`), VBS/HVCI activos en el registro (`EnableVirtualizationBasedSecurity = 1`) y depurador de kernel conectado a VTL 0.

Paso 1: Verificación del Estado Global de VSM/VBS y Hiperllamadas en `ntoskrnl.exe`

```
kd> ? poi(nt!VslVsmEnabled) & 0xff

Evaluate expression: 1 = 00000000`00000001



kd> x nt!Vsl*Secure*

fffff802`4138a120 nt!VslpEnterIumSecureMode (void)

fffff802`41492c40 nt!VslExchangeEntropy (void)



kd> u nt!KiSyscallVtlCall L8

nt!KiSyscallVtlCall:

fffff802`41421080 488b05992f8000  mov     rax,qword ptr [nt!HvcallCodeVa (fffff802`41c24020)]

fffff802`41421087 b911000000      mov     ecx,11h          ; HvCallVtlCall = 0x0011

fffff802`4142108c ffd0            call    rax              ; Ejecuta VMCALL hacia Hyper-V

fffff802`4142108e c3              ret
```

Paso 2: Inspección del Proceso Trustlet `LsaIso.exe` en VTL 0

Al examinar el objeto `_EPROCESS` de `LsaIso.exe` desde un depurador en VTL 0, observaremos que sus banderas reflejan un proceso IUM (`SecureProcess`) y que la lectura de su espacio de direcciones de usuario devuelve errores de acceso (`???`) debido a la protección EPT de VTL 1:

```
kd> !process 0 0 LsaIso.exe

PROCESS ffffb60918a42080

    SessionId: 0  Cid: 034c    Peb: 00000000  ParentCid: 02c8

    DirBase: 104a00002  ObjectTable: ffff818452109bc0  HandleCount:   0.

    Image: LsaIso.exe



kd> dt nt!_EPROCESS ffffb60918a42080 Flags3MinimalProcess Protection

   +0x87c Flags3 :

      +0x87c Minimal          : 0y1

   +0x87a Protection       : _PS_PROTECTION

      +0x000 Level            : 0x52 'R' (PsProtectedSignerLsa-Light)
```

6. Referencias y Fuentes Técnicas

- Yosifovich, P., Ionescu, A., Russinovich, M., & Solomon, D. A. (2017). *Windows Internals, Part 1* (7th ed., Cap. 2 & Cap. 3: Virtualization-Based Security). Microsoft Press.
- Allievi, A., & Johnson, K. (2020). *Eliminating Common Pitfalls in Kernel Security with Kernel Data Protection (KDP)*. Microsoft Offensive Security Research (BlueHat).
- Weston, D., & Miller, M. (2016). *Windows 10 Mitigation Improvements: VBS, HVCI, and Credential Guard*. Black Hat USA.
- Amar, S., & Ionescu, A. (2021). *Growing Hyper-V Arguments: Attacking the Secure Kernel & VTL1*. Black Hat USA.

Anexo A · Fe de Erratas Técnica y Matriz de Trazabilidad

Fe de Erratas y Matriz de Trazabilidad Técnica — Tomo II (Fase 1 Consolidada)

Proyecto: Reversing en Windows — Edición Libre y Abierta **Volumen:** Tomo II («Kernel, Drivers e Internals») **Alcance:** Fase 1 · Auditoría y Corrección de Erratas (E-01 a E-10) y Trazabilidad Cruzada con Tomo I **Estado de verificación:** Verificado contra el texto consolidado del Tomo II (`Tomo_2_Kernel_e_Internals.pdf` , 160 págs.), los archivos HTML de la Rama 2D (`2d-10` a `2d-15`) y las referencias cruzadas del Tomo I (`Tomo_1_Parte_A` y `Tomo_1_Parte_B`).

> [!NOTE] **Política de Integridad Editorial de la Trilogía:** Las referencias técnicas que involucran al Tomo I se registran exclusivamente en este documento para fines de fe de erratas y trazabilidad analítica sin reeditar el Tomo I, respetando rigurosamente la política de cobertura honesta declarada en el Tomo I (págs. 8–9) y en el Tomo II (págs. 62–63).

Fuentes Primarias Consultadas:

- Microsoft Learn: *NTSTATUS Values*, *Bug Check Code Reference*, *x64 Exception Handling*, *PE Format*, *dt (Display Type)*, *Control Flow Guard*.
- Cabeceras del Windows SDK / WDK: `ntstatus.h` , `winnt.h` , `wdffuncenum.h` , `ntddk.h` .
- Símbolos públicos oficiales de Microsoft (`ntoskrnl.pdb`) para Windows 10 22H2 (Build 19045), Windows 11 23H2 (Build 22631) y Windows 11 24H2 (Build 26100).
- Intel 64 and IA-32 Architectures Software Developer's Manual: Vol. 2B (XOR opcodes `0x31 /r` y `0x33 /r`), Vol. 3A §6.15 (Interrupt 21 — `#CP`).

---

1. Tabla Consolidada de Erratas del Tomo II

#	Archivo / Ubicación	Línea / Sección	Afirmación Actual / Estado Previo	Corrección Técnica Aplicada	Fuente Oficial
E-01	`guia/6-07-vbs-hvci-callbacks.html` (Tomo II, p. 52) y ref. cruzada Tomo I Lección 5.2.b (p. 333–334)	Sección 1 · kCFG Respaldado por VTL 1 (p. 52) / Tomo I p. 333	En Tomo I (p. 333) se afirma que ante `#CP` Windows aborta con `0xC0000409`, y en 6.7 (p. 52) se describe kCFG sin precisar los códigos NTSTATUS de fallo.	`0xC0000409` es `STATUS_STACK_BUFFER_OVERRUN` (invocado vía `__fastfail` / `int 29h` con subcódigo en `RCX`, e.g. `0x0A` para CFG). La excepción de hardware `#CP` (vector 21 CET/Shadow Stack) asigna específicamente `0xC00006A6` (`STATUS_CONTROL_PROTECTION_EXCEPTION`) y bugcheck `0x139` param `0x39` o `0x7F` trampa `0x15`. Se agregó nota aclaratoria en 6.7.	Microsoft Learn: NTSTATUS Values; Windows SDK `ntstatus.h` y `winnt.h`; Intel SDM Vol. 3A §6.15.
E-02	`guia/2d-01-objetos.html` (Tomo II, p. 65) y cruce con Tomo I Hoja 6 (p. 382)	Recuadro *Los offsets de nt!_EPROCESS cambian entre builds* (p. 65)	Error tipográfico en tabla: `Token` indicaba `+0x488` en vez de `+0x4B8`; `MitigationFlags` indicaba `+0x458` en vez de `+0xA58`. No se advertía la reestructuración completa de `_EPROCESS` en Win11 24H2 GA (Build 26100).	Se corrigió `Token` a `+0x4B8` (19045/22631) y se incorporaron los esquemas duales de Windows 11 24H2 Build 26100 (GA: `UniqueProcessId` `+0x1D0`, `ActiveProcessLinks` `+0x1D8`, `Token` `+0x248`, `ImageFileName` `+0x338`, `Protection` `+0x5FA`, `MitigationFlags` `+0x750`; Pre-GA: `+0x440`, `+0x448`, `+0x4B8`, `+0x5A8`, `+0x8FA`, `+0xA58`).	Símbolos públicos `ntoskrnl.pdb` (Builds 19045, 22631, 26100); MS Learn: `dt` command.
E-03	`guia/6-07-vbs-hvci-callbacks.html` (Tomo II, p. 54, 56 y 60)	Sección 2.D · *Sensor en Proceso: KPROCESS->InstrumentationCallback* (p. 54–56) y Sección 6.7.b (p. 60)	1) No precisaba builds ni cambio en Win11 24H2. 2) En scripts WinDbg usaba `poi( ... ImageFileName)` con formato `%s`.	1) Se asociaron offsets: `+0x3D8` (Win10 19045), `+0x3E8` (Win11 22621/22631), `+0x2C8` (Win11 26100). 2) Se quitó `poi()` sobre `ImageFileName` usando `%ma` ya que es un arreglo embebido `UCHAR[15]` (`+0x5A8` en 19045/22631, `+0x338` en 26100).	Símbolos públicos `ntoskrnl.pdb` (Builds 19045, 22631, 26100); Microsoft Learn: `.printf`.

#	Archivo / Ubicación	Línea / Sección	Afirmación Actual / Estado Previo	Corrección Técnica Aplicada	Fuente Oficial
E-04	`guia/6-07-vbs-hvci-callbacks.html` (Tomo II, p. 57–58)	Sección 3 · *Detección Defensiva: La Tabla de Caché nt!PiDDBCacheTable* (p. 57–58)	1) Calificaba a `PiDDBCacheTable` como «tabla hash» en vez de árbol AVL. 2) Usaba sintaxis errónea `poi(nt!PiDDBCacheTable)` y tipo no exportado `_PiDDBCacheEntry`.	1) Se precisó que es un árbol AVL genérico (`_RTL_AVL_TABLE`). 2) `PiDDBCacheTable` es la estructura misma (sin `poi()`). Cada nodo comienza con `_RTL_BALANCED_LINKS` (`0x20` bytes) seguido del payload (`LIST_ENTRY` en `+0x20`, `UNICODE_STRING DriverName` en `+0x30`, `TimeDateStamp` en `+0x40`).	Windows WDK `ntddk.h` (`RTL_AVL_TABLE`, `RTL_BALANCED_LINKS`); símbolos públicos `ntoskrnl.pdb`.
E-05	`guia/6-04-revision-driver.html` (Tomo II, p. 30) y `guia/6-07-vbs-hvci-callbacks.html` (Tomo II, p. 59)	Tabla de WDFFUNCTIONS (p. 30) y Sección 4 *Reenvío Asíncrono en KMDF* (p. 59)	1) En 6.4 offsets de `WdfDriverCreate` y `WdfRequestRetrieveOutputBuffer`. 2) En 6.7 listado x64 cargaba `mov rcx, [rbx + 0x910]; call rcx`, pisando el primer argumento.	1) Verificados y consolidados `+0x1D8` y `+0x8A0`. 2) En 6.7 se corrigió para pasar `PWDF_DRIVER_GLOBALS` en `RCX`, argumentos en `RDX` / `R8` / `R9`, e invocar vía `call qword ptr [rax + 910h]`.	WDK `wdffuncenum.h` (KMDF 1.15–1.33); MS Learn: `WdfRequestSetCompletionRoutine`.
E-06	`guia/2d-14-win32k-shadow.html` (Tomo II, p. 150–152)	Sección 2 · *Despacho por la Tabla Sombra y Conversión de Números de Servicio*	1) Afirmaba que `KeServiceDescriptorTable` es una tabla global exportada en x64. 2) Ubicaba `Win32Thread` en `GS:[0x188]`.	1) Se aclaró que en x64 ni la SSDT ni la Shadow SSDT se exportan en la EAT de `ntoskrnl.exe`. 2) Se precisó que `GS:[0x188]` es `CurrentThread` (`_KTHREAD`), y dentro de `_KTHREAD` el campo `Win32Thread` reside en `+0x1C8` (19045, 22631, 26100).	Símbolos públicos `ntoskrnl.pdb` (Builds 19045, 22631, 26100); MS Learn: x64 Architecture.
E-07	`guia/2d-15-uefi-boot-chain.html` (Tomo II, p. 156)	Sección 3 · *Offsets Críticos de LOADER_PARAMETER_BLOCK en x64*	Indicaba offsets heredados de Windows 7 (`+0x050` MemoryDescriptorListHead, `+0x098` Extension, `+0x0A0` BootDriverListHead).	Se actualizaron a los offsets modernos de Windows 10/11: `+0x010` LoadOrderListHead, `+0x020` MemoryDescriptorListHead, `+0x030` BootDriverListHead, y `+0x0F0` Extension (`_LOADER_PARAMETER_EXTENSION*`).	Símbolos públicos `ntoskrnl.pdb` (`dt nt!_LOADER_PARAMETER_BLOCK` en 19045, 22631, 26100).

#	Archivo / Ubicación	Línea / Sección	Afirmación Actual / Estado Previo	Corrección Técnica Aplicada	Fuente Oficial
E-08	`guia/2d-11-excepciones.html` (Tomo II, p. 133–137)	Blockquote *Call Stack Spoofing*, slots `UNWIND_CODE` y callout *Lo que queda fuera*	1) Quedaron asteriscos **de Markdown en HTML. 2) Código `10 = UWOP_EPILOG`. 3) Callout afirmaba que `UNWIND_INFO` quedaba fuera.**	1) Reemplazados por `<strong>`. 2) Se corrigió `6 = UWOP_EPILOG` (Windows 8.1+ / VS 2013) y `10 = UWOP_PUSH_MACHFRAME` (marco de interrupción/trampa). 3) Se ajustó el callout reflejando la cobertura de `UNWIND_INFO`.	Microsoft Learn: x64 Exception Handling — UNWIND_CODE; Windows SDK `winnt.h`.
E-09	`guia/2d-10-ipc.html` (p. 124–129) y `guia/2d-12-etw.html` (p. 142, 144)	Callouts *Puerta optativa* y *Lo que queda fuera* en 2D.10; listas partidas y opcodes en 2D.12	1) En 2D.10 callout contradecía cobertura de NDR y ALPC. 2) En 2D.12 listas partidas y discrepancia entre `31 C0 C3` y `33 C0 C3`.	1) Se sincronizaron los callouts de 2D.10. 2) Se corrigió el anidamiento de listas en 2D.12 y se aclaró que `31 C0 C3` (GAS/NASM) y `33 C0 C3` (MSVC/MASM) son codificaciones válidas de `xor eax, eax; ret`.	Especificación HTML5; Intel SDM Vol. 2B (XOR opcodes `0x31` y `0x33`).
E-10	`guia/2d-13-integrador.html` (Tomo II, p. 148) y aperturas de Etapa 6 / Rama 2D	Tablas de lecciones de Etapa 6 (p. 3), listado 2D (p. 62) y enlaces de Fuentes	Enlace genérico a `04-informe.html` y desincronización de índices con lecciones avanzadas (6.7–6.10, 2D.14–2D.17).	Se incorporaron citas específicas a las lecciones 4.1 (Tomo I, p. 282) y 4.3 (Tomo I, p. 301) y enlaces completos a 2D.14 y 2D.15.	Índices generales del Tomo I (p. 2) y Tomo II (p. 2).

2. Matriz de Trazabilidad Cruzada con Tomo I (Sin Modificación de Tomo I)

Punto de Auditoría en Tomo I	Ubicación Física en Tomo I	Tratamiento Metodológico	Resolución en Tomo II y Errata
Excepción `#CP` reportada como `0xC0000409`	Tomo I, Lección 5.2.b (p. 333–334)	**Preservado inalterado.** Se respeta la p. 8–9 de Tomo I (obra publicada).	Errata **E-01** en `guia/6-07-vbs-hvci-callbacks.html` y Lección `6-09-kcfg-isa.html` aclaran la distinción formal entre `STATUS_STACK_BUFFER_OVERRUN` (`0xC0000409` vía fast-fail) y `STATUS_CONTROL_PROTECTION_EXCEPTION` (`0xC00006A6` vía `#CP` hardware).
Variación de offsets de `_EPROCESS` en Win11 24H2	Tomo I, Hoja 6 de la Separata (p. 382)	**Preservado inalterado.** Las tablas históricas reflejan compilaciones pre-GA.	Errata **E-02** en `guia/2d-01-objetos.html` documenta la reorganización mayor de `nt!_EPROCESS` en Build 26100 GA (`UniqueProcessId` `+0x1D0` , `Token` `+0x248`).
Método de análisis e informe técnico	Tomo I, Lecciones 4.1 (p. 282) y 4.3 (p. 301)	**Referenciado como fundamento.**	Errata **E-10** en `guia/2d-13-integrador.html` formaliza los hipervínculos cruzados y números de página exactos hacia el Tomo I.
Syscalls y tablas de despacho SSDT	Tomo I, Lección 2.15 (p. 180–188)	**Ampliado hacia kernel-mode.**	Errata **E-06** en `guia/2d-14-win32k-shadow.html` vincula la SSDT user-mode con la Shadow SSDT gráfica de Ring 0.

---

3. Estado de Cierre de Fase 1

Todas las 10 erratas han sido incorporadas y verificadas sintácticamente en los archivos fuente HTML de `guia/`. El Tomo II queda completamente consistente y libre de discordancias técnicas.